Path: miniprogram/app.js

```
---------------------------------------------------------------------------
const USER_INFO_KEY = 'userInfo'
const LOGOUT_FLAG_KEY = 'manualLoggedOut'
const RESET_KEYS = [
  USER_INFO_KEY,
  LOGOUT_FLAG_KEY,
  'selectedRegion',
  'userLocation',
  'recentRegionList',
]

function getSettingAsync() {
  return new Promise((resolve, reject) => {
    wx.getSetting({
      success: resolve,
      fail: reject,
    })
  })
}

function getLocationAsync(timeout = 8000) {
  return new Promise((resolve, reject) => {
    const timer = setTimeout(() => {
      reject(new Error('getLocation timeout'))
    }, timeout)

    wx.getLocation({
      type: 'gcj02',
      success: (res) => {
        clearTimeout(timer)
        resolve(res)
      },
      fail: (err) => {
        clearTimeout(timer)
        reject(err)
      },
    })
  })
}

App({
  globalData: {
    userInfo: null,
    templateId: {},
  },

  onLaunch(options) {
    wx.cloud.init({
      env: 'cloud1-3ghmr5ki7b1172fe',
      traceUser: true,
    })

    this.bootstrap(options)
  },

  onShow() {
    this.refreshUserLocationCache()
  },

  async bootstrap(options = {}) {
    try {
```

```
      const targetUrl = this.getLaunchTarget()

      if (targetUrl) {
        setTimeout(() => {
          wx.reLaunch({ url: targetUrl })
        }, 80)
      }
    } catch (err) {
      console.error('bootstrap failed', err)
    }
  },

  async refreshUserLocationCache() {
    try {
      const setting = await getSettingAsync()
      if (!setting || !setting.authSetting || !setting.authSetting['scope.userLocation'] ) {
        return
      }

      const locationRes = await getLocationAsync()
      const userLocation = {
        latitude: locationRes.latitude,
        longitude: locationRes.longitude,
      }

      wx.setStorageSync('userLocation', userLocation)

      const currentUserInfo = this.globalData.userInfo || wx.getStorageSync(USER_INFO_KEY)
      if (currentUserInfo && currentUserInfo.openid) {
        const nextUserInfo = {
          ...currentUserInfo,
          userLocation,
          locationAuthorized: true,
        }
        this.globalData.userInfo = nextUserInfo
        wx.setStorageSync(USER_INFO_KEY, nextUserInfo)
      }
    } catch (err) {
      console.warn('refreshUserLocationCache failed', err)
    }
  },

  getLaunchTarget() {
    const agreePrivacy = wx.getStorageSync('agreePrivacy')
    if (!agreePrivacy) {
      return '/pages/welcome/welcome'
    }

    if (wx.getStorageSync(LOGOUT_FLAG_KEY)) {
      this.globalData.userInfo = null
      return '/pages/home/home'
    }

    const userInfo = this.getUserInfo()
    if (!userInfo) {
      return '/pages/home/home'
    }

    this.globalData.userInfo = userInfo
    return this.getPostLoginTarget(userInfo)
```

```
  },

  getPostLoginTarget(userInfo) {
    if (userInfo.onboardingCompleted) {
      return '/pages/home/home'
    }

    if (!userInfo.profileCompleted) {
      return '/pages/registerProfile/registerProfile?mode=register'
    }

    if (!userInfo.locationChoiceMade) {
      return '/pages/permissionSettings/permissionSettings?mode=onboarding'
    }

    if (!userInfo.dnaCompleted) {
      return '/pages/dnaTag/dnaTag?mode=setup'
    }

    return '/pages/home/home'
  },

  setUserInfo(userInfo) {
    if (!userInfo || !userInfo.openid) {
      this.clearUserInfo({ markLoggedOut: false })
      return
    }

    const currentUserInfo = this.globalData.userInfo || wx.getStorageSync(USER_INFO_KEY) || {}
    const mergedUserInfo = {
      ...currentUserInfo,
      ...userInfo,
    }

    if (!userInfo.avatarPreviewUrl && currentUserInfo.avatarPreviewUrl && currentUserInfo.avatarUrl === mergedUserInfo.avatarUrl) {
      mergedUserInfo.avatarPreviewUrl = currentUserInfo.avatarPreviewUrl
    }

    if (!userInfo.avatarResolvedUrl && currentUserInfo.avatarResolvedUrl && currentUserInfo.avatarUrl === mergedUserInfo.avatarUrl) {
      mergedUserInfo.avatarResolvedUrl = currentUserInfo.avatarResolvedUrl
    }

    this.globalData.userInfo = mergedUserInfo
    wx.setStorageSync(USER_INFO_KEY, mergedUserInfo)
    wx.removeStorageSync(LOGOUT_FLAG_KEY)
  },

  clearUserInfo(options = {}) {
    const { markLoggedOut = true } = options
    this.globalData.userInfo = null
    wx.removeStorageSync(USER_INFO_KEY)
    if (markLoggedOut) {
      wx.setStorageSync(LOGOUT_FLAG_KEY, true)
    } else {
      wx.removeStorageSync(LOGOUT_FLAG_KEY)
    }
  },

  getUserInfo() {
```

```
    if (wx.getStorageSync(LOGOUT_FLAG_KEY)) {
      return null
    }

    const userInfo = this.globalData.userInfo || wx.getStorageSync(USER_INFO_KEY)
    if (!userInfo || !userInfo.openid) {
      return null
    }

    return userInfo
  },

  hasActiveSession(options = {}) {
    const userInfo = this.getUserInfo()
    if (!userInfo) {
      return false
    }

    return true
  },

  clearLocalTestData() {
    this.globalData.userInfo = null
    RESET_KEYS.forEach((key) => {
      wx.removeStorageSync(key)
    })
  },
})
```

Path: miniprogram/app.json

```
{
  "pages": [
    "pages/home/home",
    "pages/message/message",
    "pages/cart/cart",
    "pages/me/me",
    "pages/welcome/welcome",
    "pages/login/login",
    "pages/notePublish/notePublish",
    "pages/noteDetail/noteDetail",
    "pages/messageInteraction/messageInteraction",
    "pages/messageMerchant/messageMerchant",
    "pages/messageMerchantChat/messageMerchantChat",
    "pages/messagePlatform/messagePlatform",
    "pages/messagePlatformSupport/messagePlatformSupport",
    "pages/messagePlatformGuarantee/messagePlatformGuarantee",
    "pages/messageBuddyApply/messageBuddyApply",
    "pages/messageBuddyApplyChat/messageBuddyApplyChat",
    "pages/messageConversation/messageConversation",
    "pages/permissionSettings/permissionSettings",
    "pages/registerProfile/registerProfile",
    "pages/dnaTag/dnaTag",
    "pages/buddyIntent/buddyIntent",
    "pages/activityDetail/activityDetail",
    "pages/productDetail/productDetail",
    "pages/scenicDetail/scenicDetail",
    "pages/hotelDetail/hotelDetail",
    "pages/search/search",
    "pages/searchResults/searchResults",
    "pages/askXiaohe/askXiaohe",
    "pages/askXiaoheChat/askXiaoheChat",
    "pages/regionSelect/regionSelect",
    "pages/book/book",
    "pages/order/order",
    "pages/travelerSelect/travelerSelect",
    "pages/travelerEdit/travelerEdit",
    "pages/myActivities/myActivities",
    "pages/activityOrderDetail/activityOrderDetail",
    "pages/activityReview/activityReview",
    "pages/myOrders/myOrders",
    "pages/reviewCenter/reviewCenter",
    "pages/favorites/favorites",
    "pages/favoritesEdit/favoritesEdit",
    "pages/ruralFun/ruralFun",
    "pages/academy/academy",
    "pages/knowledgeDetail/knowledgeDetail",
    "pages/hotel/hotel",
    "pages/mall/mall",
    "pages/qaCenter/qaCenter"
  ],
  "permission": {
    "scope.userLocation": {
      "desc": "\u7528\u4e8e\u4e3a\u60a8\u63a8\u8350\u9644\u8fd1\u9152\u5e97\u6c11\u5bbf\u3001\u6d3b\u52a8\u4e0e\u666f\u70b9\uff0c\u5e76\u663e\u793a\u5f53\u524d\u6240\u5728\u57ce\u5e02"
    }
  },
  "requiredPrivateInfos": [
    "getLocation",
    "chooseLocation"
```

```
  ],
  "window": {
    "navigationBarTitleText": "\u519c\u65c5e\u7ad9",
    "navigationBarBackgroundColor": "#F8F8F8",
    "navigationBarTextStyle": "black"
  },
  "lazyCodeLoading": "requiredComponents",
  "tabBar": {
    "color": "#999999",
    "selectedColor": "#57984a",
    "list": [
      { "pagePath": "pages/home/home", "text": "\u9996\u9875" },
      { "pagePath": "pages/message/message", "text": "\u6d88\u606f" },
      { "pagePath": "pages/cart/cart", "text": "\u8d2d\u7269\u8f66" },
      { "pagePath": "pages/me/me", "text": "\u6211\u7684" }
    ]
  }
}
```

Path: miniprogram/app.wxss

```
/**app.wxss**/
:root {
  --theme-page-bg: #f6faf4;
  --theme-panel-bg: #ffffff;
  --theme-soft-bg: #edf5ea;
  --theme-soft-bg-strong: #e4f0df;
  --theme-primary: #57984a;
  --theme-primary-dark: #27692a;
  --theme-primary-light: #dff0dc;
  --theme-text-main: #1f2a1f;
  --theme-text-sub: #667566;
  --theme-text-muted: #99a598;
  --theme-border: #dbe8d7;
  --theme-danger: #d9544f;
}

.container {
  display: flex;
  flex-direction: column;
  align-items: center;
  box-sizing: border-box;
}

button {
  background: initial;
}

button:focus{
  outline: 0;
}

button::after{
  border: none;
}


page {
  background: var(--theme-page-bg);
  color: var(--theme-text-main);
  display: flex;
  flex-direction: column;
  justify-content: flex-start;
}
```

Path: miniprogram/envList.js

```
const envList = [];
const isMac = false;
module.exports = {
  envList,
  isMac
};
```

Path: miniprogram/sitemap.json

```
1 {
2   "desc": "                 https://developers.weixin.qq.com/miniprogram/dev/framework/s
  itemap.html",
3   "rules": [{
4     "action": "allow",
5     "page": "*"
6   }]
7 }
```

Path: miniprogram/config/agent.js

```
module.exports = {
  yuxiaoheBotId: "agent-yuxiaohe-1grmumm967563411",
  yuxiaoheEnvId: "cloud1-3ghmr5ki7b1172fe",
  yuxiaoheServiceName: "agent-yuxiaohe-1grmumm967563411",
  yuxiaoheHttpBaseUrl: "https://cloud1-3ghmr5ki7b1172fe-1403917845.ap-shanghai.app.tcloudbase.com"
}
```

Path: miniprogram/components/cloudTipModal/index.js

```
Component({

  /**
   *
   */
  data: {
    showTip: false,
  },
  properties: {
    showTipProps: Boolean,
    title:String,
    content:String
  },
  observers: {
    showTipProps: function(showTipProps) {
      this.setData({
        showTip: showTipProps
      });
    }
  },
  methods: {
    onClose(){
      this.setData({
        showTip: !this.data.showTip
      });
    },
  }
});
```

Path: miniprogram/components/cloudTipModal/index.json

```
1 {
2   "usingComponents": {},
3   "component": true
4 }
```

Path: miniprogram/components/cloudTipModal/index.wxml

```
<!--miniprogram/components/cloudTipModal/index.wxml-->
<!--  wx:if="{{showUploadTip}}" -->
<view class="install_tip" wx:if="{{showTip}}">
  <view class="install_tip_back"></view>
  <view class="install_tip_detail">
    <image class="install_tip_close" bind:tap="onClose" src="../../images/icons/close.png"/>
    <view class="install_tip_detail_title">{{title}}</view>
    <view class="install_tip_detail_tip">{{content}}</view>
  </view>
</view>
```

Path: miniprogram/components/cloudTipModal/index.wxss

```
.install_tip_back {
  position: fixed;
  top: 0;
  right: 0;
  bottom: 0;
  left: 0;
  background-color: rgba(0,0,0,0.4);
  z-index: 1;
}
.install_tip_close{
  position:absolute;
  right: 10rpx;
  top: 10rpx;
  width: 32px;
  height: 32px;
  /* background-color: red; */
}
.install_tip_detail {
  position: fixed;
  background-color: white;
  right: 0;
  bottom: 0;
  left: 0;
  border-radius: 40rpx 40rpx 0 0;
  padding: 50rpx 50rpx 100rpx 50rpx;
  z-index: 9;
}

.install_tip_detail_title {
  font-weight: 400;
  font-size: 40rpx;
  text-align: center;
}

.install_tip_detail_tip {
  font-size: 25rpx;
  color: rgba(0,0,0,0.4);
  margin-top: 20rpx;
  text-align: left;
}
.install_tip_detail_buttons {
  padding-top: 50rpx;
  display: flex;
}
.install_tip_detail_button {
  color: #07C160;
  font-weight: 500;
  background-color: rgba(0,0,0,0.1);
  width: 40%;
  text-align: center;
  /* height: 90rpx; */
  /* line-height: 90rpx; */
  border-radius: 10rpx;
  margin: 0 auto;
}

.install_tip_detail_button_primary {
  background-color: #07C160;
  color: #fff;
}
```

Path: miniprogram/utils/activityCoverTags.js

```
---------------------------------------------------------------------------
const GENERIC_TAGS = new Set([
  '\u4e61\u6751',
  '\u519c\u65c5',
  '\u6587\u65c5',
  '\u4f53\u9a8c',
  '\u6d3b\u52a8',
  '\u7f8e\u98df',
  '\u73a9\u6cd5',
  '\u65b9\u5f0f',
  '\u51fa\u884c',
  '\u51fa\u6e38',
  '\u5fae\u5ea6\u5047',
  '\u5468\u672b\u77ed\u9014',
  '\u6444\u5f71',
  '\u89c2\u5149',
  '\u624b\u4f5c',
  '\u975e\u9057',
  '\u91c7\u6458',
  '\u82b1\u6d77',
  '\u7530\u56ed',
  '\u7267\u573a',
  '\u6c11\u5bbf',
  '\u9152\u5e97',
  '\u5065\u5eb7\u517b\u751f',
  '\u4eb2\u5b50\u7814\u5b66',
  '\u51fa\u7247\u6253\u5361',
])

const NOISE_WORDS = [
  '\u4f53\u9a8c',
  '\u4e4b\u65c5',
  '\u4e4b\u884c',
  '\u4e3b\u9898',
  '\u6587\u5316',
  '\u98ce\u60c5',
  '\u9650\u5b9a',
  '\u6d3b\u52a8',
  '\u7ebf\u8def',
  '\u5fae\u5ea6\u5047',
  '\u6162\u6e38',
  '\u89c2\u5149',
  '\u6253\u5361',
  '\u73a9\u6cd5',
  '\u65b9\u5f0f',
  '\u9002\u5408',
  '\u53ef\u9009',
  '\u4f18\u5148',
  '\u53ef\u62fc\u8f66',
  '\u62fc\u8f66',
  '\u8f7b\u677e',
]

const NONE_STAY_PATTERNS = [
  '\u4e0d\u542b\u4f4f\u5bbf',
  '\u65e0\u4f4f\u5bbf',
  '\u4e0d\u542b\u4f4f',
  '\u65e0\u4f4f',
]

function normalizeText(value = '') {
```

```
  return String(value || '').trim()
}

function normalizeList(value) {
  return Array.isArray(value) ? value.map(normalizeText).filter(Boolean) : []
}

function containsAny(text, patterns) {
  return patterns.some((pattern) => text.includes(pattern))
}

function normalizeCompareText(value) {
  let text = normalizeText(value)
  text = text.replace(/\u534a\u65e5/gu, '\u534a\u5929')
  text = text.replace(/\u4e00\u65e5/gu, '1\u5929')
  text = text.replace(/\u519c\u5bb6\u5348\u9910/gu, '\u519c\u5bb6\u98ce\u5473\u9910')
  text = text.replace(/\u519c\u5bb6\u665a\u9910/gu, '\u519c\u5bb6\u98ce\u5473\u9910')
  text = text.replace(/\u519c\u5bb6\u996d/gu, '\u519c\u5bb6\u98ce\u5473\u9910')
  text = text.replace(/\u4e61\u6751\u5348\u9910/gu, '\u519c\u5bb6\u98ce\u5473\u9910')
  text = text.replace(/\u5de5\u574a/gu, '\u624b\u4f5c')
  text = text.replace/(\u624b\u4f5c)+/gu, '\u624b\u4f5c')
  text = text.replace(/[\u00b7\u30fb\u3001\uff0c,\s\/| -]+/gu, '')
  NOISE_WORDS.forEach((word) => {
    text = text.split(word).join('')
  })
  return text
}

function isSimilarTag(candidate, existingList) {
  const normalizedCandidate = normalizeCompareText(candidate)
  if (!normalizedCandidate) {
    return false
  }

  return existingList.some((item) => {
    const normalizedItem = normalizeCompareText(item)
    if (!normalizedItem) {
      return false
    }

    return (
      normalizedItem === normalizedCandidate ||
      normalizedItem.includes(normalizedCandidate) ||
      normalizedCandidate.includes(normalizedItem)
    )
  })
}

function pushCandidate(target, candidate, excludedList = [], max = 5) {
  const text = normalizeText(candidate)
  if (!text || target.length >= max || GENERIC_TAGS.has(text)) {
    return
  }

  if (isSimilarTag(text, excludedList) || isSimilarTag(text, target)) {
    return
  }

  target.push(text)
}

function appendCandidates(target, candidates, excludedList, max, limit) {
```

```
  const local = []
  candidates.forEach((candidate) => {
    pushCandidate(local, candidate, excludedList, limit)
  })
  local.forEach((candidate) => {
    pushCandidate(target, candidate, excludedList, max)
  })
}

function buildDurationTag(days, durationText) {
  if (typeof days === 'number' && days >= 2) {
    return `${days}\u5929${days - 1}\u665a`
  }

  if (typeof days === 'number' && days > 0 && days < 1) {
    return normalizeText(durationText) || '\u534a\u5929'
  }

  if (typeof days === 'number' && days > 0) {
    return `${days}\u5929`
  }

  return normalizeText(durationText) || '1\u5929'
}

function collectServiceTags(activity, target, excludedList, max) {
  const transportText = normalizeText(activity.transport)
  const pickupPoint = normalizeText(activity.pickupPoint)
  const candidates = []

  if (transportText) {
    if (transportText.includes('\u4fdd\u59c6\u8f66')) {
      candidates.push('\u4fdd\u59c6\u8f66')
    }
    if (transportText.includes('\u4e00\u5355\u4e00\u56e2')) {
      candidates.push('\u4e00\u5355\u4e00\u56e2')
    }
    if (containsAny(transportText, ['\u4e13\u8f66', '\u5305\u8f66'])) {
      candidates.push('\u4e13\u8f66\u670d\u52a1')
    }
    if (containsAny(transportText, ['\u5546\u52a1\u8f66'])) {
      candidates.push('\u5546\u52a1\u8f66\u63a5\u9001')
    }
    if (containsAny(transportText, ['\u5c0f\u5df4', '\u5927\u5df4', '\u5df4\u58eb', '\u65c5\u6e38\u5df4\u58eb', '\u56e2\u961f\u5df4\u58eb'])) {
      candidates.push('\u5df4\u58eb\u63a5\u9001')
    }
    if (containsAny(transportText, ['\u81ea\u9a7e'])) {
      candidates.push('\u81ea\u9a7e\u53ef\u9009')
    }
  }

  if (activity.pickup) {
    if (containsAny(pickupPoint, ['\u9ad8\u94c1', '\u706b\u8f66\u7ad9', '\u8f66\u7ad9', '\u7ad9'])) {
      candidates.push('\u7ad9\u70b9\u63a5\u9001')
    } else if (containsAny(pickupPoint, ['\u9152\u5e97', '\u6c11\u5bbf'])) {
      candidates.push('\u4f4f\u5bbf\u63a5\u9001')
    } else if (containsAny(pickupPoint, ['\u5546\u5708', '\u5730\u94c1', '\u57ce\u5173', '\u65b0\u533a'])) {
      candidates.push('\u5546\u5708\u63a5\u9001')
    } else {
```

```
      candidates.push('\u5b9a\u70b9\u63a5\u9001')
    }
  }

  appendCandidates(target, candidates, excludedList, max, 2)
}

function collectStayTags(activity, target, excludedList, max) {
  const stayText = normalizeText(activity.stay)
  if (!stayText || containsAny(stayText, NONE_STAY_PATTERNS)) {
    return
  }
  const candidates = []

  if (containsAny(stayText, ['\u7cbe\u54c1\u6c11\u5bbf', '\u7279\u8272\u6c11\u5bbf'])) {
    candidates.push('\u7279\u8272\u6c11\u5bbf')
  } else if (stayText.includes('\u6c11\u5bbf')) {
    candidates.push('\u6c11\u5bbf\u53ef\u9009')
  }

  if (containsAny(stayText, ['\u9152\u5e97', '\u53cc\u5e8a', '\u5927\u5e8a'])) {
    candidates.push('\u9152\u5e97\u53ef\u9009')
  }

  if (containsAny(stayText, ['\u5e10\u7bf7', '\u9732\u8425'])) {
    candidates.push('\u5e10\u7bf7\u9732\u8425')
  }

  if (containsAny(stayText, ['\u6728\u5c4b', '\u5c0f\u9662'])) {
    candidates.push('\u4e61\u91ce\u6728\u5c4b')
  }

  appendCandidates(target, candidates, excludedList, max, 1)
}

function collectFoodTags(textPool, target, excludedList, max) {
  const joined = textPool.join(' ')
  const candidates = []

  if (joined.includes('\u6cb3\u5dde') && containsAny(joined, ['\u4e61\u5473\u5bb4', '\u98ce\u5473\u5bb4'])) {
    candidates.push('\u6cb3\u5dde\u4e61\u5473\u5bb4')
  }

  if (joined.includes('\u767e\u5408') && containsAny(joined, ['\u5168\u5e2d', '\u5bb4'])) {
    candidates.push('\u767e\u5408\u5168\u5e2d\u5bb4')
  } else if (joined.includes('\u767e\u5408') && containsAny(joined, ['\u7f8e\u98df', '\u98ce\u5473', '\u751c\u54c1', '\u624b\u4f5c'])) {
    candidates.push('\u767e\u5408\u98ce\u5473')
  }

  if (joined.includes('\u73ab\u7470') && containsAny(joined, ['\u5bb4', '\u665a\u9910', '\u5348\u9910', '\u98ce\u5473'])) {
    candidates.push('\u73ab\u7470\u98ce\u5473\u5bb4')
  }

  if (containsAny(joined, ['\u519c\u5bb6\u5348\u9910', '\u519c\u5bb6\u996d', '\u519c\u5bb6\u9910', '\u4e61\u6751\u665a\u9910'])) {
    candidates.push('\u519c\u5bb6\u98ce\u5473\u9910')
  }

```

```
  if (containsAny(joined, ['\u8017\u725b\u9178\u5976', '\u9178\u5976'])) {
    candidates.push('\u8017\u725b\u9178\u5976')
  }

  if (containsAny(joined, ['\u85cf\u5f0f\u5976\u8336', '\u5976\u8336'])) {
    candidates.push('\u85cf\u5f0f\u5976\u8336')
  }

  appendCandidates(target, candidates, excludedList, max, 1)
}

function normalizeHighlightTag(value) {
  let text = normalizeText(value)
  if (!text) {
    return ''
  }

  if (text.includes('\u767e\u5408') && text.includes('\u5c0f\u5361')) {
    return '\u767e\u5408\u5c0f\u5361'
  }

  text = text.replace(/^[0-9]+\.\s*/u, '')
  text = text
    .replace(/^\u8d60\u9001/u, '')
    .replace(/^\u9644\u8d60/u, '')
    .replace(/^\u5b9a\u5236/u, '')
    .replace(/\u4f53\u9a8c$/u, '')
    .replace(/\u6253\u5361$/u, '')
    .replace(/\u89c2\u5149$/u, '')
    .replace(/\u6162\u6e38$/u, '')
    .replace(/\u5165\u4f4f$/u, '')
    .replace(/\u4f11\u95f2$/u, '')
    .replace(/\u8bb2\u89e3$/u, '')
    .replace(/\u540c\u6846$/u, '')
    .trim()

  if (!text || text.length > 8 || GENERIC_TAGS.has(text)) {
    return ''
  }

  return text
}

function mapSpecificTag(text) {
  const value = normalizeText(text)
  if (!value) {
    return ''
  }

  if (value.includes('\u590f\u6cb3') && value.includes('\u7267\u573a')) return '\u590f\u6cb3\u7267\u573a'
  if (value.includes('\u9647\u5357') && containsAny(value, ['\u8336\u4e61', '\u8336\u5c71', '\u8336\u56ed'])) return '\u9647\u5357\u8336\u4e61'
  if (value.includes('\u82e6\u6c34') && value.includes('\u73ab\u7470')) return '\u82e6\u6c34\u73ab\u7470'
  if (value.includes('\u6c38\u767b') && value.includes('\u73ab\u7470')) return '\u6c38\u767b\u73ab\u7470'
  if (value.includes('\u767e\u5408') && containsAny(value, ['\u91c7\u6316', '\u91c7\u6458', '\u79cd\u690d'])) return '\u767e\u5408\u91c7\u6316'
  if (value.includes('\u8349\u8393') && containsAny(value, ['\u91c7\u6458', '\u679c\u56ed', '\u6e29\u5ba4'])) return '\u8349\u8393\u91c7\u6458'
  if (value.includes('\u68a8\u56ed')) return '\u68a8\u56ed\u91c7\u6458'
```

```
  if (value.includes('\u679c\u56ed') && value.includes('\u91c7\u6458')) return '\u679c\u56ed\u91c7\u6458'
  if (value.includes('\u4e39\u971e') && value.includes('\u5f92\u6b65')) return '\u4e39\u971e\u5f92\u6b65'
  if (value.includes('\u4e39\u971e')) return '\u4e39\u971e\u89c2\u666f'
  if (value.includes('\u82b1\u7530')) return '\u82b1\u7530\u6253\u5361'
  if (value.includes('\u82b1\u6d77')) return '\u82b1\u6d77\u6444\u5f71'
  if (containsAny(value, ['\u8336\u4e61', '\u8336\u5c71'])) return '\u8336\u4e61\u6162\u6e38'
  if (value.includes('\u8336\u56ed')) return '\u8336\u56ed\u624b\u4f5c'
  if (value.includes('\u975e\u9057') && value.includes('\u5de5\u574a')) return '\u975e\u9057\u5de5\u574a'
  if (value.includes('\u53e4\u6751')) return '\u53e4\u6751\u6f2b\u6e38'
  if (value.includes('\u7530\u56ed') && containsAny(value, ['\u6444\u5f71', '\u98ce\u5149', '\u89c2\u666f'])) return '\u7530\u56ed\u89c2\u666f'
  if (value.includes('\u7530\u56ed') && value.includes('\u82b1')) return '\u7530\u56ed\u82b1\u6d77'
  if (value.includes('\u9732\u8425')) return '\u5c71\u91ce\u9732\u8425'
  if (value.includes('\u6c11\u5bbf')) return '\u4e61\u91ce\u6c11\u5bbf'
  if (value.includes('\u624b\u4f5c')) return normalizeHighlightTag(value) || '\u624b\u4f5c\u5de5\u574a'
  if (value.includes('\u91c7\u6458')) return normalizeHighlightTag(value)

  return normalizeHighlightTag(value)
}

function collectSpecificTags(activity, target, excludedList, max) {
  const candidates = normalizeList(activity.highlights)
    .concat([
      normalizeText(activity.locationName),
      normalizeText(activity.title),
      normalizeText(activity.summary),
    ])
    .map(mapSpecificTag)
    .filter(Boolean)

  candidates.forEach((item) => pushCandidate(target, item, excludedList, max))
}

function collectFallbackTags(activity, target, excludedList, max) {
  normalizeList(activity.tags)
    .map(normalizeHighlightTag)
    .filter(Boolean)
    .forEach((item) => pushCandidate(target, item, excludedList, max))
}

function buildActivityCoverTags(activity = {}, options = {}) {
  const maxTotal = options.maxTotal || 6
  const maxExtra = Math.max(0, maxTotal - 1)
  const durationTag = buildDurationTag(activity.days, activity.durationText)
  const excludedList = [durationTag]
    .concat(normalizeList(activity.travelModeTags))
    .concat(normalizeList(activity.playTags))
  const textPool = normalizeList(activity.highlights).concat([
    normalizeText(activity.locationName),
    normalizeText(activity.title),
    normalizeText(activity.summary),
    normalizeText(activity.content),
    normalizeText(activity.detail),
  ])
  const coverTags = []

```

```
  collectServiceTags(activity, coverTags, excludedList, maxExtra)
  collectStayTags(activity, coverTags, excludedList, maxExtra)
  collectFoodTags(textPool, coverTags, excludedList, maxExtra)
  collectSpecificTags(activity, coverTags, excludedList, maxExtra)
  collectFallbackTags(activity, coverTags, excludedList, maxExtra)

  return {
    durationTag,
    tags: coverTags.slice(0, maxExtra),
    combinedTags: [durationTag].concat(coverTags.slice(0, maxExtra)),
  }
}

module.exports = {
  buildActivityCoverTags,
}
```

Path: miniprogram/utils/askConversationStore.js

```
   1 const ASK_CONVERSATION_STORAGE_KEY = "askXiaoheConversations"
   2 const MAX_CONVERSATIONS = 20
   3
   4 function normalizeText(value) {
   5   return String(value || "").trim()
   6 }
   7
   8 function createConversationId() {
   9   return `conv_${Date.now()}_${Math.floor(Math.random() * 100000)}`
  10 }
  11
  12 function buildConversationTitle(firstQuestion = "") {
  13   const text = normalizeText(firstQuestion)
  14   if (!text) return "      "
  15
  16   if (/  /.test(text) && /(  |  |  |  |  )/.test(text)) {
  17     return "          "
  18   }
  19
  20   if (/(  |  )/.test(text) && /(  |  |  |  )/.test(text)) {
  21     return "          "
  22   }
  23
  24   if (/(  |  |  |  |  |  )/.test(text)) {
  25     return "        "
  26   }
  27
  28   if (/(  |  |   |  |  )/.test(text)) {
  29     return "        "
  30   }
  31
  32   if (/(  |  |  )/.test(text)) {
  33     return "          "
  34   }
  35
  36   const compact = text.replace(/[ ? ! ,   ; :\s]/g, "")
  37   return compact.slice(0, 14) || "      "
  38 }
  39
  40 function safeRead() {
  41   try {
  42     return wx.getStorageSync(ASK_CONVERSATION_STORAGE_KEY) || []
  43   } catch (error) {
  44     return []
  45   }
  46 }
  47
  48 function safeWrite(list) {
  49   wx.setStorageSync(ASK_CONVERSATION_STORAGE_KEY, Array.isArray(list) ? list : [])
  50 }
  51
  52 function listConversations() {
  53   return safeRead()
  54     .filter((item) => item && item.id)
  55     .sort((a, b) => Number(b.updatedAt || 0) - Number(a.updatedAt || 0))
  56 }
  57
  58 function getConversationById(id) {
  59   return listConversations().find((item) => item.id === id) || null
  60 }
```

```

function saveConversation(conversation) {
  if (!conversation || !conversation.id) return null

  const current = listConversations().filter((item) => item.id !== conversation.id)
  const next = [
    {
      id: conversation.id,
      title: normalizeText(conversation.title) || "      ",
      firstQuestion: normalizeText(conversation.firstQuestion),
      createdAt: conversation.createdAt || Date.now(),
      updatedAt: conversation.updatedAt || Date.now(),
      messages: Array.isArray(conversation.messages) ? conversation.messages : [],
      source: normalizeText(conversation.source),
      skillMode: normalizeText(conversation.skillMode),
      genericPreferences: conversation.genericPreferences || {},
      currentTaskState: conversation.currentTaskState || null
    }
  ].concat(current).slice(0, MAX_CONVERSATIONS)

  safeWrite(next)
  return next[0]
}

function removeConversation(id) {
  const next = listConversations().filter((item) => item.id !== id)
  safeWrite(next)
  return next
}

function clearConversations() {
  safeWrite([])
}

module.exports = {
  ASK_CONVERSATION_STORAGE_KEY,
  buildConversationTitle,
  clearConversations,
  createConversationId,
  getConversationById,
  listConversations,
  removeConversation,
  saveConversation
}
```

Path: miniprogram/utils/collectionStore.js

```
---------------------------------------------------------------------------
const FAVORITES_KEY = 'unifiedFavorites'
const FOOTPRINTS_KEY = 'unifiedFootprints'

const CATEGORY_ALL = 'all'
const CITY_ALL = 'all'

const TYPE_CONFIG = {
  activity: {
    label: '  ',
    path: '/pages/activityDetail/activityDetail',
  },
  product: {
    label: '  ',
    path: '/pages/productDetail/productDetail',
  },
  scenic: {
    label: '  ',
    path: '/pages/scenicDetail/scenicDetail',
  },
  hotel: {
    label: '  /  ',
    path: '/pages/hotelDetail/hotelDetail',
  },
  knowledge: {
    label: '  ',
    path: '/pages/knowledgeDetail/knowledgeDetail',
  },
}

function safeList(value) {
  return Array.isArray(value) ? value : []
}

function getStorageList(key) {
  try {
    return safeList(wx.getStorageSync(key))
  } catch (error) {
    console.error('[collectionStore] getStorageList failed', key, error)
    return []
  }
}

function setStorageList(key, list) {
  try {
    wx.setStorageSync(key, safeList(list))
  } catch (error) {
    console.error('[collectionStore] setStorageList failed', key, error)
  }
}

function sortByLatest(list, fieldName) {
  return safeList(list).sort((left, right) => Number(right[fieldName] || 0) - Number(left[fieldName] || 0))
}

function buildKey(type, id) {
  return `${type || ''}::${id || ''}`
}

function createTargetUrl(type, id) {
```

```
  const config = TYPE_CONFIG[type] || {}
  return config.path && id ? `${config.path}?id=${id}` : ''
}

function normalizeText(value = '') {
  return String(value || '').trim()
}

function normalizeRecord(input = {}) {
  const type = normalizeText(input.type)
  const id = normalizeText(input.id)
  const config = TYPE_CONFIG[type] || {}
  const key = buildKey(type, id)
  const normalizedBadgeText = normalizeText(input.badgeText)
  const badgeText = type === 'knowledge'
    ? config.label
    : (normalizedBadgeText || config.label || '')

  return {
    key,
    type,
    id,
    categoryLabel: config.label || '',
    title: normalizeText(input.title) || '     ',
    cover: normalizeText(input.cover),
    city: normalizeText(input.city) || '    ',
    regionText: normalizeText(input.regionText),
    summary: normalizeText(input.summary),
    priceText: normalizeText(input.priceText),
    metaText: normalizeText(input.metaText),
    badgeText,
    author: normalizeText(input.author),
    statsText: normalizeText(input.statsText),
    scoreText: normalizeText(input.scoreText),
    targetUrl: normalizeText(input.targetUrl) || createTargetUrl(type, id),
    updatedAt: Number(input.updatedAt || 0),
    favoritedAt: Number(input.favoritedAt || 0),
    visitedAt: Number(input.visitedAt || 0),
  }
}

function getFavorites() {
  return sortByLatest(
    getStorageList(FAVORITES_KEY).map((item) => normalizeRecord(item)),
    'favoritedAt'
  )
}

function getFootprints() {
  return sortByLatest(
    getStorageList(FOOTPRINTS_KEY).map((item) => normalizeRecord(item)),
    'visitedAt'
  )
}

function isFavorited(type, id) {
  const targetKey = buildKey(type, id)
  return getFavorites().some((item) => item.key === targetKey)
}

function toggleFavorite(record = {}) {
  const normalized = normalizeRecord(record)
```

```
  const currentList = getFavorites()
  const exists = currentList.some((item) => item.key === normalized.key)

  if (exists) {
    const nextList = currentList.filter((item) => item.key !== normalized.key)
    setStorageList(FAVORITES_KEY, nextList)
    return false
  }

  const nextItem = {
    ...normalized,
    favoritedAt: Date.now(),
    updatedAt: Date.now(),
  }
  const nextList = [nextItem, ...currentList.filter((item) => item.key !== normalized.key)]
  setStorageList(FAVORITES_KEY, nextList)
  return true
}

function recordFootprint(record = {}) {
  const normalized = normalizeRecord(record)
  if (!normalized.type || !normalized.id) {
    return
  }

  const currentList = getFootprints().filter((item) => item.key !== normalized.key)
  const nextItem = {
    ...normalized,
    visitedAt: Date.now(),
    updatedAt: Date.now(),
  }
  setStorageList(FOOTPRINTS_KEY, [nextItem, ...currentList])
}

function clearFootprints() {
  setStorageList(FOOTPRINTS_KEY, [])
}

function deleteFavorites(keys = []) {
  const keySet = new Set(safeList(keys).filter(Boolean))
  if (!keySet.size) {
    return
  }

  const nextList = getFavorites().filter((item) => !keySet.has(item.key))
  setStorageList(FAVORITES_KEY, nextList)
}

module.exports = {
  CATEGORY_ALL,
  CITY_ALL,
  TYPE_CONFIG,
  getFavorites,
  getFootprints,
  isFavorited,
  toggleFavorite,
  deleteFavorites,
  recordFootprint,
  clearFootprints,
}
```

Path: miniprogram/utils/knowledgeArticle.js

```
---------------------------------------------------------------------------
const { resolveMediaSource } = require('./mediaAssets')

const CLOUD_ENV_ID = 'cloud1-3ghmr5ki7b1172fe.636c-cloud1-3ghmr5ki7b1172fe-1403917845'
const CLOUD_KNOWLEDGE_COVER_PREFIX = `cloud://${CLOUD_ENV_ID}/knowledge/covers`
const KNOWLEDGE_COVER_FILE_MAP = {
  'agri-001': 'spring-vegetable.png',
  'agri-002': 'village-chicken.png',
  'agri-003': 'corn-disease.png',
  'agri-004': 'bee-guide.png',
  'agri-005': 'rice-brand.png',
  'culture-001': 'summer-palace.png',
  'culture-002': 'intangible-heritage.png',
  'culture-003': 'folk-custom.png',
  'culture-004': 'travel-tips.png',
}

function normalizeText(value = '') {
  return String(value || '').trim()
}

function normalizeArray(value) {
  return Array.isArray(value) ? value : []
}

function normalizeContentType(type = '') {
  const value = normalizeText(type).toLowerCase()
  if (value === 'h2' || value === 'heading') {
    return 'heading'
  }
  if (value === 'img' || value === 'image') {
    return 'image'
  }
  return 'paragraph'
}

function getFileName(value = '') {
  const normalized = normalizeText(value)
  if (!normalized) {
    return ''
  }

  const segments = normalized.split('/')
  return normalizeText(segments[segments.length - 1])
}

function buildKnowledgeCoverFileID(fileName = '') {
  const normalized = getFileName(fileName)
  return normalized ? `${CLOUD_KNOWLEDGE_COVER_PREFIX}/${normalized}` : ''
}

function normalizeKnowledgeCover(article = {}) {
  const cover = normalizeText(article.cover)
  if (/^(cloud|https?):\/\//i.test(cover)) {
    return cover
  }

  const coverFileName = getFileName(cover) || KNOWLEDGE_COVER_FILE_MAP[normalizeText(art
icle.id)]
  return buildKnowledgeCoverFileID(coverFileName)
}
```

```

async function resolveKnowledgeArticleMedia(article = {}) {
  const content = await Promise.all(
    normalizeArray(article.content).map(async (item = {}) => {
      const type = normalizeContentType(item.type)
      if (type === 'image') {
        const src = await resolveMediaSource(item.src || item.image || '', '')
        return {
          ...item,
          type,
          src,
        }
      }

      return {
        ...item,
        type,
        text: normalizeText(item.text),
      }
    })
  )

  return {
    ...article,
    tags: normalizeArray(article.tags).filter(Boolean),
    cover: await resolveMediaSource(normalizeKnowledgeCover(article), ''),
    content,
  }
}

module.exports = {
  normalizeText,
  normalizeArray,
  normalizeContentType,
  normalizeKnowledgeCover,
  resolveKnowledgeArticleMedia,
}
```

Path: miniprogram/utils/mediaAssets.js

```
const ACTIVITY_MEDIA_MAP = {
  'lz-yuzhong-strawberry-family-day': {
    aliases: ['  ', '  ', '  '],
    localCover: '/images/activities/lz-yuzhong-strawberry-family-day.jpg',
  },
  'lz-gaolan-country-photo-day': {
    aliases: ['  ', '  ', '  '],
    localCover: '/images/activities/lz-gaolan-country-photo-day.jpg',
  },
  'lz-yongdeng-rose-weekend': {
    aliases: ['  ', '  ', '  '],
    localCover: '/images/activities/lz-yongdeng-rose-weekend.jpg',
  },
  'lz-suburb-farm-study-camp': {
    aliases: ['  ', '  ', '  '],
    localCover: '/images/activities/lz-suburb-farm-study-camp.jpg',
  },
  'lz-bailihe-handmade-food-tour': {
    aliases: ['  ', '  ', '  '],
    localCover: '/images/activities/lz-bailihe-handmade-food-tour.jpg',
  },
  'lz-kushui-rose-culture-day': {
    aliases: ['    ', '   ', '    '],
    localCover: '/images/nav-academy.png',
  },
  'lz-kushui-danxia-hike-day': {
    aliases: ['    ', '    ', '   '],
    localCover: '/images/nav-academy.png',
  },
  'lz-xigu-baihe-culture-day': {
    aliases: ['   ', '    ', '    '],
    localCover: '/images/nav-academy.png',
  },
  'lz-gaolan-shichuan-pear-garden-day': {
    aliases: ['  ', '   ', '  '],
    localCover: '/images/nav-academy.png',
  },
  'lz-qilihe-shenjialing-red-green-fusion': {
    aliases: ['   ', '    ', '    '],
    localCover: '/images/nav-academy.png',
  },
  'lz-bali-huazhaizi-peach-garden': {
    aliases: ['   ', '    ', '    '],
    localCover: '/images/nav-academy.png',
  },
  'tianshui-qinan-fruit-pick-day': {
    aliases: ['  ', '  ', '  '],
    localCover: '/images/activities/tianshui-qinan-fruit-pick-day.jpg',
  },
  'zhangye-rural-homestay-weekend': {
    aliases: ['  ', '  ', '  '],
    localCover: '/images/activities/zhangye-rural-homestay-weekend.jpg',
  },
  'longnan-tea-hill-folk-experience': {
    aliases: ['  ', '  ', '  '],
    localCover: '/images/activities/longnan-tea-hill-folk-experience.jpg',
  },
  'linxia-food-culture-village-tour': {
    aliases: ['  ', '  ', '  '],
    localCover: '/images/activities/linxia-food-culture-village-tour.jpg',
```

```
  },
  'gannan-ranch-life-light-tour': {
    aliases: ['  ', '  ', '  '],
    localCover: '/images/activities/gannan-ranch-life-light-tour.jpg',
    cloudCover: 'cloud://cloud1-3ghmr5ki7b1172fe.636c-cloud1-3ghmr5ki7b1172fe-1403917845/activities/covers/gannan-ranch-life-light-tour.jpg',
    cloudBanner: 'cloud://cloud1-3ghmr5ki7b1172fe.636c-cloud1-3ghmr5ki7b1172fe-1403917845/activities/banners/gannan-ranch-life-light-tour.jpg',
    cloudGallery: [
      'cloud://cloud1-3ghmr5ki7b1172fe.636c-cloud1-3ghmr5ki7b1172fe-1403917845/activities/gallery/gannan-ranch-life-light-tour-1.jpg',
      'cloud://cloud1-3ghmr5ki7b1172fe.636c-cloud1-3ghmr5ki7b1172fe-1403917845/activities/gallery/gannan-ranch-life-light-tour-2.jpg',
    ],
  },
}

const DEFAULT_ACTIVITY_KEY = 'lz-yuzhong-strawberry-family-day'
const CLOUD_FILE_CACHE = Object.create(null)
const CLOUD_ACTIVITY_PREFIX = 'cloud://cloud1-3ghmr5ki7b1172fe.636c-cloud1-3ghmr5ki7b1172fe-1403917845/activities'
const MAX_GALLERY_COUNT = 6

function normalizeText(value = '') {
  return String(value || '').trim()
}

function isLocalMedia(src = '') {
  return normalizeText(src).startsWith('/images/')
}

function isRemoteMedia(src = '') {
  return /^https?:\/\//i.test(normalizeText(src))
}

function isCloudFileId(src = '') {
  return /^cloud:\/\//i.test(normalizeText(src))
}

function isRenderableMedia(src = '') {
  return isLocalMedia(src) || isRemoteMedia(src)
}

function findActivityKey(activity = {}) {
  const seedKey = normalizeText(activity.seedKey)
  if (seedKey && ACTIVITY_MEDIA_MAP[seedKey]) {
    return seedKey
  }

  const title = normalizeText(activity.title)
  if (!title) {
    return DEFAULT_ACTIVITY_KEY
  }

  const matchedKey = Object.keys(ACTIVITY_MEDIA_MAP).find((key) => {
    const aliases = ACTIVITY_MEDIA_MAP[key].aliases || []
    return aliases.some((alias) => title.includes(alias))
  })

  return matchedKey || DEFAULT_ACTIVITY_KEY
}

```

```
function getActivityMediaConfig(activity = {}) {
  const key = typeof activity === 'string' ? activity : findActivityKey(activity)
  const config = ACTIVITY_MEDIA_MAP[key] || ACTIVITY_MEDIA_MAP[DEFAULT_ACTIVITY_KEY]
  return {
    key,
    ...config,
  }
}

function buildCloudCover(key) {
  return `${CLOUD_ACTIVITY_PREFIX}/covers/${key}.jpg`
}

function buildCloudBanner(key) {
  return `${CLOUD_ACTIVITY_PREFIX}/banners/${key}.jpg`
}

function buildCloudGallery(key) {
  return Array.from({ length: MAX_GALLERY_COUNT }, (_, index) =>
    `${CLOUD_ACTIVITY_PREFIX}/gallery/${key}-${index + 1}.jpg`
  )
}

function getRawActivityCover(activity = {}) {
  const config = getActivityMediaConfig(activity)
  return config.cloudCover || buildCloudCover(config.key)
}

function getRawActivityBanner(activity = {}) {
  const config = getActivityMediaConfig(activity)
  return config.cloudBanner || buildCloudBanner(config.key)
}

function getRawActivityGallery(activity = {}) {
  const config = getActivityMediaConfig(activity)
  return (Array.isArray(config.cloudGallery) && config.cloudGallery.length)
    ? config.cloudGallery
    : buildCloudGallery(config.key)
}

function resolveCloudFile(fileID) {
  if (!fileID) {
    return Promise.resolve('')
  }

  if (CLOUD_FILE_CACHE[fileID]) {
    return Promise.resolve(CLOUD_FILE_CACHE[fileID])
  }

  return new Promise((resolve) => {
    wx.cloud.getTempFileURL({
      fileList: [fileID],
      success: (res) => {
        const fileInfo = (((res || {}).fileList || [])[0] || {})
        const tempUrl = fileInfo.tempFileURL || ''
        console.log('[mediaAssets] getTempFileURL success', {
          fileID,
          fileInfo,
          status: fileInfo.status,
          errMsg: fileInfo.errMsg,
          code: fileInfo.code,
          message: fileInfo.message,
```

```
          tempFileURL: tempUrl,
        })
        if (tempUrl && fileInfo.status !== -1) {
          CLOUD_FILE_CACHE[fileID] = tempUrl
        }
        resolve(tempUrl || '')
      },
      fail: (err) => {
        console.error('[mediaAssets] getTempFileURL fail', {
          fileID,
          err,
        })
        resolve('')
      },
    })
  })
}

async function resolveMediaSource(src, fallback = '') {
  const value = normalizeText(src)
  if (!value) {
    return fallback
  }

  if (isRenderableMedia(value)) {
    return value
  }

  if (isCloudFileId(value)) {
    const tempUrl = await resolveCloudFile(value)
    return tempUrl || fallback
  }

  return fallback
}

async function resolveMediaList(list = [], fallback = '') {
  const resolved = await Promise.all((list || []).map((item) => resolveMediaSource(item, ''')))
  const filtered = resolved.filter(Boolean)
  return filtered.length ? filtered : (fallback ? [fallback] : [])
}

async function resolveActivityCover(activity = {}) {
  const config = getActivityMediaConfig(activity)
  const fallback = config.localCover || ACTIVITY_MEDIA_MAP[DEFAULT_ACTIVITY_KEY].localCover
  const raw = getRawActivityCover(activity)
  return resolveMediaSource(raw, fallback)
}

async function resolveActivityBanner(activity = {}) {
  const fallback = await resolveActivityCover(activity)
  return resolveMediaSource(getRawActivityBanner(activity), fallback)
}

async function resolveActivityGallery(activity = {}) {
  const fallback = await resolveActivityCover(activity)
  const rawGallery = getRawActivityGallery(activity)
  const gallery = await resolveMediaList(rawGallery, fallback)
  console.log('[mediaAssets] resolveActivityGallery', {
    key: getActivityMediaConfig(activity).key,
```

```
    rawGallery,
    resolvedCount: gallery.length,
    gallery,
  })
  return gallery
}

module.exports = {
  ACTIVITY_MEDIA_MAP,
  getActivityMediaConfig,
  isLocalMedia,
  isRemoteMedia,
  isCloudFileId,
  isRenderableMedia,
  resolveMediaSource,
  resolveMediaList,
  resolveActivityCover,
  resolveActivityBanner,
  resolveActivityGallery,
}
```

Path: miniprogram/utils/messageStore.js

```
---------------------------------------------------------------------------
const STORAGE_KEY = 'messageCenterMockState'

function normalizeList(value) {
  return Array.isArray(value) ? value : []
}

function clone(value) {
  return JSON.parse(JSON.stringify(value))
}

function sortByUpdatedAt(list) {
  return normalizeList(list).sort((left, right) => Number(right.updatedAt || 0) - Number
(left.updatedAt || 0))
}

function createSeedState() {
  const now = Date.now()
  const minute = 60 * 1000
  const hour = 60 * minute
  const day = 24 * hour

  return {
    interactiveFilters: [
      { key: 'all', label: '    ' },
      { key: 'like', label: ' ' },
      { key: 'mention', label: '  ' },
      { key: 'comment', label: '  ' },
    ],
    interactions: [
      {
        id: 'interaction-1',
        type: 'mention',
        actorName: '    ',
        actorTag: '  ',
        avatarText: ' ',
        avatarColor: '#ffb547',
        actionText: '    ',
        content: '                          ',
        targetTitle: '       ',
        updatedAt: now - 2 * hour,
      },
      {
        id: 'interaction-2',
        type: 'like',
        actorName: '    ',
        actorTag: '    ',
        avatarText: ' ',
        avatarColor: '#ff6f91',
        actionText: '        ',
        content: '    “        ”       ',
        targetTitle: '        ',
        updatedAt: now - 5 * hour,
      },
      {
        id: 'interaction-3',
        type: 'comment',
        actorName: '     ',
        actorTag: '  ',
        avatarText: ' ',
        avatarColor: '#6e9eff',
```

```
        actionText: '    ',
        content: '“                ”',
        targetTitle: '      ',
        updatedAt: now - day,
      },
    ],
    newFollowers: [
      {
        id: 'follower-1',
        userName: '     ',
        avatarText: ' ',
        avatarColor: '#ff8b5c',
        followAt: now - day,
      },
      {
        id: 'follower-2',
        userName: '    ',
        avatarText: ' ',
        avatarColor: '#4da1ff',
        followAt: now - 2 * day,
      },
      {
        id: 'follower-3',
        userName: '    ',
        avatarText: ' ',
        avatarColor: '#ff6ab0',
        followAt: now - 3 * day,
      },
    ],
    noteStories: [
      {
        id: 'note-story-1',
        userName: '      ',
        avatarText: ' ',
        avatarColor: '#ff6fb5',
        noteId: 'note-1',
        noteTitle: '         ',
        noteSummary: '                                ',
        publishAt: now - 18 * minute,
      },
      {
        id: 'note-story-2',
        userName: '      ',
        avatarText: ' ',
        avatarColor: '#5f9dff',
        noteId: 'note-2',
        noteTitle: '        ',
        noteSummary: '                           ',
        publishAt: now - 45 * minute,
      },
      {
        id: 'note-story-3',
        userName: '      ',
        avatarText: ' ',
        avatarColor: '#45b36b',
        noteId: 'note-3',
        noteTitle: '       ',
        noteSummary: '                             ',
        publishAt: now - 2 * hour,
      },
      {
        id: 'note-story-4',
```

```
      userName: '    ',
      avatarText: ' ',
      avatarColor: '#ff8b5c',
      noteId: 'note-4',
      noteTitle: '         ',
      noteSummary: '                            ',
      publishAt: now - 5 * hour,
    },
  ],
  merchantConversations: [
    {
      id: 'merchant-1',
      shopName: '     ',
      avatarText: ' ',
      avatarColor: '#ff9f43',
      preview: '                        ',
      updatedAt: now - 40 * minute,
      unread: 2,
    },
    {
      id: 'merchant-2',
      shopName: '    ',
      avatarText: ' ',
      avatarColor: '#6c5ce7',
      preview: '                      ',
      updatedAt: now - 6 * hour,
      unread: 0,
    },
  ],
  merchantMessages: {
    'merchant-1': [
      {
        id: 'merchant-1-msg-1',
        role: 'merchant',
        text: '                ',
        updatedAt: now - 80 * minute,
      },
      {
        id: 'merchant-1-msg-2',
        role: 'merchant',
        text: '                        ',
        updatedAt: now - 40 * minute,
      },
    ],
    'merchant-2': [
      {
        id: 'merchant-2-msg-1',
        role: 'merchant',
        text: '                      ',
        updatedAt: now - 6 * hour,
      },
    ],
  },
  platformSupportConversation: {
    id: 'platform-support',
    title: '    ',
    avatarText: ' ',
    avatarColor: '#3c9f5b',
    updatedAt: now - 3 * hour,
    unread: 1,
  },
  platformSupportMessages: [
```

```
      {
        id: 'platform-support-1',
        role: 'platform',
        text: '                                ',
        updatedAt: now - 3 * hour,
      },
      {
        id: 'platform-support-2',
        role: 'platform',
        text: '                            ',
        updatedAt: now - 2.8 * hour,
      },
    ],
    platformGuarantees: [
      {
        id: 'guarantee-1',
        title: '      ',
        subtitle: '         ',
        content: '                                        ',
        orderTitle: '         ',
        orderNo: 'TRIP20260326001',
        actionText: '      ',
        updatedAt: now - 4 * hour,
      },
      {
        id: 'guarantee-2',
        title: '      ',
        subtitle: '       ',
        content: '                                      ',
        orderTitle: '         ',
        orderNo: 'HOTEL20260318009',
        actionText: '      ',
        updatedAt: now - 2 * day,
      },
    ],
    buddyApplications: [
      {
        id: 'buddy-apply-1',
        userName: '      ',
        avatarText: ' ',
        avatarColor: '#ff6fb5',
        preview: '                      ',
        updatedAt: now - 25 * minute,
        unread: 2,
        messages: [
          {
            id: 'buddy-apply-1-msg-1',
            role: 'other',
            text: '                     ',
            updatedAt: now - 25 * minute,
          },
          {
            id: 'buddy-apply-1-msg-2',
            role: 'other',
            text: '               ',
            updatedAt: now - 23 * minute,
          },
        ],
      },
      {
        id: 'buddy-apply-2',
        userName: '      ',
```

```
        avatarText: ' ',
        avatarColor: '#ff9a3d',
        preview: '                        ',
        updatedAt: now - 3 * hour,
        unread: 1,
        messages: [
          {
            id: 'buddy-apply-2-msg-1',
            role: 'other',
            text: '                        ',
            updatedAt: now - 3 * hour,
          },
        ],
      },
    ],
    formalConversations: [
      {
        id: 'conversation-1',
        title: '      ',
        avatarText: ' ',
        avatarColor: '#5f9dff',
        preview: '                  ',
        updatedAt: now - 50 * minute,
        unread: 1,
        kind: 'social',
      },
      {
        id: 'conversation-2',
        title: '      ',
        avatarText: ' ',
        avatarColor: '#45b36b',
        preview: '                ',
        updatedAt: now - 8 * hour,
        unread: 0,
        kind: 'social',
      },
    ],
    formalMessages: {
      'conversation-1': [
        {
          id: 'conversation-1-msg-1',
          role: 'other',
          text: '                  ',
          updatedAt: now - 50 * minute,
        },
      ],
      'conversation-2': [
        {
          id: 'conversation-2-msg-1',
          role: 'other',
          text: '                ',
          updatedAt: now - 8 * hour,
        },
      ],
    },
  }
}

function mergeWithSeedState(cachedState = {}) {
  const seed = createSeedState()
  return {
    ...seed,
```

```
    ...cachedState,
    interactiveFilters: normalizeList(cachedState.interactiveFilters).length ? cachedSta
te.interactiveFilters : seed.interactiveFilters,
    interactions: normalizeList(cachedState.interactions).length ? cachedState.interacti
ons : seed.interactions,
    newFollowers: normalizeList(cachedState.newFollowers).length ? cachedState.newFollow
ers : seed.newFollowers,
    noteStories: normalizeList(cachedState.noteStories).length ? cachedState.noteStories
 : seed.noteStories,
    merchantConversations: normalizeList(cachedState.merchantConversations).length ? cac
hedState.merchantConversations : seed.merchantConversations,
    merchantMessages: cachedState.merchantMessages && typeof cachedState.merchantMessage
s === 'object'
      ? { ...seed.merchantMessages, ...cachedState.merchantMessages }
      : seed.merchantMessages,
    platformSupportConversation: cachedState.platformSupportConversation || seed.platfor
mSupportConversation,
    platformSupportMessages: normalizeList(cachedState.platformSupportMessages).length ?
 cachedState.platformSupportMessages : seed.platformSupportMessages,
    platformGuarantees: normalizeList(cachedState.platformGuarantees).length ? cachedSta
te.platformGuarantees : seed.platformGuarantees,
    buddyApplications: normalizeList(cachedState.buddyApplications).length ? cachedState
.buddyApplications : seed.buddyApplications,
    formalConversations: normalizeList(cachedState.formalConversations).length ? cachedS
tate.formalConversations : seed.formalConversations,
    formalMessages: cachedState.formalMessages && typeof cachedState.formalMessages ===
'object'
      ? { ...seed.formalMessages, ...cachedState.formalMessages }
      : seed.formalMessages,
  }
}

function readStorageState() {
  try {
    const state = wx.getStorageSync(STORAGE_KEY)
    return state && typeof state === 'object' ? state : null
  } catch (error) {
    console.error('[messageStore] read state failed', error)
    return null
  }
}

function writeStorageState(state) {
  try {
    wx.setStorageSync(STORAGE_KEY, state)
  } catch (error) {
    console.error('[messageStore] write state failed', error)
  }
}

function ensureState() {
  const cached = readStorageState()
  if (cached) {
    const merged = mergeWithSeedState(cached)
    writeStorageState(merged)
    return merged
  }

  const state = createSeedState()
  writeStorageState(state)
  return state
}
```

```

function getMessageState() {
  return clone(ensureState())
}

function setMessageState(nextState) {
  writeStorageState(clone(nextState))
}

function formatMonthDay(timestamp) {
  const date = new Date(Number(timestamp || 0))
  const month = String(date.getMonth() + 1).padStart(2, '0')
  const day = String(date.getDate()).padStart(2, '0')
  return `${month}/${day}`
}

function formatConversationTime(timestamp) {
  const value = Number(timestamp || 0)
  if (!value) return ''

  const date = new Date(value)
  const now = new Date()
  const sameDay = (
    date.getFullYear() === now.getFullYear() &&
    date.getMonth() === now.getMonth() &&
    date.getDate() === now.getDate()
  )

  if (sameDay) {
    const hours = String(date.getHours()).padStart(2, '0')
    const minutes = String(date.getMinutes()).padStart(2, '0')
    return `${hours}:${minutes}`
  }

  return formatMonthDay(value)
}

function buildHomeRows(state) {
  const safeState = state || ensureState()
  const buddyApplications = normalizeList(safeState.buddyApplications).filter((item) =>
item.direction)
  const buddyUnread = buddyApplications.reduce((sum, item) => sum + Number(item.unread |
| 0), 0)
  const latestBuddy = sortByUpdatedAt(buddyApplications)[0]

  const specialRows = [
    {
      id: 'buddy-home',
      kind: 'entry',
      entryType: 'buddy',
      title: '    ',
      subtitle: latestBuddy ? latestBuddy.preview : '              ',
      updatedAt: latestBuddy ? latestBuddy.updatedAt : Date.now() - 3000,
      unread: buddyUnread,
      avatarText: ' ',
      avatarColor: '#6e89ff',
    },
  ]

  const conversationRows = sortByUpdatedAt(safeState.formalConversations)
    .filter((item) => item.kind === 'buddy')
    .map((item) => ({
```

```
    ...item,
    kind: 'conversation',
  }))

  return specialRows.concat(conversationRows)
}

function buildStoryList(state) {
  const safeState = state || ensureState()
  const relatedNames = uniqueList(
    normalizeList(safeState.buddyApplications)
      .map((item) => item.userName)
      .concat(normalizeList(safeState.formalConversations).map((item) => item.title))
      .map((item) => String(item || '').trim())
      .filter(Boolean)
  )

  return clone(
    sortByUpdatedAt(safeState.noteStories)
      .filter((item) => relatedNames.includes(String(item.userName || '').trim()))
      .map((item) => ({
        id: item.id,
        title: item.userName,
        avatarUrl: item.avatarUrl || '',
        avatarText: item.avatarText,
        avatarColor: item.avatarColor,
        unread: 0,
        noteId: item.noteId,
        noteTitle: item.noteTitle,
      }))
  )
}

function getInteractiveList(filterKey) {
  const state = ensureState()
  const filter = String(filterKey || 'all')
  const source = normalizeList(state.interactions)
  if (filter === 'all') return clone(source)
  return clone(source.filter((item) => item.type === filter))
}

function markInteractionsRead() {
  const state = ensureState()
  state.interactions = normalizeList(state.interactions).map((item) => ({
    ...item,
    read: true,
  }))
  state.newFollowers = normalizeList(state.newFollowers).map((item) => ({
    ...item,
    read: true,
  }))
  writeStorageState(state)
}

function getNewFollowers() {
  return clone(sortByUpdatedAt(ensureState().newFollowers))
}

function getMerchantConversations() {
  return clone(sortByUpdatedAt(ensureState().merchantConversations))
}

```

```
480 function getMerchantConversation(id) {
481   const state = ensureState()
482   const conversation = normalizeList(state.merchantConversations).find((item) => item.id
     === id)
483   const messages = clone((state.merchantMessages && state.merchantMessages[id]) || [])
484   return {
485     conversation: clone(conversation || null),
486     messages,
487   }
488 }
489
490 function sendMerchantMessage(id, text) {
491   const content = String(text || '').trim()
492   if (!content) return null
493
494   const state = ensureState()
495   const list = normalizeList(state.merchantConversations)
496   const target = list.find((item) => item.id === id)
497   if (!target) return null
498
499   const now = Date.now()
500   const nextMessage = {
501     id: `${id}-user-${now}`,
502     role: 'user',
503     text: content,
504     updatedAt: now,
505   }
506
507   state.merchantMessages[id] = normalizeList(state.merchantMessages[id]).concat(nextMess
    age)
508   target.preview = content
509   target.updatedAt = now
510   writeStorageState(state)
511   return nextMessage
512 }
513
514 function getPlatformSupport() {
515   const state = ensureState()
516   return {
517     conversation: clone(state.platformSupportConversation),
518     messages: clone(state.platformSupportMessages),
519   }
520 }
521
522 function sendPlatformSupportMessage(text) {
523   const content = String(text || '').trim()
524   if (!content) return null
525
526   const state = ensureState()
527   const now = Date.now()
528   const nextMessage = {
529     id: `platform-user-${now}`,
530     role: 'user',
531     text: content,
532     updatedAt: now,
533   }
534
535   state.platformSupportMessages = normalizeList(state.platformSupportMessages).concat(ne
    xtMessage)
536   state.platformSupportConversation.updatedAt = now
537   writeStorageState(state)
538   return nextMessage
```

```
539 }
540
541 function getPlatformGuarantees() {
542   return clone(sortByUpdatedAt(ensureState().platformGuarantees))
543 }
544
545 function getBuddyApplications() {
546   return clone(sortByUpdatedAt(ensureState().buddyApplications).filter((item) => item.di
    rection))
547 }
548
549 function getBuddyApplication(id) {
550   const state = ensureState()
551   const target = normalizeList(state.buddyApplications).find((item) => item.id === id)
552   return clone(target || null)
553 }
554
555 function createBuddyApplicationFromMatch(payload = {}) {
556   const userName = String(payload.userName || '').trim()
557   const openingText = String(payload.openingText || '').trim()
558   if (!userName || !openingText) {
559     return null
560   }
561
562   const state = ensureState()
563   const now = Date.now()
564   const id = `buddy-apply-local-${now}`
565   const application = {
566     id,
567     userName,
568     avatarUrl: payload.avatarUrl || '',
569     avatarText: payload.avatarText || userName.slice(0, 1) || ' ',
570     avatarColor: payload.avatarColor || '#6e89ff',
571     preview: openingText,
572     draftText: openingText,
573     updatedAt: now,
574     unread: 0,
575     direction: payload.direction || 'outgoing',
576     matchScore: Number(payload.matchScore || 0),
577     matchReason: payload.matchReason || '',
578     messages: [],
579   }
580
581   state.buddyApplications = [application].concat(
582     normalizeList(state.buddyApplications).filter((item) => item.id !== id)
583   )
584   writeStorageState(state)
585   return clone(application)
586 }
587
588 function replyToBuddyApplication(id, text) {
589   const content = String(text || '').trim()
590   if (!content) return { conversationId: '', conversation: null }
591
592   const state = ensureState()
593   const target = normalizeList(state.buddyApplications).find((item) => item.id === id)
594   if (!target) {
595     return { conversationId: '', conversation: null }
596   }
597
598   const now = Date.now()
599   const conversationId = `conversation-${id}`
```

```
600   const nextMessages = normalizeList(target.messages).concat({
601     id: `${id}-reply-${now}`,
602     role: 'user',
603     text: content,
604     updatedAt: now,
605   })
606
607   state.buddyApplications = normalizeList(state.buddyApplications).filter((item) => item
    .id !== id)
608   state.formalConversations = normalizeList(state.formalConversations)
609     .filter((item) => item.id !== conversationId)
610     .concat({
611       id: conversationId,
612       title: target.userName,
613       avatarUrl: target.avatarUrl || '',
614       avatarText: target.avatarText,
615       avatarColor: target.avatarColor,
616       preview: content,
617       updatedAt: now,
618       unread: 0,
619       kind: 'buddy',
620     })
621   state.formalMessages[conversationId] = nextMessages
622   writeStorageState(state)
623
624   return {
625     conversationId,
626     conversation: clone(state.formalConversations.find((item) => item.id === conversatio
    nId) || null),
627   }
628 }
629
630 function getFormalConversation(id) {
631   const state = ensureState()
632   return {
633     conversation: clone(normalizeList(state.formalConversations).find((item) => item.id
    === id) || null),
634     messages: clone((state.formalMessages && state.formalMessages[id]) || []),
635   }
636 }
637
638 function sendFormalMessage(id, text) {
639   const content = String(text || '').trim()
640   if (!content) return null
641
642   const state = ensureState()
643   const target = normalizeList(state.formalConversations).find((item) => item.id === id)
644   if (!target) return null
645
646   const now = Date.now()
647   const nextMessage = {
648     id: `${id}-user-${now}`,
649     role: 'user',
650     text: content,
651     updatedAt: now,
652   }
653
654   state.formalMessages[id] = normalizeList(state.formalMessages[id]).concat(nextMessage)
655   target.preview = content
656   target.updatedAt = now
657   writeStorageState(state)
658   return nextMessage
```

```
}

function getNoteStories() {
  return clone(sortByUpdatedAt(ensureState().noteStories))
}

function getNoteById(id) {
  const target = normalizeList(ensureState().noteStories).find((item) => item.noteId ===
id)
  return clone(target || null)
}

function markMerchantConversationRead(id) {
  const state = ensureState()
  const target = normalizeList(state.merchantConversations).find((item) => item.id === i
d)
  if (!target) return
  target.unread = 0
  writeStorageState(state)
}

function markPlatformSupportRead() {
  const state = ensureState()
  if (!state.platformSupportConversation) return
  state.platformSupportConversation.unread = 0
  writeStorageState(state)
}

function markBuddyApplicationRead(id) {
  const state = ensureState()
  const target = normalizeList(state.buddyApplications).find((item) => item.id === id)
  if (!target) return
  target.unread = 0
  writeStorageState(state)
}

function markFormalConversationRead(id) {
  const state = ensureState()
  const target = normalizeList(state.formalConversations).find((item) => item.id === id)
  if (!target) return
  target.unread = 0
  writeStorageState(state)
}

module.exports = {
  buildHomeRows,
  buildStoryList,
  formatConversationTime,
  getInteractiveList,
  markInteractionsRead,
  getMessageState,
  getNewFollowers,
  getMerchantConversations,
  getMerchantConversation,
  markMerchantConversationRead,
  sendMerchantMessage,
  getPlatformSupport,
  markPlatformSupportRead,
  sendPlatformSupportMessage,
  getPlatformGuarantees,
  getBuddyApplications,
  getBuddyApplication,
```

```
  createBuddyApplicationFromMatch,
  markBuddyApplicationRead,
  getNoteStories,
  getNoteById,
  replyToBuddyApplication,
  getFormalConversation,
  markFormalConversationRead,
  sendFormalMessage,
  setMessageState,
}
```

Path: miniprogram/pages/home/home.js

```
---------------------------------------------------------------------
   1 const {
   2   isRenderableMedia,
   3   resolveActivityBanner,
   4   resolveActivityCover,
   5 } = require('../../utils/mediaAssets')
   6 const { buildActivityCoverTags } = require('../../utils/activityCoverTags')
   7
   8 const SAFE_PRODUCT_COVER = '/images/default-goods-image.png'
   9
  10 const DEFAULT_SEARCH_HINTS = [
  11   '    ',
  12   '    ',
  13   '     ',
  14   '    ',
  15   '    ',
  16   '    ',
  17   '    ',
  18 ]
  19
  20 const NAV_ITEMS = [
  21   {
  22     key: 'fun',
  23     title: '    ',
  24     icon: '/images/nav-fun.svg',
  25     action: 'goRuralFun',
  26   },
  27   {
  28     key: 'academy',
  29     title: '    ',
  30     icon: '/images/nav-academy.png',
  31     action: 'goAcademy',
  32   },
  33   {
  34     key: 'mall',
  35     title: '    ',
  36     icon: '/images/nav-mall.png',
  37     action: 'goMall',
  38   },
  39   {
  40     key: 'hotel',
  41     title: '    ',
  42     icon: '/images/nav-hotel.png',
  43     action: 'goHotel',
  44   },
  45 ]
  46
  47 const REGION_THEME_LIBRARY = [
  48   {
  49     matchKeywords: ['  ', '  ', '  ', '  '],
  50     themes: [
  51       {
  52         id: 'rose',
  53         title: '     ',
  54         period: '    ',
  55         description: '                            ',
  56         activityKeywords: ['  ', '  ', '  ', '  ', '  '],
  57         products: ['    ', '   ', '   ', '    ', '    '],
  58       },
  59       {
  60         id: 'lily',
```

```
        title: '     ',
        period: '    ',
        description: '                        ',
        activityKeywords: ['  ', '  ', '  ', '  ', '  '],
        products: ['   ', '   ', '     ', '    ', '   '],
      },
      {
        id: 'farm',
        title: '      ',
        period: '    ',
        description: '                      ',
        activityKeywords: ['  ', '  ', '  ', '  ', '  '],
        products: ['    ', '    ', '    ', '    '],
      },
    ],
  },
  {
    matchKeywords: ['  ', '  '],
    themes: [
      {
        id: 'fruit',
        title: '     ',
        period: '    ',
        description: '                         ',
        activityKeywords: ['  ', '  ', '  ', '  '],
        products: ['    ', '  ', '  ', '  ', '  '],
      },
      {
        id: 'study',
        title: '     ',
        period: '    ',
        description: '                       ',
        activityKeywords: ['  ', '  ', '  ', '  ', '  '],
        products: ['    ', '     ', '     ', '     '],
      },
      {
        id: 'folk',
        title: '     ',
        period: '    ',
        description: '                        ',
        activityKeywords: ['  ', '  ', '  ', '  '],
        products: ['    ', '    ', '    ', '    '],
      },
    ],
  },
  {
    matchKeywords: ['  ', '  ', '  '],
    themes: [
      {
        id: 'grassland',
        title: '     ',
        period: '    ',
        description: '                        ',
        activityKeywords: ['  ', '  ', '  ', '  ', '  '],
        products: ['    ', '    ', '   ', '    '],
      },
      {
        id: 'folk',
        title: '     ',
        period: '    ',
        description: '                        ',
        activityKeywords: ['  ', '  ', '  ', '  '],
```

```
123         products: ['    ', '    ', '     ', '    '],
124       },
125       {
126         id: 'light',
127         title: '     ',
128         period: '    ',
129         description: '                      ',
130         activityKeywords: ['  ', '  ', '  ', '  '],
131         products: ['     ', '    ', '     ', '    '],
132       },
133     ],
134   },
135   {
136     matchKeywords: ['  ', '  ', '  '],
137     themes: [
138       {
139         id: 'homestay',
140         title: '     ',
141         period: '    ',
142         description: '                      ',
143         activityKeywords: ['  ', '  ', '  ', '  '],
144         products: ['     ', '      ', '    ', '    '],
145       },
146       {
147         id: 'food',
148         title: '     ',
149         period: '    ',
150         description: '                      ',
151         activityKeywords: ['  ', '  ', '  ', '  '],
152         products: ['      ', '    ', '    ', '    '],
153       },
154       {
155         id: 'photo',
156         title: '     ',
157         period: '    ',
158         description: '                      ',
159         activityKeywords: ['  ', '  ', '  ', '  '],
160         products: ['     ', '    ', '    ', '     '],
161       },
162     ],
163   },
164 ]
165
166 const DEFAULT_THEME_GROUP = {
167   themes: [
168     {
169       id: 'season',
170       title: '     ',
171       period: '    ',
172       description: '                      ',
173       activityKeywords: ['  ', '  ', '  ', '  '],
174       products: ['    ', '    ', '     ', '    '],
175     },
176     {
177       id: 'craft',
178       title: '     ',
179       period: '    ',
180       description: '                    ',
181       activityKeywords: ['  ', '  ', '  ', '  '],
182       products: ['    ', '  ', '    ', '    '],
183     },
184     {
```

```
      id: 'slow',
      title: '     ',
      period: '    ',
      description: '                        ',
      activityKeywords: ['  ', '  ', '  ', '  '],
      products: ['    ', '   ', '     ', '    '],
    },
  ],
}

const THEME_CONTENT_OVERRIDES = {
  '     ': {
    activitySeedKeys: [
      'lz-yongdeng-rose-weekend',
      'lz-kushui-rose-culture-day',
      'lz-kushui-danxia-hike-day',
    ],
    productCards: [
      {
        seedKey: 'lz-rose-jam',
        title: '       ',
        price: 56,
        sold: 93,
        tags: ['    ', '   '],
      },
      {
        title: '        ',
        price: 79,
        sold: 41,
        tags: ['    ', '    '],
      },
    ],
  },
  '     ': {
    activitySeedKeys: [
      'lz-bailihe-handmade-food-tour',
      'lz-xigu-baihe-culture-day',
    ],
    productCards: [
      {
        seedKey: 'lz-baihe-gift-box',
        title: '        ',
        price: 88,
        sold: 126,
        tags: ['    ', '    '],
      },
      {
        title: '        ',
        price: 49,
        sold: 58,
        tags: ['    ', '  '],
      },
    ],
  },
  '      ': {
    activitySeedKeys: [
      'lz-yuzhong-strawberry-family-day',
      'lz-suburb-farm-study-camp',
      'lz-gaolan-country-photo-day',
      'lz-gaolan-shichuan-pear-garden-day',
    ],
    productCards: [
```

```
      {
        title: '        ',
        price: 69,
        sold: 84,
        tags: ['    ', '    '],
      },
      {
        title: '        ',
        price: 39,
        sold: 67,
        tags: ['    ', '    '],
      },
    ],
  },
}

function normalizeText(value = '') {
  return String(value || '').trim()
}

function trimRegionSuffix(name = '') {
  return normalizeText(name)
    .replace(/(     |   |  | )$/u, '')
    .replace(/( | | | )$/u, '')
}

function getRegionDisplayText(source) {
  if (!source) {
    return '  '
  }

  const displayName = normalizeText(source.displayName || source.label)
  const district = trimRegionSuffix(source.district)
  const city = trimRegionSuffix(source.city)
  const province = trimRegionSuffix(source.province)
  const locationText = normalizeText(source.locationText || source.address)
  return displayName || district || city || province || locationText || '  '
}

function pickThemeGroup(regionName) {
  const regionText = normalizeText(regionName)
  return REGION_THEME_LIBRARY.find((group) =>
    group.matchKeywords.some((keyword) => regionText.includes(keyword))
  ) || DEFAULT_THEME_GROUP
}

function getThemeOverride(theme = {}) {
  return THEME_CONTENT_OVERRIDES[normalizeText(theme.title)] || null
}

function buildThemeActivities(activityList, theme) {
  const override = getThemeOverride(theme)
  const activitySeedKeys = (override && override.activitySeedKeys) || []
  if (activitySeedKeys.length) {
    const activityMap = activityList.reduce((result, activity) => {
      result[activity.seedKey] = activity
      return result
    }, {})

    return activitySeedKeys
      .map((seedKey) => activityMap[seedKey])
      .filter(Boolean)
```

```
  }

  const keywords = theme.activityKeywords || []
  return activityList.filter((activity) => {
    const textList = [normalizeText(activity.title), ...(activity.rawTags || []).map(normalizeText)]
    return textList.some((text) => keywords.some((keyword) => text.includes(keyword)))
  }).slice(0, 2)
}

function normalizeProduct(product = {}) {
  const categoryTags = Array.isArray(product.categoryTags) ? product.categoryTags : []
  const tags = Array.isArray(product.tags) ? product.tags : []
  return {
    id: product._id || '',
    seedKey: product.seedKey || '',
    title: product.title || '     ',
    cover: product.cover || SAFE_PRODUCT_COVER,
    price: Number(product.price) || 0,
    priceText: product.price ? `¥${product.price}` : '    ',
    sold: Number(product.sold) || 0,
    soldText: product.sold ? `${product.sold}   ` : '    ',
    tags: [...categoryTags, ...tags].slice(0, 3),
  }
}

function buildThemeProducts(productList, theme) {
  const override = getThemeOverride(theme)
  const configuredCards = (override && override.productCards) || []
  const productMap = productList.reduce((result, product) => {
    result[product.seedKey] = product
    return result
  }, {})

  if (configuredCards.length) {
    return configuredCards.map((card, index) => {
      const matchedProduct = card.seedKey ? productMap[card.seedKey] : null
      return {
        _cardKey: `${theme.id}-product-${index}`,
        id: matchedProduct ? matchedProduct.id : '',
        seedKey: (matchedProduct && matchedProduct.seedKey) || card.seedKey || '',
        title: (matchedProduct && matchedProduct.title) || card.title,
        cover: (matchedProduct && matchedProduct.cover) || card.cover || SAFE_PRODUCT_COVER,
        priceText: matchedProduct ? matchedProduct.priceText : `¥${card.price}`,
        soldText: matchedProduct ? matchedProduct.soldText : `${card.sold}   `,
        tags: (matchedProduct && matchedProduct.tags && matchedProduct.tags.length)
          ? matchedProduct.tags
          : (card.tags || []),
      }
    })
  }

  return (theme.products || []).map((title, index) => ({
    _cardKey: `${theme.id}-product-${index}`,
    id: '',
    seedKey: '',
    title,
    cover: SAFE_PRODUCT_COVER,
    priceText: '    ',
    soldText: theme.period || '    ',
    tags: [theme.title],
```

```
  }))
}

Page({
  data: {
    regionName: '  ',
    statusBarHeight: 20,
    menuRect: null,
    searchHints: DEFAULT_SEARCH_HINTS,
    searchHintIndex: 0,
    currentSearchHint: DEFAULT_SEARCH_HINTS[0],
    navItems: NAV_ITEMS,
    bannerList: [],
    activityList: [],
    productList: [],
    themeTabs: [],
    themeActive: 0,
    themeList: [],
    currentTheme: null,
  },

  onLoad() {
    this.initNavMetrics()
    this.syncRegionName()
    this.startSearchHintRotation()
    this.loadActivities()
    this.loadProducts()
  },

  onShow() {
    this.syncRegionName()
    this.startSearchHintRotation()
  },

  onHide() {
    this.clearSearchHintTimer()
  },

  onUnload() {
    this.clearSearchHintTimer()
  },

  initNavMetrics() {
    try {
      const systemInfo = wx.getSystemInfoSync()
      const menuRect = wx.getMenuButtonBoundingClientRect()
      this.setData({
        statusBarHeight: systemInfo.statusBarHeight || 20,
        menuRect,
      })
    } catch (err) {
      this.setData({
        statusBarHeight: 20,
        menuRect: null,
      })
    }
  },

  startSearchHintRotation() {
    this.clearSearchHintTimer()
    this.searchHintTimer = setInterval(() => {
      const { searchHints, searchHintIndex } = this.data
```

```
      if (!searchHints.length) {
        return
      }

      const nextIndex = (searchHintIndex + 1) % searchHints.length
      this.setData({
        searchHintIndex: nextIndex,
        currentSearchHint: searchHints[nextIndex],
      })
    }, 5000)
  },

  clearSearchHintTimer() {
    if (this.searchHintTimer) {
      clearInterval(this.searchHintTimer)
      this.searchHintTimer = null
    }
  },

  syncRegionName() {
    const app = getApp()
    const userInfo = app.getUserInfo ? app.getUserInfo() : null
    const selectedRegion = wx.getStorageSync('selectedRegion') || null
    const regionName = getRegionDisplayText(selectedRegion || userInfo)

    this.setData({ regionName }, () => this.refreshThemeSection())
  },

  async loadActivities() {
    try {
      const res = await wx.cloud.callFunction({
        name: 'getactivities',
      })

      const list = (res.result && res.result.list) || []
      const mapped = await Promise.all(
        list.map(async (item) => {
          const coverTagInfo = buildActivityCoverTags(item)
          return {
            id: item._id,
            seedKey: item.seedKey || '',
            title: item.title || '     ',
            cover: await resolveActivityCover(item),
            dayText: coverTagInfo.durationTag,
            tags: coverTagInfo.tags,
            priceText: item.priceFrom || item.price ? `¥${item.priceFrom || item.price}`
: '    ',
            soldText: item.traveledCount ? `${item.traveledCount}    ` : '    ',
            rawTags: item.tags || [],
          }
        })
      )

      const preferredBanners = mapped.slice(0, 3)
      const bannerList = await Promise.all(
        preferredBanners.map(async (item, index) => ({
          _id: `activity-banner-${item.seedKey || index}`,
          type: 'activity',
          targetId: item.id,
          image: await resolveActivityBanner(item),
        }))
      )
```

```

      this.setData({
        bannerList: bannerList.filter((item) => isRenderableMedia(item.image)),
        activityList: mapped,
      }, () => this.refreshThemeSection())
    } catch (err) {
      console.error('load activities failed', err)
      wx.showToast({
        title: '    ',
        icon: 'none',
      })
    }
  },

  async loadProducts() {
    try {
      const db = wx.cloud.database()
      const res = await db.collection('products').limit(50).get()
      const productList = ((res && res.data) || []).map((item) => normalizeProduct(item)
)

      this.setData({
        productList,
      }, () => this.refreshThemeSection())
    } catch (err) {
      console.error('load products failed', err)
      this.refreshThemeSection()
    }
  },

  formatDays(days) {
    if (!days) {
      return '1 '
    }
    if (days === 0.5) {
      return '0.5 '
    }
    return `${days} `
  },

  refreshThemeSection() {
    const { regionName, activityList, productList, themeActive } = this.data
    const themeGroup = pickThemeGroup(regionName)
    const themeList = themeGroup.themes.map((theme) => ({
      ...theme,
      activities: buildThemeActivities(activityList, theme),
      products: buildThemeProducts(productList, theme),
    }))
    const nextActive = themeList[themeActive] ? themeActive : 0

    this.setData({
      themeTabs: themeList.map((item) => item.title),
      themeList,
      themeActive: nextActive,
      currentTheme: themeList[nextActive] || null,
    })
  },

  changeThemeTab(e) {
    const idx = Number(e.currentTarget.dataset.idx)
    this.setData({
      themeActive: idx,
```

```
      currentTheme: this.data.themeList[idx] || null,
    })
  },

  goSearch() {
    const hint = this.data.currentSearchHint || ''
    wx.navigateTo({
      url: `/pages/search/search?hint=${encodeURIComponent(hint)}`,
    })
  },

  executeSearch() {
    const keyword = this.data.currentSearchHint || ''
    wx.navigateTo({
      url: `/pages/searchResults/searchResults?keyword=${encodeURIComponent(keyword)}`,
    })
  },

  onNavTap(e) {
    const action = e.currentTarget.dataset.action
    if (action && typeof this[action] === 'function') {
      this[action]()
    }
  },

  goBannerTarget(e) {
    const { type, id } = e.currentTarget.dataset
    if (!type || !id) {
      wx.showToast({
        title: '       ',
        icon: 'none',
      })
      return
    }

    if (type === 'activity') {
      wx.navigateTo({ url: `/pages/activityDetail/activityDetail?id=${id}` })
      return
    }

    if (type === 'product') {
      wx.navigateTo({ url: `/pages/productDetail/productDetail?id=${id}` })
      return
    }

    if (type === 'scenic') {
      wx.navigateTo({ url: `/pages/scenicDetail/scenicDetail?id=${id}` })
      return
    }
  },

  goActivityDetail(e) {
    const id = e.currentTarget.dataset.id
    if (!id) {
      return
    }

    wx.navigateTo({
      url: `/pages/activityDetail/activityDetail?id=${id}`,
    })
  },

```

```
  goProductDetail(e) {
    const id = e.currentTarget.dataset.id
    if (!id) {
      wx.showToast({
        title: '         ',
        icon: 'none',
      })
      return
    }

    wx.navigateTo({
      url: `/pages/productDetail/productDetail?id=${id}`,
    })
  },

  goRegion() {
    wx.navigateTo({
      url: '/pages/regionSelect/regionSelect',
    })
  },

  onScanTap() {
    wx.showToast({
      title: '  /        ',
      icon: 'none',
    })
  },

  goRuralFun() {
    wx.navigateTo({
      url: '/pages/ruralFun/ruralFun',
    })
  },

  goAcademy() {
    wx.navigateTo({
      url: '/pages/academy/academy',
    })
  },

  goHotel() {
    wx.navigateTo({
      url: '/pages/hotel/hotel',
    })
  },

  goMall() {
    wx.navigateTo({
      url: '/pages/mall/mall',
    })
  },

  goNotePublish() {
    wx.navigateTo({
      url: '/pages/notePublish/notePublish',
    })
  },
})
```

Path: miniprogram/pages/home/home.json

```
1 {
2   "navigationStyle": "custom"
3 }
```

Path: miniprogram/pages/home/home.wxml

```
<view class="page">
  <view class="customNav" style="padding-top: {{statusBarHeight}}px;">
    <view class="navCapsuleRow">
      <view class="locationInfo" catchtap="goRegion">
        <view class="locationIcon"></view>
        <text class="locationText">{{regionName}}</text>
        <text class="locationArrow">▼</text>
      </view>

      <view class="capsuleSpacer" wx:if="{{menuRect}}"></view>
    </view>

    <view class="searchRow">
      <view class="searchBox">
        <view class="searchInputArea" bindtap="goSearch">
          <text class="searchInputText">{{currentSearchHint}}</text>
        </view>
        <view class="searchAction" catchtap="executeSearch">  </view>
      </view>

      <view class="scanAction" bindtap="onScanTap">
        <view class="scanIcon"></view>
      </view>
    </view>
  </view>

  <view class="nav">
    <view
      wx:for="{{navItems}}"
      wx:key="key"
      class="navItem"
      data-action="{{item.action}}"
      bindtap="onNavTap"
    >
      <image class="navIconImg" src="{{item.icon}}" mode="aspectFit"></image>
      <view class="navText">{{item.title}}</view>
    </view>
  </view>

  <view class="bannerWrap" wx:if="{{bannerList.length > 0}}">
    <swiper
      class="bannerSwiper"
      indicator-dots="{{true}}"
      indicator-color="rgba(255,255,255,0.45)"
      indicator-active-color="#ffffff"
      autoplay="{{true}}"
      interval="3000"
      duration="500"
      circular="{{true}}"
    >
      <swiper-item wx:for="{{bannerList}}" wx:key="_id">
        <view class="bannerItem" data-type="{{item.type}}" data-id="{{item.targetId}}" b
indtap="goBannerTarget">
          <image src="{{item.image}}" mode="aspectFill" class="bannerImg" />
        </view>
      </swiper-item>
    </swiper>
  </view>

  <view class="bannerWrap" wx:else>
```

```
    <view class="bannerItem">
      <image src="/images/activities/lz-yuzhong-strawberry-family-day.jpg" mode="aspectFill" class="bannerImg" />
    </view>
  </view>

  <view class="themeSection">
    <view class="sectionHead">
      <view class="hotTitle">
        <text class="line"></text>
        <text class="titleText">    </text>
      </view>
    </view>

    <scroll-view class="themeTabsScroll" scroll-x="{{true}}" enable-flex="{{true}}" show-scrollbar="{{false}}">
      <view class="hotTabs">
        <view
          wx:for="{{themeTabs}}"
          wx:key="*this"
          class="tab {{themeActive===index ? 'active' : ''}}"
          data-idx="{{index}}"
          bindtap="changeThemeTab"
        >
          {{item}}
        </view>
      </view>
    </scroll-view>

    <view class="themePanel" wx:if="{{currentTheme}}">
      <view class="themePanelHead">
        <view class="themeTitleRow">
          <text class="themePeriod">{{currentTheme.period}}</text>
        </view>
        <text class="themeDesc">{{currentTheme.description}}</text>
      </view>

      <view class="themeBlock">
        <text class="themeBlockTitle">    </text>
        <view wx:if="{{currentTheme.activities.length > 0}}">
          <view
            wx:for="{{currentTheme.activities}}"
            wx:key="id"
            class="themeActivityCard"
            data-id="{{item.id}}"
            bindtap="goActivityDetail"
          >
            <view class="themeActivityCoverWrap">
              <image src="{{item.cover}}" class="themeActivityCover" mode="aspectFill" />
            </view>
            <view class="themeActivityBody">
              <text class="themeActivityName">{{item.title}}</text>
              <view class="themeActivityMeta">
                <text class="dayTag">{{item.dayText}}</text>
                <text wx:for="{{item.tags}}" wx:key="*this" class="productTag">{{item}}</text>
              </view>
              <view class="themeActivityFooter">
                <text class="themeActivityPrice">{{item.priceText}}</text>
                <text class="themeActivitySold">{{item.soldText}}</text>
              </view>
```

```
            </view>
          </view>
        </view>
        <view class="emptyState" wx:else>             </view>
      </view>

      <view class="themeBlock">
        <text class="themeBlockTitle">    </text>
        <view wx:if="{{currentTheme.products.length > 0}}" class="themeProductList">
          <view
            wx:for="{{currentTheme.products}}"
            wx:key="_cardKey"
            class="themeProductCard"
            data-id="{{item.id}}"
            bindtap="goProductDetail"
          >
            <image src="{{item.cover}}" class="themeProductCover" mode="aspectFill" />
            <view class="themeProductBody">
              <text class="themeProductName">{{item.title}}</text>
              <view class="themeProductTags">
                <text wx:for="{{item.tags}}" wx:key="*this" class="productTag">{{item}}<
/text>
              </view>
              <view class="themeProductFooter">
                <text class="themeProductPrice">{{item.priceText}}</text>
                <text class="themeProductSold">{{item.soldText}}</text>
              </view>
            </view>
          </view>
        </view>
        <view class="emptyState" wx:else>             </view>
      </view>
    </view>
  </view>

  <view class="publishFloat" bindtap="goNotePublish">
    <view class="publishFloatPlus">+</view>
    <view class="publishFloatText">    </view>
  </view>
</view>
```

Path: miniprogram/pages/home/home.wxss

```
.page {
  min-height: 100vh;
  background: #f6faf4;
  padding-bottom: 30rpx;
}

.customNav {
  background: #f6faf4;
  padding-left: 24rpx;
  padding-right: 24rpx;
  padding-bottom: 14rpx;
}

.navCapsuleRow {
  min-height: 88rpx;
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.locationInfo {
  display: flex;
  align-items: center;
  gap: 12rpx;
  color: #1f2a1f;
  max-width: 70%;
}

.locationIcon {
  width: 24rpx;
  height: 24rpx;
  border: 4rpx solid #1f2a1f;
  border-radius: 50%;
  position: relative;
}

.locationIcon::after {
  content: '';
  position: absolute;
  left: 50%;
  bottom: -12rpx;
  width: 4rpx;
  height: 14rpx;
  background: #1f2a1f;
  transform: translateX(-50%);
  border-radius: 999rpx;
}

.locationText {
  font-size: 30rpx;
  font-weight: 700;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.locationArrow {
  font-size: 24rpx;
  flex-shrink: 0;
}
```

```
.capsuleSpacer {
  width: 180rpx;
  height: 64rpx;
}

.searchRow {
  display: flex;
  align-items: center;
  gap: 16rpx;
  margin-top: 6rpx;
}

.searchBox {
  flex: 1;
  min-height: 82rpx;
  padding: 0 10rpx 0 0;
  border-radius: 999rpx;
  background: #ffffff;
  border: 3rpx solid #d7e8d3;
  display: flex;
  align-items: center;
  box-shadow: 0 14rpx 28rpx rgba(54, 102, 61, 0.08);
}

.searchInputArea {
  flex: 1;
  min-width: 0;
  min-height: 82rpx;
  display: flex;
  align-items: center;
  padding: 0 24rpx 0 26rpx;
}

.searchInputText {
  color: #8ea08b;
  font-size: 26rpx;
  font-weight: 400;
}

.searchAction {
  flex-shrink: 0;
  min-width: 96rpx;
  padding: 14rpx 22rpx;
  border-radius: 999rpx;
  background: #57984a;
  color: #ffffff;
  font-size: 26rpx;
  text-align: center;
  font-weight: 400;
}

.scanAction {
  width: 82rpx;
  height: 82rpx;
  border-radius: 22rpx;
  background: #edf5ea;
  box-shadow: 0 14rpx 28rpx rgba(54, 102, 61, 0.08);
  display: flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
```

```
}

.scanIcon {
  width: 40rpx;
  height: 40rpx;
  position: relative;
  border-left: 4rpx solid transparent;
}

.scanIcon::before,
.scanIcon::after {
  content: '';
  position: absolute;
  width: 14rpx;
  height: 14rpx;
  border: 4rpx solid #1f2a1f;
}

.scanIcon::before {
  left: 0;
  top: 0;
  border-right: none;
  border-bottom: none;
}

.scanIcon::after {
  right: 0;
  bottom: 0;
  border-left: none;
  border-top: none;
}

.nav {
  margin: 18rpx 24rpx 24rpx;
  background: #ffffff;
  border-radius: 22rpx;
  padding: 26rpx 6rpx;
  display: flex;
  justify-content: space-between;
  box-shadow: 0 16rpx 32rpx rgba(53, 88, 57, 0.06);
}

.navItem {
  width: 25%;
  display: flex;
  flex-direction: column;
  align-items: center;
}

.navIconImg {
  width: 88rpx;
  height: 88rpx;
  display: block;
}

.navText {
  margin-top: 14rpx;
  font-size: 26rpx;
  color: #41513f;
  text-align: center;
}

```

```
.bannerWrap {
  margin: 0 24rpx 24rpx;
}

.bannerSwiper {
  width: 100%;
  height: 360rpx;
}

.bannerItem {
  width: 100%;
  height: 100%;
  border-radius: 28rpx;
  overflow: hidden;
}

.bannerImg {
  width: 100%;
  height: 100%;
  display: block;
}

.themeSection {
  margin: 0 24rpx;
}

.sectionHead {
  margin-bottom: 18rpx;
}

.hotTitle {
  display: flex;
  align-items: center;
}

.line {
  width: 8rpx;
  height: 34rpx;
  background: #57984a;
  border-radius: 999rpx;
  margin-right: 12rpx;
}

.titleText {
  font-size: 34rpx;
  font-weight: 700;
  color: #1f2a1f;
}

.themeTabsScroll {
  width: 100%;
  margin-bottom: 12rpx;
}

.hotTabs {
  display: inline-flex;
  gap: 16rpx;
  padding-bottom: 2rpx;
}

.tab {
  padding: 10rpx 22rpx;
```

```
  border-radius: 22rpx;
  background: #eef6ee;
  color: #5f7260;
  font-size: 28rpx;
  white-space: nowrap;
}

.tab.active {
  background: #57984a;
  color: #ffffff;
}

.themePanel {
  background: #ffffff;
  border-radius: 24rpx;
  padding: 28rpx 24rpx;
  margin-bottom: 20rpx;
}

.themePanelHead {
  margin-bottom: 28rpx;
}

.themeTitleRow {
  display: flex;
  align-items: center;
  gap: 14rpx;
  margin-bottom: 12rpx;
}

.themePeriod {
  display: inline-flex;
  padding: 8rpx 18rpx;
  border-radius: 999rpx;
  background: #edf5ea;
  color: #5f7260;
  font-size: 22rpx;
}

.themeDesc {
  font-size: 26rpx;
  color: #667566;
  line-height: 1.7;
}

.themeBlock {
  margin-bottom: 26rpx;
}

.themeBlock:last-child {
  margin-bottom: 0;
}

.themeBlockTitle {
  display: block;
  font-size: 28rpx;
  font-weight: 600;
  color: #233123;
  margin-bottom: 18rpx;
}

.themeActivityCard {
```

```
  display: flex;
  align-items: stretch;
  background: #f7fbf5;
  border-radius: 20rpx;
  overflow: hidden;
  margin-bottom: 18rpx;
  height: 276rpx;
}

.themeActivityCard:last-child {
  margin-bottom: 0;
}

.themeActivityCoverWrap {
  width: 180rpx;
  height: 276rpx;
  flex-shrink: 0;
  overflow: hidden;
}

.themeActivityCover {
  width: 100%;
  height: 100%;
  flex-shrink: 0;
  display: block;
}

.themeActivityBody {
  flex: 1;
  min-width: 0;
  padding: 18rpx 20rpx;
  display: flex;
  flex-direction: column;
}

.themeActivityName {
  display: block;
  font-size: 28rpx;
  font-weight: 600;
  color: #1f2a1f;
  line-height: 1.5;
  display: -webkit-box;
  -webkit-line-clamp: 2;
  -webkit-box-orient: vertical;
  overflow: hidden;
}

.themeActivityMeta {
  display: flex;
  flex-wrap: wrap;
  gap: 12rpx;
  margin-top: 14rpx;
}

.dayTag {
  padding: 8rpx 18rpx;
  border-radius: 999rpx;
  background: #57984a;
  color: #ffffff;
  font-size: 22rpx;
}

```

```
.themeActivityFooter {
  margin-top: auto;
  padding-top: 18rpx;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16rpx;
}

.themeActivityPrice {
  font-size: 30rpx;
  line-height: 1;
  color: #57984a;
  font-weight: 700;
}

.themeActivitySold {
  font-size: 22rpx;
  color: #8ea08b;
  text-align: right;
}

.themeProductList {
  display: flex;
  flex-direction: column;
  gap: 18rpx;
}

.themeProductCard {
  display: flex;
  align-items: stretch;
  background: #f7fbf5;
  border-radius: 20rpx;
  overflow: hidden;
  height: 180rpx;
}

.themeProductCover {
  width: 180rpx;
  height: 180rpx;
  flex-shrink: 0;
}

.themeProductBody {
  flex: 1;
  min-width: 0;
  padding: 18rpx 20rpx;
  display: flex;
  flex-direction: column;
}

.themeProductName {
  font-size: 28rpx;
  line-height: 1.5;
  color: #1f2a1f;
  font-weight: 600;
  display: -webkit-box;
  -webkit-line-clamp: 2;
  -webkit-box-orient: vertical;
  overflow: hidden;
}

```

```
.themeProductTags {
  display: flex;
  flex-wrap: wrap;
  gap: 12rpx;
  margin-top: 14rpx;
}

.productTag {
  padding: 8rpx 18rpx;
  border-radius: 999rpx;
  background: #edf5ea;
  color: #426046;
  font-size: 22rpx;
}

.themeProductFooter {
  margin-top: auto;
  padding-top: 18rpx;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16rpx;
}

.themeProductPrice {
  font-size: 30rpx;
  line-height: 1;
  color: #57984a;
  font-weight: 700;
}

.themeProductSold {
  font-size: 22rpx;
  color: #8ea08b;
  text-align: right;
}

.emptyState {
  padding: 28rpx 0;
  font-size: 26rpx;
  color: #8ea08b;
}

.publishFloat {
  position: fixed;
  right: 28rpx;
  bottom: 120rpx;
  width: 132rpx;
  height: 132rpx;
  border-radius: 50%;
  background: linear-gradient(135deg, #5ea94f 0%, #3f8a3e 100%);
  box-shadow: 0 18rpx 36rpx rgba(63, 138, 62, 0.28);
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  color: #ffffff;
  z-index: 20;
}

.publishFloatPlus {
  font-size: 48rpx;
```

```
  line-height: 1;
  font-weight: 600;
}

.publishFloatText {
  margin-top: 6rpx;
  font-size: 22rpx;
  line-height: 1.2;
}


```

Path: miniprogram/pages/login/login.js

```
---------------------------------------------------------------------
const app = getApp()

const TEXT = {
  loginFailed: '\u767b\u5f55\u5931\u8d25',
  loginRetry: '\u767b\u5f55\u5931\u8d25\uff0c\u8bf7\u91cd\u8bd5',
  unsupportedLoginMethod: '\u4e0d\u652f\u6301\u7684\u767b\u5f55\u65b9\u5f0f',
  brandName: '\u519c\u65c5e\u7ad9',
  tagline: '\u4e61\u91ce\u98ce\u7269\u4e0e\u4e2a\u6027\u5316\u65c5\u7a0b\uff0c\u4ece\u8fd9\u91cc\u5f00\u59cb',
  heroTitle: '\u6388\u6743\u767b\u5f55\u540e\uff0c\u5feb\u901f\u5f00\u542f\u4f60\u7684\u4e13\u5c5e\u519c\u65c5\u4f53\u9a8c',
  titleWechat: '\u6b22\u8fce\u6765\u5230\u519c\u65c5e\u7ad9',
  descBeforeLogin: '\u4e00\u952e\u767b\u5f55\u540e\uff0c\u5373\u53ef\u7ee7\u7eed\u5b8c\u5584\u4f60\u7684\u519c\u65c5\u504f\u597d\u4e0e\u4e13\u5c5e\u884c\u7a0b\u3002',
  btnWechatLogin: '\u5fae\u4fe1\u4e00\u952e\u767b\u5f55',
  agreementSummary: '\u5df2\u9605\u8bfb\u5e76\u540c\u610f\u300a\u519c\u65c5e\u7ad9\u4e2a\u4eba\u4fe1\u606f\u4fdd\u62a4\u6307\u5f15\u300b\u3001\u300a\u519c\u65c5e\u7ad9\u7528\u6237\u670d\u52a1\u8bf4\u660e\u300b',
  footer: '\u767b\u5f55\u540e\u5c06\u7ee7\u7eed\u5b8c\u6210\u5b9a\u4f4d\u3001\u8d44\u6599\u548c\u519c\u65c5 DNA \u8bbe\u7f6e\u3002',
}

function mapCloudMessage(message) {
  const messageMap = {
    UNSUPPORTED_LOGIN_METHOD: TEXT.unsupportedLoginMethod,
  }
  return messageMap[message] || message
}

Page({
  data: {
    text: {
      brandName: TEXT.brandName,
      tagline: TEXT.tagline,
      heroTitle: TEXT.heroTitle,
      titleWechat: TEXT.titleWechat,
      descBeforeLogin: TEXT.descBeforeLogin,
      btnWechatLogin: TEXT.btnWechatLogin,
      agreementSummary: TEXT.agreementSummary,
      footer: TEXT.footer,
    },
    loading: false,
  },

  onShow() {
    const agreePrivacy = wx.getStorageSync('agreePrivacy')
    if (!agreePrivacy) {
      wx.redirectTo({ url: '/pages/welcome/welcome' })
      return
    }
    this.syncFromStorage()
  },

  syncFromStorage() {
    const currentUser = app.getUserInfo() || null
    if (currentUser && currentUser.openid) {
      this.finishLogin(currentUser)
    }
  },

```

```
  async onWechatLogin() {
    this.setData({ loading: true })
    try {
      const loginRes = await wx.cloud.callFunction({
        name: 'login',
        data: { method: 'wechat' },
      })

      if (!loginRes.result.success) {
        wx.showToast({ title: mapCloudMessage(loginRes.result.message) || TEXT.loginFail
ed, icon: 'none' })
        return
      }

      app.setUserInfo(loginRes.result.userInfo)
      this.finishLogin(loginRes.result.userInfo)
    } catch (err) {
      console.error('[login] wechat login failed', err)
      wx.showToast({ title: TEXT.loginRetry, icon: 'none' })
    } finally {
      this.setData({ loading: false })
    }
  },

  finishLogin(userInfo) {
    app.setUserInfo(userInfo)
    const target = app.getPostLoginTarget(userInfo)
    if (target === '/pages/home/home') {
      wx.switchTab({ url: target })
      return
    }
    wx.redirectTo({ url: target })
  },
})
```

Path: miniprogram/pages/login/login.json

```
{
  "navigationBarTitleText": "  "
}
```

Path: miniprogram/pages/login/login.wxml

```
<view class="page">
  <view class="ambient ambientLeft"></view>
  <view class="ambient ambientRight"></view>
  <view class="grain"></view>

  <view class="content">
    <view class="brandBlock">
      <image class="brandAvatar" src="/images/login-brand.png" mode="aspectFill"></image>
      <view class="brandName">{{text.brandName}}</view>
      <view class="brandTagline">{{text.tagline}}</view>
    </view>

    <view class="heroBlock">
      <view class="heroTitle">{{text.titleWechat}}</view>
      <view class="heroDesc">{{text.descBeforeLogin}}</view>
    </view>

    <view class="formCard">
      <view class="cardTop">
        <view class="cardBadge">AgriTravel</view>
        <view class="cardHint">{{text.heroTitle}}</view>
      </view>

      <button
        class="primaryButton"
        bindtap="onWechatLogin"
        loading="{{loading}}"
        disabled="{{loading}}"
      >
        {{text.btnWechatLogin}}
      </button>

      <view class="agreement">{{text.agreementSummary}}</view>
    </view>
  </view>

  <view class="footer">{{text.footer}}</view>
</view>
```

Path: miniprogram/pages/login/login.wxss

```
.page {
  position: relative;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  background:
    radial-gradient(circle at top left, rgba(186, 218, 183, 0.42) 0, rgba(186, 218, 183, 0) 36%),
    radial-gradient(circle at 85% 22%, rgba(225, 239, 210, 0.68) 0, rgba(225, 239, 210, 0) 28%),
    linear-gradient(180deg, #f7fbf5 0%, #eef6ea 58%, #f8fbf7 100%);
  overflow: hidden;
}

.ambient {
  position: absolute;
  border-radius: 999rpx;
  filter: blur(16rpx);
  opacity: 0.9;
}

.ambientLeft {
  top: 220rpx;
  left: -120rpx;
  width: 360rpx;
  height: 360rpx;
  background: rgba(213, 232, 193, 0.78);
}

.ambientRight {
  top: 640rpx;
  right: -90rpx;
  width: 280rpx;
  height: 280rpx;
  background: rgba(194, 222, 184, 0.72);
}

.grain {
  position: absolute;
  inset: 0;
  background:
    linear-gradient(115deg, rgba(255, 255, 255, 0.18) 0, rgba(255, 255, 255, 0) 34%),
    linear-gradient(290deg, rgba(255, 255, 255, 0.12) 0, rgba(255, 255, 255, 0) 38%);
  pointer-events: none;
}

.content {
  position: relative;
  z-index: 1;
  flex: 1;
  display: flex;
  flex-direction: column;
  justify-content: center;
  padding: 90rpx 40rpx 24rpx;
}

.brandBlock {
  display: flex;
  flex-direction: column;
  align-items: center;
```

```
  text-align: center;
}

.brandAvatar {
  width: 168rpx;
  height: 168rpx;
  border-radius: 50%;
  border: 8rpx solid rgba(255, 255, 255, 0.92);
  box-shadow: 0 24rpx 56rpx rgba(65, 109, 60, 0.16);
}

.brandName {
  margin-top: 28rpx;
  font-size: 56rpx;
  font-weight: 700;
  color: #23472a;
  letter-spacing: 2rpx;
}

.brandTagline {
  margin-top: 14rpx;
  font-size: 26rpx;
  color: #70856f;
  line-height: 1.7;
}

.heroBlock {
  margin-top: 88rpx;
  text-align: center;
}

.heroTitle {
  font-size: 52rpx;
  font-weight: 700;
  color: #203624;
  line-height: 1.34;
}

.heroDesc {
  margin-top: 22rpx;
  font-size: 28rpx;
  line-height: 1.8;
  color: #697d67;
  padding: 0 16rpx;
}

.formCard {
  margin-top: 72rpx;
  padding: 42rpx 36rpx 40rpx;
  border-radius: 36rpx;
  background: rgba(255, 255, 255, 0.92);
  border: 2rpx solid rgba(223, 235, 218, 0.95);
  box-shadow: 0 30rpx 64rpx rgba(49, 92, 53, 0.1);
  backdrop-filter: blur(12rpx);
}

.cardTop {
  margin-bottom: 34rpx;
  text-align: center;
}

.cardBadge {
```

```
  display: inline-flex;
  align-items: center;
  justify-content: center;
  min-width: 168rpx;
  height: 50rpx;
  padding: 0 24rpx;
  border-radius: 999rpx;
  background: #edf5ea;
  color: #57984a;
  font-size: 22rpx;
  font-weight: 600;
  letter-spacing: 1rpx;
}

.cardHint {
  margin-top: 18rpx;
  font-size: 28rpx;
  font-weight: 600;
  color: #37553b;
  line-height: 1.6;
}

.primaryButton {
  width: 100%;
  height: 94rpx;
  line-height: 94rpx;
  border: none;
  border-radius: 999rpx;
  background: linear-gradient(135deg, #57984a 0%, #57984a 100%);
  color: #ffffff;
  font-size: 32rpx;
  font-weight: 700;
  box-shadow: 0 16rpx 36rpx rgba(87, 152, 74, 0.22);
}

.primaryButton[disabled] {
  background: #b7c8b4;
  box-shadow: none;
}

.agreement {
  margin-top: 28rpx;
  text-align: center;
  font-size: 24rpx;
  line-height: 1.8;
  color: #80907e;
}

.agreementStrong {
  color: #57984a;
}

.agreementJoin,
.agreementPrefix {
  color: #80907e;
}

.footer {
  position: relative;
  z-index: 1;
  padding: 18rpx 44rpx 48rpx;
  font-size: 22rpx;
```

```
  line-height: 1.7;
  text-align: center;
  color: #81917f;
}
```

Path: miniprogram/pages/activityDetail/activityDetail.js

```
const {
  resolveActivityCover,
  resolveActivityGallery,
} = require('../../utils/mediaAssets')
const { buildActivityCoverTags } = require('../../utils/activityCoverTags')
const {
  isFavorited,
  toggleFavorite,
  recordFootprint,
} = require('../../utils/collectionStore')

Page({
  data: {
    detail: null,
    activityId: '',
    favorited: false,
    statusBarHeight: 20,
    favoriteIcon: '/images/icons/favorite-heart-outline.png',
    favoriteIconFilled: '/images/icons/favorite-heart-filled.png',
    text: {
      location: '    ',
      price: '    ',
      transport: '    ',
      stay: '    ',
      travelMode: '    ',
      playMode: '    ',
      suitableGroups: '    ',
      highlights: '    ',
      itinerary: '    ',
      info: '    ',
      pickup: '    ',
      pickupPoint: '    ',
      address: '    ',
      phone: '    ',
      duration: '    ',
      region: '    ',
      description: '    ',
      gallery: '    ',
      priceLabel: '   ',
      bookNow: '    ',
      empty: '      ',
    },
  },

  onLoad(options) {
    this.initNavMetrics()
    const id = options.id || ''
    this.setData({ activityId: id })
    if (id) {
      this.loadDetail(id)
    }
  },

  onShow() {
    const { activityId } = this.data
    if (activityId) {
      this.loadDetail(activityId)
    }
  },

```

```
  initNavMetrics() {
    try {
      const systemInfo = wx.getSystemInfoSync()
      this.setData({
        statusBarHeight: systemInfo.statusBarHeight || 20,
      })
    } catch (error) {
      this.setData({
        statusBarHeight: 20,
      })
    }
  },

  async loadDetail(id) {
    wx.showLoading({ title: '    ' })
    try {
      const res = await wx.cloud.callFunction({
        name: 'getactivitydetail',
        data: { id },
      })
      const rawDetail = res.result?.data || null
      const detail = await this.normalizeDetail(rawDetail)
      const favorited = isFavorited('activity', detail && detail._id)
      if (detail) {
        recordFootprint(this.buildCollectionRecord(detail))
      }

      this.setData({
        detail,
        favorited,
      })
    } catch (error) {
      console.error('[activityDetail] load failed', error)
      wx.showToast({
        title: '    ',
        icon: 'none',
      })
    } finally {
      wx.hideLoading()
    }
  },

  async normalizeDetail(detail) {
    if (!detail) {
      return null
    }

    const cover = await resolveActivityCover(detail)
    const gallery = await resolveActivityGallery(detail)
    const tags = Array.isArray(detail.tags) ? detail.tags : []
    const travelModeTags = Array.isArray(detail.travelModeTags) ? detail.travelModeTags : []
    const playTags = Array.isArray(detail.playTags) ? detail.playTags : []
    const suitableGroups = Array.isArray(detail.suitableGroups) ? detail.suitableGroups : []
    const highlights = Array.isArray(detail.highlights) ? detail.highlights : []
    const itinerary = Array.isArray(detail.itinerary) ? detail.itinerary : []
    const regionText = [detail.province, detail.city, detail.district].filter(Boolean).join(' · ')
    const coverTagInfo = buildActivityCoverTags(detail)
    const priceValue = detail.priceFrom || detail.price || 0

```

```
    return {
      ...detail,
      cover,
      gallery,
      tags,
      travelModeTags,
      playTags,
      suitableGroups,
      highlights,
      itinerary,
      summary: detail.summary || detail.content || '',
      detailText: detail.detail || detail.content || '',
      regionText,
      durationText: coverTagInfo.durationTag,
      coverTags: coverTagInfo.tags,
      priceValue,
      priceText: priceValue ? `¥${priceValue}` : '  ',
      locationText: detail.locationName || regionText || '     ',
      transportText: detail.transport || '   ',
      stayText: detail.stay || '    ',
      pickupText: detail.pickup ? '    ' : '    ',
      pickupPointText: detail.pickupPoint || '',
      addressText: detail.address || detail.locationName || regionText || '   ',
      phoneText: detail.merchantPhone || detail.contactPhone || detail.phone || detail.tel || '   ',
      traveledCount: detail.traveledCount || 0,
    }
  },

  buildCollectionRecord(detail) {
    if (!detail) {
      return null
    }

    return {
      type: 'activity',
      id: detail._id,
      title: detail.title,
      cover: detail.cover,
      city: detail.city || detail.province || detail.locationText,
      regionText: detail.regionText || detail.locationText,
      summary: detail.summary,
      priceText: detail.priceText,
      metaText: detail.durationText,
      badgeText: '  ',
      statsText: `${detail.traveledCount || 0}    `,
    }
  },

  toggleFavorite() {
    const { detail } = this.data
    if (!detail || !detail._id) {
      return
    }

    const favorited = toggleFavorite(this.buildCollectionRecord(detail))
    this.setData({ favorited })
    wx.showToast({
      title: favorited ? '   ' : '     ',
      icon: 'none',
    })
  },
```

```

  previewGallery(e) {
    const current = e.currentTarget.dataset.src
    const urls = (this.data.detail && this.data.detail.gallery) || []
    if (!current || !urls.length) {
      return
    }

    wx.previewImage({
      current,
      urls,
    })
  },

  goBack() {
    if (getCurrentPages().length > 1) {
      wx.navigateBack()
      return
    }

    wx.switchTab({
      url: '/pages/home/home',
    })
  },

  handleBook() {
    const { detail } = this.data
    if (!detail || !detail._id) {
      wx.showToast({
        title: '            ',
        icon: 'none',
      })
      return
    }

    wx.navigateTo({
      url: `/pages/book/book?id=${detail._id}`,
    })
  },
})
```

Path: miniprogram/pages/activityDetail/activityDetail.json

```
1 {
2   "navigationStyle": "custom"
3 }
```

Path: miniprogram/pages/activityDetail/activityDetail.wxml

```
<view class="page" wx:if="{{detail}}">
  <view class="heroWrap">
    <swiper
      class="heroSwiper"
      indicator-dots="{{detail.gallery.length > 1}}"
      indicator-color="rgba(255,255,255,0.42)"
      indicator-active-color="#ffffff"
      autoplay="{{detail.gallery.length > 1}}"
      touchable="{{detail.gallery.length > 1}}"
      interval="3200"
      duration="500"
      circular="{{detail.gallery.length > 1}}"
    >
      <swiper-item wx:for="{{detail.gallery}}" wx:key="*this">
        <image class="hero" src="{{item}}" mode="aspectFill" data-src="{{item}}" bindtap="previewGallery" />
      </swiper-item>
    </swiper>
    <view class="heroMask"></view>

    <view class="customNav" style="padding-top: {{statusBarHeight}}px;">
      <view class="backBtn" bindtap="goBack">
        <view class="backArrow"></view>
      </view>
    </view>
  </view>

  <view class="content">
    <view class="title">{{detail.title}}</view>
    <view class="summary" wx:if="{{detail.summary}}">{{detail.summary}}</view>

    <view class="metaRow">
      <view class="metaPrimary">{{detail.durationText}}</view>
      <view class="metaTag" wx:for="{{detail.coverTags}}" wx:key="*this">{{item}}</view>
    </view>

    <view class="infoGrid">
      <view class="infoCard">
        <view class="infoLabel">{{text.location}}</view>
        <view class="infoValue">{{detail.locationText}}</view>
        <view class="infoSub" wx:if="{{detail.regionText}}">{{detail.regionText}}</view>
      </view>
      <view class="infoCard">
        <view class="infoLabel">{{text.price}}</view>
        <view class="infoValue priceValue">{{detail.priceText}}</view>
        <view class="infoSub">{{detail.traveledCount}}     </view>
      </view>
      <view class="infoCard">
        <view class="infoLabel">{{text.transport}}</view>
        <view class="infoValue">{{detail.transportText}}</view>
      </view>
      <view class="infoCard">
        <view class="infoLabel">{{text.stay}}</view>
        <view class="infoValue">{{detail.stayText}}</view>
      </view>
    </view>

    <view class="section" wx:if="{{detail.travelModeTags.length}}">
      <view class="sectionTitle">{{text.travelMode}}</view>
      <view class="chipWrap">
```

```
        <view class="chip chipWarm" wx:for="{{detail.travelModeTags}}" wx:key="*this">{{item}}</view>
      </view>
    </view>

    <view class="section" wx:if="{{detail.playTags.length}}">
      <view class="sectionTitle">{{text.playMode}}</view>
      <view class="chipWrap">
        <view class="chip" wx:for="{{detail.playTags}}" wx:key="*this">{{item}}</view>
      </view>
    </view>

    <view class="section" wx:if="{{detail.highlights.length}}">
      <view class="sectionTitle">{{text.highlights}}</view>
      <view class="listCard">
        <view class="bulletItem" wx:for="{{detail.highlights}}" wx:key="*this">
          <view class="bulletDot"></view>
          <view class="bulletText">{{item}}</view>
        </view>
      </view>
    </view>

    <view class="section" wx:if="{{detail.itinerary.length}}">
      <view class="sectionTitle">{{text.itinerary}}</view>
      <view class="listCard">
        <view class="timelineItem" wx:for="{{detail.itinerary}}" wx:key="*this">
          <view class="timelineIndex">{{index + 1}}</view>
          <view class="timelineText">{{item}}</view>
        </view>
      </view>
    </view>

    <view class="section">
      <view class="sectionTitle">{{text.info}}</view>
      <view class="detailCard">
        <view class="detailRow">
          <text class="detailLabel">{{text.pickup}}</text>
          <text class="detailValue">{{detail.pickupText}}</text>
        </view>
        <view class="detailRow" wx:if="{{detail.pickupPointText}}">
          <text class="detailLabel">{{text.pickupPoint}}</text>
          <text class="detailValue">{{detail.pickupPointText}}</text>
        </view>
        <view class="detailRow">
          <text class="detailLabel">{{text.address}}</text>
          <text class="detailValue">{{detail.addressText}}</text>
        </view>
        <view class="detailRow">
          <text class="detailLabel">{{text.phone}}</text>
          <text class="detailValue">{{detail.phoneText}}</text>
        </view>
        <view class="detailRow">
          <text class="detailLabel">{{text.duration}}</text>
          <text class="detailValue">{{detail.durationText}}</text>
        </view>
        <view class="detailRow">
          <text class="detailLabel">{{text.region}}</text>
          <text class="detailValue">{{detail.regionText || detail.locationText}}</text>
        </view>
      </view>
    </view>

```

```
    <view class="section" wx:if="{{detail.detailText}}">
      <view class="sectionTitle">{{text.description}}</view>
      <view class="descriptionCard">{{detail.detailText}}</view>
    </view>

    <view class="section" wx:if="{{detail.gallery.length > 1}}">
      <view class="sectionTitle">{{text.gallery}}</view>
      <scroll-view class="galleryScroll" scroll-x enhanced show-scrollbar="{{false}}">
        <image
          class="galleryImage"
          wx:for="{{detail.gallery}}"
          wx:key="*this"
          src="{{item}}"
          mode="aspectFill"
          data-src="{{item}}"
          bindtap="previewGallery"
        />
      </scroll-view>
    </view>
  </view>

  <view class="bottomBar">
    <view class="bottomMain">
      <view class="priceBox">
        <text class="priceLabel">{{text.priceLabel}}</text>
        <text class="priceNum">{{detail.priceText}}</text>
      </view>
      <view class="favoriteBtn" bindtap="toggleFavorite">
        <image class="favoriteIcon" src="{{favorited ? favoriteIconFilled : favoriteIcon}}" mode="aspectFit" />
      </view>
    </view>
    <button class="bookBtn" bindtap="handleBook">{{text.bookNow}}</button>
  </view>
</view>

<view wx:else class="emptyState">{{text.empty}}</view>
```

Path: miniprogram/pages/activityDetail/activityDetail.wxss

```
.page {
  min-height: 100vh;
  background: linear-gradient(180deg, #f6faf4 0%, #edf5ea 100%);
  padding-bottom: 220rpx;
  box-sizing: border-box;
}

.heroWrap {
  position: relative;
  width: 100%;
  height: 520rpx;
  overflow: hidden;
}

.heroSwiper {
  width: 100%;
  height: 100%;
}

.hero {
  width: 100%;
  height: 100%;
  display: block;
}

.heroMask {
  position: absolute;
  inset: 0;
  background: linear-gradient(180deg, rgba(20, 20, 20, 0.28) 0%, rgba(20, 20, 20, 0.08) 36%, rgba(20, 20, 20, 0) 68%);
  pointer-events: none;
}

.customNav {
  position: fixed;
  left: 24rpx;
  right: auto;
  top: 0;
  padding-left: 0;
  padding-right: 0;
  z-index: 20;
  pointer-events: none;
}

.backBtn {
  width: 72rpx;
  height: 72rpx;
  border-radius: 50%;
  background: rgba(14, 38, 47, 0.72);
  display: flex;
  align-items: center;
  justify-content: center;
  pointer-events: auto;
}

.backArrow {
  width: 22rpx;
  height: 22rpx;
  border-left: 4rpx solid #ffffff;
  border-bottom: 4rpx solid #ffffff;
```

```
  transform: rotate(45deg);
  margin-left: 8rpx;
}

.content {
  position: relative;
  z-index: 1;
  margin-top: 34rpx;
  padding: 8rpx 24rpx 32rpx;
}

.title {
  font-size: 48rpx;
  line-height: 1.3;
  font-weight: 700;
  color: #1f2a1f;
}

.summary {
  margin-top: 14rpx;
  font-size: 28rpx;
  line-height: 1.7;
  color: #667566;
}

.metaRow {
  margin-top: 20rpx;
  display: flex;
  flex-wrap: wrap;
  gap: 12rpx;
}

.metaPrimary,
.metaTag,
.chip {
  padding: 8rpx 18rpx;
  border-radius: 999rpx;
  font-size: 24rpx;
}

.metaPrimary {
  background: #edf5ea;
  color: #57984a;
}

.metaTag {
  background: rgba(255, 255, 255, 0.86);
  color: #667566;
}

.infoGrid {
  margin-top: 26rpx;
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 16rpx;
}

.infoCard,
.listCard,
.detailCard,
.descriptionCard {
  background: rgba(255, 255, 255, 0.92);
```

```
  border-radius: 28rpx;
  box-shadow: 0 12rpx 30rpx rgba(87, 73, 54, 0.06);
}

.infoCard {
  padding: 22rpx 22rpx 24rpx;
}

.infoLabel {
  font-size: 24rpx;
  color: #8ea08b;
}

.infoValue {
  margin-top: 12rpx;
  font-size: 28rpx;
  line-height: 1.5;
  color: #1f2a1f;
  font-weight: 600;
}

.priceValue {
  color: #57984a;
}

.infoSub {
  margin-top: 10rpx;
  font-size: 22rpx;
  line-height: 1.5;
  color: #8ea08b;
}

.section {
  margin-top: 28rpx;
}

.sectionTitle {
  margin-bottom: 16rpx;
  font-size: 32rpx;
  font-weight: 700;
  color: #1f2a1f;
}

.chipWrap {
  display: flex;
  flex-wrap: wrap;
  gap: 12rpx;
}

.chip {
  background: rgba(255, 255, 255, 0.9);
  color: #667566;
}

.chipWarm {
  background: #edf5ea;
  color: #57984a;
}

.chipSoft {
  background: #edf5ea;
  color: #426046;
```

```
}

.listCard {
  padding: 16rpx 20rpx;
}

.bulletItem,
.timelineItem {
  display: flex;
  align-items: flex-start;
  gap: 14rpx;
  padding: 12rpx 0;
}

.bulletDot {
  width: 12rpx;
  height: 12rpx;
  margin-top: 12rpx;
  border-radius: 50%;
  background: #57984a;
  flex-shrink: 0;
}

.bulletText,
.timelineText {
  flex: 1;
  font-size: 28rpx;
  line-height: 1.7;
  color: #233123;
}

.timelineIndex {
  width: 42rpx;
  height: 42rpx;
  border-radius: 50%;
  background: #e6f2e3;
  color: #57984a;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 24rpx;
  font-weight: 700;
  flex-shrink: 0;
}

.detailCard {
  padding: 16rpx 22rpx;
}

.detailRow {
  display: flex;
  justify-content: space-between;
  gap: 24rpx;
  padding: 14rpx 0;
  border-bottom: 1rpx solid rgba(0, 0, 0, 0.05);
}

.detailRow:last-child {
  border-bottom: none;
}

.detailLabel {
```

```
  color: #8ea08b;
  font-size: 26rpx;
}

.detailValue {
  flex: 1;
  text-align: right;
  color: #233123;
  font-size: 26rpx;
}

.descriptionCard {
  padding: 24rpx;
  color: #233123;
  font-size: 28rpx;
  line-height: 1.85;
  white-space: pre-line;
}

.galleryScroll {
  white-space: nowrap;
}

.galleryImage {
  display: inline-block;
  width: 250rpx;
  height: 180rpx;
  margin-right: 16rpx;
  border-radius: 22rpx;
}

.bottomBar {
  position: fixed;
  left: 0;
  right: 0;
  bottom: 0;
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 24rpx;
  padding: 8rpx 24rpx calc(env(safe-area-inset-bottom) + 8rpx);
  background: rgba(255, 255, 255, 0.98);
  box-shadow: 0 -8rpx 28rpx rgba(55, 42, 26, 0.08);
  z-index: 999;
  pointer-events: auto;
}

.bottomMain {
  flex: 1;
  display: flex;
  align-items: center;
  gap: 20rpx;
}

.priceBox {
  display: flex;
  flex-direction: column;
  gap: 6rpx;
  flex: 1;
}

.priceLabel {
```

```
  font-size: 20rpx;
  color: #8ea08b;
}

.priceNum {
  font-size: 40rpx;
  font-weight: 700;
  color: #57984a;
}

.favoriteBtn {
  width: 80rpx;
  height: 80rpx;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
}

.favoriteIcon {
  width: 64rpx;
  height: 64rpx;
}

.bookBtn {
  min-width: 240rpx;
  height: 64rpx;
  line-height: 64rpx;
  border-radius: 999rpx;
  background: #57984a;
  color: #ffffff;
  font-size: 26rpx;
  font-weight: 600;
  padding: 0 28rpx;
}

.emptyState {
  padding: 80rpx 32rpx;
  text-align: center;
  color: #99a598;
  font-size: 28rpx;
}
```

Path: miniprogram/pages/book/book.js

```
Page({

  data: {

    activityId: "",

    today: "",

    travelDate: "",

    adultCount: 0,

    childCount: 0

  },


  onLoad(options) {

    const today = this.formatDate(new Date())

    this.setData({

      today,

      //        id   activityId
      activityId: options.id || options.activityId || ""

    })

  },


  formatDate(date) {

    const y = date.getFullYear()
    const m = (date.getMonth() + 1).toString().padStart(2, '0')
    const d = date.getDate().toString().padStart(2, '0')

    return `${y}-${m}-${d}`

  },


  onDateChange(e) {

    this.setData({

      travelDate: e.detail.value

    })

  },


  addAdult() {

    this.setData({

      adultCount: this.data.adultCount + 1
```

```

    })

  },


  minusAdult() {

    if (this.data.adultCount <= 0) return

    this.setData({

      adultCount: this.data.adultCount - 1

    })

  },


  addChild() {

    this.setData({

      childCount: this.data.childCount + 1

    })

  },


  minusChild() {

    if (this.data.childCount <= 0) return

    this.setData({

      childCount: this.data.childCount - 1

    })

  },


  goNext() {

    const { activityId, travelDate, adultCount, childCount } = this.data

    if (!activityId) {
      return wx.showToast({
        title: '  ID        ',
        icon: 'none'
      })
    }

    if (!travelDate) {

      return wx.showToast({

        title: '       ',

        icon: 'none'

```

```
      })

    }

    if (adultCount + childCount <= 0) {

      return wx.showToast({

        title: '       ',

        icon: 'none'

      })

    }


    const nonce = Date.now()

    wx.navigateTo({

      url: `/pages/order/order?activityId=${activityId}&date=${travelDate}&adult=${adult
Count}&child=${childCount}&nonce=${nonce}`

    })

  }

})
```

Path: miniprogram/pages/book/book.json

```
{
  "navigationBarTitleText": "    /    "
}
```

Path: miniprogram/pages/book/book.wxml

```
<view class="container">

<!--      -->
<view class="card">
  <view class="title">       </view>

  <picker mode="date"
          value="{{travelDate}}"
          start="{{today}}"
          bindchange="onDateChange">

    <view class="picker">
      <text wx:if="{{travelDate}}">{{travelDate}}</text>
      <text wx:else class="placeholder">      </text>
    </view>

  </picker>
</view>


<!--      -->
<view class="card">

  <view class="title">       </view>

  <!--    -->
  <view class="row">

    <text>  </text>

    <view class="counter">

      <button class="btn"
              bindtap="minusAdult"
              disabled="{{adultCount<=0}}">-</button>

      <text class="num">{{adultCount}}</text>

      <button class="btn"
              bindtap="addAdult">+</button>

    </view>

  </view>


  <!--    -->
  <view class="row">

    <text>  </text>

    <view class="counter">

      <button class="btn"
              bindtap="minusChild"
              disabled="{{childCount<=0}}">-</button>

      <text class="num">{{childCount}}</text>

      <button class="btn"
```

```
              bindtap="addChild">+</button>

    </view>

  </view>

  <view class="child-tip">
            12
  </view>

</view>


<!--     -->
<view class="bottom">

  <button class="next"
          bindtap="goNext">

  </button>

</view>

</view>
```

Path: miniprogram/pages/book/book.wxss

```
.container{
  padding:30rpx;
  background:#f6faf4;
  min-height:100vh;
}

.card{
  background:#ffffff;
  border-radius:16rpx;
  padding:30rpx;
  margin-bottom:30rpx;
}

.title{
  font-size:30rpx;
  font-weight:bold;
  margin-bottom:20rpx;
}

.picker{
  padding:20rpx;
  border:1px solid #dbe8d7;
  border-radius:10rpx;
}

.placeholder{
  color:#99a598;
}

.row{
  display:flex;
  justify-content:space-between;
  align-items:center;
  margin:20rpx 0;
}

.counter{
  display:flex;
  align-items:center;
}

.btn{
  width:60rpx;
  height:60rpx;
  line-height:60rpx;
}

.num{
  width:60rpx;
  text-align:center;
}

.child-tip{
  color:red;
  font-size:24rpx;
}

.bottom{
  position:fixed;
  bottom:0;
```

```
  left:0;
  right:0;
  padding:20rpx;
  background:#ffffff;
}

.next{
  background:#57984a;
  color:#ffffff;
}
```

Path: miniprogram/pages/hotel/hotel.js

```
---------------------------------------------------------------------
const db = wx.cloud.database()
const { resolveMediaSource } = require('../../utils/mediaAssets')

const SAFE_HOTEL_COVER = '/images/nav-hotel.png'

function normalizeText(value = '') {
  return String(value || '').trim()
}

function trimRegionSuffix(name = '') {
  return normalizeText(name)
    .replace(/(     |   |  | )$/u, '')
    .replace(/( | | | )$/u, '')
}

function getDisplayCity(source) {
  if (!source) {
    return '  '
  }

  const city = trimRegionSuffix(source.city)
  const district = trimRegionSuffix(source.district)
  const province = trimRegionSuffix(source.province)
  const displayName = trimRegionSuffix(source.displayName || source.label || '')
  return city || district || province || displayName || '  '
}

function buildDisplayPrice(price) {
  const value = Number(price || 0)
  return value > 0 ? `¥${value} ` : '    '
}

Page({
  data: {
    locationCity: '  ',
    hotelList: [],
  },

  onLoad() {
    this.syncLocationCity()
    this.loadHotels()
  },

  onShow() {
    this.syncLocationCity()
  },

  syncLocationCity() {
    const app = getApp()
    const userInfo = app.getUserInfo ? app.getUserInfo() : null
    const selectedRegion = wx.getStorageSync('selectedRegion') || null

    this.setData({
      locationCity: getDisplayCity(selectedRegion || userInfo),
    })
  },

  async loadHotels() {
    try {
      const res = await db.collection('hotels')
```

```
        .where({ status: true })
        .orderBy('sort', 'asc')
        .get()

      const hotelList = await Promise.all((res.data || []).map(async (item) => ({
        ...item,
        sourceType: item.sourceType || 'demo',
        cover: await resolveMediaSource(item.cover, SAFE_HOTEL_COVER),
        displayPrice: buildDisplayPrice(item.price),
      })))

      this.setData({
        hotelList,
      })
    } catch (err) {
      console.error('load hotels failed', err)
      wx.showToast({
        title: '      ',
        icon: 'none',
      })
    }
  },

  openHotelDetail(e) {
    const id = e.currentTarget.dataset.id
    if (!id) return

    wx.navigateTo({
      url: `/pages/hotelDetail/hotelDetail?id=${id}`,
    })
  },
})
```

Path: miniprogram/pages/hotel/hotel.json

```
1 {
2   "navigationBarTitleText": "    "
3 }
```

Path: miniprogram/pages/hotel/hotel.wxml

```
<view class="page">
  <view class="searchWrap">
    <view class="searchBox">
      <view class="city">{{locationCity}}</view>
      <view class="searchDivider"></view>
      <view class="searchPlaceholder">  /  /   </view>
    </view>
  </view>

  <view class="filterBar">
    <view class="filterItem">    </view>
    <view class="filterItem">    </view>
    <view class="filterItem">  /  </view>
    <view class="filterItem">  </view>
  </view>

  <view class="hotelList">
    <view
      wx:for="{{hotelList}}"
      wx:key="_id"
      class="hotelCard"
      data-id="{{item._id}}"
      bindtap="openHotelDetail"
    >
      <image class="hotelImg" src="{{item.cover}}" mode="aspectFill"></image>

      <view class="hotelBody">
        <view class="hotelName">{{item.name}}</view>

        <view class="scoreRow" wx:if="{{Number(item.score) > 0}}">
          <text class="score">{{item.score}} </text>
          <text class="comment">{{item.commentCount}}  </text>
        </view>

        <view class="desc">{{item.distanceText}} · {{item.locationText}}</view>

        <view class="tagRow">
          <text wx:for="{{item.tags}}" wx:key="*this" class="tag">{{item}}</text>
        </view>

        <view class="priceRow">
          <text class="price">{{item.displayPrice}}</text>
        </view>
      </view>
    </view>
  </view>

  <view wx:if="{{hotelList.length===0}}" class="empty">

  </view>
</view>
```

Path: miniprogram/pages/hotel/hotel.wxss

```
.page{
  min-height:100vh;
  background:#f6faf4;
  padding-bottom:30rpx;
}

.searchWrap{
  padding:24rpx;
  background:#ffffff;
}

.searchBox{
  height:84rpx;
  background:#ffffff;
  border:2rpx solid #cfe9e8;
  border-radius:999rpx;
  display:flex;
  align-items:center;
  padding:0 24rpx;
}

.city{
  color:#57984a;
  font-size:28rpx;
  font-weight:600;
  flex-shrink:0;
}

.searchDivider{
  width:1rpx;
  height:34rpx;
  background:#dbe8d7;
  margin:0 20rpx;
}

.searchPlaceholder{
  color:#8ea08b;
  font-size:28rpx;
}

.filterBar{
  background:#ffffff;
  display:flex;
  justify-content:space-around;
  padding:24rpx 0;
  margin-bottom:16rpx;
}

.filterItem{
  font-size:30rpx;
  font-weight:600;
  color:#1f2a1f;
}

.hotelList{
  padding:0 24rpx;
}

.hotelCard{
  background:#ffffff;
```

```
  border-radius:24rpx;
  overflow:hidden;
  margin-bottom:20rpx;
  box-shadow:0 10rpx 28rpx rgba(161, 132, 103, 0.08);
}

.hotelImg{
  width:100%;
  height:260rpx;
  display:block;
}

.hotelBody{
  padding:22rpx;
}

.hotelName{
  font-size:34rpx;
  font-weight:700;
  color:#1f2a1f;
  margin-bottom:12rpx;
}

.scoreRow{
  display:flex;
  align-items:center;
  margin-bottom:10rpx;
}

.score{
  color:#57984a;
  font-size:30rpx;
  font-weight:700;
  margin-right:16rpx;
}

.comment{
  color:#667566;
  font-size:26rpx;
}

.desc{
  color:#667566;
  font-size:26rpx;
  line-height:1.7;
  margin-bottom:14rpx;
}

.tagRow{
  display:flex;
  flex-wrap:wrap;
  gap:12rpx;
  margin-bottom:18rpx;
}

.tag{
  padding:8rpx 16rpx;
  background:#edf5ea;
  border-radius:999rpx;
  color:#667566;
  font-size:22rpx;
}
```

```

.priceRow{
  display:flex;
  justify-content:flex-end;
}

.price{
  color:#57984a;
  font-size:38rpx;
  font-weight:700;
}

.empty{
  text-align:center;
  color:#99a598;
  margin-top:100rpx;
  font-size:28rpx;
}
```

Path: miniprogram/pages/hotelDetail/hotelDetail.js

```
---------------------------------------------------------------------------
   1 const DEFAULT_HOTEL_COVER = '/images/nav-hotel.png'
   2 const {
   3   resolveMediaSource,
   4   resolveMediaList,
   5 } = require('../../utils/mediaAssets')
   6 const {
   7   isFavorited,
   8   toggleFavorite,
   9   recordFootprint,
  10 } = require('../../utils/collectionStore')
  11
  12 function normalizeArray(value) {
  13   return Array.isArray(value) ? value.filter(Boolean) : []
  14 }
  15
  16 Page({
  17   data: {
  18     id: '',
  19     detail: null,
  20     favorited: false,
  21     favoriteIcon: '/images/icons/favorite-heart-outline.png',
  22     favoriteIconFilled: '/images/icons/favorite-heart-filled.png',
  23     text: {
  24       empty: '        ',
  25       location: '     ',
  26       price: '    ',
  27       score: '    ',
  28       facilities: '    ',
  29       roomTypes: '    ',
  30       highlights: '    ',
  31       suitableGroups: '    ',
  32       description: '    ',
  33       gallery: '    ',
  34       bookNow: '    ',
  35     },
  36   },
  37
  38   onLoad(options) {
  39     const id = options.id || ''
  40     this.setData({ id })
  41     if (id) {
  42       this.loadDetail(id)
  43     }
  44   },
  45
  46   onShow() {
  47     const { detail } = this.data
  48     if (detail && detail._id) {
  49       this.setData({
  50         favorited: isFavorited('hotel', detail._id),
  51       })
  52     }
  53   },
  54
  55   async loadDetail(id) {
  56     wx.showLoading({ title: '    ' })
  57     try {
  58       const db = wx.cloud.database()
  59       const res = await db.collection('hotels').doc(id).get()
  60       const detail = await this.normalizeDetail(res.data || null)
```

```
      const favorited = isFavorited('hotel', detail && detail._id)
      if (detail) {
        recordFootprint(this.buildCollectionRecord(detail))
      }

      this.setData({
        detail,
        favorited,
      })
    } catch (error) {
      console.error('[hotelDetail] load failed', error)
      wx.showToast({
        title: '    ',
        icon: 'none',
      })
    } finally {
      wx.hideLoading()
    }
  },

  async normalizeDetail(detail) {
    if (!detail) {
      return null
    }

    const cover = await resolveMediaSource(detail.cover, DEFAULT_HOTEL_COVER)
    const gallery = await resolveMediaList(normalizeArray(detail.gallery), cover)
    const tags = normalizeArray(detail.tags).slice(0, 6)
    const facilities = normalizeArray(detail.facilities)
    const roomTypes = normalizeArray(detail.roomTypes)
    const highlights = normalizeArray(detail.highlights)
    const suitableGroups = normalizeArray(detail.suitableGroups)
    const regionText = [detail.province, detail.city, detail.district].filter(Boolean).j
oin(' · ')
    const commentCount = Number(detail.commentCount || 0)
    const price = Number(detail.price || 0)
    const score = Number(detail.score || 0)

    return {
      ...detail,
      cover,
      gallery,
      tags,
      facilities,
      roomTypes,
      highlights,
      suitableGroups,
      regionText,
      locationText: detail.locationText || detail.address || regionText || '     ',
      addressText: detail.address || '',
      priceText: price ? `¥${price} ` : '    ',
      scoreText: score ? `${score}  · ${commentCount}   ` : '     ',
      titleScoreText: score ? `${score}  · ${commentCount}   ` : '',
      descriptionText: detail.description || detail.desc || detail.summary || '',
    }
  },

  buildCollectionRecord(detail) {
    if (!detail) {
      return null
    }

```

```
    return {
      type: 'hotel',
      id: detail._id,
      title: detail.name || detail.title,
      cover: detail.cover,
      city: detail.city || detail.province || detail.locationText,
      regionText: detail.regionText || detail.locationText,
      summary: detail.summary || detail.descriptionText,
      priceText: detail.priceText,
      metaText: detail.scoreText,
      badgeText: '  /  ',
      scoreText: detail.scoreText,
    }
  },

  toggleFavorite() {
    const { detail } = this.data
    if (!detail || !detail._id) {
      return
    }

    const favorited = toggleFavorite(this.buildCollectionRecord(detail))
    this.setData({ favorited })
    wx.showToast({
      title: favorited ? '   ' : '     ',
      icon: 'none',
    })
  },

  previewGallery(e) {
    const current = e.currentTarget.dataset.src
    const urls = (this.data.detail && this.data.detail.gallery) || []
    if (!current || !urls.length) {
      return
    }

    wx.previewImage({
      current,
      urls,
    })
  },

  handleContact() {
    wx.showToast({
      title: '          ',
      icon: 'none',
    })
  },
})
```

Path: miniprogram/pages/hotelDetail/hotelDetail.json

```
{
  "navigationBarTitleText": "    "
}
```

Path: miniprogram/pages/hotelDetail/hotelDetail.wxml

```
<view class="page" wx:if="{{detail}}">
  <swiper
    class="heroSwiper"
    indicator-dots="{{detail.gallery.length > 1}}"
    autoplay="{{detail.gallery.length > 1}}"
    circular="{{detail.gallery.length > 1}}"
    interval="4000"
    duration="500"
  >
    <swiper-item wx:for="{{detail.gallery}}" wx:key="*this">
      <image class="hero" src="{{item}}" mode="aspectFill" data-src="{{item}}" bindtap="previewGallery" />
    </swiper-item>
  </swiper>

  <view class="content">
    <view class="titleRow">
      <view class="title">{{detail.name || detail.title}}</view>
      <view class="titleMeta" wx:if="{{detail.titleScoreText}}">{{detail.titleScoreText}}</view>
    </view>
    <view class="summary" wx:if="{{detail.summary}}">{{detail.summary}}</view>

    <view class="tagWrap" wx:if="{{detail.tags.length}}">
      <view class="tag" wx:for="{{detail.tags}}" wx:key="*this">{{item}}</view>
    </view>

    <view class="infoGrid">
      <view class="infoCard">
        <view class="infoLabel">{{text.location}}</view>
        <view class="infoValue">{{detail.locationText}}</view>
        <view class="infoSub" wx:if="{{detail.addressText}}">{{detail.addressText}}</view>
        <view class="infoSub" wx:if="{{detail.regionText}}">{{detail.regionText}}</view>
      </view>
    </view>

    <view class="section" wx:if="{{detail.highlights.length}}">
      <view class="sectionTitle">{{text.highlights}}</view>
      <view class="card">
        <view class="bulletItem" wx:for="{{detail.highlights}}" wx:key="*this">
          <view class="dot"></view>
          <view class="bulletText">{{item}}</view>
        </view>
      </view>
    </view>

    <view class="section" wx:if="{{detail.facilities.length}}">
      <view class="sectionTitle">{{text.facilities}}</view>
      <view class="chipWrap">
        <view class="chip" wx:for="{{detail.facilities}}" wx:key="*this">{{item}}</view>
      </view>
    </view>

    <view class="section" wx:if="{{detail.roomTypes.length}}">
      <view class="sectionTitle">{{text.roomTypes}}</view>
      <view class="card">
        <view class="bulletItem" wx:for="{{detail.roomTypes}}" wx:key="*this">
          <view class="dot"></view>
          <view class="bulletText">{{item}}</view>
```

```
        </view>
      </view>
    </view>

    <view class="section" wx:if="{{detail.suitableGroups.length}}">
      <view class="sectionTitle">{{text.suitableGroups}}</view>
      <view class="chipWrap">
        <view class="chip soft" wx:for="{{detail.suitableGroups}}" wx:key="*this">{{item}}</view>
      </view>
    </view>

    <view class="section" wx:if="{{detail.descriptionText}}">
      <view class="sectionTitle">{{text.description}}</view>
      <view class="card plainText">{{detail.descriptionText}}</view>
    </view>

    <view class="section" wx:if="{{detail.gallery.length}}">
      <view class="sectionTitle">{{text.gallery}}</view>
      <scroll-view class="galleryScroll" scroll-x enhanced show-scrollbar="{{false}}">
        <image
          class="galleryImage"
          wx:for="{{detail.gallery}}"
          wx:key="*this"
          src="{{item}}"
          mode="aspectFill"
          data-src="{{item}}"
          bindtap="previewGallery"
        />
      </scroll-view>
    </view>
  </view>

  <view class="bottomBar">
    <view class="bottomMain">
      <view class="priceBox">
        <view class="priceLabel">{{text.price}}</view>
        <view class="priceNum">{{detail.priceText}}</view>
      </view>
      <view class="favoriteBtn" bindtap="toggleFavorite">
        <image class="favoriteIcon" src="{{favorited ? favoriteIconFilled : favoriteIcon}}" mode="aspectFit" />
      </view>
    </view>
    <button class="actionBtn" bindtap="handleContact">{{text.bookNow}}</button>
  </view>
</view>

<view wx:else class="emptyState">{{text.empty}}</view>
```

Path: miniprogram/pages/hotelDetail/hotelDetail.wxss

```
.page {
  min-height: 100vh;
  background: #f6faf4;
  padding-bottom: 180rpx;
}

.heroSwiper,
.hero {
  width: 100%;
  height: 460rpx;
  display: block;
}

.content {
  margin-top: -36rpx;
  background: #f6faf4;
  border-radius: 36rpx 36rpx 0 0;
  padding: 32rpx 28rpx 60rpx;
  position: relative;
  z-index: 1;
}

.title {
  font-size: 52rpx;
  line-height: 1.22;
  font-weight: 700;
  color: #1f2a1f;
}

.titleRow {
  display: flex;
  align-items: baseline;
  justify-content: space-between;
  gap: 20rpx;
}

.titleRow .title {
  flex: 1;
  min-width: 0;
}

.titleMeta {
  flex-shrink: 0;
  font-size: 24rpx;
  line-height: 1.4;
  color: #8ea08b;
}

.summary {
  margin-top: 18rpx;
  font-size: 30rpx;
  line-height: 1.75;
  color: #667566;
}

.tagWrap,
.chipWrap {
  display: flex;
  flex-wrap: wrap;
  gap: 16rpx;
```

```
  margin-top: 24rpx;
}

.tag,
.chip {
  padding: 14rpx 24rpx;
  border-radius: 999rpx;
  font-size: 24rpx;
  color: #57984a;
  background: #eaf5e7;
}

.chip.soft {
  color: #667566;
  background: #eef4eb;
}

.infoGrid {
  margin-top: 28rpx;
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 18rpx;
}

.infoCard:first-child {
  grid-column: span 2;
}

.infoCard,
.card {
  background: #ffffff;
  border-radius: 28rpx;
  padding: 28rpx;
  box-shadow: 0 12rpx 36rpx rgba(161, 132, 103, 0.08);
}

.infoLabel,
.sectionTitle {
  font-size: 28rpx;
  color: #8ea08b;
}

.infoValue {
  margin-top: 16rpx;
  font-size: 40rpx;
  line-height: 1.32;
  color: #1f2a1f;
  font-weight: 700;
}

.infoSub {
  margin-top: 12rpx;
  font-size: 26rpx;
  line-height: 1.6;
  color: #8ea08b;
}

.priceText,
.bottomPrice {
  color: #57984a;
}

```

```
.section {
  margin-top: 34rpx;
}

.card {
  margin-top: 18rpx;
}

.bulletItem {
  display: flex;
  align-items: flex-start;
  gap: 16rpx;
}

.bulletItem + .bulletItem {
  margin-top: 18rpx;
}

.dot {
  width: 14rpx;
  height: 14rpx;
  margin-top: 12rpx;
  border-radius: 50%;
  background: #57984a;
  flex-shrink: 0;
}

.bulletText,
.plainText {
  font-size: 30rpx;
  line-height: 1.8;
  color: #2f2922;
}

.galleryScroll {
  margin-top: 18rpx;
  white-space: nowrap;
}

.galleryImage {
  width: 360rpx;
  height: 260rpx;
  border-radius: 24rpx;
  margin-right: 18rpx;
  display: inline-block;
}

.bottomBar {
  position: fixed;
  left: 0;
  right: 0;
  bottom: 0;
  z-index: 99;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 24rpx;
  padding: 18rpx 24rpx calc(env(safe-area-inset-bottom) + 18rpx);
  background: rgba(255, 253, 250, 0.96);
  box-shadow: 0 -10rpx 30rpx rgba(115, 91, 67, 0.08);
}

```

```
.bottomMain {
  flex: 1;
  display: flex;
  align-items: center;
  gap: 20rpx;
}

.priceBox {
  display: flex;
  flex-direction: column;
  gap: 8rpx;
  flex: 1;
}

.priceLabel {
  font-size: 22rpx;
  color: #8ea08b;
}

.priceNum {
  font-size: 42rpx;
  line-height: 1;
  font-weight: 700;
  color: #57984a;
}

.favoriteBtn {
  width: 80rpx;
  height: 80rpx;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
}

.favoriteIcon {
  width: 64rpx;
  height: 64rpx;
}

.actionBtn {
  margin: 0;
  min-width: 260rpx;
  height: 76rpx;
  line-height: 76rpx;
  border-radius: 999rpx;
  background: #57984a;
  color: #ffffff;
  font-size: 30rpx;
  font-weight: 600;
  padding: 0;
  border: none;
}

.actionBtn::after {
  border: none;
}

.emptyState {
  min-height: 100vh;
  display: flex;
  align-items: center;
```

```
  justify-content: center;
  padding: 60rpx;
  font-size: 30rpx;
  color: #8ea08b;
}
```

Path: miniprogram/pages/mall/mall.js

```
---------------------------------------------------------------------------
const db = wx.cloud.database()
const { resolveMediaSource } = require('../../utils/mediaAssets')

const SAFE_PRODUCT_COVER = '/images/default-goods-image.png'

async function normalizeProduct(item = {}) {
  const cover = await resolveMediaSource(item.cover, SAFE_PRODUCT_COVER)
  return {
    id: item._id || item.id || '',
    title: item.title || item.name || '     ',
    cover,
    priceText: item.price ? ` ${item.price}` : '    ',
    soldText: item.sold ? `${item.sold}   ` : '    ',
    sourceType: item.sourceType || 'demo',
  }
}

Page({
  data: {
    goodsList: [
      {
        id: '1',
        title: '      ',
        cover: SAFE_PRODUCT_COVER,
        priceText: ' 39.9',
        soldText: '126   ',
        sourceType: 'demo',
      },
      {
        id: '2',
        title: '     5  ',
        cover: SAFE_PRODUCT_COVER,
        priceText: ' 56',
        soldText: '89   ',
        sourceType: 'demo',
      },
      {
        id: '3',
        title: '      ',
        cover: SAFE_PRODUCT_COVER,
        priceText: ' 29.9',
        soldText: '203   ',
        sourceType: 'demo',
      },
      {
        id: '4',
        title: '      ',
        cover: SAFE_PRODUCT_COVER,
        priceText: ' 68',
        soldText: '75   ',
        sourceType: 'demo',
      },
    ],
  },

  onLoad() {
    this.loadRealProducts()
  },

  async loadRealProducts() {
```

```
    try {
      const res = await db.collection('products').limit(50).get()
      const productList = await Promise.all((res.data || []).map((item) => normalizeProduct(item)))
      this.setData({
        goodsList: [
          ...productList,
          ...this.data.goodsList,
        ],
      })
    } catch (error) {
      console.error('[mall] load real products failed', error)
      wx.showToast({
        title: '      ',
        icon: 'none',
      })
    }
  },

  goProductDetail(e) {
    const id = e.currentTarget.dataset.id
    if (!id) {
      wx.showToast({
        title: '        ',
        icon: 'none',
      })
      return
    }

    wx.navigateTo({
      url: `/pages/productDetail/productDetail?id=${id}`,
    })
  },
})
```

Path: miniprogram/pages/mall/mall.json

```
1 {
2   "navigationBarTitleText": "    "
3 }
```

Path: miniprogram/pages/mall/mall.wxml

```
<view class="page">
  <view class="searchWrap">
    <view class="searchBox">    </view>
  </view>

  <view class="goodsList">
    <view
      wx:for="{{goodsList}}"
      wx:key="id"
      class="goodsCard"
      data-id="{{item.id}}"
      bindtap="goProductDetail"
    >
      <image class="goodsImg" src="{{item.cover}}" mode="aspectFill"></image>
      <text class="goodsName">{{item.title}}</text>
      <view class="goodsBottom">
        <text class="goodsPrice">{{item.priceText}}</text>
        <text class="goodsSold">{{item.soldText}}</text>
      </view>
    </view>
  </view>
</view>
```

Path: miniprogram/pages/mall/mall.wxss

```
.page{
  min-height:100vh;
  background:#f6faf4;
  padding-bottom:30rpx;
}

.searchWrap{
  padding:24rpx;
  background:#ffffff;
  margin-bottom: 20rpx;
}

.searchBox{
  height:80rpx;
  border-radius:999rpx;
  background:#edf5ea;
  color:#57984a;
  display:flex;
  align-items:center;
  justify-content:center;
  padding:0 24rpx;
  font-size:28rpx;
  font-weight: 600;
}

.goodsList{
  padding:0 24rpx;
  display:flex;
  flex-wrap:wrap;
  justify-content:space-between;
}

.goodsCard{
  width:48%;
  background:#ffffff;
  border-radius:22rpx;
  overflow:hidden;
  margin-bottom:20rpx;
}

.goodsImg{
  width:100%;
  height:260rpx;
  display:block;
}

.goodsName{
  font-size:28rpx;
  color:#1f2a1f;
  line-height:1.6;
  padding:16rpx 16rpx 0;
  min-height:90rpx;
}

.goodsBottom{
  display:flex;
  justify-content:space-between;
  align-items:center;
  padding:14rpx 16rpx 20rpx;
}
```

```

.goodsPrice{
  color:#57984a;
  font-size:32rpx;
  font-weight:700;
}

.goodsSold{
  color:#99a598;
  font-size:22rpx;
}
```

Path: miniprogram/pages/productDetail/productDetail.js

```
---------------------------------------------------------------------------
const { resolveMediaSource, resolveMediaList } = require('../../utils/mediaAssets')
const {
  isFavorited,
  toggleFavorite,
  recordFootprint,
} = require('../../utils/collectionStore')

Page({
  data: {
    id: '',
    detail: null,
    favorited: false,
    statusBarHeight: 20,
    favoriteIcon: '/images/icons/favorite-heart-outline.png',
    favoriteIconFilled: '/images/icons/favorite-heart-filled.png',
    text: {
      empty: '      ',
      origin: '    ',
      price: '    ',
      sold: '    ',
      highlights: '    ',
      description: '    ',
      gallery: '    ',
      buyNow: '   ',
    },
  },

  onLoad(options) {
    this.initNavMetrics()
    const id = options.id || ''
    this.setData({ id })
    if (id) {
      this.loadDetail(id)
    }
  },

  onShow() {
    const { detail } = this.data
    if (detail && detail._id) {
      this.setData({
        favorited: isFavorited('product', detail._id),
      })
    }
  },

  initNavMetrics() {
    try {
      const systemInfo = wx.getSystemInfoSync()
      this.setData({
        statusBarHeight: systemInfo.statusBarHeight || 20,
      })
    } catch (error) {
      this.setData({
        statusBarHeight: 20,
      })
    }
  },

  async loadDetail(id) {
    wx.showLoading({ title: '   ' })
```

```
    try {
      const db = wx.cloud.database()
      const res = await db.collection('products').doc(id).get()
      const detail = await this.normalizeDetail(res.data || null)
      const favorited = isFavorited('product', detail && detail._id)
      if (detail) {
        recordFootprint(this.buildCollectionRecord(detail))
      }
      this.setData({ detail, favorited })
    } catch (error) {
      console.error('[productDetail] load failed', error)
      wx.showToast({
        title: '    ',
        icon: 'none',
      })
    } finally {
      wx.hideLoading()
    }
  },

  async normalizeDetail(detail) {
    if (!detail) {
      return null
    }

    const cover = await resolveMediaSource(detail.cover, '/images/nav-mall.png')
    const gallery = await resolveMediaList(detail.gallery || [], cover)
    const tags = []
      .concat(Array.isArray(detail.categoryTags) ? detail.categoryTags : [])
      .concat(Array.isArray(detail.tags) ? detail.tags : [])
      .slice(0, 6)
    const regionText = [detail.province, detail.city, detail.district].filter(Boolean).j
oin(' · ')
    const price = Number(detail.price || 0)

    return {
      ...detail,
      cover,
      gallery,
      tags,
      regionText,
      locationText: detail.locationName || regionText || '     ',
      priceText: price ? `¥${price}` : '    ',
      soldText: detail.sold ? `${detail.sold}   ` : '     ',
      highlights: Array.isArray(detail.highlights) ? detail.highlights : [],
      descriptionText: detail.content || detail.summary || '',
    }
  },

  buildCollectionRecord(detail) {
    if (!detail) {
      return null
    }

    return {
      type: 'product',
      id: detail._id,
      title: detail.title,
      cover: detail.cover,
      city: detail.city || detail.province || detail.locationText,
      regionText: detail.regionText || detail.locationText,
      summary: detail.summary || detail.descriptionText,
```

```
      priceText: detail.priceText,
      metaText: detail.soldText,
      badgeText: '  ',
    }
  },

  toggleFavorite() {
    const { detail } = this.data
    if (!detail || !detail._id) {
      return
    }

    const favorited = toggleFavorite(this.buildCollectionRecord(detail))
    this.setData({ favorited })
    wx.showToast({
      title: favorited ? '   ' : '     ',
      icon: 'none',
    })
  },

  previewGallery(e) {
    const current = e.currentTarget.dataset.src
    const urls = (this.data.detail && this.data.detail.gallery) || []
    if (!current || !urls.length) {
      return
    }

    wx.previewImage({
      current,
      urls,
    })
  },

  goBack() {
    if (getCurrentPages().length > 1) {
      wx.navigateBack()
      return
    }

    wx.switchTab({
      url: '/pages/home/home',
    })
  },

  handleBuy() {
    wx.showToast({
      title: '                ',
      icon: 'none',
    })
  },
})
```

Path: miniprogram/pages/productDetail/productDetail.json

```
1 {
2   "navigationBarTitleText": "    "
3 }
```

Path: miniprogram/pages/productDetail/productDetail.wxml

```
<view class="page" wx:if="{{detail}}">
  <view class="heroWrap">
    <swiper
      class="heroSwiper"
      indicator-dots="{{detail.gallery.length > 1}}"
      indicator-color="rgba(255,255,255,0.42)"
      indicator-active-color="#ffffff"
      autoplay="{{detail.gallery.length > 1}}"
      touchable="{{detail.gallery.length > 1}}"
      interval="3200"
      duration="500"
      circular="{{detail.gallery.length > 1}}"
    >
      <swiper-item wx:for="{{detail.gallery}}" wx:key="*this">
        <image class="hero" src="{{item}}" mode="aspectFill" data-src="{{item}}" bindtap="previewGallery" />
      </swiper-item>
    </swiper>
    <view class="heroMask"></view>

    <view class="customNav" style="padding-top: {{statusBarHeight}}px;">
      <view class="backBtn" bindtap="goBack">
        <view class="backArrow"></view>
      </view>
    </view>
  </view>

  <view class="content">
    <view class="title">{{detail.title}}</view>
    <view class="summary" wx:if="{{detail.summary}}">{{detail.summary}}</view>

    <view class="tagWrap" wx:if="{{detail.tags.length}}">
      <view class="tag" wx:for="{{detail.tags}}" wx:key="*this">{{item}}</view>
    </view>

    <view class="infoGrid">
      <view class="infoCard">
        <view class="infoLabel">{{text.origin}}</view>
        <view class="infoValue">{{detail.locationText}}</view>
        <view class="infoSub" wx:if="{{detail.regionText}}">{{detail.regionText}}</view>
      </view>
      <view class="infoCard">
        <view class="infoLabel">{{text.price}}</view>
        <view class="infoValue priceText">{{detail.priceText}}</view>
        <view class="infoSub">{{detail.soldText}}</view>
      </view>
    </view>

    <view class="section" wx:if="{{detail.highlights.length}}">
      <view class="sectionTitle">{{text.highlights}}</view>
      <view class="card">
        <view class="bulletItem" wx:for="{{detail.highlights}}" wx:key="*this">
          <view class="dot"></view>
          <view class="bulletText">{{item}}</view>
        </view>
      </view>
    </view>

    <view class="section" wx:if="{{detail.descriptionText}}">
      <view class="sectionTitle">{{text.description}}</view>
```

```
      <view class="card plainText">{{detail.descriptionText}}</view>
    </view>

    <view class="section" wx:if="{{detail.gallery.length}}">
      <view class="sectionTitle">{{text.gallery}}</view>
      <scroll-view class="galleryScroll" scroll-x enhanced show-scrollbar="{{false}}">
        <image
          class="galleryImage"
          wx:for="{{detail.gallery}}"
          wx:key="*this"
          src="{{item}}"
          mode="aspectFill"
          data-src="{{item}}"
          bindtap="previewGallery"
        />
      </scroll-view>
    </view>
  </view>

  <view class="bottomBar">
    <view class="bottomMain">
      <view class="priceBox">
        <view class="priceLabel">{{text.price}}</view>
        <view class="priceNum">{{detail.priceText}}</view>
      </view>
      <view class="favoriteBtn" bindtap="toggleFavorite">
        <image class="favoriteIcon" src="{{favorited ? favoriteIconFilled : favoriteIcon}}" mode="aspectFit" />
      </view>
    </view>
    <button class="buyBtn" bindtap="handleBuy">{{text.buyNow}}</button>
  </view>
</view>

<view wx:else class="emptyState">{{text.empty}}</view>
```

Path: miniprogram/pages/productDetail/productDetail.wxss

```
.page {
  min-height: 100vh;
  background: linear-gradient(180deg, #f6faf4 0%, #edf5ea 100%);
  padding-bottom: 220rpx;
  box-sizing: border-box;
}

.heroWrap {
  position: relative;
  width: 100%;
  height: 520rpx;
  overflow: hidden;
}

.heroSwiper {
  width: 100%;
  height: 100%;
}

.hero {
  width: 100%;
  height: 100%;
  display: block;
}

.heroMask {
  position: absolute;
  inset: 0;
  background: linear-gradient(180deg, rgba(20, 20, 20, 0.28) 0%, rgba(20, 20, 20, 0.08) 36%, rgba(20, 20, 20, 0) 68%);
  pointer-events: none;
}

.customNav {
  position: fixed;
  left: 24rpx;
  right: auto;
  top: 0;
  padding-left: 0;
  padding-right: 0;
  z-index: 20;
  pointer-events: none;
}

.backBtn {
  width: 72rpx;
  height: 72rpx;
  border-radius: 50%;
  background: rgba(14, 38, 47, 0.72);
  display: flex;
  align-items: center;
  justify-content: center;
  pointer-events: auto;
}

.backArrow {
  width: 22rpx;
  height: 22rpx;
  border-left: 4rpx solid #ffffff;
  border-bottom: 4rpx solid #ffffff;
```

```
  transform: rotate(45deg);
  margin-left: 8rpx;
}

.content {
  position: relative;
  z-index: 1;
  margin-top: 34rpx;
  padding: 8rpx 24rpx 32rpx;
}

.title {
  font-size: 48rpx;
  line-height: 1.3;
  font-weight: 700;
  color: #1f2a1f;
}

.summary {
  margin-top: 14rpx;
  font-size: 28rpx;
  line-height: 1.7;
  color: #667566;
}

.tagWrap {
  display: flex;
  flex-wrap: wrap;
  gap: 12rpx;
  margin-top: 20rpx;
}

.tag {
  padding: 8rpx 18rpx;
  border-radius: 999rpx;
  font-size: 24rpx;
  color: #57984a;
  background: #edf5ea;
}

.infoGrid {
  margin-top: 26rpx;
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 16rpx;
}

.infoCard,
.card,
.descriptionCard {
  background: rgba(255, 255, 255, 0.92);
  border-radius: 28rpx;
  box-shadow: 0 12rpx 30rpx rgba(87, 73, 54, 0.06);
}

.infoCard,
.card {
  padding: 22rpx 22rpx 24rpx;
}

.infoLabel {
  font-size: 24rpx;
```

```
  color: #8ea08b;
}

.infoValue {
  margin-top: 12rpx;
  font-size: 28rpx;
  line-height: 1.5;
  color: #1f2a1f;
  font-weight: 600;
}

.infoSub {
  margin-top: 10rpx;
  font-size: 22rpx;
  line-height: 1.5;
  color: #8ea08b;
}

.priceText {
  color: #57984a;
}

.section {
  margin-top: 28rpx;
}

.sectionTitle {
  margin-bottom: 16rpx;
  font-size: 32rpx;
  font-weight: 700;
  color: #1f2a1f;
}

.bulletItem {
  display: flex;
  align-items: flex-start;
  gap: 14rpx;
  padding: 12rpx 0;
}

.dot {
  width: 12rpx;
  height: 12rpx;
  margin-top: 12rpx;
  border-radius: 50%;
  background: #57984a;
  flex-shrink: 0;
}

.bulletText,
.plainText {
  flex: 1;
  font-size: 28rpx;
  line-height: 1.7;
  color: #233123;
}

.descriptionCard {
  padding: 24rpx;
  color: #233123;
  font-size: 28rpx;
  line-height: 1.85;
```

```
  white-space: pre-line;
}

.galleryScroll {
  white-space: nowrap;
}

.galleryImage {
  display: inline-block;
  width: 250rpx;
  height: 180rpx;
  margin-right: 16rpx;
  border-radius: 22rpx;
}

.bottomBar {
  position: fixed;
  left: 0;
  right: 0;
  bottom: 0;
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 24rpx;
  padding: 8rpx 24rpx calc(env(safe-area-inset-bottom) + 8rpx);
  background: rgba(255, 255, 255, 0.98);
  box-shadow: 0 -8rpx 28rpx rgba(55, 42, 26, 0.08);
  z-index: 999;
}

.bottomMain {
  flex: 1;
  display: flex;
  align-items: center;
  gap: 20rpx;
}

.priceBox {
  display: flex;
  flex-direction: column;
  gap: 6rpx;
  flex: 1;
}

.priceLabel {
  font-size: 20rpx;
  color: #8ea08b;
}

.priceNum {
  font-size: 40rpx;
  font-weight: 700;
  color: #57984a;
}

.favoriteBtn {
  width: 80rpx;
  height: 80rpx;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
```

```
}

.favoriteIcon {
  width: 64rpx;
  height: 64rpx;
}

.buyBtn {
  min-width: 240rpx;
  height: 64rpx;
  line-height: 64rpx;
  border-radius: 999rpx;
  background: #57984a;
  color: #ffffff;
  font-size: 26rpx;
  font-weight: 600;
  padding: 0 28rpx;
}

.buyBtn::after {
  border: none;
}

.emptyState {
  padding: 80rpx 32rpx;
  text-align: center;
  color: #99a598;
  font-size: 28rpx;
}
```

Path: miniprogram/pages/cart/cart.js

```
Page({})
```

Path: miniprogram/pages/cart/cart.json

```
1 {
2   "navigationBarTitleText": "   "
3 }
```

Path: miniprogram/pages/cart/cart.wxml

```
<view style="padding: 24rpx;">       </view>
```

Path: miniprogram/pages/cart/cart.wxss

1

Path: miniprogram/pages/order/order.js

```
---------------------------------------------------------------------
const DEFAULT_DATA = () => ({
  activityId: '',
  travelDate: '',
  adultCount: 0,
  childCount: 0,
  totalCount: 0,
  nonce: '',
  contactName: '',
  contactPhone: '',
  contactQQ: '',
  contactWechat: '',
  travelers: [],
  travelerIds: [],
  specialRequest: '',
  emergencyEnabled: false,
  emergencyName: '',
  emergencyPhone: '',
})

function mapOrderMessage(message) {
  const messageMap = {
    ACTIVITY_ID_REQUIRED: 'Missing activity id',
    INVALID_TRAVEL_DATE: 'Please choose a valid travel date',
    INVALID_TRAVELER_COUNT: 'Traveler count is invalid',
    TRAVELER_COUNT_MISMATCH: 'Please select the required travelers',
    CONTACT_NAME_REQUIRED: 'Please fill in contact name',
    INVALID_CONTACT_PHONE: 'Invalid contact phone number',
    EMERGENCY_NAME_REQUIRED: 'Please fill in emergency contact name',
    INVALID_EMERGENCY_PHONE: 'Invalid emergency contact phone number',
    ACTIVITY_NOT_AVAILABLE: 'Activity is unavailable',
    TRAVELER_NOT_FOUND: 'Traveler data is invalid, please reselect',
    PHONE_BIND_REQUIRED: 'Please bind your phone number first',
    WECHAT_LOGIN_REQUIRED: 'Please login first',
  }
  return messageMap[message] || message || 'Submit failed'
}

Page({
  data: DEFAULT_DATA(),

  onLoad(options) {
    const adult = Number(options.adult || 0)
    const child = Number(options.child || 0)
    const activityId = options.activityId || options.id || ''

    this.setData({
      ...DEFAULT_DATA(),
      activityId,
      travelDate: options.date || '',
      adultCount: adult,
      childCount: child,
      totalCount: adult + child,
      nonce: options.nonce || '',
    })
  },

  onNameInput(e) { this.setData({ contactName: e.detail.value }) },
  onPhoneInput(e) { this.setData({ contactPhone: e.detail.value }) },
  onQQInput(e) { this.setData({ contactQQ: e.detail.value }) },
  onWechatInput(e) { this.setData({ contactWechat: e.detail.value }) },
```

```
  onSpecialInput(e) { this.setData({ specialRequest: e.detail.value }) },
  onEmergencyName(e) { this.setData({ emergencyName: e.detail.value }) },
  onEmergencyPhone(e) { this.setData({ emergencyPhone: e.detail.value }) },

  toggleEmergency(e) {
    this.setData({ emergencyEnabled: !!e.detail.value })
  },

  goSelectTravelers() {
    const need = this.data.totalCount
    wx.navigateTo({
      url: `/pages/travelerSelect/travelerSelect?need=${need}`,
      success: (res) => {
        res.eventChannel.emit('initSelected', { selectedIds: this.data.travelerIds })
        res.eventChannel.on('selectedDone', (data) => {
          const travelers = data.travelers || []
          this.setData({
            travelers,
            travelerIds: travelers.map((item) => item._id),
          })
        })
      },
    })
  },

  async submitOrder() {
    const {
      activityId, travelDate, adultCount, childCount, contactName, contactPhone,
      contactQQ, contactWechat, travelerIds, specialRequest,
      emergencyEnabled, emergencyName, emergencyPhone,
    } = this.data

    wx.showLoading({ title: 'Submitting' })
    try {
      const res = await wx.cloud.callFunction({
        name: 'activityOrder',
        data: {
          action: 'create',
          activityId,
          travelDate,
          adultCount,
          childCount,
          travelerIds,
          contact: {
            name: contactName,
            phone: contactPhone,
            qq: contactQQ,
            wechat: contactWechat,
          },
          specialRequest,
          emergency: {
            enabled: emergencyEnabled,
            name: emergencyName,
            phone: emergencyPhone,
          },
        },
      })

      if (!res.result || !res.result.success) {
        wx.showToast({ title: mapOrderMessage(res.result && res.result.message), icon: '
none' })
        return
```

```
      }

      wx.showToast({ title: 'Order created', icon: 'success' })
      setTimeout(() => wx.navigateBack({ delta: 2 }), 1200)
    } catch (error) {
      console.error('[order] submit failed', error)
      wx.showToast({ title: 'Submit failed', icon: 'none' })
    } finally {
      wx.hideLoading()
    }
  },
})
```

Path: miniprogram/pages/order/order.json

```
1 {
2   "navigationBarTitleText": "    "
3 }
```

Path: miniprogram/pages/order/order.wxml

```
<view class="container">

<!--      -->
<view class="card">
  <view class="title">    </view>

  <view class="row">
    <text>    </text>
    <text>{{travelDate}}</text>
  </view>

  <view class="row">
    <text>  </text>
    <text>{{adultCount}}  </text>
  </view>

  <view class="row">
    <text>  </text>
    <text>{{childCount}}  </text>
  </view>

</view>


<!--     -->
<view class="card">

  <view class="title">   </view>

  <view class="inputRow">
    <text class="label">*  </text>
    <input
      class="input"
      placeholder="     "
      value="{{contactName}}"
      bindinput="onNameInput"
    />
  </view>

  <view class="inputRow">
    <text class="label">*   </text>
    <input
      class="input"
      type="number"
      placeholder="      "
      value="{{contactPhone}}"
      bindinput="onPhoneInput"
    />
  </view>

  <view class="inputRow">
    <text class="label">QQ </text>
    <input
      class="input"
      placeholder="  "
      value="{{contactQQ}}"
      bindinput="onQQInput"
    />
  </view>

```

```
  <view class="inputRow">
    <text class="label">   </text>
    <input
      class="input"
      placeholder="  "
      value="{{contactWechat}}"
      bindinput="onWechatInput"
    />
  </view>

</view>


<!--     -->
<view class="card">

  <view class="titleRow">
    <text class="title">   </text>

    <button
      class="chooseBtn"
      size="mini"
      bindtap="goSelectTravelers">

    </button>
  </view>

  <view class="tip">

  </view>


  <view wx:if="{{travelers.length==0}}" class="empty">

  </view>


  <view wx:for="{{travelers}}" wx:key="_id" class="travelerRow">

    <text>   {{index+1}}</text>

    <view class="travelerInfo">
      <text>{{item.name}}</text>
      <text class="idno">{{item.idNo}}</text>
    </view>

  </view>

</view>


<!--      -->
<view class="card">

  <view class="title">    </view>

  <textarea
    class="textarea"
    placeholder="  "
    value="{{specialRequest}}"
    bindinput="onSpecialInput"
  />
```

```
</view>


<!--       -->
<view class="card">

  <view class="row">

    <text>       </text>

    <switch
      checked="{{emergencyEnabled}}"
      bindchange="toggleEmergency"
    />

  </view>


  <view wx:if="{{emergencyEnabled}}">

    <view class="inputRow">
      <text class="label">  </text>
      <input
        class="input"
        placeholder="     "
        value="{{emergencyName}}"
        bindinput="onEmergencyName"
      />
    </view>

    <view class="inputRow">
      <text class="label">   </text>
      <input
        class="input"
        type="number"
        placeholder="      "
        value="{{emergencyPhone}}"
        bindinput="onEmergencyPhone"
      />
    </view>

  </view>

</view>


<!--      -->
<view class="bottom">

  <button
    class="submit"
    bindtap="submitOrder">

  </button>

</view>

</view>
```

Path: miniprogram/pages/order/order.wxss

```
.container{
  padding:30rpx;
  background:#f6faf4;
  min-height:100vh;
}

.card{
  background:#ffffff;
  padding:30rpx;
  border-radius:16rpx;
  margin-bottom:30rpx;
}

.title{
  font-weight:bold;
  margin-bottom:20rpx;
}

.titleRow{
  display:flex;
  justify-content:space-between;
  align-items:center;
}

.row{
  display:flex;
  justify-content:space-between;
  margin:20rpx 0;
}

.inputRow{
  display:flex;
  align-items:center;
  margin:20rpx 0;
}

.label{
  width:150rpx;
}

.input{
  flex:1;
  border:1px solid #dbe8d7;
  padding:10rpx;
  border-radius:8rpx;
}

.textarea{
  border:1px solid #dbe8d7;
  border-radius:8rpx;
  padding:20rpx;
}

.tip{
  color:#99a598;
  font-size:24rpx;
  margin-bottom:10rpx;
}

.empty{
```

```
  color:#99a598;
}

.travelerRow{
  display:flex;
  justify-content:space-between;
  margin-top:20rpx;
}

.idno{
  color:#667566;
  font-size:24rpx;
}

.bottom{
  padding:30rpx 0;
}

.submit{
  background:#57984a;
  color:white;
}
```

Path: miniprogram/pages/myOrders/myOrders.js

```
---------------------------------------------------------------------
const TAB_CONFIG = [
  { key: 'all', label: 'All' },
  { key: 'pending_payment', label: 'To Pay' },
  { key: 'pending_trip', label: 'Upcoming' },
  { key: 'pending_receive', label: 'To Receive' },
  { key: 'refund_after_sale', label: 'Refunds' },
]

const DEFAULT_ORDER_COVER = '/images/activities/lz-yuzhong-strawberry-family-day.jpg'

const EMPTY_COPY = {
  all: { title: 'No orders', desc: 'Your orders will appear here.' },
  pending_payment: { title: 'No pending payments', desc: 'Unpaid orders will appear here.' },
  pending_trip: { title: 'No upcoming trips', desc: 'Paid activity orders will appear here.' },
  pending_receive: { title: 'No deliveries', desc: 'Product delivery flow is not connected yet.' },
  refund_after_sale: { title: 'No refunds', desc: 'Refunding and cancelled orders will appear here.' },
}

function pad(num) {
  return String(num).padStart(2, '0')
}

function formatDateTime(input) {
  if (!input) return 'Time unavailable'
  const date = new Date(input)
  if (Number.isNaN(date.getTime())) return 'Time unavailable'
  return `${date.getFullYear()}-${pad(date.getMonth() + 1)}-${pad(date.getDate())} ${pad(date.getHours())}:${pad(date.getMinutes())}`
}

function toMoney(value) {
  return Number(value || 0).toFixed(2)
}

function mapOrder(item) {
  const statusKey = item.displayStatusKey
  const bucketMap = {
    pending_payment: 'pending_payment',
    upcoming: 'pending_trip',
    pending_review: 'all',
    refunding: 'refund_after_sale',
    cancelled: 'refund_after_sale',
    completed: 'all',
  }

  return {
    ...item,
    orderType: 'activity',
    orderTypeText: 'Activity Order',
    title: item.activityTitle || 'Untitled Activity',
    cover: item.activityCover || item.cover || DEFAULT_ORDER_COVER,
    summary: item.travelDate ? `Travel date ${item.travelDate}` : 'Activity order',
    createdAtValue: new Date(item.createdAt || 0).getTime() || 0,
    createdAtText: formatDateTime(item.createdAt),
    totalPriceText: toMoney(item.totalPrice),
    statusBucket: bucketMap[statusKey] || 'all',
```

```
    statusClass: statusKey || 'success',
  }
}

Page({
  data: {
    tabs: TAB_CONFIG,
    currentTab: 'all',
    rawList: [],
    displayList: [],
    emptyTitle: EMPTY_COPY.all.title,
    emptyDesc: EMPTY_COPY.all.desc,
  },

  onLoad(options) {
    const tab = options.tab || 'all'
    this.setData({
      currentTab: TAB_CONFIG.some((item) => item.key === tab) ? tab : 'all',
    })
  },

  onShow() {
    this.loadOrders()
  },

  switchTab(e) {
    const { tab } = e.currentTarget.dataset
    this.setData({ currentTab: tab }, () => this.buildDisplayList())
  },

  async loadOrders() {
    wx.showLoading({ title: 'Loading' })
    try {
      const res = await wx.cloud.callFunction({
        name: 'activityOrder',
        data: { action: 'listMine', page: 1, pageSize: 100 },
      })

      if (!res.result || !res.result.success) {
        wx.showToast({ title: (res.result && res.result.message) || 'Load failed', icon: 'none' })
        return
      }

      const orders = (res.result.data || []).map(mapOrder)
      orders.sort((a, b) => (b.createdAtValue || 0) - (a.createdAtValue || 0))
      this.setData({ rawList: orders }, () => this.buildDisplayList())
    } catch (error) {
      console.error('[myOrders] load orders failed', error)
      wx.showToast({ title: 'Load failed', icon: 'none' })
    } finally {
      wx.hideLoading()
    }
  },

  buildDisplayList() {
    const { currentTab, rawList } = this.data
    let list = rawList

    if (currentTab === 'pending_payment') list = rawList.filter((item) => item.statusBucket === 'pending_payment')
    if (currentTab === 'pending_trip') list = rawList.filter((item) => item.statusBucket
```

```
    if (currentTab === 'pending_receive') list = rawList.filter((item) => item.statusBucket === 'pending_receive')
    if (currentTab === 'refund_after_sale') list = rawList.filter((item) => item.statusBucket === 'refund_after_sale')

    const copy = EMPTY_COPY[currentTab] || EMPTY_COPY.all
    this.setData({
      displayList: list,
      emptyTitle: copy.title,
      emptyDesc: copy.desc,
    })
  },

  goDetail(e) {
    const { id } = e.currentTarget.dataset
    if (!id) return
    wx.navigateTo({ url: `/pages/activityOrderDetail/activityOrderDetail?id=${id}` })
  },
})
```

Path: miniprogram/pages/myOrders/myOrders.json

```
1 {
2   "navigationBarTitleText": "    "
3 }
```

Path: miniprogram/pages/myOrders/myOrders.wxml

```
<view class="page">
  <view class="tabs">
    <view
      wx:for="{{tabs}}"
      wx:key="key"
      class="tab {{currentTab===item.key?'active':''}}"
      data-tab="{{item.key}}"
      bindtap="switchTab">
      {{item.label}}
    </view>
  </view>

  <view wx:if="{{displayList.length === 0}}" class="emptyWrap">
    <view class="emptyIcon"></view>
    <view class="emptyTitle">{{emptyTitle}}</view>
    <view class="emptyDesc">{{emptyDesc}}</view>
  </view>

  <view wx:for="{{displayList}}" wx:key="_id" class="orderCard" data-id="{{item._id}}" bindtap="goDetail">
    <view class="cardHeader">
      <view class="orderType">{{item.orderTypeText}}</view>
      <view class="status {{item.statusClass}}">{{item.displayStatusText}}</view>
    </view>

    <view class="cardBody">
      <image
        wx:if="{{item.cover}}"
        class="cover"
        src="{{item.cover}}"
        mode="aspectFill">
      </image>
      <view wx:else class="cover placeholderCover"></view>

      <view class="info">
        <view class="title">{{item.title}}</view>
        <view class="meta">{{item.summary}}</view>
        <view class="meta">{{item.createdAtText}}</view>
      </view>
    </view>

    <view class="cardFooter">
      <view class="price"> {{item.totalPriceText}}</view>
    </view>
  </view>
</view>
```

Path: miniprogram/pages/myOrders/myOrders.wxss

```
.page{
  min-height: 100vh;
  background: #f6faf4;
}

.tabs{
  position: sticky;
  top: 0;
  z-index: 10;
  display: flex;
  padding: 0 12rpx;
  background: #ffffff;
  border-bottom: 1rpx solid #e2eadf;
}

.tab{
  position: relative;
  flex: 1;
  padding: 26rpx 0 22rpx;
  text-align: center;
  font-size: 26rpx;
  color: #7f8794;
}

.tab.active{
  color: #1f2a1f;
  font-weight: 700;
}

.tab.active::after{
  content: '';
  position: absolute;
  left: 50%;
  bottom: 6rpx;
  width: 34rpx;
  height: 6rpx;
  margin-left: -17rpx;
  border-radius: 999rpx;
  background: #1f2a1f;
}

.emptyWrap{
  display: flex;
  min-height: 58vh;
  padding: 120rpx 48rpx 0;
  flex-direction: column;
  align-items: center;
  box-sizing: border-box;
}

.emptyIcon{
  width: 112rpx;
  height: 112rpx;
  border-radius: 28rpx;
  background: linear-gradient(180deg, #f5f6f8 0%, #e5efe1 100%);
}

.emptyTitle{
  margin-top: 28rpx;
  font-size: 34rpx;
```

```
  61   font-weight: 700;
  62   color: #233123;
  63 }
  64
  65 .emptyDesc{
  66   margin-top: 12rpx;
  67   font-size: 24rpx;
  68   color: #99a598;
  69 }
  70
  71 .orderCard{
  72   margin: 20rpx 24rpx 0;
  73   padding: 24rpx;
  74   border-radius: 24rpx;
  75   background: #ffffff;
  76 }
  77
  78 .cardHeader{
  79   display: flex;
  80   justify-content: space-between;
  81   align-items: center;
  82 }
  83
  84 .orderType{
  85   font-size: 24rpx;
  86   color: #8d96a3;
  87 }
  88
  89 .status{
  90   font-size: 24rpx;
  91   font-weight: 600;
  92 }
  93
  94 .status.pending_payment,
  95 .status.pending_review{
  96   color: #57984a;
  97 }
  98
  99 .status.cancelled,
 100 .status.withdrawn{
 101   color: #99a598;
 102 }
 103
 104 .status.pending_trip,
 105 .status.pending_receive,
 106 .status.signed_pending_confirm{
 107   color: #667566;
 108 }
 109
 110 .status.refunding{
 111   color: #57984a;
 112 }
 113
 114 .status.refunded,
 115 .status.success{
 116   color: #57984a;
 117 }
 118
 119 .status.reviewed{
 120   color: #2f6fed;
 121 }
 122
```

```
.cardBody{
  display: flex;
  margin-top: 20rpx;
}

.cover{
  width: 144rpx;
  height: 144rpx;
  border-radius: 20rpx;
  background: #e4eee0;
  flex-shrink: 0;
}

.placeholderCover{
  background: linear-gradient(180deg, #eef5ea 0%, #eceff4 100%);
}

.info{
  min-width: 0;
  margin-left: 20rpx;
  display: flex;
  flex: 1;
  flex-direction: column;
  justify-content: space-between;
}

.title{
  font-size: 30rpx;
  line-height: 1.5;
  color: #181d27;
  font-weight: 700;
}

.meta{
  margin-top: 10rpx;
  font-size: 24rpx;
  line-height: 1.5;
  color: #8b94a1;
}

.cardFooter{
  margin-top: 18rpx;
  display: flex;
  justify-content: flex-end;
}

.price{
  font-size: 30rpx;
  font-weight: 700;
  color: #181d27;
}

```

Path: miniprogram/pages/academy/academy.js

```
---------------------------------------------------------------------------
const { resolveKnowledgeArticleMedia } = require('../../utils/knowledgeArticle')

const DEFAULT_TAB = 'agri'

Page({
  data: {
    loading: true,
    activeTab: DEFAULT_TAB,
    currentCategory: '  ',
    categories: [],
    filteredList: [],
    heroCover: '',
  },

  onLoad() {
    this.fullList = []
    this.loadArticles()
  },

  onPullDownRefresh() {
    this.loadArticles().finally(() => wx.stopPullDownRefresh())
  },

  async loadArticles() {
    this.setData({ loading: true })
    try {
      const res = await wx.cloud.callFunction({
        name: 'getknowledgearticles',
        data: {
          channel: this.data.activeTab,
        },
      })

      const result = (res || {}).result || {}
      if (!result.ok) {
        throw new Error(result.error || 'load articles failed')
      }

      const resolvedList = await Promise.all(
        (result.list || []).map((item) => resolveKnowledgeArticleMedia(item))
      )

      const categories = Array.isArray(result.categories) && result.categories.length
        ? result.categories
        : ['  ']

      this.fullList = resolvedList
      this.setData({
        loading: false,
        categories,
        heroCover: (resolvedList[0] && resolvedList[0].cover) || '',
      })
      this.applyCategory(this.data.currentCategory)
    } catch (err) {
      console.error('[academy] load articles failed', err)
      this.fullList = []
      this.setData({
        loading: false,
        categories: ['  '],
        currentCategory: '  ',
```

```
        filteredList: [],
        heroCover: '',
      })
      wx.showToast({
        title: '      ',
        icon: 'none',
      })
    }
  },

  applyCategory(category) {
    const currentCategory = category || '  '
    const sourceList = this.fullList || []
    const filteredList = currentCategory === '  '
      ? sourceList
      : sourceList.filter((item) => (item.tags || []).includes(currentCategory))

    this.setData({
      currentCategory,
      filteredList,
    })
  },

  async switchTab(e) {
    const activeTab = e.currentTarget.dataset.tab
    if (!activeTab || activeTab === this.data.activeTab) {
      return
    }

    this.fullList = []
    this.setData({
      activeTab,
      currentCategory: '  ',
      categories: [],
      filteredList: [],
      heroCover: '',
    })

    await this.loadArticles()
  },

  switchCategory(e) {
    const currentCategory = e.currentTarget.dataset.category || '  '
    this.applyCategory(currentCategory)
  },

  goDetail(e) {
    const id = e.currentTarget.dataset.id
    if (!id) {
      return
    }

    wx.navigateTo({
      url: `/pages/knowledgeDetail/knowledgeDetail?id=${id}`,
    })
  },
})
```

Path: miniprogram/pages/academy/academy.json

```
1 {
2   "navigationBarTitleText": "    ",
3   "enablePullDownRefresh": true
4 }
```

Path: miniprogram/pages/academy/academy.wxml

```
<view class="page container-safe">
  <view class="hero">
    <image
      wx:if="{{heroCover}}"
      class="hero-bg"
      src="{{heroCover}}"
      mode="aspectFill"
    ></image>
    <view class="hero-bg hero-placeholder" wx:else></view>
    <view class="hero-mask"></view>
    <view class="hero-content">
      <view class="hero-chip">{{activeTab === 'agri' ? '    ' : '    '}}</view>
      <view class="hero-title">    </view>
      <view class="hero-subtitle">
        {{activeTab === 'agri' ? '            ' : '                '}}
      </view>
      <view class="hero-desc">
        {{activeTab === 'agri' ? '                  ' : '                      '}}
      </view>
    </view>
  </view>

  <view class="sheet">
    <view class="tab-row">
      <view class="tab {{activeTab === 'agri' ? 'active' : ''}}" data-tab="agri" bindtap="switchTab">    </view>
      <view class="tab {{activeTab === 'culture' ? 'active' : ''}}" data-tab="culture" bindtap="switchTab">    </view>
    </view>

    <scroll-view class="category-scroll" scroll-x enable-flex show-scrollbar="false">
      <view class="category-list">
        <block wx:for="{{categories}}" wx:key="index">
          <view
            class="category-chip {{currentCategory === item ? 'selected' : ''}}"
            data-category="{{item}}"
            bindtap="switchCategory"
          >
            {{item}}
          </view>
        </block>
      </view>
    </scroll-view>

    <view class="feed-head">
      <view>
        <view class="feed-title">{{activeTab === 'agri' ? '    ' : '    '}}</view>
        <view class="feed-subtitle">{{activeTab === 'agri' ? '          ' : '          '}}</view>
      </view>
      <view class="feed-count">{{filteredList.length}}    </view>
    </view>

    <view class="loading-box" wx:if="{{loading}}">      ...</view>

    <block wx:elif="{{filteredList.length}}">
      <block wx:for="{{filteredList}}" wx:key="id">
        <view class="news-card" data-id="{{item.id}}" bindtap="goDetail">
          <view class="news-main">
            <view class="news-left">
```

```
58              <view class="news-title">{{item.title}}</view>
59              <view class="news-author">{{item.author}}</view>
60              <view class="news-time">{{item.publishTime}}</view>
61              <view class="news-tag-row">
62                <block wx:for="{{item.tags}}" wx:key="index" wx:for-item="tag">
63                  <text class="news-tag">{{tag}}</text>
64                </block>
65              </view>
66            </view>
67            <view class="news-right">
68              <image class="cover" src="{{item.cover}}" mode="aspectFill"></image>
69              <view class="cover-views">{{item.views}}</view>
70            </view>
71          </view>
72          <view class="news-meta-row">
73            <view class="news-wordcount">{{item.wordCount}} </view>
74            <view class="footer-right">
75              <text class="stat-item">  {{item.likes}}</text>
76              <text class="stat-item">  {{item.comments}}</text>
77              <text class="stat-item">  {{item.shareCount}}</text>
78            </view>
79          </view>
80        </view>
81      </block>
82    </block>
83
84    <view class="empty-box" wx:else>
85      <view class="empty-title">         </view>
86      <view class="empty-text">                          </view>
87    </view>
88  </view>
89 </view>
```

Path: miniprogram/pages/academy/academy.wxss

```
.page {
  min-height: 100vh;
  background: #f5f6f7;
}

.hero {
  position: relative;
  height: 390rpx;
  overflow: hidden;
}

.hero-bg {
  width: 100%;
  height: 100%;
}

.hero-placeholder {
  background: linear-gradient(135deg, #8aa96d 0%, #5f7b4d 100%);
}

.hero-mask {
  position: absolute;
  inset: 0;
  background: linear-gradient(180deg, rgba(14, 27, 18, 0.12) 0%, rgba(10, 25, 16, 0.38) 100%);
}

.hero-content {
  position: absolute;
  left: 28rpx;
  right: 28rpx;
  bottom: 38rpx;
  color: #fff;
}

.hero-chip {
  display: inline-flex;
  align-items: center;
  height: 46rpx;
  padding: 0 22rpx;
  border-radius: 999rpx;
  background: rgba(124, 173, 98, 0.96);
  font-size: 24rpx;
  margin-bottom: 18rpx;
}

.hero-title {
  font-size: 52rpx;
  font-weight: 700;
  line-height: 1.2;
}

.hero-subtitle {
  margin-top: 10rpx;
  font-size: 28rpx;
  line-height: 1.5;
  opacity: 0.98;
}

.hero-desc {
```

```
  margin-top: 8rpx;
  font-size: 24rpx;
  line-height: 1.6;
  opacity: 0.94;
}

.sheet {
  margin: -26rpx 24rpx 0;
  padding: 28rpx 22rpx 48rpx;
  background: #fff;
  border-radius: 34rpx 34rpx 0 0;
  box-shadow: 0 -8rpx 26rpx rgba(18, 24, 29, 0.04);
}

.tab-row {
  display: flex;
  gap: 18rpx;
}

.tab {
  min-width: 180rpx;
  height: 76rpx;
  padding: 0 24rpx;
  border-radius: 20rpx;
  background: #f1f2f3;
  color: #6b6f72;
  font-size: 32rpx;
  font-weight: 600;
  display: flex;
  align-items: center;
  justify-content: center;
}

.tab.active {
  background: linear-gradient(135deg, #8aa96d 0%, #6d8a57 100%);
  color: #fff;
}

.category-scroll {
  margin-top: 22rpx;
  white-space: nowrap;
}

.category-list {
  display: inline-flex;
  gap: 14rpx;
  padding-bottom: 8rpx;
}

.category-chip {
  padding: 12rpx 22rpx;
  border-radius: 999rpx;
  background: #f2f3f5;
  color: #8b8f94;
  font-size: 24rpx;
  line-height: 1;
}

.category-chip.selected {
  background: #eaf3e4;
  color: #5e7b4c;
  font-weight: 600;
```

```
}

.feed-head {
  margin: 28rpx 2rpx 18rpx;
  display: flex;
  justify-content: space-between;
  align-items: flex-end;
}

.feed-title {
  color: #1f1f1f;
  font-size: 34rpx;
  font-weight: 700;
}

.feed-subtitle {
  margin-top: 8rpx;
  color: #8d8d8d;
  font-size: 22rpx;
}

.feed-count {
  color: #8e8e8e;
  font-size: 24rpx;
}

.loading-box,
.empty-box {
  padding: 100rpx 0;
  text-align: center;
  color: #8d8d8d;
  font-size: 26rpx;
}

.empty-title {
  font-size: 30rpx;
  font-weight: 700;
  color: #1f1f1f;
}

.empty-text {
  margin-top: 14rpx;
  font-size: 24rpx;
  color: #8d8d8d;
}

.news-card {
  padding: 24rpx;
  border-radius: 28rpx;
  background: #fafafa;
  margin-bottom: 22rpx;
  box-shadow: 0 8rpx 24rpx rgba(31, 35, 41, 0.05);
}

.news-main {
  display: flex;
  align-items: flex-start;
}

.news-left {
  flex: 1;
  min-width: 0;
```

```
  padding-right: 22rpx;
}

.news-right {
  position: relative;
  width: 210rpx;
  flex-shrink: 0;
}

.news-title {
  color: #1e1e1f;
  font-size: 32rpx;
  line-height: 1.36;
  font-weight: 700;
  display: -webkit-box;
  -webkit-line-clamp: 2;
  -webkit-box-orient: vertical;
  overflow: hidden;
}

.news-author,
.news-time,
.news-wordcount {
  color: #8d8d8d;
  font-size: 24rpx;
  line-height: 1.5;
}

.news-author {
  margin-top: 10rpx;
}

.news-time {
  margin-top: 4rpx;
}

.news-tag-row {
  margin-top: 12rpx;
  display: flex;
  flex-wrap: wrap;
  gap: 10rpx;
}

.news-tag {
  height: 40rpx;
  padding: 0 16rpx;
  border-radius: 999rpx;
  background: #edf4e8;
  color: #5d7d4d;
  font-size: 22rpx;
  line-height: 40rpx;
}

.news-meta-row {
  margin-top: 14rpx;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16rpx;
}

.news-meta-row .news-wordcount {
```

```
  margin-top: 0;
}

.cover {
  width: 210rpx;
  height: 162rpx;
  border-radius: 22rpx;
  background: #e9edf1;
}

.cover-views {
  position: absolute;
  right: 10rpx;
  bottom: 10rpx;
  min-width: 72rpx;
  height: 34rpx;
  padding: 0 12rpx;
  border-radius: 999rpx;
  background: rgba(0, 0, 0, 0.5);
  color: #fff;
  font-size: 20rpx;
  line-height: 34rpx;
  text-align: center;
}

.footer-right {
  display: flex;
  align-items: center;
  gap: 16rpx;
  flex-shrink: 0;
}

.stat-item {
  color: #7e7e7e;
  font-size: 22rpx;
}
```

Path: miniprogram/pages/knowledgeDetail/knowledgeDetail.js

```
---------------------------------------------------------------------------
const { resolveKnowledgeArticleMedia, normalizeArray } = require('../../utils/knowledgeArticle')
const {
  isFavorited,
  toggleFavorite,
  recordFootprint,
} = require('../../utils/collectionStore')

Page({
  data: {
    loading: true,
    article: null,
    liked: false,
    favorited: false,
    commentInput: '',
    commentList: [],
    favoriteIcon: '/images/icons/favorite-heart-outline.png',
    favoriteIconFilled: '/images/icons/favorite-heart-filled.png',
  },

  onLoad(options = {}) {
    this.articleId = options.id || ''
    this.loadArticle()
  },

  onShow() {
    const article = this.data.article
    if (article && article.id) {
      this.setData({
        favorited: isFavorited('knowledge', article.id),
      })
    }
  },

  async loadArticle() {
    if (!this.articleId) {
      wx.showToast({ title: '     ', icon: 'none' })
      return
    }

    this.setData({ loading: true })
    try {
      const res = await wx.cloud.callFunction({
        name: 'getknowledgearticledetail',
        data: {
          id: this.articleId,
        },
      })

      const result = (res || {}).result || {}
      if (!result.ok || !result.data) {
        throw new Error(result.error || 'article not found')
      }

      const article = await resolveKnowledgeArticleMedia(result.data)
      const likeState = wx.getStorageSync(`knowledge_like_${article.id}`) || false
      const favState = isFavorited('knowledge', article.id)
      const storedComments =
        wx.getStorageSync(`knowledge_comments_${article.id}`) || normalizeArray(article.commentList)
```

```

      recordFootprint(this.buildCollectionRecord(article))

      this.setData({
        loading: false,
        article,
        liked: likeState,
        favorited: favState,
        commentList: storedComments,
      })
    } catch (err) {
      console.error('[knowledgeDetail] load article failed', err)
      this.setData({
        loading: false,
        article: null,
      })
      wx.showToast({ title: '      ', icon: 'none' })
    }
  },

  buildCollectionRecord(article) {
    if (!article) {
      return null
    }

    const location = article.location || {}
    return {
      type: 'knowledge',
      id: article.id,
      title: article.title,
      cover: article.cover,
      city: article.city || location.city || location.name || article.author,
      regionText: location.address || location.name || '',
      summary: article.summary,
      metaText: `${article.publishTime || ''}${article.views ? ` · ${article.views}  ` : ''}`,
      badgeText: '  /  ',
      author: article.author,
      statsText: `${Number(article.likes || 0)}  · ${Number(article.favorites || 0)}  `,
    }
  },

  toggleLike() {
    const article = this.data.article
    if (!article) {
      return
    }

    const liked = !this.data.liked
    const likes = Number(article.likes || 0) + (liked ? 1 : -1)
    wx.setStorageSync(`knowledge_like_${article.id}`, liked)
    this.setData({
      liked,
      'article.likes': Math.max(likes, 0),
    })
  },

  toggleFavorite() {
    const article = this.data.article
    if (!article) {
      return
    }
```

```

    const baseFavorites = Number(article.favorites || 0)
    const favorited = toggleFavorite(this.buildCollectionRecord(article))
    this.setData({
      favorited,
      'article.favorites': Math.max(baseFavorites + (favorited ? 1 : -1), 0),
    })
  },

  onCommentInput(e) {
    this.setData({ commentInput: e.detail.value })
  },

  submitComment() {
    const article = this.data.article
    const value = String(this.data.commentInput || '').trim()
    if (!article || !value) {
      wx.showToast({ title: '       ', icon: 'none' })
      return
    }

    const comment = {
      user: '    ',
      content: value,
      time: '  ',
    }
    const commentList = [comment, ...this.data.commentList]
    wx.setStorageSync(`knowledge_comments_${article.id}`, commentList)
    this.setData({
      commentInput: '',
      commentList,
    })
    wx.showToast({ title: '    ', icon: 'success' })
  },

  focusComment() {
    wx.pageScrollTo({
      selector: '.comment-section',
      duration: 300,
    })
  },

  openMap() {
    const location = (this.data.article || {}).location
    if (!location) {
      return
    }

    wx.openLocation({
      latitude: Number(location.latitude),
      longitude: Number(location.longitude),
      name: location.name,
      address: location.address,
      scale: 16,
    })
  },

  onShareAppMessage() {
    const article = this.data.article || {}
    this.setData({
      'article.shareCount': Number(article.shareCount || 0) + 1,
    })
```

```

    return {
      title: article.title || '    ',
      path: `/pages/knowledgeDetail/knowledgeDetail?id=${article.id}`,
      imageUrl: article.cover || '',
    }
  },
})
```

Path: miniprogram/pages/knowledgeDetail/knowledgeDetail.json

```
1 {
2   "navigationBarTitleText": "    "
3 }
```

Path: miniprogram/pages/knowledgeDetail/knowledgeDetail.wxml

```
<view class="detail-page container-safe">
  <view class="loading-box" wx:if="{{loading}}">      ...</view>

  <block wx:elif="{{article}}">
    <image class="detail-cover" src="{{article.cover}}" mode="aspectFill"></image>
    <view class="detail-card">
      <view class="detail-title">{{article.title}}</view>
      <view class="meta-row">{{article.author}} · {{article.publishTime}}</view>
      <view class="meta-row light">{{article.wordCount}}   · {{article.views}}   </view>
      <view class="tag-row">
        <block wx:for="{{article.tags}}" wx:key="*this">
          <text class="tag">{{item}}</text>
        </block>
      </view>

      <view class="summary-box">{{article.summary}}</view>

      <block wx:for="{{article.content}}" wx:key="index">
        <view wx:if="{{item.type === 'heading'}}" class="content-h2">{{item.text}}</view>
        <image wx:elif="{{item.type === 'image'}}" class="content-image" src="{{item.src}}" mode="widthFix"></image>
        <view wx:else class="content-p">{{item.text}}</view>
      </block>

      <view class="location-card" wx:if="{{article.location}}" bindtap="openMap">
        <view class="location-left">
          <view class="location-title">{{article.location.name}}</view>
          <view class="location-address">{{article.location.address}}</view>
        </view>
        <view class="location-right">    </view>
      </view>

      <view class="comment-section">
        <view class="comment-head">   {{commentList.length}}</view>
        <view class="comment-input-wrap">
          <input
            class="comment-input"
            placeholder="      ..."
            value="{{commentInput}}"
            bindinput="onCommentInput"
          />
          <view class="comment-send" bindtap="submitComment">  </view>
        </view>

        <block wx:if="{{commentList.length}}">
          <block wx:for="{{commentList}}" wx:key="index">
            <view class="comment-item">
              <view class="comment-user">{{item.user}}</view>
              <view class="comment-text">{{item.content}}</view>
              <view class="comment-time">{{item.time}}</view>
            </view>
          </block>
        </block>
        <view class="no-comment" wx:else>            </view>
      </view>
    </view>

    <view class="action-bar">
      <view class="action-item" bindtap="toggleLike">{{liked ? '  ' : '  '}} {{article.l
```

```
      <view class="action-item favoriteAction" bindtap="toggleFavorite">
        <image class="favoriteActionIcon" src="{{favorited ? favoriteIconFilled : favori
teIcon}}" mode="aspectFit" />
        <text>{{favorited ? '   ' : '  '}} {{article.favorites}}</text>
      </view>
      <view class="action-item" bindtap="focusComment">   {{commentList.length}}</view>
      <button class="share-btn" open-type="share">   {{article.shareCount}}</button>
    </view>
  </block>

  <view class="empty-box" wx:else>          </view>
</view>
```

Path: miniprogram/pages/knowledgeDetail/knowledgeDetail.wxss

```
.detail-page {
  min-height: 100vh;
  background: #f7f4ef;
  padding-bottom: 140rpx;
}

.loading-box,
.empty-box {
  padding: 180rpx 40rpx;
  text-align: center;
  color: #8d8d8d;
  font-size: 28rpx;
}

.detail-cover {
  width: 100%;
  height: 420rpx;
  background: #e9edf1;
}

.detail-card {
  margin: -34rpx 24rpx 0;
  background: #fff;
  border-radius: 34rpx 34rpx 0 0;
  padding: 32rpx 28rpx 40rpx;
}

.detail-title {
  font-size: 42rpx;
  line-height: 1.45;
  font-weight: 700;
  color: #2d2925;
}

.meta-row {
  margin-top: 16rpx;
  font-size: 24rpx;
  color: #6a6259;
}

.meta-row.light {
  color: #988d80;
}

.tag-row {
  display: flex;
  flex-wrap: wrap;
  gap: 12rpx;
  margin-top: 20rpx;
}

.tag {
  padding: 8rpx 18rpx;
  border-radius: 999rpx;
  background: #eef5ea;
  color: #5f8f4e;
  font-size: 22rpx;
}

.summary-box {
```

```
  margin: 24rpx 0;
  padding: 24rpx;
  background: #fbfaf8;
  border-radius: 24rpx;
  font-size: 26rpx;
  line-height: 1.8;
  color: #5f564d;
}

.content-h2 {
  margin-top: 30rpx;
  font-size: 32rpx;
  font-weight: 700;
  color: #2f2a25;
}

.content-p {
  margin-top: 18rpx;
  font-size: 28rpx;
  line-height: 1.95;
  color: #3e372f;
  text-align: justify;
}

.content-image {
  display: block;
  width: 100%;
  margin-top: 24rpx;
  border-radius: 24rpx;
  background: #e9edf1;
}

.location-card {
  margin-top: 34rpx;
  padding: 24rpx;
  border-radius: 24rpx;
  background: #f4efe7;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.location-left {
  flex: 1;
  padding-right: 20rpx;
}

.location-title {
  font-size: 28rpx;
  font-weight: 700;
  color: #3a332d;
}

.location-address {
  margin-top: 10rpx;
  font-size: 24rpx;
  line-height: 1.7;
  color: #7d7368;
}

.location-right {
  padding: 12rpx 20rpx;
```

```
  border-radius: 999rpx;
  background: #b78366;
  color: #fff;
  font-size: 24rpx;
}

.comment-section {
  margin-top: 36rpx;
}

.comment-head {
  font-size: 30rpx;
  font-weight: 700;
  color: #2f2a25;
}

.comment-input-wrap {
  margin-top: 18rpx;
  display: flex;
  gap: 16rpx;
}

.comment-input {
  flex: 1;
  background: #f7f4ef;
  border-radius: 999rpx;
  padding: 18rpx 26rpx;
  font-size: 26rpx;
}

.comment-send {
  padding: 18rpx 28rpx;
  border-radius: 999rpx;
  background: #6c9b57;
  color: #fff;
  font-size: 26rpx;
}

.comment-item {
  margin-top: 20rpx;
  padding: 22rpx;
  border-radius: 22rpx;
  background: #fbfaf8;
}

.comment-user {
  font-size: 26rpx;
  font-weight: 700;
  color: #2e2a26;
}

.comment-text {
  margin-top: 10rpx;
  font-size: 26rpx;
  line-height: 1.7;
  color: #4e473f;
}

.comment-time {
  margin-top: 12rpx;
  font-size: 22rpx;
  color: #9a9085;
```

```
}

.no-comment {
  margin-top: 18rpx;
  font-size: 24rpx;
  color: #9a9085;
}

.action-bar {
  position: fixed;
  left: 20rpx;
  right: 20rpx;
  bottom: 20rpx;
  display: flex;
  align-items: center;
  gap: 12rpx;
  padding: 16rpx;
  background: rgba(255, 255, 255, 0.96);
  border-radius: 999rpx;
  box-shadow: 0 8rpx 28rpx rgba(0, 0, 0, 0.08);
}

.action-item {
  flex: 1;
  text-align: center;
  padding: 16rpx 0;
  border-radius: 999rpx;
  background: #f7f4ef;
  color: #4a433b;
  font-size: 24rpx;
}

.favoriteAction {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 8rpx;
}

.favoriteActionIcon {
  width: 28rpx;
  height: 28rpx;
  flex-shrink: 0;
}

.share-btn {
  flex: 1;
  margin: 0;
  padding: 16rpx 0;
  line-height: 1.3;
  border-radius: 999rpx;
  background: #b78366;
  color: #fff;
  font-size: 24rpx;
}

.share-btn::after {
  border: none;
}
```

Path: miniprogram/pages/askXiaohe/askXiaohe.js

```
Page({
  data: {
    text: {
      title: "   ",
      subtitle: "                ",
      introTitle: "           ",
      introDesc:
        "                                                                ",
      primaryAction: "      ",
      secondaryAction: "      ",
      sectionTitle: "        ",
    },
    exampleQuestions: [
      "                  ",
      "                   ",
      "               ",
    ],
  },

  goBack() {
    if (getCurrentPages().length > 1) {
      wx.navigateBack();
      return;
    }

    wx.navigateTo({
      url: "/pages/search/search",
    });
  },

  goChat() {
    wx.navigateTo({
      url: "/pages/askXiaoheChat/askXiaoheChat",
    });
  },

  goSearch() {
    wx.navigateTo({
      url: "/pages/search/search",
    });
  },

  useExample(e) {
    const question = e.currentTarget.dataset.question || "";
    if (!question) return;

    wx.navigateTo({
      url: `/pages/askXiaoheChat/askXiaoheChat?q=${encodeURIComponent(question)}`,
    });
  },
});
```

Path: miniprogram/pages/askXiaohe/askXiaohe.json

```
1 {
2   "navigationBarTitleText": "   "
3 }
```

Path: miniprogram/pages/askXiaohe/askXiaohe.wxml

```
<view class="page">
  <view class="ambient ambientOne"></view>
  <view class="ambient ambientTwo"></view>

  <view class="topbar">
    <view class="backBtn" bindtap="goBack"> </view>
    <view class="titleWrap">
      <view class="title">{{text.title}}</view>
      <view class="subtitle">{{text.subtitle}}</view>
    </view>
  </view>

  <view class="heroCard">
    <view class="heroEyebrow">   </view>
    <view class="heroTitle">{{text.introTitle}}</view>
    <view class="heroDesc">{{text.introDesc}}</view>

    <view class="actionRow">
      <view class="primaryAction" bindtap="goChat">{{text.primaryAction}}</view>
      <view class="secondaryAction" bindtap="goSearch">{{text.secondaryAction}}</view>
    </view>
  </view>

  <view class="exampleCard">
    <view class="sectionTitle">{{text.sectionTitle}}</view>
    <view class="exampleList">
      <view
        class="exampleItem"
        wx:for="{{exampleQuestions}}"
        wx:key="*this"
        data-question="{{item}}"
        bindtap="useExample"
      >
        <view class="exampleDot"></view>
        <view class="exampleText">{{item}}</view>
      </view>
    </view>
  </view>
</view>
```

Path: miniprogram/pages/askXiaohe/askXiaohe.wxss

```
.page {
  position: relative;
  min-height: 100vh;
  padding: 28rpx 24rpx 40rpx;
  box-sizing: border-box;
  background:
    radial-gradient(circle at top left, rgba(253, 210, 149, 0.35) 0, rgba(253, 210, 149, 0) 32%),
    linear-gradient(180deg, #f6f1e7 0%, #fcfaf5 38%, #ffffff 100%);
  overflow: hidden;
}

.ambient {
  position: absolute;
  border-radius: 999rpx;
  filter: blur(10rpx);
  opacity: 0.7;
}

.ambientOne {
  top: 96rpx;
  right: -60rpx;
  width: 220rpx;
  height: 220rpx;
  background: rgba(134, 173, 96, 0.16);
}

.ambientTwo {
  top: 360rpx;
  left: -40rpx;
  width: 180rpx;
  height: 180rpx;
  background: rgba(214, 141, 79, 0.14);
}

.topbar {
  position: relative;
  display: flex;
  align-items: center;
  gap: 22rpx;
  z-index: 1;
}

.backBtn {
  width: 64rpx;
  height: 64rpx;
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 28rpx;
  font-weight: 700;
  color: #2d3423;
  background: rgba(255, 255, 255, 0.82);
  box-shadow: 0 10rpx 22rpx rgba(88, 75, 53, 0.08);
}

.titleWrap {
  display: flex;
  flex-direction: column;
```

```
}

.title {
  font-size: 40rpx;
  font-weight: 700;
  color: #2f302a;
}

.subtitle {
  margin-top: 8rpx;
  font-size: 24rpx;
  color: #7b786c;
}

.heroCard,
.exampleCard {
  position: relative;
  z-index: 1;
  border-radius: 32rpx;
  background: rgba(255, 255, 255, 0.88);
  box-shadow: 0 24rpx 48rpx rgba(118, 96, 66, 0.08);
  backdrop-filter: blur(10rpx);
}

.heroCard {
  margin-top: 34rpx;
  padding: 40rpx 32rpx 36rpx;
}

.heroEyebrow {
  display: inline-flex;
  padding: 10rpx 18rpx;
  border-radius: 999rpx;
  background: rgba(113, 146, 80, 0.12);
  color: #5f7f3e;
  font-size: 22rpx;
  font-weight: 700;
  letter-spacing: 2rpx;
}

.heroTitle {
  margin-top: 24rpx;
  font-size: 44rpx;
  line-height: 1.28;
  font-weight: 700;
  color: #2d2d28;
}

.heroDesc {
  margin-top: 18rpx;
  font-size: 28rpx;
  line-height: 1.8;
  color: #666256;
}

.actionRow {
  display: flex;
  gap: 18rpx;
  margin-top: 30rpx;
}

.primaryAction,
```

```
.secondaryAction {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  min-height: 84rpx;
  padding: 0 30rpx;
  border-radius: 999rpx;
  font-size: 28rpx;
  font-weight: 700;
}

.primaryAction {
  color: #ffffff;
  background: linear-gradient(90deg, #6c8f43 0%, #88a95c 100%);
  box-shadow: 0 14rpx 26rpx rgba(108, 143, 67, 0.22);
}

.secondaryAction {
  color: #5e574a;
  background: #f2ece1;
}

.exampleCard {
  margin-top: 28rpx;
  padding: 32rpx 28rpx;
}

.sectionTitle {
  font-size: 30rpx;
  font-weight: 700;
  color: #2d2d28;
}

.exampleList {
  margin-top: 20rpx;
}

.exampleItem {
  display: flex;
  align-items: flex-start;
  gap: 16rpx;
  padding: 20rpx 0;
  border-bottom: 1rpx solid rgba(93, 85, 66, 0.08);
}

.exampleItem:last-child {
  border-bottom: none;
  padding-bottom: 0;
}

.exampleDot {
  width: 14rpx;
  height: 14rpx;
  margin-top: 14rpx;
  border-radius: 50%;
  background: #7d9d58;
  flex-shrink: 0;
}

.exampleText {
  flex: 1;
  font-size: 28rpx;
```

```
  line-height: 1.7;
  color: #5d584d;
}
```

Path: miniprogram/pages/askXiaoheChat/askXiaoheChat.js

```
---------------------------------------------------------------------------
const {
  yuxiaoheBotId,
  yuxiaoheHttpBaseUrl
} = require("../../config/agent")
const { resolveActivityCover, resolveActivityGallery } = require("../../utils/mediaAsset
s")
const { buildActivityCoverTags } = require("../../utils/activityCoverTags")
const { createBuddyApplicationFromMatch } = require("../../utils/messageStore")
const {
  buildConversationTitle,
  createConversationId,
  getConversationById,
  saveConversation
} = require("../../utils/askConversationStore")

const DEFAULT_ACTIVITY_COVER = "/images/nav-academy.png"
const DEFAULT_PRODUCT_COVER = "/images/default-goods-image.png"
const LOADING_BASE_TEXT = "           "

function normalizeSkillMode(skillMode = "") {
  return skillMode === "route_planning" ? "guide_customization" : skillMode
}

const SKILL_CONFIG = {
  guide_customization: {
    badgeName: "    ",
    placeholder: "             ",
    intro: "                                                    "
  },
  buddy_matching: {
    badgeName: "   ",
    placeholder: "             ",
    intro: "                                                        "
  },
  xiaohe_feedback: {
    badgeName: "    ",
    placeholder: "      ",
    intro: "                                        "
  }
}

const DEFAULT_BUDDY_TAGS = ["    ", "   ", "   "]

const GENERIC_BUDDY_STATUS_TAGS = ["    "]
const GENERIC_BUDDY_TAGS = ["     "]
const GENERIC_BUDDY_HIGHLIGHT_KEYWORDS = ["    ", "     ", "     "]
const GENERIC_BUDDY_PRACTICAL_VALUES = ["     ", "   "]

function normalizeText(value) {
  return String(value || "").trim()
}

function normalizeArray(value) {
  return Array.isArray(value)
    ? value.map((item) => normalizeText(item)).filter(Boolean)
    : []
}

function normalizeTextLower(value) {
  return normalizeText(value).toLowerCase()
```

```
}

function getSettingAsync() {
  return new Promise((resolve, reject) => {
    wx.getSetting({
      success: resolve,
      fail: reject
    })
  })
}

function getLocationAsync(timeout = 8000) {
  return new Promise((resolve, reject) => {
    const timer = setTimeout(() => {
      reject(new Error("getLocation timeout"))
    }, timeout)

    wx.getLocation({
      type: "gcj02",
      success: (res) => {
        clearTimeout(timer)
        resolve(res)
      },
      fail: (error) => {
        clearTimeout(timer)
        reject(error)
      }
    })
  })
}

function readSelectedRegion() {
  try {
    return wx.getStorageSync("selectedRegion") || null
  } catch (error) {
    return null
  }
}

function buildLocationContext(userInfo = {}, userLocation = null, selectedRegion = null)
{
  const region = selectedRegion || {}
  const location = userLocation || {}

  return {
    province: region.province || userInfo.province || "",
    city: region.city || userInfo.city || "",
    district: region.district || userInfo.district || "",
    displayName: region.displayName || region.title || region.label || "",
    locationText: region.locationText || region.address || "",
    latitude: location.latitude || region.latitude || "",
    longitude: location.longitude || region.longitude || ""
  }
}

function ensureBuddySession() {
  const app = getApp()
  if (app.hasActiveSession && app.hasActiveSession()) {
    return true
  }

  wx.showToast({
```

```
    title: "         ",
    icon: "none"
  })

  setTimeout(() => {
    wx.navigateTo({
      url: "/pages/login/login"
    })
  }, 250)

  return false
}

function uniqueList(list = []) {
  return Array.from(new Set((list || []).filter(Boolean)))
}

function requestAgentHttp({ path = "/", method = "GET", data = {} } = {}) {
  if (!yuxiaoheHttpBaseUrl) {
    return Promise.reject(new Error("yuxiaoheHttpBaseUrl is required"))
  }

  const normalizedPath = path.startsWith("/") ? path : `/${path}`
  return new Promise((resolve, reject) => {
    wx.request({
      url: `${yuxiaoheHttpBaseUrl}${normalizedPath}`,
      method,
      data,
      header: {
        "content-type": "application/json"
      },
      success: (res) => {
        const statusCode = Number(res && res.statusCode)
        if (statusCode >= 200 && statusCode < 300) {
          resolve(res.data)
          return
        }

        const error = new Error(`request failed with status ${statusCode}`)
        error.statusCode = statusCode
        error.responseData = res && res.data
        try {
          console.error("[askXiaoheChat] requestAgentHttp non-2xx", {
            path: normalizedPath,
            method,
            statusCode,
            responseData: res && res.data
          })
        } catch (logError) {
          console.warn("[askXiaoheChat] inspect requestAgentHttp non-2xx failed", logError)
        }
        reject(error)
      },
      fail: reject
    })
  })
}

function filterBuddyStatusTag(tag = "") {
  const text = normalizeText(tag)
  return GENERIC_BUDDY_STATUS_TAGS.includes(text) ? "" : text
```

```
}

function filterBuddyTags(tags = []) {
  return uniqueList(normalizeArray(tags).filter((tag) => !GENERIC_BUDDY_TAGS.includes(tag)))
}

function filterBuddyHighlights(items = []) {
  return normalizeArray(items).filter(
    (item) => !GENERIC_BUDDY_HIGHLIGHT_KEYWORDS.some((keyword) => item.includes(keyword))
  )
}

function filterBuddyPracticalInfo(list = []) {
  return (Array.isArray(list) ? list : []).filter((item) => {
    const value = normalizeText(item && item.value)
    return value && !GENERIC_BUDDY_PRACTICAL_VALUES.includes(value)
  })
}

function buildBuddyDisplaySummary(item = {}) {
  const tags = filterBuddyTags(item.tags)
  if (tags.length) {
    return `     ${tags.slice(0, 3).join(" ")}`
  }

  const practicalInfo = filterBuddyPracticalInfo(item.practicalInfo)
  const locationInfo = practicalInfo.find((entry) => normalizeText(entry && entry.label).includes("  "))
  if (locationInfo && locationInfo.value) {
    return `       ${locationInfo.value}`
  }

  return "                                    "
}

function buildBuddySystemReason(item = {}) {
  const parts = []
  const statusTag = filterBuddyStatusTag(item.statusTag)
  const tags = filterBuddyTags(item.tags)

  if (statusTag) {
    parts.push(statusTag)
  }

  if (tags.length) {
    parts.push(`     ${Math.min(tags.length, 3)}  `)
  }

  if (item.matchScore) {
    parts.push(`    ${item.matchScore}%`)
  }

  return `       ${parts.join(" + ") || "                      "}`
}

function normalizeBuddyRecommendation(item = {}) {
  const tags = filterBuddyTags(item.tags)
  const practicalInfo = filterBuddyPracticalInfo(item.practicalInfo)

  return {
```

```
    ...item,
    statusTag: filterBuddyStatusTag(item.statusTag),
    summary: normalizeBuddySummaryText(item.summary, normalizeText(item.region)),
    matchReason: normalizeText(item.matchReason) || buildBuddySystemReason({ ...item, tags }),
    reasonTitle: "      ",
    tags,
    playItems: filterBuddyHighlights(item.playItems),
    practicalInfo,
  }
}

function trimRegionSuffix(value = "") {
  return normalizeText(value).replace(/(     |   |   |  | | | | | | )$/u, "")
}

function buildBuddyAvatarText(name = "") {
  const text = normalizeText(name)
  return text ? text.slice(0, 1).toUpperCase() : " "
}

function buildBuddyAvatarColor(name = "") {
  const palette = ["#7A8CFF", "#8B6BFF", "#4FA7A7", "#F08A5D", "#6C9A8B"]
  const text = normalizeText(name)
  if (!text) return palette[0]

  const codeSum = text.split("").reduce((sum, char) => sum + char.charCodeAt(0), 0)
  return palette[codeSum % palette.length]
}

function buildBuddyOpeningText(item = {}) {
  const name = normalizeText(item.title || item.nickname) || "    "
  const region = normalizeText(item.region)
  const matchReason = normalizeText(item.matchReason)
  if (region && matchReason) {
    return `   ${name}         ${region}           “${matchReason}”
        `
  }
  if (region) {
    return `   ${name}         ${region}                           `
  }
  return `   ${name}                                `
}

function buildBuddyPracticalInfoFromCandidate(item = {}) {
  const info = []
  const region = normalizeText(item.region)
  if (region && !/   |   |  /u.test(region)) {
    info.push({ label: "    ", value: region })
  }
  return info
}

function buildBuddyHighlightsFromCandidate(item = {}) {
  const tags = filterBuddyTags(item.tags)
  if (tags.length) {
    return tags.slice(0, 3)
  }

  return ["      ", "              "]
}

```

```
function normalizeBuddySummaryText(summary = "", region = "") {
  const text = normalizeText(summary)
  if (
    !text ||
    /       |          |     |    /u.test(text)
  ) {
    if (region && !/   |   |  /u.test(region)) {
      return `      ${region}                    `
    }
    return "                          "
  }

  return text
}

function createMessage(role, text, extra = {}) {
  return {
    id: `${role}-${Date.now()}-${Math.floor(Math.random() * 1000)}`,
    role,
    text,
    recommendations: [],
    tips: "",
    guessQuestions: [],
    followUp: "",
    channelLabel: "",
    ...extra
  }
}

function buildSkillContext(skillMode = "") {
  if (!skillMode || !SKILL_CONFIG[skillMode]) {
    return null
  }

  return {
    mode: skillMode,
    title: SKILL_CONFIG[skillMode].badgeName,
    collected: {}
  }
}

const MAINLINE_BY_SKILL_MODE = {
  buddy_matching: "buddy_matching",
  guide_customization: "guide_customization",
  xiaohe_feedback: "xiaohe_feedback"
}

function createEmptyTaskState(mainline = "") {
  return {
    mainline: normalizeText(mainline),
    subType: "",
    collected: {},
    missingField: "",
    lastAskedField: "",
    candidateIds: [],
    feedbackType: "",
    intentConfidence: mainline ? 1 : 0
  }
}

function createInitialTaskState(skillMode = "") {
  const mainline = MAINLINE_BY_SKILL_MODE[normalizeText(skillMode)] || ""
```

```
  return createEmptyTaskState(mainline)
}

function buildTaskStateFromBackend(result = {}, fallbackState = {}) {
  const collected = result && result.collected && typeof result.collected === "object"
    ? { ...result.collected }
    : { ...(fallbackState.collected || {}) }
  const missingField = result && result.missingField && typeof result.missingField === "object"
    ? normalizeText(result.missingField.key || result.missingField.label)
    : normalizeText(result && result.missingField)

  return {
    ...createEmptyTaskState(normalizeText(result && result.mainline)),
    ...fallbackState,
    mainline: normalizeText(result && result.mainline) || normalizeText(fallbackState.mainline),
    collected,
    missingField,
    lastAskedField: normalizeText(result && result.lastAskedField) || missingField || normalizeText(fallbackState.lastAskedField),
    candidateIds: normalizeArray((result.candidates || []).map((item) => item && item.id)),
    feedbackType: normalizeText(result && result.feedbackType) || normalizeText(fallbackState.feedbackType),
    intentConfidence: 1
  }
}

function buildGuideCardIntro(taskState = {}) {
  const collected = (taskState && taskState.collected) || {}
  const time = normalizeText(collected.time)
  const relationship = normalizeText(collected.relationship)
  const destination = normalizeText(collected.destination)
  const budget = normalizeText(collected.budget)

  const segments = [
    time ? `${time}  ` : "",
    relationship ? ` ${relationship}` : "",
    destination ? ` ${destination}` : "",
    budget ? `   ${budget}` : ""
  ].filter(Boolean)

  if (!segments.length) {
    return "                        "
  }

  return `       ${segments.join(" ")}                    `
}

function buildGuideWarmTips(taskState = {}) {
  const collected = (taskState && taskState.collected) || {}
  const destination = normalizeText(collected.destination) || "   "
  const time = normalizeText(collected.time)
  const relationship = normalizeText(collected.relationship)
  const budget = normalizeText(collected.budget)

  const tips = []

  if (time) {
    tips.push(`${time}  ${destination}                              `)
  } else {
```

```
    tips.push(` ${destination}                    `)
  }

  if (relationship) {
    tips.push(`       ${relationship}                           `)
  } else if (budget) {
    tips.push(`   ${budget}                           `)
  } else {
    tips.push("                                 ")
  }

  return ["    "]
    .concat(tips.slice(0, 2).map((item, index) => `${index + 1}. ${item}`))
    .join("\n")
}

function buildGuideFallbackRecommendation(item = {}) {
  const type = normalizeText(item && item.type)
  const sourceId = normalizeText(item && item.id)
  if (!type || !sourceId) return null

  const practicalInfo = []
  const region = normalizeText(item && item.region)
  const priceText = normalizeText(item && item.priceText)

  if (region) {
    practicalInfo.push({ label: "    ", value: region })
  }

  if (priceText) {
    practicalInfo.push({ label: "    ", value: priceText })
  }

  return {
    id: `${type}-${sourceId}`,
    sourceId,
    type,
    title: normalizeText(item && item.title) || "    ",
    summary: normalizeText(item && item.summary) || "                ",
    gallery: [],
    tags: normalizeArray(item && item.tags).slice(0, 4),
    playItems: [],
    practicalInfo
  }
}

function buildGuideFocusItems(taskState = {}, candidate = {}, item = {}, type = "") {
  const collected = (taskState && taskState.collected) || {}
  const destination = normalizeText(collected.destination)
  const time = normalizeText(collected.time)
  const relationship = normalizeText(collected.relationship)
  const budget = normalizeText(collected.budget)
  const region = normalizeText(candidate.region || item.region || item.locationName)
  const tags = uniqueList(
    normalizeArray(item.highlights)
      .concat(normalizeArray(item.itinerary))
      .concat(normalizeArray(item.playTags))
      .concat(normalizeArray(item.categoryTags))
      .concat(normalizeArray(item.tags))
      .concat(normalizeArray(item.suitableGroups))
      .concat(normalizeArray(candidate.tags))
  )
```

```

  const focusItems = []

  if (destination && region) {
    focusItems.push(`    ${region}     ${destination}       `)
  }

  if (relationship) {
    focusItems.push(`  ${type === "product" ? "  " : "  "}    ${relationship}
    `)
  }

  if (time) {
    focusItems.push(`${time}                                `)
  }

  if (budget) {
    const priceText = normalizeText(candidate.priceText || item.priceText || item.price)
    focusItems.push(priceText ? `       ${priceText}        ${budget}        ` : `
      ${budget}           `)
  }

  tags.forEach((tag) => {
    if (focusItems.length < 4 && normalizeText(tag)) {
      focusItems.push(normalizeText(tag))
    }
  })

  return uniqueList(focusItems).slice(0, 4)
}

function buildGuidePracticalInfo(taskState = {}, candidate = {}, item = {}, type = "") {
  const collected = (taskState && taskState.collected) || {}
  const destination = normalizeText(collected.destination)
  const time = normalizeText(collected.time)
  const relationship = normalizeText(collected.relationship)
  const region = normalizeText(candidate.region || item.region || item.locationName || i
tem.address)
  const priceText = normalizeText(candidate.priceText || item.priceText || item.price)

  const info = []

  if (region) {
    info.push({ label: "    ", value: region })
  } else if (destination) {
    info.push({ label: "    ", value: destination })
  }

  if (priceText) {
    info.push({ label: type === "hotel" ? "    " : "    ", value: priceText })
  }

  if (time) {
    info.push({ label: "    ", value: `${time}              ` })
  }

  if (relationship) {
    info.push({ label: "    ", value: ` ${relationship}                 ` })
  }

  if (!info.length) {
    if (type === "activity") return getActivityPracticalInfo(item)
```

```
    if (type === "scenic") return getScenicPracticalInfo(item)
    if (type === "product") return getProductPracticalInfo(item)
  }

  return info.slice(0, 4)
}

async function buildActivityRecommendationCard(item = {}, candidate = {}, taskState = {}) {
  const coverTagInfo = buildActivityCoverTags(item)
  const playItems = buildGuideFocusItems(taskState, candidate, item, "activity")
  const gallery = await resolveActivityGallery(item)

  return {
    id: normalizeText(item._id || item.id || candidate.id),
    sourceId: normalizeText(item._id || item.id || candidate.id),
    type: "activity",
    title: normalizeText(item.title || candidate.title) || "    ",
    summary: normalizeText(candidate.summary || item.summary || item.content || item.detail) || "            ",
    gallery: Array.isArray(gallery) ? gallery.filter(Boolean).slice(0, 2) : [],
    tags: coverTagInfo.combinedTags.slice(0, 4),
    playItems: playItems.length ? playItems : ["       "],
    practicalInfo: buildGuidePracticalInfo(taskState, candidate, item, "activity"),
    cover: await resolveActivityCover(item)
  }
}

function buildScenicRecommendationCard(item = {}, candidate = {}, taskState = {}) {
  const cover = normalizeText(item.cover) || DEFAULT_ACTIVITY_COVER
  const gallery = Array.isArray(item.gallery) && item.gallery.length
    ? item.gallery.filter(Boolean).slice(0, 2)
    : [cover]
  const playItems = buildGuideFocusItems(taskState, candidate, item, "scenic")

  return {
    id: normalizeText(item._id || item.id || candidate.id),
    sourceId: normalizeText(item._id || item.id || candidate.id),
    type: "scenic",
    title: normalizeText(item.locationName || item.title || candidate.title) || "    ",
    summary: normalizeText(candidate.summary || item.summary || item.content || item.detail) || "            ",
    gallery,
    tags: uniqueList(normalizeArray(item.playTags).concat(normalizeArray(item.tags))).slice(0, 4),
    playItems: playItems.length ? playItems : ["       "],
    practicalInfo: buildGuidePracticalInfo(taskState, candidate, item, "scenic"),
    cover
  }
}

function buildProductRecommendationCard(item = {}, candidate = {}, taskState = {}) {
  const cover = normalizeText(item.cover) || DEFAULT_PRODUCT_COVER
  const playItems = buildGuideFocusItems(taskState, candidate, item, "product")

  return {
    id: normalizeText(item._id || item.id || candidate.id),
    sourceId: normalizeText(item._id || item.id || candidate.id),
    type: "product",
    title: normalizeText(item.title || candidate.title) || "    ",
    summary: normalizeText(candidate.summary || item.summary || item.content || item.detail) || "            ",
```

```
    gallery: [cover],
    tags: uniqueList(normalizeArray(item.categoryTags).concat(normalizeArray(item.tags)).slice(0, 4),
    playItems: playItems.length ? playItems : ["       "],
    practicalInfo: buildGuidePracticalInfo(taskState, candidate, item, "product"),
    cover
  }
}

function buildHotelRecommendationCard(item = {}, candidate = {}, taskState = {}) {
  const cover = normalizeText(item.cover || item.image || item.poster) || DEFAULT_ACTIVITY_COVER
  const gallery = Array.isArray(item.images) && item.images.length
    ? item.images.filter(Boolean).slice(0, 2)
    : [cover]

  return {
    id: normalizeText(item._id || item.id || candidate.id),
    sourceId: normalizeText(item._id || item.id || candidate.id),
    type: "hotel",
    title: normalizeText(item.name || item.title || candidate.title) || "    ",
    summary: normalizeText(candidate.summary || item.summary || item.description || item.detail) || "            ",
    gallery,
    tags: uniqueList(normalizeArray(item.tags)).slice(0, 4),
    playItems: buildGuideFocusItems(taskState, candidate, item, "hotel"),
    practicalInfo: buildGuidePracticalInfo(taskState, candidate, item, "hotel"),
    cover
  }
}

function readCollectedText(collected = {}, keys = []) {
  for (const key of keys) {
    const value = normalizeText(collected && collected[key])
    if (value) return value
  }
  return ""
}

function extractTaskDestination(text = "", collected = {}) {
  const existing = readCollectedText(collected, ["destination"])
  if (existing) return existing

  const normalized = normalizeText(text)
  const patterns = [
    / ([\u4e00-\u9fa5A-Za-z0-9]{2,12})/u,
    / ([\u4e00-\u9fa5A-Za-z0-9]{2,12})/u,
    / ([\u4e00-\u9fa5A-Za-z0-9]{2,12})(?: | |  |  )/u
  ]

  for (const pattern of patterns) {
    const matched = normalized.match(pattern)
    if (matched && matched[1]) return matched[1]
  }

  if (/^[\u4e00-\u9fa5A-Za-z0-9]{2,12}$/u.test(normalized)) {
    return normalized
  }

  return ""
}
```

```
function extractTaskTime(text = "", collected = {}) {
  const existing = readCollectedText(collected, ["time"])
  if (existing) return existing

  const matched = normalizeText(text).match(
    /(  |  |  |  |  |  |  |  |  |  |  |  | [        ]|\d{1,2} \d{1,2} |\d{1,2} )/u
  )
  return matched && matched[1] ? matched[1] : ""
}

function extractBuddyCompanionPreference(text = "", collected = {}) {
  const existing = readCollectedText(collected, ["companionPreference"])
  if (existing) return existing

  const normalized = normalizeText(text)
  const directPatterns = [
    /(    |    |    |    |    |   |     )/u,
    /(   |   |   |   |   |    |    |   |   )/u
  ]

  for (const pattern of directPatterns) {
    const matched = normalized.match(pattern)
    if (matched && matched[1]) return matched[1]
  }

  const normalizedGender =
    /(  |   )/u.test(normalized) ? "  " : /(  |   )/u.test(normalized) ? "  " : ""
  const normalizedTrait =
    /(   |   |  )/u.test(normalized)
      ? "   "
      : /(   |   )/u.test(normalized)
        ? "   "
        : /   /u.test(normalized)
          ? "   "
          : /   /u.test(normalized)
            ? "   "
            : /   /u.test(normalized)
              ? "   "
              : /   /u.test(normalized)
                ? "   "
                : /  /u.test(normalized)
                  ? "    "
                  : /  /u.test(normalized)
                    ? "    "
                    : ""

  if (normalizedTrait && normalizedGender) {
    return `${normalizedTrait} ${normalizedGender}`
  }

  if (normalizedTrait) return normalizedTrait
  if (normalizedGender) return normalizedGender

  return ""
}

function extractGuidePeopleCount(text = "", collected = {}) {
  const existing = readCollectedText(collected, ["peopleCount"])
  if (existing) return existing

  const normalized = normalizeText(text)
  const matched = normalized.match(/([           \d]+(?: | ))/u)
```

```
  if (matched && matched[1]) return matched[1]

  if (/^  /u.test(normalized)) {
    return "2 "
  }

  return ""
}

function extractGuideRelationship(text = "", collected = {}) {
  const existing = readCollectedText(collected, ["relationship"])
  if (existing) return existing

  const normalized = normalizeText(text)
  const matched = normalized.match(/(  |  |  |  |  |  |  |  |   |  |   |   |  |  |  )/u)
  if (matched && matched[1]) {
    return /(   |   |  |  |  )/u.test(matched[1]) ? "  " : matched[1]
  }

  if (/(   |   |  |  |  )/u.test(normalized)) {
    return "  "
  }

  return ""
}

function extractGuideBudget(text = "", collected = {}) {
  const existing = readCollectedText(collected, ["budget"])
  if (existing) return existing

  const matched = normalizeText(text).match(/(\d{2,5}\s*(?: | |w| ))/u)
  return matched && matched[1] ? matched[1] : ""
}

function buildBuddyCollectedFields(text = "", prevCollected = {}, location = {}) {
  return {
    departure:
      readCollectedText(prevCollected, ["departure"]) ||
      normalizeText(location && (location.city || location.displayName)) ||
      "",
    destination: extractTaskDestination(text, prevCollected),
    time: extractTaskTime(text, prevCollected),
    companionPreference: extractBuddyCompanionPreference(text, prevCollected)
  }
}

function buildGuideCollectedFields(text = "", prevCollected = {}) {
  return {
    time: extractTaskTime(text, prevCollected),
    peopleCount: extractGuidePeopleCount(text, prevCollected),
    relationship: extractGuideRelationship(text, prevCollected),
    budget: extractGuideBudget(text, prevCollected),
    destination: extractTaskDestination(text, prevCollected)
  }
}

function pickMissingField(mainline = "", collected = {}) {
  const fieldOrderMap = {
    buddy_matching: ["destination", "time", "departure", "companionPreference"],
    guide_customization: ["destination", "time", "peopleCount", "relationship", "budget"
  ]
  }
```

```
780
781   const order = fieldOrderMap[mainline] || []
782   return order.find((field) => !normalizeText(collected && collected[field])) || ""
783 }
784
785 function classifyFeedbackType(text = "") {
786   const source = normalizeText(text)
787   if (!source) return ""
788   if (/(  |  |   |    |  )/u.test(source)) return "platform_suggestion"
789   if (/(  | |bug|  |   |   |  )/u.test(source)) return "product_feedback"
790   if (/( |  |  |  |emo|  | )/u.test(source)) return "emotion_support"
791   return "experience_complaint"
792 }
793
794 function inferMainlineFromText(text = "") {
795   const source = normalizeText(text)
796   if (!source) return ""
797   if (/(  |  |  |  |  |   |   |   |     |    |  |    )/u.test(source)) {
798     return "weather_location"
799   }
800   if (/(  |  |   |      |    |   )/u.test(source)) {
801     return "buddy_matching"
802   }
803   if (/(  |  |  |   |  |  |  | .+ )/u.test(source)) {
804     return "guide_customization"
805   }
806   if (/(  |  |  |  |  |  |  |  |emo|  |  |   | |  )/u.test(source)) {
807     return "xiaohe_feedback"
808   }
809   return ""
810 }
811
812 function buildSkillOpeningQuestion(skillMode = "") {
813   if (skillMode === "guide_customization") {
814     return "      “    ”
                                  "
815   }
816
817   if (skillMode === "buddy_matching") {
818     return "      “   ”       “  ”
                                    "
819   }
820
821   if (skillMode === "xiaohe_feedback") {
822     return "      “    ”                                                    "
823   }
824
825   return ""
826 }
827
828 function toAgentHistory(messages = []) {
829   return (messages || [])
830     .map((item) => {
831       const content = normalizeText(item && item.text)
832       if (!content) return null
833       return {
834         role: item.role === "user" ? "user" : "assistant",
835         content
836       }
837     })
838     .filter(Boolean)
839     .slice(-12)
```

```
}

function detectRecommendationType(question = "") {
  const text = normalizeText(question)
  if (!text) return ""

  if (/(  |  |  |   |  |  |  )/u.test(text)) return "product"
  if (/(  |  |  |  |  |  |  |  |  |  |  )/u.test(text)) return "scenic"
  if (/(  |  |  |  |  |  |  |  )/u.test(text)) return "activity"

  return ""
}

function buildIntentTokensByType(question = "", type = "") {
  const source = normalizeText(question)
  const keywordMap = {
    activity: ["  ", "  ", "  ", "  ", "  ", "  ", "  ", "  ", "  ", "  "],
    scenic: ["  ", "  ", "  ", "  ", "  ", "  ", "  ", "  ", "  ", "  "],
    product: ["  ", "   ", "  ", "  ", "  ", "  ", "  ", "  ", "  ", "  "]
  }

  return uniqueList((keywordMap[type] || []).filter((keyword) => source.includes(keyword
))) }

function buildLocationTokens(location = {}) {
  return uniqueList([
    trimRegionSuffix(location.province),
    trimRegionSuffix(location.city),
    trimRegionSuffix(location.district),
    trimRegionSuffix(location.displayName),
    trimRegionSuffix(location.locationText)
  ].filter(Boolean))
}

function joinActivitySearchText(item = {}) {
  return normalizeTextLower([
    item.title,
    item.summary,
    item.content,
    item.detail,
    item.locationName,
    item.province,
    item.city,
    item.district,
    item.transport,
    item.stay
  ]
    .concat(normalizeArray(item.tags))
    .concat(normalizeArray(item.travelModeTags))
    .concat(normalizeArray(item.playTags))
    .concat(normalizeArray(item.suitableGroups))
    .concat(normalizeArray(item.highlights))
    .concat(normalizeArray(item.itinerary))
    .join(" "))
}

function joinScenicSearchText(item = {}) {
  return normalizeTextLower([
    item.title,
    item.locationName,
    item.summary,
```

```
    item.content,
    item.detail,
    item.province,
    item.city,
    item.district,
    item.tips,
    item.openTime
  ]
    .concat(normalizeArray(item.tags))
    .concat(normalizeArray(item.playTags))
    .concat(normalizeArray(item.suitableGroups))
    .concat(normalizeArray(item.highlights))
    .join(" "))
}

function joinProductSearchText(item = {}) {
  return normalizeTextLower([
    item.title,
    item.locationName,
    item.summary,
    item.content,
    item.detail,
    item.province,
    item.city,
    item.district
  ]
    .concat(normalizeArray(item.tags))
    .concat(normalizeArray(item.categoryTags))
    .concat(normalizeArray(item.highlights))
    .concat(normalizeArray(item.suitableGroups))
    .join(" "))
}

function scoreByQuestion(text, question = "", location = {}, type = "") {
  const intentTokens = buildIntentTokensByType(question, type)
  const locationTokens = buildLocationTokens(location)
  let score = 0

  intentTokens.forEach((token) => {
    if (text.includes(normalizeTextLower(token))) {
      score += 12
    }
  })

  locationTokens.forEach((token, index) => {
    if (token && text.includes(normalizeTextLower(token))) {
      score += 16 - Math.min(index * 2, 8)
    }
  })

  if (/(  |  )/u.test(normalizeText(question)) && !locationTokens.length) {
    score -= 4
  }

  return score
}

function getActivityPlayItems(item = {}) {
  return uniqueList(
    normalizeArray(item.highlights)
      .concat(normalizeArray(item.itinerary))
      .concat(normalizeArray(item.playTags))
```

```
      .concat(normalizeArray(item.suitableGroups))
  ).slice(0, 4)
}

function getActivityPracticalInfo(item = {}) {
  const addressText = normalizeText(
    item.locationName || [item.province, item.city, item.district].filter(Boolean).join(" · ")
  ) || "   "
  const phoneText = normalizeText(item.merchantPhone || item.contactPhone || item.phone || item.tel) || "   "

  return [
    { label: "  ", value: addressText },
    { label: "  ", value: phoneText }
  ]
}

function getScenicPlayItems(item = {}) {
  return uniqueList(
    normalizeArray(item.highlights)
      .concat(normalizeArray(item.playTags))
      .concat(normalizeArray(item.suitableGroups))
  ).slice(0, 4)
}

function getScenicPracticalInfo(item = {}) {
  const addressText = normalizeText(
    item.locationName || [item.province, item.city, item.district].filter(Boolean).join(" · ")
  ) || "   "
  const openTimeText = normalizeText(item.openTime) || "       "

  return [
    { label: "  ", value: addressText },
    { label: "    ", value: openTimeText }
  ]
}

function getProductPlayItems(item = {}) {
  return uniqueList(
    normalizeArray(item.highlights)
      .concat(normalizeArray(item.categoryTags))
      .concat(normalizeArray(item.tags))
      .concat(normalizeArray(item.suitableGroups))
  ).slice(0, 4)
}

function getProductPracticalInfo(item = {}) {
  const originText = normalizeText(
    item.locationName || [item.province, item.city, item.district].filter(Boolean).join(" · ")
  ) || "   "
  const priceText = item.price
    ? `¥${item.price}`
    : (item.priceFrom ? `¥${item.priceFrom} ` : "    ")

  return [
    { label: "  ", value: originText },
    { label: "    ", value: priceText }
  ]
}
```

```
function buildBuddyPromptSummary(question = "", userInfo = {}) {
  const dnaTags = normalizeArray(userInfo.dnaTags).slice(0, 3)
  const profileText = dnaTags.length ? `           “${dnaTags.join(" ")}”` : ""
  const questionText = normalizeText(question) || "        "
  return `    “${questionText}”${profileText}                        `
}

Page({
  data: {
    statusBarHeight: 20,
    text: {
      brand: "   ",
      fallbackQuestion: "               ",
      inputPlaceholder: "              "
    },
    source: "search_input",
    skillMode: "",
    question: "",
    conversationId: "",
    showSkillIntro: false,
    showSkillBadge: false,
    skillBadgeName: "",
    inputPlaceholder: "              ",
    introText: "",
    quickOptions: [],
    messages: [],
    inputValue: "",
    isAiLoading: false,
    currentTaskState: createEmptyTaskState(""),
    genericPreferences: {
      distance: "",
      budget: "",
      detailLevel: ""
    }
  },

  onLoad(options) {
    this.initNavMetrics()

    const source = options && options.source ? options.source : "search_input"
    const skillMode = normalizeSkillMode(options && options.skillMode ? options.skillMode : "")
    const skillConfig = SKILL_CONFIG[skillMode] || null
    const shouldShowSkillIntro = source === "skill_entry" && !!skillConfig && skillMode !== "buddy_matching"
    const question = options && options.q ? decodeURIComponent(options.q) : ""
    const conversationId = options && options.conversationId ? decodeURIComponent(options.conversationId) : ""

    if (conversationId) {
      this.restoreConversation(conversationId, source, skillMode, skillConfig)
      return
    }

    const initialQuestion = source === "skill_entry"
      ? (skillConfig ? skillConfig.badgeName : this.data.text.brand)
      : (question || this.data.text.fallbackQuestion)
    const messages = this.buildMessages({ source, question: initialQuestion })
    const nextConversationId = createConversationId()

    this.setData({
```

```
      source,
      skillMode,
      question: initialQuestion,
      conversationId: nextConversationId,
      showSkillIntro: shouldShowSkillIntro,
      showSkillBadge: source === "skill_entry" && !!skillConfig,
      skillBadgeName: skillConfig ? skillConfig.badgeName : "",
      inputPlaceholder: skillConfig ? skillConfig.placeholder : this.data.text.inputPlac
eholder,
      introText: shouldShowSkillIntro && skillConfig ? skillConfig.intro : "",
      quickOptions: [],
      messages,
      currentTaskState: createInitialTaskState(skillMode),
      genericPreferences: {
        distance: "",
        budget: "",
        detailLevel: ""
      }
    })

    this.persistConversation(nextConversationId, messages, initialQuestion)

    if (source === "skill_entry") {
      const openingQuestion = buildSkillOpeningQuestion(skillMode)
      this.requestGenericAnswer(openingQuestion, {
        contextPayload: {
          mode: "skill",
          skillContext: buildSkillContext(skillMode) || {}
        }
      })
      return
    }

    this.requestGenericAnswer(initialQuestion)
  },

  onUnload() {
    this.stopLoadingAnimation()
  },

  initNavMetrics() {
    try {
      const systemInfo = wx.getSystemInfoSync()
      this.setData({ statusBarHeight: systemInfo.statusBarHeight || 20 })
    } catch (error) {
      this.setData({ statusBarHeight: 20 })
    }
  },

  restoreConversation(conversationId, source, skillMode, skillConfig) {
    const conversation = getConversationById(conversationId)
    if (!conversation) {
      wx.showToast({
        title: "         ",
        icon: "none"
      })
      wx.navigateBack()
      return
    }

    this.setData({
      source,
```

```
      skillMode,
      question: normalizeText(conversation.firstQuestion),
      conversationId,
      showSkillIntro: false,
      showSkillBadge: false,
      skillBadgeName: skillConfig ? skillConfig.badgeName : "",
      inputPlaceholder: skillConfig ? skillConfig.placeholder : this.data.text.inputPlac
eholder,
      introText: "",
      quickOptions: [],
      messages: Array.isArray(conversation.messages) ? conversation.messages : [],
      currentTaskState: conversation.currentTaskState || createInitialTaskState(skillMod
e),
      genericPreferences: {
        ...(conversation.genericPreferences || {
          distance: "",
          budget: "",
          detailLevel: ""
        })
      }
    })
  },

  buildMessages({ source, question }) {
    if (source === "skill_entry") {
      return []
    }
    return [createMessage("user", question)]
  },

  persistConversation(conversationId, messages, firstQuestion) {
    if (!conversationId) return

    saveConversation({
      id: conversationId,
      title: buildConversationTitle(firstQuestion),
      firstQuestion,
      createdAt: getConversationById(conversationId)?.createdAt || Date.now(),
      updatedAt: Date.now(),
      messages,
      source: this.data.source,
      skillMode: this.data.skillMode,
      genericPreferences: this.data.genericPreferences,
      currentTaskState: this.data.currentTaskState
    })
  },

  syncCurrentConversation() {
    const { conversationId, messages } = this.data
    if (!conversationId || !messages.length) return

    const firstUserMessage = messages.find((item) => item.role === "user")
    const firstQuestion = normalizeText(firstUserMessage && firstUserMessage.text) || no
rmalizeText(this.data.question)
    if (!firstQuestion) return

    this.persistConversation(conversationId, messages, firstQuestion)
  },

  getCurrentSkillContext(extraSkillContext = null) {
    const baseSkillContext = buildSkillContext(this.data.skillMode) || {}
    const currentTaskState = this.data.currentTaskState || createEmptyTaskState("")
```

```
    const mergedSkillContext = {
      ...baseSkillContext,
      ...(extraSkillContext || {})
    }

    return {
      ...mergedSkillContext,
      collected: {
        ...(mergedSkillContext.collected || {}),
        ...(currentTaskState.collected || {})
      }
    }
  },

  buildNextTaskState(question, location = null) {
    const currentTaskState = this.data.currentTaskState || createEmptyTaskState("")
    const inferredMainline = inferMainlineFromText(question)
    const skillMainline =
      MAINLINE_BY_SKILL_MODE[this.data.skillMode] ||
      currentTaskState.mainline ||
      inferredMainline ||
      ""
    const nextState = {
      ...createEmptyTaskState(skillMainline),
      ...currentTaskState,
      mainline: skillMainline || currentTaskState.mainline || ""
    }

    const text = normalizeText(question)
    if (!text) {
      return nextState
    }

    if (nextState.mainline === "buddy_matching") {
      const collected = buildBuddyCollectedFields(text, nextState.collected, location)
      return {
        ...nextState,
        collected,
        missingField: pickMissingField("buddy_matching", collected),
        lastAskedField: currentTaskState.missingField || currentTaskState.lastAskedField || ""
      }
    }

    if (nextState.mainline === "guide_customization") {
      const collected = buildGuideCollectedFields(text, nextState.collected)
      return {
        ...nextState,
        collected,
        missingField: pickMissingField("guide_customization", collected),
        lastAskedField: currentTaskState.missingField || currentTaskState.lastAskedField || ""
      }
    }

    if (nextState.mainline === "xiaohe_feedback") {
      return {
        ...nextState,
        feedbackType: classifyFeedbackType(text)
      }
    }

```

```
    return nextState
  },

  updateCurrentTaskState(question, location = null) {
    const nextTaskState = this.buildNextTaskState(question, location)
    this.setData({ currentTaskState: nextTaskState })
    return nextTaskState
  },

  goBack() {
    if (getCurrentPages().length > 1) {
      wx.navigateBack()
      return
    }

    wx.navigateTo({ url: "/pages/search/search?mode=ask" })
  },

  onInputChange(e) {
    this.setData({ inputValue: e.detail.value || "" })
  },

  submitInput() {
    const value = normalizeText(this.data.inputValue)
    if (!value) return
    this.handleUserReply(value)
  },

  useQuickOption(e) {
    const question = normalizeText(e.currentTarget.dataset.question)
    if (!question) return
    this.handleUserReply(question)
  },

  handleUserReply(value) {
    const nextMessages = this.data.messages.concat(createMessage("user", value))
    this.setData({
      messages: nextMessages,
      inputValue: ""
    })
    this.syncCurrentConversation()

    if (this.data.source === "skill_entry") {
      this.requestGenericAnswer(value, {
        contextPayload: {
          mode: "skill",
          skillContext: buildSkillContext(this.data.skillMode) || {}
        }
      })
      return
    }

    this.replyGeneric(value)
  },

  async replyGeneric(value) {
    if (!value) return
    const normalizedValue = normalizeText(value)

    if (normalizedValue.includes("  ") || normalizedValue.includes("   ")) {
      this.setData({
        genericPreferences: {
```

```
          ...this.data.genericPreferences,
          distance: "near"
        }
      })
      this.appendAiMessage("                              ")
      return
    }

    if (normalizedValue.includes("  ") || normalizedValue.includes("   ") || normalizedV
alue.includes("  ")) {
      this.setData({
        genericPreferences: {
          ...this.data.genericPreferences,
          budget: "value"
        }
      })
      this.appendAiMessage("                          ")
      return
    }

    if (/^(    |     |    |    |    |    |    )$/u.test(normalizedValue)) {
      this.setData({
        genericPreferences: {
          ...this.data.genericPreferences,
          detailLevel: "detailed"
        }
      })
      this.appendAiMessage("                            ")
      return
    }

    await this.requestGenericAnswer(value)
  },

  appendAiMessage(text, extra = {}) {
    const nextMessages = this.data.messages.concat(createMessage("ai", text, extra))
    this.setData({ messages: nextMessages })
    this.syncCurrentConversation()
  },

  appendAiPlaceholder() {
    const message = createMessage("ai", "", {
      recommendations: [],
      tips: "",
      guessQuestions: [],
      followUp: ""
    })

    const nextMessages = this.data.messages.concat(message)
    this.setData({ messages: nextMessages })
    this.syncCurrentConversation()
    return message.id
  },

  updateMessageById(id, updates = {}) {
    const messages = (this.data.messages || []).map((message) => {
      if (message.id !== id) return message
      return { ...message, ...updates }
    })

    this.setData({ messages })
    this.syncCurrentConversation()
```

```
  },

  startLoadingAnimation(messageId) {
    this.stopLoadingAnimation()
    const frames = ["", ".", "..", "..."]
    let frameIndex = 0

    this.updateMessageById(messageId, {
      text: LOADING_BASE_TEXT
    })

    this.loadingMessageTimer = setInterval(() => {
      frameIndex = (frameIndex + 1) % frames.length
      this.updateMessageById(messageId, {
        text: `${LOADING_BASE_TEXT}${frames[frameIndex]}`
      })
    }, 450)
  },

  stopLoadingAnimation() {
    if (this.loadingMessageTimer) {
      clearInterval(this.loadingMessageTimer)
      this.loadingMessageTimer = null
    }
  },

  async requestGenericAnswer(question, options = {}) {
    if (!question || this.data.isAiLoading) return

    if (!yuxiaoheBotId) {
      this.appendAiMessage("                          ")
      return
    }

    if (!wx.cloud || !wx.cloud.extend || !wx.cloud.extend.AI || !wx.cloud.extend.AI.bot) {
      this.appendAiMessage("                              3.7.1     ")
      return
    }

    this.setData({ isAiLoading: true })
    const aiMessageId = this.appendAiPlaceholder()
    const recommendationType = detectRecommendationType(question)
    const latestLocation = await this.ensureFreshLocationContext()
    const currentTaskState = this.data.currentTaskState || createInitialTaskState(this.data.skillMode)
    const contextPayload = {
      ...(options.contextPayload || {}),
      location: latestLocation,
      currentTaskState,
      skillContext: {
        ...this.getCurrentSkillContext(options.contextPayload && options.contextPayload.skillContext),
        collected: {
          ...(currentTaskState.collected || {})
        }
      }
    }
    const shouldRenderStructuredAnswer = !!recommendationType && !currentTaskState.mainline
    this.startLoadingAnimation(aiMessageId)

```

```
    try {
      const groundedUiPromise = shouldRenderStructuredAnswer
        ? this.buildGroundedRecommendationUi(question, contextPayload)
        : Promise.resolve({ introText: "", recommendations: [], tips: "" })
      const conversationStatePromise = this.fetchConversationState(question, contextPayl
oad).catch((error) => {
        console.warn("[askXiaoheChat] fetch conversation state failed", error)
        return null
      })
      const res = await wx.cloud.extend.AI.bot.sendMessage({
        data: {
          botId: yuxiaoheBotId,
          msg: question,
          history: toAgentHistory(this.data.messages),
          contextPayload: this.buildGenericPayload(question, contextPayload)
        }
      })
      try {
        console.log("[askXiaoheChat] sendMessage response keys", Object.keys(res || {}))
        console.log("[askXiaoheChat] sendMessage response", res)
      } catch (logError) {
        console.warn("[askXiaoheChat] inspect sendMessage response failed", logError)
      }

      let answer = ""
      for await (const chunk of res.textStream) {
        answer += chunk
        this.stopLoadingAnimation()
        this.updateMessageById(aiMessageId, {
          text: answer
        })
      }

      const conversationState = await conversationStatePromise
      try {
        console.log("[askXiaoheChat] conversation state", conversationState)
      } catch (logError) {
        console.warn("[askXiaoheChat] inspect conversation state failed", logError)
      }
      if (conversationState) {
        const nextTaskState = buildTaskStateFromBackend(conversationState, currentTaskSt
ate)
        this.setData({ currentTaskState: nextTaskState })
      }

      const taskStateFromBackend = buildTaskStateFromBackend(conversationState, currentT
askState)
      const recommendations =
        conversationState && conversationState.mainline === "buddy_matching"
          ? this.buildBuddyRecommendations(conversationState && conversationState.candid
ates, taskStateFromBackend)
          : await this.buildGuideRecommendations(
              conversationState && conversationState.candidates,
              taskStateFromBackend
            )
      try {
        console.log("[askXiaoheChat] structured recommendation summary", {
          mainline: conversationState && conversationState.mainline,
          candidateCount: Array.isArray(conversationState && conversationState.candidate
s)
            ? conversationState.candidates.length
            : 0,
```

```
          recommendationCount: Array.isArray(recommendations) ? recommendations.length : 0,
          firstRecommendation: Array.isArray(recommendations) && recommendations.length
            ? recommendations[0]
            : null
        })
      } catch (logError) {
        console.warn("[askXiaoheChat] inspect structured recommendations failed", logError)
      }
      if (
        conversationState &&
        conversationState.mainline === "guide_customization" &&
        recommendations.length
      ) {
        this.updateMessageById(aiMessageId, {
          text: buildGuideCardIntro(taskStateFromBackend),
          recommendations,
          tips: buildGuideWarmTips(taskStateFromBackend, recommendations),
          guessQuestions: [],
          followUp: ""
        })
        return
      }

      if (
        conversationState &&
        conversationState.mainline === "buddy_matching" &&
        recommendations.length
      ) {
        this.updateMessageById(aiMessageId, {
          text: buildBuddyPromptSummary(question),
          recommendations,
          tips: [
            "     ",
            "1.                        ",
            "2.                       "
          ].join("\n"),
          guessQuestions: [],
          followUp: ""
        })
        return
      }

      const groundedUi = await groundedUiPromise
      if (shouldRenderStructuredAnswer && groundedUi.recommendations && groundedUi.recommendations.length) {
        this.updateMessageById(aiMessageId, {
          text: groundedUi.introText || "",
          recommendations: groundedUi.recommendations || [],
          tips: groundedUi.tips || "",
          guessQuestions: [],
          followUp: ""
        })
        return
      }

      this.updateMessageById(aiMessageId, {
        text: normalizeText(answer) || "                        ",
        recommendations: [],
        tips: "",
        guessQuestions: [],
```

```
        followUp: ""
      })
    } catch (error) {
      console.error("[askXiaoheChat] generic failed", error)
      this.updateMessageById(aiMessageId, {
        text: "                            ",
        recommendations: [],
        tips: "",
        guessQuestions: [],
        followUp: ""
      })
    } finally {
      this.stopLoadingAnimation()
      this.setData({ isAiLoading: false })
      this.syncCurrentConversation()
    }
  },

  async ensureFreshLocationContext() {
    const app = getApp()
    let userInfo = {}

    try {
      userInfo = (app && typeof app.getUserInfo === "function" && app.getUserInfo()) || {}
    } catch (error) {
      userInfo = {}
    }

    const selectedRegion = readSelectedRegion()
    let nextUserLocation = null

    try {
      nextUserLocation = wx.getStorageSync("userLocation") || userInfo.userLocation || null
    } catch (error) {
      nextUserLocation = userInfo.userLocation || null
    }

    try {
      const setting = await getSettingAsync()
      if (setting && setting.authSetting && setting.authSetting["scope.userLocation"]) {
        const locationRes = await getLocationAsync()
        nextUserLocation = {
          latitude: locationRes.latitude,
          longitude: locationRes.longitude
        }
        wx.setStorageSync("userLocation", nextUserLocation)

        if (userInfo && userInfo.openid && app && typeof app.setUserInfo === "function") {
          app.setUserInfo({
            ...userInfo,
            userLocation: nextUserLocation,
            locationAuthorized: true
          })
          userInfo = (typeof app.getUserInfo === "function" && app.getUserInfo()) || userInfo
        }
      }
    } catch (error) {
      console.warn("[askXiaoheChat] refresh location before request failed", error)
```

```
    }

    return buildLocationContext(userInfo, nextUserLocation, selectedRegion)
  },

  buildGenericPayload(question, extraPayload = {}) {
    const app = getApp()
    let userInfo = {}
    let cachedUserLocation = null
    let selectedRegion = null

    try {
      userInfo = (app && typeof app.getUserInfo === "function" && app.getUserInfo()) || {}
    } catch (error) {
      userInfo = {}
    }

    try {
      cachedUserLocation = wx.getStorageSync("userLocation") || (userInfo && userInfo.userLocation) || null
    } catch (error) {
      cachedUserLocation = null
    }

    selectedRegion = readSelectedRegion()

    const baseLocation = buildLocationContext(userInfo, cachedUserLocation, selectedRegion)
    const mergedLocation = {
      ...baseLocation,
      ...(extraPayload.location || {})
    }
    const currentTaskState = extraPayload.currentTaskState || this.data.currentTaskState || createEmptyTaskState("")
    const skillContext = {
      ...this.getCurrentSkillContext(extraPayload.skillContext),
      collected: {
        ...(this.getCurrentSkillContext(extraPayload.skillContext).collected || {}),
        ...(currentTaskState.collected || {})
      }
    }

    return {
      mode: extraPayload.mode || "generic",
      source: this.data.source || "search_input",
      question,
      location: mergedLocation,
      userProfile: {
        nickname: userInfo.nickName || userInfo.nickname || "",
        dnaTags: userInfo.dnaTags || [],
        residentCity: userInfo.city || "",
        commonDeparture: userInfo.commonDeparture || userInfo.city || "",
        travelStyle: userInfo.travelStyle || "",
        budgetRange: userInfo.budgetRange || ""
      },
      preferences: this.data.genericPreferences || {},
      skillContext,
      currentTaskState,
      history: (this.data.messages || [])
        .map((item) => ({
          role: item.role === "ai" ? "assistant" : item.role,
```

```
          text: item.text
        }))
        .filter((item) => item.role && item.text)
        .slice(-20),
      context: {
        source: this.data.source || "search_input"
      },
      ...extraPayload
    }
  },

  async fetchActivitiesForChat() {
    try {
      const res = await wx.cloud.callFunction({ name: "getactivities" })
      return res && res.result && Array.isArray(res.result.list) ? res.result.list : []
    } catch (error) {
      console.error("[askXiaoheChat] fetch activities failed", error)
      return []
    }
  },

  async fetchScenicsForChat() {
    try {
      const db = wx.cloud.database()
      const res = await db.collection("scenics").get()
      return Array.isArray(res && res.data) ? res.data : []
    } catch (error) {
      console.error("[askXiaoheChat] fetch scenics failed", error)
      return []
    }
  },

  async fetchProductsForChat() {
    try {
      const db = wx.cloud.database()
      const res = await db.collection("products").get()
      return Array.isArray(res && res.data) ? res.data : []
    } catch (error) {
      console.error("[askXiaoheChat] fetch products failed", error)
      return []
    }
  },

  async fetchHotelsForChat() {
    try {
      const db = wx.cloud.database()
      const res = await db.collection("hotels").get()
      return Array.isArray(res && res.data) ? res.data : []
    } catch (error) {
      console.error("[askXiaoheChat] fetch hotels failed", error)
      return []
    }
  },

  async fetchConversationState(question, contextPayload = {}) {
    const app = getApp()
    const genericPayload = this.buildGenericPayload(question, contextPayload)
    const userInfo = (app && typeof app.getUserInfo === "function" && app.getUserInfo())
  || {}
    const cloudbaseUserId = normalizeText(userInfo && userInfo.openid)
    const payload = {
      location: genericPayload.location || {},
```

```
      currentTaskState: genericPayload.currentTaskState || {},
      history: Array.isArray(genericPayload.history) ? genericPayload.history.slice(-20) : []
    }
    try {
      console.log("[askXiaoheChat] fetchConversationState payload", payload)
      console.log("[askXiaoheChat] fetchConversationState payload json", JSON.stringify(payload))
      console.log("[askXiaoheChat] fetchConversationState cloudbaseUserId", cloudbaseUserId)
    } catch (logError) {
      console.warn("[askXiaoheChat] inspect fetchConversationState payload failed", logError)
    }
    const result = await requestAgentHttp({
      path: "/api/conversation-state",
      method: "POST",
      data: {
        question,
        contextPayload: payload,
        cloudbaseUserId
      }
    })

    return result && result.data ? result.data : result
  },

  async buildGroundedRecommendationUi(question, extraPayload = {}) {
    const recommendationType = detectRecommendationType(question)
    if (!recommendationType) {
      return {
        introText: "",
        recommendations: [],
        tips: ""
      }
    }

    if (recommendationType === "activity") {
      return this.buildGroundedActivityUi(question, extraPayload)
    }

    if (recommendationType === "scenic") {
      return this.buildGroundedScenicUi(question, extraPayload)
    }

    return this.buildGroundedProductUi(question, extraPayload)
  },

  async buildGuideRecommendations(candidates = [], taskState = {}) {
    const normalizedCandidates = Array.isArray(candidates) ? candidates : []
    if (!normalizedCandidates.length) return []

    const [activities, scenics, products, hotels] = await Promise.all([
      this.fetchActivitiesForChat(),
      this.fetchScenicsForChat(),
      this.fetchProductsForChat(),
      this.fetchHotelsForChat()
    ])

    const activityMap = new Map((activities || []).map((item) => [normalizeText(item && (item._id || item.id)), item]))
    const scenicMap = new Map((scenics || []).map((item) => [normalizeText(item && (item
```

```
    const productMap = new Map((products || []).map((item) => [normalizeText(item && (item._id || item.id)), item]))
    const hotelMap = new Map((hotels || []).map((item) => [normalizeText(item && (item._id || item.id)), item]))

    const recommendations = await Promise.all(
      normalizedCandidates.map(async (candidate) => {
        const type = normalizeText(candidate && candidate.type)
        const sourceId = normalizeText(candidate && candidate.id)
        const fallback = buildGuideFallbackRecommendation(candidate)
        if (!type || !sourceId) return fallback

        if (type === "activity") {
          const matched = activityMap.get(sourceId)
          return matched ? buildActivityRecommendationCard(matched, candidate, taskState) : fallback
        }

        if (type === "scenic") {
          const matched = scenicMap.get(sourceId)
          return matched ? buildScenicRecommendationCard(matched, candidate, taskState) : fallback
        }

        if (type === "product") {
          const matched = productMap.get(sourceId)
          return matched ? buildProductRecommendationCard(matched, candidate, taskState) : fallback
        }

        if (type === "hotel") {
          const matched = hotelMap.get(sourceId)
          return matched ? buildHotelRecommendationCard(matched, candidate, taskState) : fallback
        }

        return fallback
      })
    )

    return recommendations.filter(Boolean)
  },

  buildBuddyRecommendations(candidates = [], taskState = {}) {
    const normalizedCandidates = Array.isArray(candidates) ? candidates : []
    if (!normalizedCandidates.length) return []

  return normalizedCandidates
      .map((candidate, index) => {
        const title = normalizeText(candidate && (candidate.nickname || candidate.title)) || `     ${index + 1}`
        const tags = filterBuddyTags(candidate && candidate.tags)
        const region = normalizeText(candidate && candidate.region)
        const summary = normalizeBuddySummaryText(candidate && candidate.summary, region)
        const matchReason = normalizeText(candidate && candidate.matchReason) || "
                "
        const practicalInfo = buildBuddyPracticalInfoFromCandidate(candidate)
        const playItems = buildBuddyHighlightsFromCandidate({ ...candidate, tags })

        return normalizeBuddyRecommendation({
          id: normalizeText(candidate && candidate.id) || `buddy-${index}`,
```

```
          type: "buddy",
          title,
          statusTag: "",
          summary,
          matchReason,
          reasonTitle: "    ",
          tags,
          playItems,
          practicalInfo,
          avatarUrl: "",
          avatarText: buildBuddyAvatarText(title),
          avatarColor: buildBuddyAvatarColor(title),
          actionText: "      ",
          openingText: buildBuddyOpeningText({
            ...candidate,
            title,
            region,
            matchReason,
          }),
          planSummary: "",
          matchScore: "",
          matchScoreText: "",
          taskState,
        })
      })
      .filter(Boolean)
  },

  async buildGroundedActivityUi(question, extraPayload = {}) {
    const payload = this.buildGenericPayload(question, extraPayload)
    const location = payload.location || {}
    const list = await this.fetchActivitiesForChat()
    const rankedList = list
      .map((item) => ({ item, score: scoreByQuestion(joinActivitySearchText(item), quest
ion, location, "activity") }))
      .filter((item) => item.score > 0)
      .sort((a, b) => b.score - a.score)
      .slice(0, 3)

    const recommendations = await Promise.all(
      rankedList.map(async ({ item }) => {
        const coverTagInfo = buildActivityCoverTags(item)
        const playItems = getActivityPlayItems(item)
        const gallery = await resolveActivityGallery(item)

        return {
          id: normalizeText(item._id || item.id),
          sourceId: normalizeText(item._id || item.id),
          type: "activity",
          title: normalizeText(item.title) || "    ",
          summary: normalizeText(item.summary || item.content || item.detail) || "
      ",
          gallery: Array.isArray(gallery) ? gallery.filter(Boolean).slice(0, 2) : [],
          tags: coverTagInfo.combinedTags.slice(0, 4),
          playItems: playItems.length ? playItems : ["       "],
          practicalInfo: getActivityPracticalInfo(item),
          cover: await resolveActivityCover(item)
        }
      })
    )

    return {
```

```
      introText: recommendations.length ? "                                  " : "",
      recommendations,
      tips: recommendations.length
        ? [
            "     ",
            "1.                      ",
            "2.                              "
          ].join("\n")
        : ""
    }
  },

  async buildGroundedScenicUi(question, extraPayload = {}) {
    const payload = this.buildGenericPayload(question, extraPayload)
    const location = payload.location || {}
    const list = await this.fetchScenicsForChat()
    const rankedList = list
      .map((item) => ({ item, score: scoreByQuestion(joinScenicSearchText(item), questio
n, location, "scenic") }))
      .filter((item) => item.score > 0)
      .sort((a, b) => b.score - a.score)
      .slice(0, 3)

    const recommendations = rankedList.map(({ item }) => {
      const cover = normalizeText(item.cover) || DEFAULT_ACTIVITY_COVER
      const gallery = Array.isArray(item.gallery) && item.gallery.length
        ? item.gallery.filter(Boolean).slice(0, 2)
        : [cover]
      const playItems = getScenicPlayItems(item)

      return {
        id: normalizeText(item._id || item.id),
        sourceId: normalizeText(item._id || item.id),
        type: "scenic",
        title: normalizeText(item.locationName || item.title) || "    ",
        summary: normalizeText(item.summary || item.content || item.detail) || "
    ",
        gallery,
        tags: uniqueList(normalizeArray(item.playTags).concat(normalizeArray(item.tags))
).slice(0, 4),
        playItems: playItems.length ? playItems : ["       "],
        practicalInfo: getScenicPracticalInfo(item),
        cover
      }
    })

    return {
      introText: recommendations.length ? "                                  " : "",
      recommendations,
      tips: recommendations.length
        ? [
            "     ",
            "1.                      ",
            "2.                              "
          ].join("\n")
        : ""
    }
  },

  async buildGroundedProductUi(question, extraPayload = {}) {
    const payload = this.buildGenericPayload(question, extraPayload)
    const location = payload.location || {}
```

```
    const list = await this.fetchProductsForChat()
    const rankedList = list
      .map((item) => ({ item, score: scoreByQuestion(joinProductSearchText(item), questi
on, location, "product") }))
      .filter((item) => item.score > 0)
      .sort((a, b) => b.score - a.score)
      .slice(0, 3)

    const recommendations = rankedList.map(({ item }) => {
      const cover = normalizeText(item.cover) || DEFAULT_PRODUCT_COVER
      const playItems = getProductPlayItems(item)

      return {
        id: normalizeText(item._id || item.id),
        sourceId: normalizeText(item._id || item.id),
        type: "product",
        title: normalizeText(item.title) || "    ",
        summary: normalizeText(item.summary || item.content || item.detail) || "
    ",
        gallery: [cover],
        tags: uniqueList(normalizeArray(item.categoryTags).concat(normalizeArray(item.ta
gs))).slice(0, 4),
        playItems: playItems.length ? playItems : ["       "],
        practicalInfo: getProductPracticalInfo(item),
        cover
      }
    })

    return {
      introText: recommendations.length ? "                                    " : "",
      recommendations,
      tips: recommendations.length
        ? [
            "     ",
            "1.                          ",
            "2.                     "
          ].join("\n")
        : ""
    }
  },

  async requestBuddyMatches(question) {
    if (!ensureBuddySession()) {
      return
    }

    const app = getApp()
    let userInfo = {}
    try {
      userInfo = (app && typeof app.getUserInfo === "function" && app.getUserInfo()) ||
{}
    } catch (error) {
      userInfo = {}
    }

    try {
      const result = await wx.cloud.callFunction({
        name: "userManage",
        data: {
          action: "getBuddyMatches",
          payload: {
            question: normalizeText(question) || DEFAULT_BUDDY_TAGS.join(" "),
```

```
2021            limit: 3
2022          }
2023        }
2024      })
2025
2026      const recommendations = Array.isArray(result?.result?.list)
2027        ? result.result.list.map((item) => ({
2028            ...normalizeBuddyRecommendation(item),
2029            type: "buddy",
2030            title: item.userName,
2031          }))
2032        : []
2033
2034      if (!recommendations.length) {
2035        this.appendAiMessage("                              DNA            ", {
2036          channelLabel: "   "
2037        })
2038        return
2039      }
2040
2041      this.appendAiMessage(buildBuddyPromptSummary(question, userInfo), {
2042        recommendations,
2043        tips: [
2044          "     ",
2045          "1.                        ",
2046          "2.                           "
2047        ].join("\n"),
2048        channelLabel: "   "
2049      })
2050    } catch (error) {
2051      console.error("[askXiaoheChat] get buddy matches failed", error)
2052      this.appendAiMessage("                         ", {
2053        channelLabel: "   "
2054      })
2055    }
2056  },
2057
2058  applyBuddyMatch(e) {
2059    const dataset = (e && e.currentTarget && e.currentTarget.dataset) || {}
2060    const userName = normalizeText(dataset.username)
2061    const openingText = normalizeText(dataset.opening)
2062    if (!userName || !openingText) {
2063      wx.showToast({ title: "            ", icon: "none" })
2064      return
2065    }
2066
2067    const application = createBuddyApplicationFromMatch({
2068      userName,
2069      openingText,
2070      avatarUrl: dataset.avatarurl,
2071      avatarText: dataset.avatartext,
2072      avatarColor: dataset.avatarcolor,
2073      matchScore: dataset.matchscore,
2074      matchReason: dataset.matchreason,
2075      direction: "outgoing"
2076    })
2077
2078    if (!application || !application.id) {
2079      wx.showToast({ title: "      ", icon: "none" })
2080      return
2081    }
2082
```

```
    wx.showToast({ title: "     ", icon: "success" })
    setTimeout(() => {
      wx.navigateTo({
        url: `/pages/messageBuddyApplyChat/messageBuddyApplyChat?id=${application.id}`
      })
    }, 220)
  },

  openCardDetail(e) {
    const { type, sourceid: sourceId } = e.currentTarget.dataset || {}
    try {
      console.log("[askXiaoheChat] openCardDetail dataset", e.currentTarget.dataset || {
})
    } catch (logError) {
      console.warn("[askXiaoheChat] inspect openCardDetail dataset failed", logError)
    }
    if (type === "buddy") return
    if (!sourceId) return

    const urlMap = {
      activity: `/pages/activityDetail/activityDetail?id=${sourceId}`,
      scenic: `/pages/scenicDetail/scenicDetail?id=${sourceId}`,
      product: `/pages/productDetail/productDetail?id=${sourceId}`,
      hotel: `/pages/hotelDetail/hotelDetail?id=${sourceId}`
    }

    const targetUrl = urlMap[type]
    if (!targetUrl) return

    wx.navigateTo({ url: targetUrl })
  }
})
```

Path: miniprogram/pages/askXiaoheChat/askXiaoheChat.json

```
1 {
2   "navigationStyle": "custom"
3 }
```

Path: miniprogram/pages/askXiaoheChat/askXiaoheChat.wxml

```
<view class="page">
  <view class="customNav" style="padding-top: {{statusBarHeight}}px;">
    <view class="topbar">
      <view class="backBtn" bindtap="goBack">
        <view class="backArrow"></view>
      </view>
      <view class="brandWrap">
        <image class="brandIconImage" src="/images/ask_xiaohe_logo.png" mode="aspectFill">></image>
        <view class="brandTitle">{{text.brand}}</view>
      </view>
    </view>
  </view>

  <scroll-view class="content" scroll-y show-scrollbar="{{false}}" style="padding-top: calc({{statusBarHeight}}px + 108rpx);">
    <view class="introCard" wx:if="{{showSkillIntro}}">
      {{introText}}
    </view>

    <view class="chatList">
      <view class="messageBlock {{item.role}}" wx:for="{{messages}}" wx:key="id">
        <block wx:if="{{item.role === 'user'}}">
          <view class="bubble user">
            <view class="bubbleText" wx:if="{{item.text}}">{{item.text}}</view>
          </view>
        </block>

        <block wx:elif="{{item.role === 'ai'}}">
          <view class="bubble ai aiAsideBubble" wx:if="{{item.text || item.channelLabel}}">
            <view class="channelLabel" wx:if="{{item.channelLabel}}">
              {{item.channelLabel}}
            </view>
            <view class="bubbleText" wx:if="{{item.text}}">{{item.text}}</view>
          </view>

          <view class="recommendationList" wx:if="{{item.recommendations && item.recommendations.length}}">
            <view class="recommendationCard">
              <view class="recommendationBlock" wx:for="{{item.recommendations}}" wx:key="id">
                <view class="buddyCardHead" wx:if="{{item.type === 'buddy'}}">
                  <view>
                    <view class="recommendationTitleLink">{{item.title}}</view>
                    <view class="buddyStatusTag" wx:if="{{item.statusTag}}">{{item.statusTag}}</view>
                  </view>
                  <view class="buddyScore">{{item.matchScoreText}}</view>
                </view>
                <view wx:else class="recommendationTitleLink" data-type="{{item.type}}" data-sourceid="{{item.sourceId}}" bindtap="openCardDetail">
                  {{item.title}}
                </view>
                <view class="recommendationGallery" wx:if="{{item.gallery && item.gallery.length}}">
                  <image class="recommendationGalleryImage" wx:for="{{item.gallery}}" wx:key="*this" src="{{item}}" mode="aspectFill" />
                </view>
                <view class="buddyPlan" wx:if="{{item.type === 'buddy' && item.planSumma
```

```
                <view class="recommendationSummary">{{item.summary}}</view>
                <view class="resultSection" wx:if="{{item.type === 'buddy' && item.matchReason}}">
                  <view class="resultSectionTitle">    </view>
                  <view class="resultInfoValue">{{item.matchReason}}</view>
                </view>
                <view class="resultTags" wx:if="{{item.tags && item.tags.length}}">
                  <view class="resultTag" wx:for="{{item.tags}}" wx:key="*this">{{item}}</view>
                </view>
                <view class="resultSection" wx:if="{{item.playItems && item.playItems.length}}">
                  <view class="resultSectionTitle">    </view>
                  <view class="resultBulletItem" wx:for="{{item.playItems}}" wx:key="*this">
                    <view class="resultBulletDot"></view>
                    <view class="resultBulletText">{{item}}</view>
                  </view>
                </view>
                <view class="resultSection" wx:if="{{item.practicalInfo && item.practicalInfo.length}}">
                  <view class="resultSectionTitle">    </view>
                  <view class="resultInfoRow" wx:for="{{item.practicalInfo}}" wx:key="label">
                    <text class="resultInfoLabel">{{item.label}} </text>
                    <text class="resultInfoValue">{{item.value}}</text>
                  </view>
                </view>
                <view
                  class="buddyApplyBtn"
                  wx:if="{{item.type === 'buddy' && item.actionText}}"
                  data-username="{{item.title}}"
                  data-opening="{{item.openingText}}"
                  data-avatarurl="{{item.avatarUrl}}"
                  data-avatartext="{{item.avatarText}}"
                  data-avatarcolor="{{item.avatarColor}}"
                  data-matchscore="{{item.matchScore}}"
                  data-matchreason="{{item.matchReason}}"
                  bindtap="applyBuddyMatch"
                >
                  {{item.actionText}}
                </view>
              </view>
            </view>
          </view>

          <view class="bubble ai aiAsideBubble" wx:if="{{item.tips}}">
            <view class="tipsText">{{item.tips}}</view>
          </view>

          <view class="bubble ai aiAsideBubble" wx:if="{{item.followUp}}">
            <view class="followUpText">{{item.followUp}}</view>
          </view>
        </block>

        <view class="guessBlock" wx:if="{{item.role === 'ai' && item.guessQuestions && item.guessQuestions.length}}">
          <view class="guessTitle">       </view>
          <view class="guessList">
            <view class="guessItem" wx:for="{{item.guessQuestions}}" wx:key="*this" data-question="{{item}}" bindtap="useQuickOption">
              {{item}}
```

```
            </view>
          </view>
        </view>
      </view>
    </view>

    <view class="quickOptionList" wx:if="{{quickOptions.length}}">
      <view class="quickOption" wx:for="{{quickOptions}}" wx:key="*this" data-question="{{item}}" bindtap="useQuickOption">
        {{item}}
      </view>
    </view>
  </scroll-view>

  <view class="composerWrap">
    <view class="skillBadge" wx:if="{{showSkillBadge}}">
      <view class="skillBadgeIconPlaceholder"></view>
      <view class="skillBadgeText">{{skillBadgeName}}</view>
    </view>
    <view class="composer">
      <input class="composerInput" value="{{inputValue}}" placeholder="{{inputPlaceholder}}" placeholder-class="composerPlaceholder" confirm-type="send" bindinput="onInputChange" bindconfirm="submitInput" />
      <view class="composerSendBtn" bindtap="submitInput">
        {{isAiLoading ? '   ' : '  '}}
      </view>
    </view>
  </view>
</view>
```

Path: miniprogram/pages/askXiaoheChat/askXiaoheChat.wxss

```
.page {
  height: 100vh;
  overflow: hidden;
  background:
    radial-gradient(circle at top right, rgba(183, 149, 255, 0.18), transparent 28%),
    radial-gradient(circle at top left, rgba(255, 215, 138, 0.2), transparent 24%),
    linear-gradient(180deg, #fbf5ff 0%, #ffffff 32%, #fafafc 100%);
  box-sizing: border-box;
  display: flex;
  flex-direction: column;
}

.customNav {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  z-index: 20;
  padding: 0 24rpx 2rpx;
  min-height: 92rpx;
  background:
    radial-gradient(circle at top right, rgba(183, 149, 255, 0.18), transparent 28%),
    radial-gradient(circle at top left, rgba(255, 215, 138, 0.2), transparent 24%),
    linear-gradient(180deg, rgba(251, 245, 255, 0.98) 0%, rgba(255, 255, 255, 0.94) 76%, rgba(255, 255, 255, 0) 100%);
  box-sizing: border-box;
  backdrop-filter: blur(18rpx) saturate(120%);
  -webkit-backdrop-filter: blur(18rpx) saturate(120%);
}

.topbar {
  display: flex;
  align-items: center;
  gap: 20rpx;
  min-height: 70rpx;
  padding-bottom: 0;
}

.backBtn {
  width: 60rpx;
  height: 60rpx;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
}

.backArrow {
  width: 20rpx;
  height: 20rpx;
  border-left: 4rpx solid #1f1f1f;
  border-bottom: 4rpx solid #1f1f1f;
  transform: rotate(45deg);
}

.brandWrap {
  display: flex;
  align-items: center;
  gap: 12rpx;
  min-width: 0;
```

```
}

.brandIconImage {
  width: 66rpx;
  height: 66rpx;
  border-radius: 50%;
  display: block;
  flex-shrink: 0;
}

.brandTitle {
  font-size: 32rpx;
  font-weight: 700;
  color: #232323;
  letter-spacing: 0.5rpx;
}

.content {
  flex: 1;
  height: 100vh;
  padding: 24rpx 24rpx 240rpx;
  box-sizing: border-box;
}

.introCard {
  padding: 28rpx 30rpx;
  border-radius: 28rpx;
  background: rgba(244, 241, 255, 0.92);
  color: #655b8f;
  font-size: 28rpx;
  line-height: 1.75;
  text-align: center;
  box-shadow: 0 14rpx 28rpx rgba(140, 104, 255, 0.08);
}

.chatList {
  margin-top: 22rpx;
  display: flex;
  flex-direction: column;
  gap: 18rpx;
}

.messageBlock {
  display: flex;
  flex-direction: column;
  gap: 12rpx;
  width: 100%;
}

.messageBlock.user {
  align-items: flex-end;
}

.messageBlock.ai {
  align-items: flex-start;
}

.bubble {
  padding: 22rpx 24rpx;
  border-radius: 28rpx;
  font-size: 28rpx;
  line-height: 1.7;
```

```
  box-sizing: border-box;
  max-width: 66%;
}

.bubble.user {
  background: rgba(241, 235, 255, 0.98);
  color: #52468d;
}

.bubble.ai {
  background: rgba(255, 255, 255, 0.96);
  color: #000000;
  box-shadow: 0 16rpx 32rpx rgba(140, 104, 255, 0.08);
}

.aiAsideBubble {
  max-width: 66%;
}

.bubbleText {
  white-space: pre-wrap;
  color: #000000;
  word-break: break-all;
}

.channelLabel {
  display: inline-flex;
  align-items: center;
  margin-bottom: 12rpx;
  padding: 6rpx 14rpx;
  border-radius: 999rpx;
  background: rgba(126, 99, 255, 0.1);
  color: #6d56d6;
  font-size: 28rpx;
  line-height: 1.5;
  font-weight: 700;
}

.recommendationList {
  display: flex;
  flex-direction: column;
  gap: 16rpx;
  align-items: stretch;
  width: 100%;
}

.recommendationCard {
  overflow: hidden;
  width: 100%;
  padding: 28rpx 30rpx;
  border-radius: 28rpx;
  background: rgba(255, 255, 255, 0.97);
  box-shadow: 0 16rpx 32rpx rgba(140, 104, 255, 0.08);
  box-sizing: border-box;
}

.recommendationBlock + .recommendationBlock {
  margin-top: 24rpx;
  padding-top: 24rpx;
  border-top: 2rpx solid rgba(164, 136, 255, 0.14);
}

```

```
.buddyCardHead {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 18rpx;
}

.recommendationSummary {
  margin-top: 16rpx;
  color: #000000;
  font-size: 28rpx;
  line-height: 1.7;
  word-break: break-all;
}

.resultTags {
  margin-top: 14rpx;
  display: flex;
  flex-wrap: wrap;
  gap: 10rpx;
}

.buddyStatusTag {
  margin-top: 10rpx;
  color: #7f67e8;
  font-size: 24rpx;
  line-height: 1.4;
}

.buddyScore {
  flex-shrink: 0;
  padding: 10rpx 16rpx;
  border-radius: 999rpx;
  background: rgba(126, 99, 255, 0.12);
  color: #6d56d6;
  font-size: 24rpx;
  font-weight: 700;
}

.buddyPlan {
  margin-top: 14rpx;
  color: #53478f;
  font-size: 26rpx;
  line-height: 1.6;
}

.resultTag {
  padding: 8rpx 14rpx;
  border-radius: 999rpx;
  background: rgba(241, 235, 255, 0.95);
  color: #6a55ca;
  font-size: 22rpx;
}

.resultSection {
  margin-top: 18rpx;
  padding-top: 16rpx;
  border-top: 2rpx solid rgba(164, 136, 255, 0.14);
}

.resultSectionTitle {
  margin-bottom: 12rpx;
```

```
  color: #000000;
  font-size: 28rpx;
  font-weight: 700;
}

.resultBulletItem {
  display: flex;
  align-items: flex-start;
  gap: 10rpx;
  margin-top: 8rpx;
}

.resultBulletDot {
  width: 10rpx;
  height: 10rpx;
  margin-top: 13rpx;
  border-radius: 50%;
  background: #8b6bff;
  flex-shrink: 0;
}

.resultBulletText {
  color: #000000;
  font-size: 28rpx;
  line-height: 1.7;
  word-break: break-all;
}

.resultInfoRow {
  margin-top: 8rpx;
  font-size: 28rpx;
  line-height: 1.7;
  color: #000000;
  word-break: break-all;
}

.resultInfoLabel {
  font-weight: 700;
  color: #000000;
}

.resultInfoValue {
  color: #000000;
  word-break: break-all;
}

.buddyApplyBtn {
  margin-top: 20rpx;
  height: 78rpx;
  border-radius: 22rpx;
  background: linear-gradient(90deg, #7da6ff 0%, #b46cff 100%);
  color: #ffffff;
  font-size: 28rpx;
  font-weight: 700;
  display: flex;
  align-items: center;
  justify-content: center;
}

.tipsText,
.followUpText {
  color: #000000;
```

```
  font-size: 28rpx;
  line-height: 1.7;
  white-space: pre-wrap;
  word-break: break-all;
}

.guessBlock {
  margin-top: 8rpx;
  padding-left: 6rpx;
}

.guessTitle {
  margin-bottom: 14rpx;
  color: #000000;
  font-size: 28rpx;
  line-height: 1.4;
}

.guessList {
  display: flex;
  flex-direction: column;
  gap: 12rpx;
  align-items: flex-start;
}

.guessItem {
  display: inline-flex;
  align-items: center;
  max-width: 100%;
  padding: 16rpx 22rpx;
  border-radius: 20rpx;
  background: rgba(244, 241, 255, 0.95);
  color: #5a4d8e;
  font-size: 28rpx;
  line-height: 1.55;
  box-sizing: border-box;
  box-shadow: 0 8rpx 20rpx rgba(140, 104, 255, 0.05);
}

.quickOptionList {
  margin-top: 26rpx;
  display: flex;
  flex-direction: column;
  gap: 16rpx;
}

.quickOption {
  align-self: flex-start;
  max-width: 84%;
  padding: 18rpx 24rpx;
  border-radius: 22rpx;
  background: rgba(244, 241, 255, 0.95);
  color: #4a4280;
  font-size: 28rpx;
  line-height: 1.6;
}

.composerWrap {
  position: fixed;
  left: 0;
  right: 0;
  bottom: 0;
```

```
  padding: 0 24rpx calc(env(safe-area-inset-bottom) + 18rpx);
  background: linear-gradient(180deg, rgba(255, 255, 255, 0) 0%, rgba(253, 250, 255, 0.96) 26%, #ffffff 100%);
}

.skillBadge {
  display: inline-flex;
  align-items: center;
  gap: 10rpx;
  margin-left: 12rpx;
  margin-bottom: 12rpx;
  padding: 12rpx 18rpx;
  border-radius: 999rpx;
  background: rgba(255, 255, 255, 0.96);
  border: 2rpx solid rgba(122, 93, 234, 0.28);
  box-shadow: 0 10rpx 20rpx rgba(140, 104, 255, 0.08);
}

.skillBadgeIconPlaceholder {
  width: 28rpx;
  height: 28rpx;
  border-radius: 10rpx;
  border: 2rpx dashed rgba(122, 93, 234, 0.45);
  background: rgba(244, 241, 255, 1);
}

.skillBadgeText {
  color: #232323;
  font-size: 28rpx;
  font-weight: 600;
}

.composer {
  display: flex;
  align-items: center;
  gap: 10rpx;
  min-height: 92rpx;
  padding: 8rpx 16rpx 8rpx 24rpx;
  border-radius: 999rpx;
  background: rgba(255, 255, 255, 0.98);
  border: 2rpx solid rgba(122, 93, 234, 0.4);
  box-shadow: 0 16rpx 28rpx rgba(140, 104, 255, 0.08);
}

.composerInput {
  flex: 1;
  min-width: 0;
  font-size: 28rpx;
  color: #1f1f1f;
}

.composerPlaceholder {
  color: #b6b6b6;
}

.composerSendBtn {
  min-width: 132rpx;
  height: 60rpx;
  padding: 0 22rpx;
  border-radius: 24rpx;
  background: linear-gradient(90deg, #7da6ff 0%, #b46cff 100%);
  color: #ffffff;
```

```
  font-size: 24rpx;
  font-weight: 600;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
}

.recommendationTitleLink {
  display: inline-block;
  color: #5f73ff;
  font-size: 28rpx;
  font-weight: 700;
  line-height: 1.7;
}

.recommendationGallery {
  margin-top: 18rpx;
  display: flex;
  flex-wrap: wrap;
  gap: 12rpx;
}

.recommendationGalleryImage {
  width: 210rpx;
  height: 210rpx;
  border-radius: 18rpx;
  background: #ece5ff;
  flex: 0 0 210rpx;
}
```

Path: miniprogram/pages/me/me.js

```
---------------------------------------------------------------------------
   1 const app = getApp()
   2 const { resolveMediaSource } = require('../../utils/mediaAssets')
   3
   4 const DEFAULT_LOGGED_IN_AVATAR = '/images/avatar.png'
   5 const GUEST_AVATAR = '/images/me/guest-avatar.png'
   6
   7 const TEXT = {
   8   defaultNickname: 'e   ',
   9   guestHint: '            ',
  10   authLogin: '    ',
  11   editProfile: '      ',
  12   myDNA: '  DNA',
  13   dnaPlaceholder: '             ',
  14   dnaPlaceholderGuest: '           ',
  15   myOrders: '    ',
  16   allOrders: '  ',
  17   favorites: '    ',
  18   history: '    ',
  19   customerService: '    ',
  20   systemSettings: '    ',
  21   clearLoginState: '       ',
  22   clearLocalTestData: '        ',
  23   systemSettingsCancel: '  ',
  24   logout: '    ',
  25   historyDeveloping: '       ',
  26   customerServiceDeveloping: '       ',
  27   systemSettingsDeveloping: '       ',
  28   clearDone: '   ',
  29   logoutSuccess: '     ',
  30 }
  31
  32 const ORDER_ENTRIES = [
  33   {
  34     key: 'pending_payment',
  35     tab: 'pending_payment',
  36     label: '   ',
  37     icon: '/images/me/order-status/pending-payment.png',
  38   },
  39   {
  40     key: 'pending_trip',
  41     tab: 'pending_trip',
  42     label: '   ',
  43     icon: '/images/me/order-status/pending-trip.png',
  44   },
  45   {
  46     key: 'pending_receive',
  47     tab: 'pending_receive',
  48     label: '   ',
  49     icon: '/images/me/order-status/pending-receive.png',
  50   },
  51   {
  52     key: 'review',
  53     tab: 'pending',
  54     label: '  ',
  55     icon: '/images/me/order-status/review.png',
  56   },
  57   {
  58     key: 'refund_after_sale',
  59     tab: 'refund_after_sale',
  60     label: '  /  ',
```

```
    icon: '/images/me/order-status/refund-after-sale.png',
  },
]

Page({
  data: {
    text: TEXT,
    orderEntries: ORDER_ENTRIES,
    statusBarHeight: 20,
    loggedIn: false,
    userInfo: {
      avatarUrl: GUEST_AVATAR,
      nickName: '',
    },
    dnaTags: [],
    dnaPlaceholderText: TEXT.dnaPlaceholderGuest,
  },

  onLoad() {
    this.initNavMetrics()
  },

  onShow() {
    this.refreshUser()
  },

  initNavMetrics() {
    try {
      const systemInfo = wx.getSystemInfoSync()
      this.setData({
        statusBarHeight: systemInfo.statusBarHeight || 20,
      })
    } catch (err) {
      this.setData({
        statusBarHeight: 20,
      })
    }
  },

  async refreshUser() {
    const rawUserInfo = app.getUserInfo() || {}
    const loggedIn = !!rawUserInfo.openid
    let avatarUrl = GUEST_AVATAR

    if (loggedIn) {
      const previewAvatar = rawUserInfo.avatarPreviewUrl || ''
      const cachedResolvedAvatar = rawUserInfo.avatarResolvedUrl || ''
      const resolvedAvatar = await resolveMediaSource(rawUserInfo.avatarUrl, cachedResolvedAvatar || DEFAULT_LOGGED_IN_AVATAR)
      avatarUrl = previewAvatar || resolvedAvatar || cachedResolvedAvatar || DEFAULT_LOGGED_IN_AVATAR
    }

    const nickName = loggedIn ? (rawUserInfo.nickName || TEXT.defaultNickname) : ''
    const dnaTags = loggedIn && Array.isArray(rawUserInfo.dnaTags) ? rawUserInfo.dnaTags : []

    this.setData({
      loggedIn,
      userInfo: {
        avatarUrl,
        nickName,
```

```
      },
      dnaTags,
      dnaPlaceholderText: loggedIn ? TEXT.dnaPlaceholder : TEXT.dnaPlaceholderGuest,
    })

    if (loggedIn && rawUserInfo.avatarPreviewUrl) {
      const resolvedAvatar = await resolveMediaSource(rawUserInfo.avatarUrl, '')
      if (resolvedAvatar) {
        const nextUserInfo = {
          ...rawUserInfo,
          avatarPreviewUrl: '',
          avatarResolvedUrl: resolvedAvatar,
        }
        app.setUserInfo(nextUserInfo)
        this.setData({
          'userInfo.avatarUrl': resolvedAvatar,
        })
      }
    } else if (loggedIn && rawUserInfo.avatarUrl && avatarUrl && avatarUrl !== rawUserInfo.avatarResolvedUrl && avatarUrl !== DEFAULT_LOGGED_IN_AVATAR) {
      app.setUserInfo({
        ...rawUserInfo,
        avatarResolvedUrl: avatarUrl,
      })
    }
  },

  onAvatarError() {
    this.setData({
      'userInfo.avatarUrl': DEFAULT_LOGGED_IN_AVATAR,
    })
  },

  onUserBoxTap() {
    if (!this.data.loggedIn) {
      this.goLogin()
    }
  },

  goLogin() {
    wx.navigateTo({
      url: '/pages/login/login',
    })
  },

  goEditProfile() {
    wx.navigateTo({
      url: '/pages/registerProfile/registerProfile?mode=edit',
    })
  },

  goMyDNATags() {
    if (!this.data.loggedIn) {
      this.goLogin()
      return
    }

    wx.navigateTo({
      url: '/pages/dnaTag/dnaTag?mode=edit',
    })
  },

```

```
  goAllOrders() {
    wx.navigateTo({
      url: '/pages/myOrders/myOrders?tab=all',
    })
  },

  goOrderTab(e) {
    const { tab } = e.currentTarget.dataset
    if (tab === 'pending') {
      wx.navigateTo({
        url: '/pages/reviewCenter/reviewCenter?tab=pending',
      })
      return
    }

    wx.navigateTo({
      url: `/pages/myOrders/myOrders?tab=${tab || 'all'}`,
    })
  },

  goFavorites() {
    wx.navigateTo({
      url: '/pages/favorites/favorites',
    })
  },

  goHistory() {
    wx.showToast({
      title: TEXT.historyDeveloping,
      icon: 'none',
    })
  },

  goCustomerService() {
    wx.showToast({
      title: TEXT.customerServiceDeveloping,
      icon: 'none',
    })
  },

  goSystemSetting() {
    wx.showActionSheet({
      itemList: [TEXT.clearLoginState, TEXT.clearLocalTestData],
      success: (res) => {
        if (res.tapIndex === 0) {
          app.clearUserInfo()
          wx.showToast({
            title: TEXT.clearDone,
            icon: 'success',
          })
          setTimeout(() => {
            wx.reLaunch({
              url: '/pages/home/home',
            })
          }, 250)
          return
        }

        if (res.tapIndex === 1) {
          app.clearLocalTestData()
          wx.showToast({
            title: TEXT.clearDone,
```

```
            icon: 'success',
          })
          setTimeout(() => {
            wx.reLaunch({
              url: '/pages/home/home',
            })
          }, 250)
        }
      },
      fail: () => {},
    })
  },

  logout() {
    app.clearUserInfo()
    this.setData({
      loggedIn: false,
      userInfo: {
        avatarUrl: GUEST_AVATAR,
        nickName: '',
      },
      dnaTags: [],
    })

    wx.showToast({
      title: TEXT.logoutSuccess,
      icon: 'success',
    })

    setTimeout(() => {
      wx.reLaunch({
        url: '/pages/home/home',
      })
    }, 300)
  },
})
```

Path: miniprogram/pages/me/me.json

```
1 {
2   "navigationStyle": "custom"
3 }
```

Path: miniprogram/pages/me/me.wxml

```
<view class="page">
  <view class="topSpacer" style="padding-top: {{statusBarHeight}}px;"></view>

  <view class="header">
    <view class="userBox {{loggedIn ? 'userBoxLoggedIn' : 'userBoxGuest'}}">
      <image
        class="avatar"
        src="{{userInfo.avatarUrl}}"
        mode="aspectFill"
        bindtap="onUserBoxTap"
        binderror="onAvatarError"
      ></image>

      <view class="userInfo">
        <block wx:if="{{loggedIn}}">
          <view class="nickname">{{userInfo.nickName}}</view>
          <view class="edit" bindtap="goEditProfile">{{text.editProfile}}</view>
        </block>

        <block wx:else>
          <view class="guestRow">
            <view class="guestHint" bindtap="onUserBoxTap">{{text.guestHint}}</view>
            <view class="loginButton" bindtap="goLogin">{{text.authLogin}}</view>
          </view>
        </block>
      </view>
    </view>
  </view>

  <view class="dnaTagsBox" bindtap="goMyDNATags">
    <view class="dnaTagsLabel">{{text.myDNA}}</view>
    <view wx:if="{{dnaTags.length > 0}}" class="dnaTags">
      <view class="tag" wx:for="{{dnaTags}}" wx:key="*this">{{item}}</view>
    </view>
    <view wx:else class="dnaPlaceholder">{{dnaPlaceholderText}}</view>
  </view>

  <view class="orderSection">
    <view class="orderHeader">
      <view class="orderTitle">{{text.myOrders}}</view>
      <view class="orderAll" bindtap="goAllOrders">
        <text>{{text.allOrders}}</text>
        <text class="orderAllArrow">></text>
      </view>
    </view>

    <view class="orderIcons">
      <view
        class="orderItem"
        wx:for="{{orderEntries}}"
        wx:key="key"
        data-tab="{{item.tab}}"
        bindtap="goOrderTab"
      >
        <image class="orderIcon" src="{{item.icon}}" mode="aspectFit"></image>
        <text class="orderLabel">{{item.label}}</text>
      </view>
    </view>
  </view>

```

```
  <view class="menu">
    <view class="menuItem" bindtap="goFavorites">
      <text class="menuText">{{text.favorites}}</text>
      <text class="arrow">></text>
    </view>

    <view class="menuItem" bindtap="goCustomerService">
      <text class="menuText">{{text.customerService}}</text>
      <text class="arrow">></text>
    </view>

    <view class="menuItem" bindtap="goSystemSetting">
      <text class="menuText">{{text.systemSettings}}</text>
      <text class="arrow">></text>
    </view>

    <view class="menuItem logout" bindtap="logout">
      <text class="menuText">{{text.logout}}</text>
      <text class="arrow">></text>
    </view>
  </view>
</view>
```

Path: miniprogram/pages/me/me.wxss

```
.page {
  min-height: 100vh;
  background: #f6faf4;
  padding-bottom: 30rpx;
}

.topSpacer {
  height: 72rpx;
}

.header {
  padding: 0 24rpx 20rpx;
}

.dnaTagsBox {
  margin: 12rpx 24rpx 18rpx;
  background: #ffffff;
  border-radius: 20rpx;
  padding: 20rpx;
}

.dnaTagsLabel {
  font-size: 30rpx;
  color: #1f2a1f;
  margin-bottom: 12rpx;
  font-weight: 400;
}

.dnaTags {
  display: flex;
  flex-wrap: wrap;
  gap: 8rpx;
}

.dnaPlaceholder {
  font-size: 24rpx;
  color: #8ea08b;
  line-height: 1.6;
}

.buddyIntentSummary {
  font-size: 24rpx;
  color: #426046;
  line-height: 1.7;
}

.tag {
  background: #edf5ea;
  border: 1rpx solid #57984a;
  color: #57984a;
  padding: 6rpx 12rpx;
  border-radius: 16rpx;
  font-size: 24rpx;
}

.userBox {
  display: flex;
  align-items: center;
}

```

```
.userBoxLoggedIn {
  padding: 18rpx 8rpx 6rpx 8rpx;
}

.userBoxGuest {
  padding: 18rpx 8rpx 6rpx 8rpx;
}

.avatar {
  width: 124rpx;
  height: 124rpx;
  border-radius: 999rpx;
  background: #e5efe1;
  flex-shrink: 0;
}

.userInfo {
  margin-left: 22rpx;
  flex: 1;
}

.nickname {
  font-size: 36rpx;
  font-weight: 700;
  color: #1f2a1f;
}

.edit {
  margin-top: 10rpx;
  font-size: 24rpx;
  color: #99a598;
}

.guestRow {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 20rpx;
}

.guestHint {
  flex: 1;
  font-size: 27rpx;
  font-weight: 500;
  color: #1f2a1f;
  line-height: 1.45;
}

.loginButton {
  flex-shrink: 0;
  min-width: 120rpx;
  height: 62rpx;
  padding: 0 18rpx;
  border-radius: 16rpx;
  background: #57984a;
  color: #ffffff;
  font-size: 24rpx;
  font-weight: 600;
  display: flex;
  align-items: center;
  justify-content: center;
  box-shadow: 0 8rpx 18rpx rgba(87, 152, 74, 0.18);
```

```
}

.orderSection {
  margin: 12rpx 24rpx 18rpx;
  background: #ffffff;
  border-radius: 20rpx;
  padding: 24rpx 20rpx 18rpx;
}

.orderHeader {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 28rpx;
}

.orderTitle {
  font-size: 30rpx;
  font-weight: 400;
  color: #1f2a1f;
}

.orderAll {
  display: flex;
  align-items: center;
  font-size: 24rpx;
  color: #8ea08b;
}

.orderAllArrow {
  margin-left: 8rpx;
  font-size: 30rpx;
}

.orderIcons {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 8rpx;
}

.orderItem {
  flex: 1;
  display: flex;
  flex-direction: column;
  align-items: center;
}

.orderIcon {
  width: 58rpx;
  height: 58rpx;
}

.orderLabel {
  margin-top: 14rpx;
  font-size: 24rpx;
  line-height: 1.4;
  color: #1f2a1f;
  text-align: center;
}

.menu {
```

```
  margin: 0 24rpx;
  background: #ffffff;
  border-radius: 20rpx;
  overflow: hidden;
}

.menuItem {
  padding: 26rpx 22rpx;
  display: flex;
  align-items: center;
  justify-content: space-between;
  border-bottom: 1rpx solid #e3ece0;
}

.menuItem:last-child {
  border-bottom: none;
}

.menuText {
  font-size: 28rpx;
  color: #1f2a1f;
}

.arrow {
  font-size: 34rpx;
  color: #bbb;
  margin-left: 12rpx;
}

.logout .menuText {
  color: #d9544f;
}
```

Path: cloudfunctions/login/index.js

```
   1 const cloud = require('wx-server-sdk')
   2
   3 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })
   4 const db = cloud.database()
   5
   6 function buildNewUser(openid) {
   7   return {
   8     openid,
   9     nickName: '',
  10     avatarUrl: '',
  11     phoneNumber: '',
  12     hasBoundPhone: true,
  13     gender: 'unknown',
  14     birthDate: '',
  15     dnaTags: [],
  16     buddyTags: [],
  17     buddyIntent: {
  18       availability: '',
  19       buddyType: '',
  20       acceptCarpool: '',
  21       groupPreference: '',
  22     },
  23     buddyIntentCompleted: false,
  24     role: 'user',
  25     profileCompleted: false,
  26     dnaCompleted: false,
  27     onboardingCompleted: false,
  28     locationAuthorized: false,
  29     locationChoiceMade: false,
  30     userLocation: null,
  31     createdAt: db.serverDate(),
  32     updatedAt: db.serverDate(),
  33   }
  34 }
  35
  36 async function getUserByOpenid(openid) {
  37   const res = await db.collection('users').where({ openid }).limit(1).get()
  38   return res.data[0] || null
  39 }
  40
  41 async function ensureWechatUser(openid) {
  42   const existingUser = await getUserByOpenid(openid)
  43   if (existingUser) {
  44     if (existingUser.hasBoundPhone === false) {
  45       await db.collection('users').doc(existingUser._id).update({
  46         data: {
  47           hasBoundPhone: true,
  48           updatedAt: db.serverDate(),
  49         },
  50       })
  51       const fresh = await db.collection('users').doc(existingUser._id).get()
  52       return { success: true, userInfo: fresh.data, isNewUser: false }
  53     }
  54     return { success: true, userInfo: existingUser, isNewUser: false }
  55   }
  56
  57   const newUser = buildNewUser(openid)
  58   const addRes = await db.collection('users').add({ data: newUser })
  59   return {
  60     success: true,
```

```
    userInfo: { _id: addRes._id, ...newUser },
    isNewUser: true,
  }
}

exports.main = async (event = {}) => {
  const { method } = event
  const { OPENID } = cloud.getWXContext()

  try {
    if (method === 'wechat') return await ensureWechatUser(OPENID)
    return { success: false, message: 'UNSUPPORTED_LOGIN_METHOD' }
  } catch (err) {
    console.error('[login cloud] failed', method, err)
    return {
      success: false,
      message: err.code || err.message || 'LOGIN_FAILED',
    }
  }
}
```

Path: cloudfunctions/login/package.json

```
1 {
2   "name": "login",
3   "version": "1.0.0",
4   "dependencies": {
5     "wx-server-sdk": "latest"
6   }
7 }
```

Path: cloudfunctions/getactivities/index.js

```
1 const cloud = require('wx-server-sdk')
2
3 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })
4 const db = cloud.database()
5
6 exports.main = async () => {
7   const res = await db.collection('activities')
8     .where({ status: 'published' })
9     .orderBy('updatedAt', 'desc')
10    .limit(20)
11    .get()
12
13  return { list: res.data }
14 }
```

Path: cloudfunctions/getactivities/package.json

```
1 {
2   "name": "getactivities",
3   "version": "1.0.0",
4   "dependencies": {
5     "wx-server-sdk": "latest"
6   }
7 }
```

Path: cloudfunctions/getactivitydetail/index.js

```
1 const cloud = require('wx-server-sdk')
2
3 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })
4 const db = cloud.database()
5
6 exports.main = async (event) => {
7   const { id } = event
8   if (!id) return { ok: false, error: 'missing id' }
9
10   const res = await db.collection('activities').doc(id).get()
11   return { ok: true, data: res.data }
12 }
```

Path: cloudfunctions/getactivitydetail/package.json

```
1 {
2   "name": "getactivitydetail",
3   "version": "1.0.0",
4   "dependencies": {
5     "wx-server-sdk": "latest"
6   }
7 }
```

Path: cloudfunctions/activityOrder/index.js

```
---------------------------------------------------------------------
   1 const cloud = require('wx-server-sdk')
   2
   3 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })
   4 const db = cloud.database()
   5 const _ = db.command
   6
   7 const STATUS_TEXT = {
   8   pending_payment: '\u5f85\u652f\u4ed8',
   9   upcoming: '\u5f85\u51fa\u884c',
  10   pending_review: '\u5f85\u8bc4\u4ef7',
  11   refunding: '\u9000\u6b3e\u4e2d',
  12   cancelled: '\u5df2\u53d6\u6d88',
  13   completed: '\u4ea4\u6613\u6210\u529f',
  14 }
  15
  16 function normalizeText(value) {
  17   return String(value || '').trim()
  18 }
  19
  20 function normalizeNumber(value, fallback = 0) {
  21   const num = Number(value)
  22   return Number.isFinite(num) ? num : fallback
  23 }
  24
  25 function normalizePhone(value) {
  26   return normalizeText(value).replace(/\s+/g, '')
  27 }
  28
  29 function toDateStart(dateText) {
  30   const normalized = normalizeText(dateText)
  31   if (!normalized) return null
  32   const timestamp = new Date(`${normalized} 00:00:00`).getTime()
  33   return Number.isFinite(timestamp) ? timestamp : null
  34 }
  35
  36 function buildOrderStatus(order, now = Date.now()) {
  37   const rawStatus = normalizeText(order.status)
  38   const travelTs = toDateStart(order.travelDate)
  39   const activityDays = Math.max(1, normalizeNumber(order.activityDays || order.days, 1))
  40   const endTs = travelTs ? travelTs + activityDays * 24 * 60 * 60 * 1000 : null
  41   const after7Days = endTs ? endTs + 7 * 24 * 60 * 60 * 1000 : null
  42   const paymentDeadline = normalizeNumber(order.paymentDeadline, 0)
  43   const reviewed = !!order.reviewed
  44
  45   if (rawStatus === 'pending_payment') {
  46     if (paymentDeadline && now > paymentDeadline) {
  47       return { key: 'cancelled', text: STATUS_TEXT.cancelled }
  48     }
  49     return { key: 'pending_payment', text: STATUS_TEXT.pending_payment }
  50   }
  51
  52   if (rawStatus === 'refunding') {
  53     return { key: 'refunding', text: STATUS_TEXT.refunding }
  54   }
  55
  56   if (rawStatus === 'cancelled') {
  57     return { key: 'cancelled', text: STATUS_TEXT.cancelled }
  58   }
  59
  60   if (rawStatus === 'completed') {
```

```
    return { key: 'completed', text: STATUS_TEXT.completed }
  }

  if (rawStatus === 'upcoming') {
    if (endTs && now < endTs) {
      return { key: 'upcoming', text: STATUS_TEXT.upcoming }
    }
    if (after7Days && now <= after7Days && !reviewed) {
      return { key: 'pending_review', text: STATUS_TEXT.pending_review }
    }
    return { key: 'completed', text: STATUS_TEXT.completed }
  }

  if (rawStatus === 'pending_review') {
    if (after7Days && now <= after7Days && !reviewed) {
      return { key: 'pending_review', text: STATUS_TEXT.pending_review }
    }
    return { key: 'completed', text: STATUS_TEXT.completed }
  }

  return { key: 'completed', text: STATUS_TEXT.completed }
}

function buildActions(statusKey, order) {
  return {
    canPay: statusKey === 'pending_payment',
    canCancel: statusKey === 'pending_payment',
    canRefund: statusKey === 'upcoming' || statusKey === 'pending_review',
    canReview: statusKey === 'pending_review' && !order.reviewed,
  }
}

async function expirePendingOrderIfNeeded(order) {
  if (!order || order.status !== 'pending_payment') {
    return order
  }

  const paymentDeadline = normalizeNumber(order.paymentDeadline, 0)
  if (!paymentDeadline || Date.now() <= paymentDeadline) {
    return order
  }

  await db.collection('activityOrders').doc(order._id).update({
    data: {
      status: 'cancelled',
      cancelReason: '\u8d85\u65f6\u672a\u652f\u4ed8\u81ea\u52a8\u53d6\u6d88',
      cancelledAt: db.serverDate(),
      updatedAt: db.serverDate(),
    },
  })

  const fresh = await db.collection('activityOrders').doc(order._id).get()
  return fresh.data
}

function normalizeTraveler(item) {
  return {
    _id: item._id,
    name: normalizeText(item.name),
    idType: normalizeText(item.idType),
    idNo: normalizeText(item.idNo),
  }
```

```
}

function presentOrder(order) {
  const status = buildOrderStatus(order)
  const actions = buildActions(status.key, order)
  const totalPrice = normalizeNumber(order.unitPrice, 0) * normalizeNumber(order.totalCo
unt, 0)

  return {
    ...order,
    displayStatusKey: status.key,
    displayStatusText: status.text,
    totalPrice,
    actions,
    canRefundAfterTrip: status.key === 'pending_review',
  }
}

async function getCurrentUser(openid) {
  const res = await db.collection('users').where({ openid }).limit(1).get()
  return res.data[0] || null
}

async function requireCurrentUser(openid, options = {}) {
  const user = await getCurrentUser(openid)
  if (!user) {
    const error = new Error('WECHAT_LOGIN_REQUIRED')
    error.code = 'WECHAT_LOGIN_REQUIRED'
    throw error
  }

  return user
}

async function getOrderOwnedByUser(orderId, openid) {
  const id = normalizeText(orderId)
  if (!id) {
    const error = new Error('ORDER_ID_REQUIRED')
    error.code = 'ORDER_ID_REQUIRED'
    throw error
  }

  const res = await db.collection('activityOrders').doc(id).get()
  const order = res.data || null
  if (!order) {
    const error = new Error('ORDER_NOT_FOUND')
    error.code = 'ORDER_NOT_FOUND'
    throw error
  }

  const ownerOpenid = normalizeText(order.ownerOpenid || order.openid || order._openid)
  if (ownerOpenid !== openid) {
    const error = new Error('ORDER_NOT_FOUND')
    error.code = 'ORDER_NOT_FOUND'
    throw error
  }

  return expirePendingOrderIfNeeded(order)
}

async function createOrder(event, openid) {
  await requireCurrentUser(openid)
```

```

  const activityId = normalizeText(event.activityId)
  const travelDate = normalizeText(event.travelDate)
  const adultCount = normalizeNumber(event.adultCount, 0)
  const childCount = normalizeNumber(event.childCount, 0)
  const totalCount = adultCount + childCount
  const contact = event.contact || {}
  const specialRequest = normalizeText(event.specialRequest)
  const emergency = event.emergency || {}
  const travelerIds = Array.isArray(event.travelerIds)
    ? Array.from(new Set(event.travelerIds.map((item) => normalizeText(item)).filter(Boolean)))
    : []

  if (!activityId) {
    throw Object.assign(new Error('ACTIVITY_ID_REQUIRED'), { code: 'ACTIVITY_ID_REQUIRED' })
  }
  if (!travelDate || !toDateStart(travelDate)) {
    throw Object.assign(new Error('INVALID_TRAVEL_DATE'), { code: 'INVALID_TRAVEL_DATE' })
  }
  if (adultCount < 0 || childCount < 0 || totalCount <= 0) {
    throw Object.assign(new Error('INVALID_TRAVELER_COUNT'), { code: 'INVALID_TRAVELER_COUNT' })
  }
  if (travelerIds.length !== totalCount) {
    throw Object.assign(new Error('TRAVELER_COUNT_MISMATCH'), { code: 'TRAVELER_COUNT_MISMATCH' })
  }

  const contactName = normalizeText(contact.name)
  const contactPhone = normalizePhone(contact.phone)
  if (!contactName) {
    throw Object.assign(new Error('CONTACT_NAME_REQUIRED'), { code: 'CONTACT_NAME_REQUIRED' })
  }
  if (!/^1[3-9]\d{9}$/.test(contactPhone)) {
    throw Object.assign(new Error('INVALID_CONTACT_PHONE'), { code: 'INVALID_CONTACT_PHONE' })
  }

  const emergencyEnabled = !!emergency.enabled
  const emergencyName = normalizeText(emergency.name)
  const emergencyPhone = normalizePhone(emergency.phone)
  if (emergencyEnabled) {
    if (!emergencyName) {
      throw Object.assign(new Error('EMERGENCY_NAME_REQUIRED'), { code: 'EMERGENCY_NAME_REQUIRED' })
    }
    if (!/^1[3-9]\d{9}$/.test(emergencyPhone)) {
      throw Object.assign(new Error('INVALID_EMERGENCY_PHONE'), { code: 'INVALID_EMERGENCY_PHONE' })
    }
  }

  const activityRes = await db.collection('activities').doc(activityId).get()
  const activity = activityRes.data || null
  if (!activity || normalizeText(activity.status) !== 'published') {
    throw Object.assign(new Error('ACTIVITY_NOT_AVAILABLE'), { code: 'ACTIVITY_NOT_AVAILABLE' })
  }
```

```

  const travelerRes = await db.collection('travelers').where({
    _id: _.in(travelerIds),
  }).get()
  const travelers = (travelerRes.data || []).filter((item) => {
    const ownerOpenid = normalizeText(item.ownerOpenid || item.openid || item._openid)
    return ownerOpenid === openid
  })

  if (travelers.length !== travelerIds.length) {
    throw Object.assign(new Error('TRAVELER_NOT_FOUND'), { code: 'TRAVELER_NOT_FOUND' })
  }

  const travelersById = new Map(travelers.map((item) => [item._id, item]))
  const orderedTravelers = travelerIds.map((id) => normalizeTraveler(travelersById.get(id)))

  const now = Date.now()
  const paymentDeadline = now + 30 * 60 * 1000

  const payload = {
    ownerOpenid: openid,
    openid,
    _openid: openid,
    activityId,
    activityTitle: normalizeText(activity.title) || '\u672a\u547d\u540d\u6d3b\u52a8',
    activityDays: Math.max(1, normalizeNumber(activity.days, 1)),
    activityCover: normalizeText(activity.cover),
    unitPrice: normalizeNumber(activity.price, 0),
    travelDate,
    adultCount,
    childCount,
    totalCount,
    contact: {
      name: contactName,
      phone: contactPhone,
      qq: normalizeText(contact.qq),
      wechat: normalizeText(contact.wechat),
    },
    travelerIds,
    travelers: orderedTravelers,
    specialRequest,
    emergency: {
      enabled: emergencyEnabled,
      name: emergencyName,
      phone: emergencyEnabled ? emergencyPhone : '',
    },
    status: 'pending_payment',
    paymentDeadline,
    paidAt: null,
    reviewed: false,
    reviewedAt: null,
    review: null,
    refundStatus: '',
    cancelledAt: null,
    cancelReason: '',
    createdAt: db.serverDate(),
    updatedAt: db.serverDate(),
  }

  const addRes = await db.collection('activityOrders').add({ data: payload })
  const fresh = await db.collection('activityOrders').doc(addRes._id).get()
```

```

  return {
    success: true,
    order: presentOrder(fresh.data),
  }
}

async function listMine(event, openid) {
  await requireCurrentUser(openid)

  const page = Math.max(1, normalizeNumber(event.page, 1))
  const pageSize = Math.min(100, Math.max(1, normalizeNumber(event.pageSize, 20)))
  const skip = (page - 1) * pageSize

  const res = await db.collection('activityOrders')
    .where({ ownerOpenid: openid })
    .orderBy('createdAt', 'desc')
    .skip(skip)
    .limit(pageSize)
    .get()

  const normalized = []
  for (const item of (res.data || [])) {
    const fresh = await expirePendingOrderIfNeeded(item)
    normalized.push(presentOrder(fresh))
  }

  return {
    success: true,
    data: normalized,
  }
}

async function detail(event, openid) {
  await requireCurrentUser(openid)
  const order = await getOrderOwnedByUser(event.id, openid)
  return {
    success: true,
    data: presentOrder(order),
  }
}

async function mockPay(event, openid) {
  await requireCurrentUser(openid)
  const order = await getOrderOwnedByUser(event.id, openid)
  const status = buildOrderStatus(order)

  if (status.key !== 'pending_payment') {
    throw Object.assign(new Error('ORDER_CANNOT_PAY'), { code: 'ORDER_CANNOT_PAY' })
  }

  await db.collection('activityOrders').doc(order._id).update({
    data: {
      status: 'upcoming',
      paidAt: db.serverDate(),
      updatedAt: db.serverDate(),
    },
  })

  const fresh = await db.collection('activityOrders').doc(order._id).get()
  return {
    success: true,
```

```
    data: presentOrder(fresh.data),
  }
}

async function cancel(event, openid) {
  await requireCurrentUser(openid)
  const order = await getOrderOwnedByUser(event.id, openid)
  const status = buildOrderStatus(order)

  if (status.key !== 'pending_payment') {
    throw Object.assign(new Error('ORDER_CANNOT_CANCEL'), { code: 'ORDER_CANNOT_CANCEL' })
  }

  await db.collection('activityOrders').doc(order._id).update({
    data: {
      status: 'cancelled',
      cancelReason: '\u7528\u6237\u624b\u52a8\u53d6\u6d88',
      cancelledAt: db.serverDate(),
      updatedAt: db.serverDate(),
    },
  })

  const fresh = await db.collection('activityOrders').doc(order._id).get()
  return {
    success: true,
    data: presentOrder(fresh.data),
  }
}

async function requestRefund(event, openid) {
  await requireCurrentUser(openid)
  const order = await getOrderOwnedByUser(event.id, openid)
  const status = buildOrderStatus(order)

  if (!(status.key === 'upcoming' || status.key === 'pending_review')) {
    throw Object.assign(new Error('ORDER_CANNOT_REFUND'), { code: 'ORDER_CANNOT_REFUND' })
  }

  await db.collection('activityOrders').doc(order._id).update({
    data: {
      status: 'refunding',
      refundStatus: 'refunding',
      updatedAt: db.serverDate(),
    },
  })

  const fresh = await db.collection('activityOrders').doc(order._id).get()
  return {
    success: true,
    data: presentOrder(fresh.data),
  }
}

async function submitReview(event, openid) {
  await requireCurrentUser(openid)
  const order = await getOrderOwnedByUser(event.id, openid)
  const status = buildOrderStatus(order)
  const score = Math.min(5, Math.max(1, normalizeNumber(event.score, 5)))
  const content = normalizeText(event.content)

```

```
  if (status.key !== 'pending_review') {
    throw Object.assign(new Error('ORDER_CANNOT_REVIEW'), { code: 'ORDER_CANNOT_REVIEW' })
  }
  if (!content) {
    throw Object.assign(new Error('REVIEW_CONTENT_REQUIRED'), { code: 'REVIEW_CONTENT_REQUIRED' })
  }

  await db.collection('activityOrders').doc(order._id).update({
    data: {
      reviewed: true,
      reviewedAt: db.serverDate(),
      review: {
        score,
        content,
      },
      status: 'completed',
      updatedAt: db.serverDate(),
    },
  })

  const fresh = await db.collection('activityOrders').doc(order._id).get()
  return {
    success: true,
    data: presentOrder(fresh.data),
  }
}

exports.main = async (event = {}) => {
  const { OPENID } = cloud.getWXContext()
  const action = normalizeText(event.action)

  try {
    if (action === 'create') return await createOrder(event, OPENID)
    if (action === 'listMine') return await listMine(event, OPENID)
    if (action === 'detail') return await detail(event, OPENID)
    if (action === 'mockPay') return await mockPay(event, OPENID)
    if (action === 'cancel') return await cancel(event, OPENID)
    if (action === 'requestRefund') return await requestRefund(event, OPENID)
    if (action === 'submitReview') return await submitReview(event, OPENID)

    return {
      success: false,
      message: 'UNSUPPORTED_ACTION',
    }
  } catch (error) {
    console.error('[activityOrder] failed', action, error)
    return {
      success: false,
      message: error.code || error.message || 'ACTIVITY_ORDER_FAILED',
    }
  }
}
```

Path: cloudfunctions/activityOrder/package.json

```
1 {
2   "name": "activityOrder",
3   "version": "1.0.0",
4   "dependencies": {
5     "wx-server-sdk": "latest"
6   }
7 }
```

Path: cloudfunctions/getknowledgearticles/index.js

```
---------------------------------------------------------------------------
   1 const cloud = require('wx-server-sdk')
   2
   3 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })
   4 const db = cloud.database()
   5
   6 const COLLECTION_NAME = 'knowledgeArticles'
   7 const MAX_LIMIT = 100
   8 const CLOUD_ENV_ID = 'cloud1-3ghmr5ki7b1172fe.636c-cloud1-3ghmr5ki7b1172fe-1403917845'
   9 const CLOUD_KNOWLEDGE_COVER_PREFIX = `cloud://${CLOUD_ENV_ID}/knowledge/covers`
  10 const KNOWLEDGE_COVER_FILE_MAP = {
  11   'agri-001': 'spring-vegetable.png',
  12   'agri-002': 'village-chicken.png',
  13   'agri-003': 'corn-disease.png',
  14   'agri-004': 'bee-guide.png',
  15   'agri-005': 'rice-brand.png',
  16   'culture-001': 'summer-palace.png',
  17   'culture-002': 'intangible-heritage.png',
  18   'culture-003': 'folk-custom.png',
  19   'culture-004': 'travel-tips.png',
  20 }
  21
  22 function normalizeText(value = '') {
  23   return String(value || '').trim()
  24 }
  25
  26 function normalizeArray(value) {
  27   return Array.isArray(value) ? value : []
  28 }
  29
  30 function getFileName(value = '') {
  31   const normalized = normalizeText(value)
  32   if (!normalized) {
  33     return ''
  34   }
  35
  36   const segments = normalized.split('/')
  37   return normalizeText(segments[segments.length - 1])
  38 }
  39
  40 function buildKnowledgeCoverFileID(fileName = '') {
  41   const normalized = getFileName(fileName)
  42   return normalized ? `${CLOUD_KNOWLEDGE_COVER_PREFIX}/${normalized}` : ''
  43 }
  44
  45 function normalizeKnowledgeCover(doc = {}) {
  46   const cover = normalizeText(doc.cover)
  47   if (/^(cloud|https?):\/\//i.test(cover)) {
  48     return cover
  49   }
  50
  51   const coverFileName = getFileName(cover) || KNOWLEDGE_COVER_FILE_MAP[normalizeText(doc
     ._id)]
  52   return buildKnowledgeCoverFileID(coverFileName)
  53 }
  54
  55 function parseTime(value) {
  56   if (!value) {
  57     return 0
  58   }
  59
```

```
  if (typeof value === 'number') {
    return value
  }

  if (value instanceof Date) {
    return value.getTime()
  }

  if (typeof value === 'string') {
    const timestamp = Date.parse(value.replace(/-/g, '/'))
    return Number.isNaN(timestamp) ? 0 : timestamp
  }

  if (typeof value === 'object') {
    if (typeof value.getTime === 'function') {
      return value.getTime()
    }
    if (value.$date) {
      return parseTime(value.$date)
    }
  }

  return 0
}

function buildListItem(doc = {}) {
  return {
    id: doc._id,
    channel: normalizeText(doc.channel) || 'agri',
    title: normalizeText(doc.title),
    publishTime: normalizeText(doc.publishTime),
    tags: normalizeArray(doc.tags).filter(Boolean),
    wordCount: Number(doc.wordCount) || 0,
    summary: normalizeText(doc.summary),
    cover: normalizeKnowledgeCover(doc),
    views: Number(doc.views) || 0,
    likes: Number(doc.likes) || 0,
    favorites: Number(doc.favorites) || 0,
    comments: Number(doc.comments) || 0,
    shareCount: Number(doc.shareCount) || 0,
    author: normalizeText(doc.author),
    location: doc.location || null,
  }
}

function sortArticles(list = []) {
  return [...list].sort((a, b) => {
    const pinnedDiff = Number(b.isPinned) - Number(a.isPinned)
    if (pinnedDiff !== 0) {
      return pinnedDiff
    }

    const timeDiff = parseTime(b.publishTime) - parseTime(a.publishTime)
    if (timeDiff !== 0) {
      return timeDiff
    }

    return parseTime(b.updatedAt) - parseTime(a.updatedAt)
  })
}

exports.main = async (event = {}) => {
```

```
  const channel = normalizeText(event.channel) || 'agri'
  const res = await db.collection(COLLECTION_NAME)
    .where({
      status: 'published',
      channel,
    })
    .limit(MAX_LIMIT)
    .get()

  const list = sortArticles(res.data || [])
  const categories = ['  ', ...new Set(
    list.flatMap((item) => normalizeArray(item.tags).filter(Boolean))
  )]

  return {
    ok: true,
    channel,
    categories,
    list: list.map(buildListItem),
  }
}
```

Path: cloudfunctions/getknowledgearticles/package.json

```
{
  "name": "getknowledgearticles",
  "version": "1.0.0",
  "description": "          ",
  "main": "index.js",
  "dependencies": {
    "wx-server-sdk": "latest"
  }
}
```

Path: cloudfunctions/getknowledgearticledetail/index.js

```
---------------------------------------------------------------------------
const cloud = require('wx-server-sdk')

cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })
const db = cloud.database()
const _ = db.command

const COLLECTION_NAME = 'knowledgeArticles'
const CLOUD_ENV_ID = 'cloud1-3ghmr5ki7b1172fe.636c-cloud1-3ghmr5ki7b1172fe-1403917845'
const CLOUD_KNOWLEDGE_COVER_PREFIX = `cloud://${CLOUD_ENV_ID}/knowledge/covers`
const KNOWLEDGE_COVER_FILE_MAP = {
  'agri-001': 'spring-vegetable.png',
  'agri-002': 'village-chicken.png',
  'agri-003': 'corn-disease.png',
  'agri-004': 'bee-guide.png',
  'agri-005': 'rice-brand.png',
  'culture-001': 'summer-palace.png',
  'culture-002': 'intangible-heritage.png',
  'culture-003': 'folk-custom.png',
  'culture-004': 'travel-tips.png',
}

function normalizeText(value = '') {
  return String(value || '').trim()
}

function normalizeArray(value) {
  return Array.isArray(value) ? value : []
}

function getFileName(value = '') {
  const normalized = normalizeText(value)
  if (!normalized) {
    return ''
  }

  const segments = normalized.split('/')
  return normalizeText(segments[segments.length - 1])
}

function buildKnowledgeCoverFileID(fileName = '') {
  const normalized = getFileName(fileName)
  return normalized ? `${CLOUD_KNOWLEDGE_COVER_PREFIX}/${normalized}` : ''
}

function normalizeKnowledgeCover(article = {}) {
  const cover = normalizeText(article.cover)
  if (/^(cloud|https?):\/\//i.test(cover)) {
    return cover
  }

  const coverFileName = getFileName(cover) || KNOWLEDGE_COVER_FILE_MAP[normalizeText(art
icle._id)]
  return buildKnowledgeCoverFileID(coverFileName)
}

exports.main = async (event = {}) => {
  const id = normalizeText(event.id)
  if (!id) {
    return { ok: false, error: 'missing id' }
  }
```

```

  const docRes = await db.collection(COLLECTION_NAME).doc(id).get()
  const article = docRes.data || null
  if (!article || article.status !== 'published') {
    return { ok: false, error: 'article not found' }
  }

  await db.collection(COLLECTION_NAME).doc(id).update({
    data: {
      views: _.inc(1),
      updatedAt: new Date(),
    },
  })

  return {
    ok: true,
    data: {
      id: article._id,
      channel: normalizeText(article.channel) || 'agri',
      title: normalizeText(article.title),
      publishTime: normalizeText(article.publishTime),
      tags: normalizeArray(article.tags).filter(Boolean),
      wordCount: Number(article.wordCount) || 0,
      summary: normalizeText(article.summary),
      cover: normalizeKnowledgeCover(article),
      views: Number(article.views) + 1,
      likes: Number(article.likes) || 0,
      favorites: Number(article.favorites) || 0,
      comments: Number(article.comments) || 0,
      shareCount: Number(article.shareCount) || 0,
      author: normalizeText(article.author),
      content: normalizeArray(article.content),
      location: article.location || null,
      commentList: normalizeArray(article.commentList),
    },
  }
}
```

Path: cloudfunctions/getknowledgearticledetail/package.json

```
1 {
2   "name": "getknowledgearticledetail",
3   "version": "1.0.0",
4   "description": "           ",
5   "main": "index.js",
6   "dependencies": {
7     "wx-server-sdk": "latest"
8   }
9 }
```

Path: cloudfunctions/getMyOrders/index.js

```
   1 const cloud = require('wx-server-sdk')
   2 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })
   3 const db = cloud.database()
   4
   5 exports.main = async (event, context) => {
   6   const { OPENID } = cloud.getWXContext()
   7   const { status, page = 1, pageSize = 20 } = event || {}
   8
   9   const where = { _openid: OPENID }
  10
  11   const allowed = new Set(['reserved', 'ongoing', 'cancelled', 'finished'])
  12   if (status && allowed.has(status)) where.status = status
  13
  14   const skip = (Number(page) - 1) * Number(pageSize)
  15
  16   const res = await db.collection('activityOrders')
  17     .where(where)
  18     .orderBy('createdAt', 'desc')
  19     .skip(skip)
  20     .limit(Number(pageSize))
  21     .get()
  22
  23   return { data: res.data }
  24 }
```

Path: cloudfunctions/getMyOrders/package.json

```
1 {
2   "name": "getMyOrders",
3   "version": "1.0.0",
4   "dependencies": {
5     "wx-server-sdk": "^3.0.0"
6   }
7 }
```

Path: cloudfunctions/getMyOrderCounts/index.js

```
1 const cloud = require('wx-server-sdk')
2 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })
3 const db = cloud.database()
4
5 function ymd(date) {
6   const d = new Date(date)
7   const pad = (x) => String(x).padStart(2, '0')
8   return `${d.getFullYear()}-${pad(d.getMonth()+1)}-${pad(d.getDate())}`
9 }
10
11 function classify(order) {
12   if (order.status === 'cancelled' || order.cancelled === true) return 'cancelled'
13   const td = order.travelDate
14   if (!td) return 'comfirmed'
15   const today = ymd(new Date())
16   if (td > today) return 'comfirmed'
17   if (td === today) return 'ongoing'
18   return 'completed'
19 }
20
21 exports.main = async (event, context) => {
22   const { OPENID } = cloud.getWXContext()
23
24   const res = await db.collection('activityOrders')
25     .where({ _openid: OPENID })
26     .get()
27
28   const counts = { comfirmed: 0, ongoing: 0, cancelled: 0, completed: 0 }
29   for (const o of (res.data || [])) {
30     const s = classify(o)
31     counts[s] = (counts[s] || 0) + 1
32   }
33   return counts
34 }
```

Path: cloudfunctions/getMyOrderCounts/package.json

```
1 {
2   "name": "getMyOrderCounts",
3   "version": "1.0.0",
4   "dependencies": {
5     "wx-server-sdk": "^3.0.0"
6   }
7 }
```

Path: cloudfunctions/userManage/index.js

```
   1 const https = require('https')
   2 const cloud = require('wx-server-sdk')
   3
   4 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })
   5 const db = cloud.database()
   6 const TENCENT_MAP_KEY = process.env.TENCENT_MAP_KEY || process.env.QQ_MAP_KEY || ''
   7
   8 async function getCurrentUserRecord() {
   9   const { OPENID } = cloud.getWXContext()
  10   const res = await db.collection('users').where({ openid: OPENID }).limit(1).get()
  11
  12   if (!res.data.length) {
  13     return null
  14   }
  15
  16   return res.data[0]
  17 }
  18
  19 async function saveAndFetchUser(userId, data) {
  20   await db.collection('users').doc(userId).update({
  21     data: {
  22       ...data,
  23       updatedAt: db.serverDate(),
  24     },
  25   })
  26
  27   const fresh = await db.collection('users').doc(userId).get()
  28   return fresh.data
  29 }
  30
  31 function requestJson(url) {
  32   return new Promise((resolve, reject) => {
  33     https
  34       .get(url, (res) => {
  35         let raw = ''
  36         res.on('data', (chunk) => {
  37           raw += chunk
  38         })
  39         res.on('end', () => {
  40           try {
  41             resolve(JSON.parse(raw))
  42           } catch (error) {
  43             reject(error)
  44           }
  45         })
  46       })
  47       .on('error', reject)
  48   })
  49 }
  50
  51 function normalizeNumber(value) {
  52   const number = Number(value)
  53   return Number.isFinite(number) ? number : null
  54 }
  55
  56 function normalizeText(value) {
  57   return typeof value === 'string' ? value.trim() : ''
  58 }
  59
  60 function normalizeArray(value) {
```

```
  return Array.isArray(value)
    ? value.map((item) => normalizeText(item)).filter(Boolean)
    : []
}

function uniqueList(list = []) {
  return Array.from(new Set((list || []).filter(Boolean)))
}

function trimRegionSuffix(value) {
  return normalizeText(value).replace(/(     |   |   |  | | | | | | )$/u, '')
}

function getSharedTags(currentUser = {}, candidate = {}) {
  const currentTags = new Set(normalizeArray(currentUser.dnaTags))
  return normalizeArray(candidate.dnaTags).filter((tag) => currentTags.has(tag))
}

function deriveBuddyTagsFromIntent(intent = {}) {
  const normalized = normalizeBuddyIntent(intent)
  const tags = []

  if (normalized.buddyType === 'casual') tags.push('    ')
  if (normalized.buddyType === 'photo') tags.push('    ')
  if (normalized.buddyType === 'parent_child') tags.push('    ')
  if (normalized.buddyType === 'free_travel') tags.push('     ')
  if (normalized.acceptCarpool === 'yes') tags.push('    ')
  if (normalized.groupPreference === 'two') tags.push('    ')
  if (normalized.groupPreference === 'small_group') tags.push('3-4   ')
  if (normalized.groupPreference === 'flexible') tags.push('    ')

  return uniqueList(tags)
}

function getBuddyTags(user = {}) {
  const directTags = normalizeArray(user.buddyTags)
  if (directTags.length) {
    return directTags
  }
  return deriveBuddyTagsFromIntent(user.buddyIntent)
}

function getSharedBuddyTags(currentUser = {}, candidate = {}) {
  const currentTags = new Set(getBuddyTags(currentUser))
  return getBuddyTags(candidate).filter((tag) => currentTags.has(tag))
}

function buildBuddyExplanation(currentUser = {}, candidate = {}, sharedTags = []) {
  const parts = []
  const sharedBuddyTags = getSharedBuddyTags(currentUser, candidate)
  const currentCity = trimRegionSuffix(currentUser.city)
  const candidateCity = trimRegionSuffix(candidate.city)
  const currentProvince = trimRegionSuffix(currentUser.province)
  const candidateProvince = trimRegionSuffix(candidate.province)

  if (currentCity && candidateCity && currentCity === candidateCity) {
    parts.push('  ')
  } else if (currentProvince && candidateProvince && currentProvince === candidateProvin
ce) {
    parts.push('  ')
  }

```

```
  if (sharedBuddyTags.length) {
    parts.push(`       ${Math.min(sharedBuddyTags.length, 3)}  `)
  }

  if (sharedTags.length) {
    parts.push(`DNA      ${Math.min(sharedTags.length, 3)}  `)
  }

  if (candidate.profileCompleted && candidate.dnaCompleted) {
    parts.push('     ')
  }

  return parts
}

function buildBuddyRealSummary(candidate = {}) {
  const buddyTags = getBuddyTags(candidate)
  if (buddyTags.length) {
    return `     ${buddyTags.slice(0, 3).join(' ')}`
  }

  const cityText = trimRegionSuffix(candidate.city) || trimRegionSuffix(candidate.provin
ce) || ''
  if (cityText) {
    return `       ${cityText}`
  }

  return '                                    '
}

function getAgeFromBirthDate(value) {
  const text = normalizeText(value)
  if (!text) return null
  const date = new Date(text)
  if (Number.isNaN(date.getTime())) return null

  const now = new Date()
  let age = now.getFullYear() - date.getFullYear()
  const monthDiff = now.getMonth() - date.getMonth()
  const dayDiff = now.getDate() - date.getDate()
  if (monthDiff < 0 || (monthDiff === 0 && dayDiff < 0)) {
    age -= 1
  }
  return age >= 0 ? age : null
}

function getAgeRangeLabel(age) {
  if (age === null) return ''
  if (age < 18) return '18   '
  if (age < 25) return '18-24 '
  if (age < 35) return '25-34 '
  if (age < 45) return '35-44 '
  if (age < 60) return '45-59 '
  return '60   '
}

function inferBuddyType(tags = []) {
  const joined = normalizeArray(tags).join(' ')
  if (joined.includes('  ')) return '    '
  if (joined.includes('  ')) return '    '
  if (joined.includes('   ') || joined.includes('    ')) return '     '
  if (joined.includes('  ') || joined.includes('  ')) return '      '
```

```
  return '     '
}

function getBuddyTypeLabel(value) {
  const map = {
    casual: '      ',
    photo: '    ',
    parent_child: '    ',
    free_travel: '     ',
  }
  return map[normalizeText(value)] || ''
}

function pickStatusTag(currentUser = {}, candidate = {}, sharedTags = []) {
  const currentCity = trimRegionSuffix(currentUser.city)
  const candidateCity = trimRegionSuffix(candidate.city)
  if (currentCity && candidateCity && currentCity === candidateCity) {
    return '      '
  }

  if (sharedTags.length >= 2) {
    return `${sharedTags.slice(0, 2).join(' · ')}`
  }

  return '    '
}

function buildBuddySummary(candidate = {}, sharedTags = []) {
  const cityText = trimRegionSuffix(candidate.city) || trimRegionSuffix(candidate.province) || '   '
  const tags = normalizeArray(candidate.dnaTags)
  const focusTags = sharedTags.length ? sharedTags : tags.slice(0, 3)
  const buddyType = inferBuddyType(focusTags)
  const tagText = focusTags.length ? focusTags.join(' ') : '     '
  return `  ${cityText}    ${tagText}     ${buddyType}     `
}

function buildMatchReason(currentUser = {}, candidate = {}, sharedTags = []) {
  const reasons = []
  const currentCity = trimRegionSuffix(currentUser.city)
  const candidateCity = trimRegionSuffix(candidate.city)
  const currentProvince = trimRegionSuffix(currentUser.province)
  const candidateProvince = trimRegionSuffix(candidate.province)

  if (currentCity && candidateCity && currentCity === candidateCity) {
    reasons.push('                ')
  } else if (currentProvince && candidateProvince && currentProvince === candidateProvince) {
    reasons.push('                ')
  }

  if (sharedTags.length) {
    reasons.push(`DNA      ${sharedTags.slice(0, 3).join(' ')}`)
  }

  const candidateAge = getAgeFromBirthDate(candidate.birthDate)
  const currentAge = getAgeFromBirthDate(currentUser.birthDate)
  if (candidateAge !== null && currentAge !== null && Math.abs(candidateAge - currentAge) <= 8) {
    reasons.push('                ')
  }

```

```
  return reasons.slice(0, 2).join(' ') || '                          '
}

function buildBuddyOpeningText(question, candidate = {}, sharedTags = []) {
  const text = normalizeText(question)
  const title = inferBuddyType(sharedTags.length ? sharedTags : normalizeArray(candidate.dnaTags))
  if (text) {
    return `       “${text}”                                `
  }
  return `        ${title}                      `
}

function normalizeBuddyIntent(input = {}) {
  return {
    availability: normalizeText(input.availability),
    buddyType: normalizeText(input.buddyType),
    acceptCarpool: normalizeText(input.acceptCarpool),
    groupPreference: normalizeText(input.groupPreference),
  }
}

function buildBuddyIntentFromTags(tags = []) {
  const list = normalizeArray(tags)

  return normalizeBuddyIntent({
    availability: '',
    buddyType: list.includes('    ')
      ? 'photo'
      : list.includes('    ')
        ? 'parent_child'
        : list.includes('     ')
          ? 'free_travel'
          : list.includes('    ')
            ? 'casual'
            : '',
    acceptCarpool: list.includes('    ') ? 'yes' : '',
    groupPreference: list.includes('    ')
      ? 'two'
      : list.includes('3-4   ')
        ? 'small_group'
        : list.includes('    ')
          ? 'flexible'
          : '',
  })
}

function isBuddyIntentCompleted(intent = {}) {
  const normalized = normalizeBuddyIntent(intent)
  return !!(
    normalized.availability &&
    normalized.buddyType &&
    normalized.acceptCarpool &&
    normalized.groupPreference
  )
}

function getQuestionIntentTokens(question = '') {
  const text = normalizeText(question)
  const tokens = []
  if (text.includes('  ')) tokens.push('saturday')
  if (text.includes('  ')) tokens.push('sunday')
```

```
  if (text.includes('  ')) tokens.push('weekend')
  if (text.includes('  ')) tokens.push('photo')
  if (text.includes('  ')) tokens.push('parent_child')
  if (text.includes('  ')) tokens.push('carpool_yes')
  if (text.includes('   ') || text.includes('2 ')) tokens.push('group_two')
  if (text.includes('  ') || text.includes('3') || text.includes('4')) tokens.push('group_small')
  return tokens
}

function scoreBuddyCandidate(currentUser = {}, candidate = {}, question = '') {
  let score = 45
  const sharedTags = getSharedTags(currentUser, candidate)
  const currentIntent = normalizeBuddyIntent(currentUser.buddyIntent)
  const candidateIntent = normalizeBuddyIntent(candidate.buddyIntent)
  const questionTokens = getQuestionIntentTokens(question)
  score += Math.min(sharedTags.length * 14, 42)

  const currentCity = trimRegionSuffix(currentUser.city)
  const candidateCity = trimRegionSuffix(candidate.city)
  const currentProvince = trimRegionSuffix(currentUser.province)
  const candidateProvince = trimRegionSuffix(candidate.province)

  if (currentCity && candidateCity && currentCity === candidateCity) {
    score += 24
  } else if (currentProvince && candidateProvince && currentProvince === candidateProvince) {
    score += 14
  }

  if (candidate.profileCompleted) score += 4
  if (candidate.dnaCompleted) score += 4
  if (candidate.buddyIntentCompleted) score += 10
  if (normalizeText(candidate.avatarUrl)) score += 3
  if (normalizeText(candidate.nickName)) score += 3

  const currentAge = getAgeFromBirthDate(currentUser.birthDate)
  const candidateAge = getAgeFromBirthDate(candidate.birthDate)
  if (currentAge !== null && candidateAge !== null) {
    const diff = Math.abs(currentAge - candidateAge)
    if (diff <= 5) score += 8
    else if (diff <= 10) score += 4
  }

  const candidateText = normalizeArray(candidate.dnaTags).join(' ')
  normalizeArray(String(question || '').match(/[\u4e00-\u9fa5A-Za-z0-9]{2,8}/gu)).forEach((token) => {
    if (candidateText.includes(token)) {
      score += 5
    }
  })

  if (currentIntent.availability && candidateIntent.availability && currentIntent.availability === candidateIntent.availability) {
    score += 10
  }
  if (currentIntent.buddyType && candidateIntent.buddyType && currentIntent.buddyType === candidateIntent.buddyType) {
    score += 12
  }
  if (currentIntent.acceptCarpool && candidateIntent.acceptCarpool && currentIntent.acceptCarpool === candidateIntent.acceptCarpool) {
```

```
    score += 8
  }
  if (currentIntent.groupPreference && candidateIntent.groupPreference && currentIntent.groupPreference === candidateIntent.groupPreference) {
    score += 8
  }

  if (questionTokens.includes('saturday') && candidateIntent.availability === 'saturday') score += 10
  if (questionTokens.includes('sunday') && candidateIntent.availability === 'sunday') score += 10
  if (questionTokens.includes('weekend') && (candidateIntent.availability === 'weekend' || candidateIntent.availability === 'flexible')) score += 8
  if (questionTokens.includes('photo') && candidateIntent.buddyType === 'photo') score += 12
  if (questionTokens.includes('parent_child') && candidateIntent.buddyType === 'parent_child') score += 12
  if (questionTokens.includes('carpool_yes') && candidateIntent.acceptCarpool === 'yes') score += 10
  if (questionTokens.includes('group_two') && candidateIntent.groupPreference === 'two') score += 8
  if (questionTokens.includes('group_small') && candidateIntent.groupPreference === 'small_group') score += 8

  return Math.min(score, 98)
}

function buildBuddyCandidate(currentUser = {}, candidate = {}, question = '') {
  const sharedTags = getSharedTags(currentUser, candidate)
  const score = scoreBuddyCandidate(currentUser, candidate, question)
  const candidateIntent = normalizeBuddyIntent(candidate.buddyIntent)
  const cityText = trimRegionSuffix(candidate.city) || trimRegionSuffix(candidate.province) || '   '
  const ageText = getAgeRangeLabel(getAgeFromBirthDate(candidate.birthDate))
  const tags = uniqueList(sharedTags.concat(normalizeArray(candidate.dnaTags))).slice(0, 4)
  const practicalInfo = [
    { label: '    ', value: cityText },
    { label: '    ', value: getBuddyTypeLabel(candidateIntent.buddyType) || inferBuddyType(tags) },
  ]

  if (candidateIntent.availability) {
    const availabilityMap = {
      saturday: '     ',
      sunday: '     ',
      weekend: '     ',
      flexible: '     ',
    }
    practicalInfo.push({ label: '     ', value: availabilityMap[candidateIntent.availability] || candidateIntent.availability })
  }

  if (candidateIntent.acceptCarpool) {
    practicalInfo.push({
      label: '      ',
      value: candidateIntent.acceptCarpool === 'yes' ? '    ' : '    ',
    })
  }

  if (ageText) {
    practicalInfo.push({ label: '    ', value: ageText })
```

```
  }

  return {
    id: candidate._id,
    sourceId: candidate._id,
    userName: normalizeText(candidate.nickName) || '    ',
    avatarUrl: normalizeText(candidate.avatarUrl),
    avatarText: (normalizeText(candidate.nickName) || ' ').slice(0, 1),
    avatarColor: '#6e89ff',
    matchScore: score,
    matchScoreText: `    ${score}%`,
    statusTag: '',
    summary: buildBuddyRealSummary(candidate),
    matchReason: `       ${buildBuddyExplanation(currentUser, candidate, sharedTags).joi
n(' + ') || '                     '}`,
    tags,
    playItems: uniqueList([
      `     ${sharedTags.length ? sharedTags.slice(0, 3).join(' ') : '     '}`,
      `      ${candidate.profileCompleted && candidate.dnaCompleted ? '   ' : '     '}`,
      candidateIntent.groupPreference === 'two' ? '       ' : '',
      candidateIntent.groupPreference === 'small_group' ? '   3-4   ' : '',
    ]).filter(Boolean),
    practicalInfo,
    actionText: '      ',
    openingText: buildBuddyOpeningText(question, candidate, sharedTags),
  }
}

function buildDisplayName({ displayName, poiTitle, district, city, province, locationTex
t }) {
  return (
    normalizeText(displayName) ||
    normalizeText(poiTitle) ||
    stripAdminPrefix(normalizeText(locationText)) ||
    normalizeText(district) ||
    normalizeText(city) ||
    normalizeText(province) ||
    normalizeText(locationText)
  )
}

function stripAdminPrefix(text) {
  const raw = normalizeText(text)
  if (!raw) return ''
  return raw.replace(/^(?:(?:[\u4e00-\u9fa5]{2,12})(?: |     |   |  | | | | | )){1,4}/u, '
')
}

function scorePoi(item = {}) {
  const title = normalizeText(item.title || item.name)
  if (!title) {
    return -999
  }

  let score = 0
  if (/(  |  |  |  |  |  |  |  |  |  |  |  |  |  | )$/u.test(title)) score += 12
  if (/(    |  |   |   |   |   |   )$/u.test(title)) score -= 8
  if (/( | )$/u.test(title)) score += 4
  if (title.length <= 16) score += 3
  if (title.length >= 24) score -= 2

  const distance = normalizeNumber(item._distance || item.distance)
```

```
  if (distance !== null) {
    score += Math.max(0, 6 - distance / 200)
  }

  return score
}

function pickBestPoi(pois = []) {
  if (!Array.isArray(pois) || !pois.length) {
    return {}
  }

  return pois
    .map((item) => ({ item, score: scorePoi(item) }))
    .sort((a, b) => b.score - a.score)[0].item || {}
}

function hasOwn(obj, key) {
  return Object.prototype.hasOwnProperty.call(obj || {}, key)
}

async function reverseGeocodeByTencentMap(location) {
  if (!TENCENT_MAP_KEY) {
    console.warn('[userManage] TENCENT_MAP_KEY missing, skip reverse geocode')
    return null
  }

  const latitude = normalizeNumber(location && location.latitude)
  const longitude = normalizeNumber(location && location.longitude)
  if (latitude === null || longitude === null) {
    return null
  }

  const url =
    'https://apis.map.qq.com/ws/geocoder/v1/?output=json&get_poi=1' +
    '&poi_options=policy=5;radius=1000;page_size=5;page_index=1' +
    `&location=${encodeURIComponent(`${latitude},${longitude}`)}` +
    `&key=${encodeURIComponent(TENCENT_MAP_KEY)}`

  const result = await requestJson(url)
  if (!result || result.status !== 0 || !result.result) {
    console.warn('[userManage] reverse geocode failed', result)
    return null
  }

  const addressComponent = result.result.address_component || {}
  const adInfo = result.result.ad_info || {}
  const pois = Array.isArray(result.result.pois) ? result.result.pois : []
  const nearestPoi = pickBestPoi(pois)
  const poiTitle =
    stripAdminPrefix(normalizeText(nearestPoi.title) || normalizeText(nearestPoi.name)) ||
    ''

  return {
    province: addressComponent.province || adInfo.province || '',
    city: addressComponent.city || adInfo.city || '',
    district: addressComponent.district || adInfo.district || '',
    adcode: adInfo.adcode || '',
    poiTitle,
    displayName:
      poiTitle ||
```

```
      addressComponent.street ||
      addressComponent.street_number ||
      '',
    locationText:
      result.result.address ||
      (result.result.formatted_addresses && result.result.formatted_addresses.recommend) ||
      '',
  }
}

async function placeSuggestionByTencentMap(keyword, options = {}) {
  if (!TENCENT_MAP_KEY) {
    console.warn('[userManage] TENCENT_MAP_KEY missing, skip place suggestion')
    return []
  }

  const text = normalizeText(keyword)
  if (!text) {
    return []
  }

  const params = [
    `keyword=${encodeURIComponent(text)}`,
    `key=${encodeURIComponent(TENCENT_MAP_KEY)}`,
    'output=json',
    'get_ad_info=1',
    'region_fix=0',
    'policy=1',
    'page_size=10',
  ]

  const region = normalizeText(options.region)
  if (region) {
    params.push(`region=${encodeURIComponent(region)}`)
  }

  const latitude = normalizeNumber(options.latitude)
  const longitude = normalizeNumber(options.longitude)
  if (latitude !== null && longitude !== null) {
    params.push(`location=${encodeURIComponent(`${latitude},${longitude}`)}`)
  }

  const url = `https://apis.map.qq.com/ws/place/v1/suggestion?${params.join('&')}`
  const result = await requestJson(url)
  if (!result || result.status !== 0 || !Array.isArray(result.data)) {
    console.warn('[userManage] place suggestion failed', result)
    return []
  }

  return result.data.map((item) => ({
    title: item.title || '',
    address: item.address || '',
    adcode: String(item.adcode || ''),
    province: item.province || '',
    city: item.city || '',
    district: item.district || '',
    latitude: item.location && item.location.lat,
    longitude: item.location && item.location.lng,
  }))
}
```

```
exports.main = async (event) => {
  try {
    const user = await getCurrentUserRecord()
    if (!user) {
      return {
        success: false,
        message: '     ',
      }
    }

    if (event.action === 'updateProfile') {
      const profile = event.profile || {}
      const nextUser = await saveAndFetchUser(user._id, {
        avatarUrl: profile.avatarUrl || '',
        nickName: profile.nickName || '',
        gender: profile.gender || '  ',
        birthDate: profile.birthDate || '',
        profileCompleted: true,
      })

      return {
        success: true,
        userInfo: nextUser,
      }
    }

    if (event.action === 'updateOnboarding') {
      const payload = event.payload || {}
      let resolvedLocation = null

      try {
        resolvedLocation = payload.userLocation ? await reverseGeocodeByTencentMap(payload.userLocation) : null
      } catch (error) {
        console.error('[userManage] reverse geocode error', error)
        resolvedLocation = null
      }

      const nextUser = await saveAndFetchUser(user._id, {
        locationAuthorized: !!payload.locationAuthorized,
        locationChoiceMade: !!payload.locationChoiceMade,
        userLocation: payload.userLocation || null,
        province: (resolvedLocation && resolvedLocation.province) || user.province || '',
        city: (resolvedLocation && resolvedLocation.city) || user.city || '',
        district: (resolvedLocation && resolvedLocation.district) || user.district || '',
        adcode: (resolvedLocation && resolvedLocation.adcode) || user.adcode || '',
        locationText: (resolvedLocation && resolvedLocation.locationText) || user.locationText || '',
        displayName:
          (resolvedLocation && buildDisplayName(resolvedLocation)) ||
          user.displayName ||
          '',
      })

      return {
        success: true,
        userInfo: nextUser,
      }
    }

```

```
    if (event.action === 'updateRegion') {
      const payload = event.payload || {}
      let resolvedLocation = null

      try {
        resolvedLocation = payload.userLocation ? await reverseGeocodeByTencentMap(paylo
ad.userLocation) : null
      } catch (error) {
        console.error('[userManage] updateRegion reverse geocode error', error)
        resolvedLocation = null
      }

      const nextUser = await saveAndFetchUser(user._id, {
        locationAuthorized:
          typeof payload.locationAuthorized === 'boolean'
            ? payload.locationAuthorized
            : !!user.locationAuthorized,
        locationChoiceMade: true,
        userLocation: hasOwn(payload, 'userLocation')
          ? (payload.userLocation || null)
          : (user.userLocation || null),
        province:
          normalizeText(payload.province) ||
          (resolvedLocation && resolvedLocation.province) ||
          user.province ||
          '',
        city:
          normalizeText(payload.city) ||
          (resolvedLocation && resolvedLocation.city) ||
          user.city ||
          '',
        district:
          normalizeText(payload.district) ||
          (resolvedLocation && resolvedLocation.district) ||
          user.district ||
          '',
        adcode:
          normalizeText(payload.adcode) ||
          (resolvedLocation && resolvedLocation.adcode) ||
          user.adcode ||
          '',
        locationText:
          normalizeText(payload.locationText) ||
          (resolvedLocation && resolvedLocation.locationText) ||
          user.locationText ||
          '',
        displayName:
          buildDisplayName({
            displayName: payload.displayName,
            poiTitle: resolvedLocation && resolvedLocation.poiTitle,
            district:
              normalizeText(payload.district) ||
              (resolvedLocation && resolvedLocation.district) ||
              user.district ||
              '',
            city:
              normalizeText(payload.city) ||
              (resolvedLocation && resolvedLocation.city) ||
              user.city ||
              '',
            province:
              normalizeText(payload.province) ||
```

```
              (resolvedLocation && resolvedLocation.province) ||
              user.province ||
              '',
            locationText:
              normalizeText(payload.locationText) ||
              (resolvedLocation && resolvedLocation.locationText) ||
              user.locationText ||
              '',
          }) ||
          user.displayName ||
          '',
      })

      return {
        success: true,
        userInfo: nextUser,
      }
    }

    if (event.action === 'searchPlaceSuggestions') {
      const payload = event.payload || {}
      const list = await placeSuggestionByTencentMap(payload.keyword, {
        region: payload.region,
        latitude: payload.latitude,
        longitude: payload.longitude,
      })

      return {
        success: true,
        list,
      }
    }

    if (event.action === 'updateDNATags') {
      const tags = Array.isArray(event.tags) ? event.tags : []
      const buddyTags = Array.isArray(event.buddyTags) ? uniqueList(event.buddyTags) : [
    ]
      const buddyIntent = buildBuddyIntentFromTags(buddyTags)
      const nextUser = await saveAndFetchUser(user._id, {
        dnaTags: tags,
        buddyTags,
        buddyIntent,
        buddyIntentCompleted: !!buddyTags.length,
        dnaCompleted: true,
        onboardingCompleted: true,
      })

      return {
        success: true,
        userInfo: nextUser,
      }
    }

    if (event.action === 'updateBuddyIntent') {
      const buddyIntent = normalizeBuddyIntent(event.buddyIntent || {})
      const buddyTags = deriveBuddyTagsFromIntent(buddyIntent)
      const nextUser = await saveAndFetchUser(user._id, {
        buddyTags,
        buddyIntent,
        buddyIntentCompleted: !!buddyTags.length,
      })

```

```
      return {
        success: true,
        userInfo: nextUser,
      }
    }

    if (event.action === 'getBuddyMatches') {
      const payload = event.payload || {}
      const question = normalizeText(payload.question)
      const limit = Math.min(Math.max(Number(payload.limit) || 3, 1), 10)
      const res = await db.collection('users').limit(100).get()
      const currentOpenid = user.openid || ''

      const list = (res.data || [])
        .filter((item) => item && item.openid && item.openid !== currentOpenid)
        .filter((item) => item.profileCompleted && item.dnaCompleted)
        .map((item) => buildBuddyCandidate(user, item, question))
        .sort((left, right) => right.matchScore - left.matchScore)
        .slice(0, limit)

      return {
        success: true,
        list,
      }
    }

    return {
      success: false,
      message: '      ',
    }
  } catch (err) {
    console.error('userManage failed', err)
    return {
      success: false,
      message: '    ',
    }
  }
}
```

Path: cloudfunctions/userManage/package.json

```
1 {
2   "name": "userManage",
3   "version": "1.0.0",
4   "dependencies": {
5     "wx-server-sdk": "latest"
6   }
7 }
```

Path: cloudfunctions/agent-yuxiaohe-1grmumm967563411/src/index.js

```
import express from "express";
import { agui, createExpressRoutes } from "@cloudbase/agent-server";
import { AdpAgent } from "@cloudbase/agent-adapter-adp";
import dotenvx from "@dotenvx/dotenvx";
import { Observable } from "rxjs";
import { randomUUID } from "crypto";
import { jwtDecode } from "jwt-decode";
import {
  WeChatAgent,
  createWxMessageHandler,
  WeChatHistoryManager,
} from "@cloudbase/agent-adapter-wx";
import {
  DetectCloudbaseUserMiddleware,
  buildAgentUserPrompt,
  buildBuddyMatchResult,
  buildConversationStateResult,
  buildGuideGroundingPayload,
  buildGuideCustomizationResult,
  buildPlatformGroundingContext,
  buildConversationWorkflow,
  buildUnifiedTaskState,
} from "./utils.js";
import {
  buildDirectAnswer,
  buildLocationQueryResult,
  buildWeatherQueryResult,
} from "./direct-answer.js";

dotenvx.config();

const PORT = Number(process.env.PORT || 9000);

class MyAgent extends AdpAgent {
  generateRequestBody({ message, fileInfos, input }) {
    return super.generateRequestBody({
      message,
      fileInfos,
      input,
    });
  }
}

class PatchedWeChatAgent extends WeChatAgent {
  constructor(config) {
    super(config);
    this.finalReplyCache = "";
  }

  handleWeChatSending(event, input) {
    if (event?.type === "TEXT_MESSAGE_CHUNK") {
      this.messageBuffer += event.delta || event.content || "";
      return;
    }

    if (event?.type === "RUN_FINISHED") {
      this.lastReply = this.messageBuffer.trim();
      if (this.lastReply) {
        this.finalReplyCache = this.lastReply;
        this.sendToWeChat(this.lastReply, input).catch(console.error);
```

```
        this.messageBuffer = "";
      }
      return;
    }

    super.handleWeChatSending(event, input);
  }

  async sendToWeChat(content, input) {
    if (content) {
      this.finalReplyCache = content;
    }
    return super.sendToWeChat(content, input);
  }

  run(input) {
    const original$ = super.run(input);

    return new Observable((subscriber) => {
      const subscription = original$.subscribe({
        next: (event) => subscriber.next(event),
        error: (error) => subscriber.error(error),
        complete: () => {
          if (!this.lastReply && this.finalReplyCache) {
            this.lastReply = this.finalReplyCache;
          }
          subscriber.complete();
        },
      });

      return () => subscription.unsubscribe();
    });
  }
}

function createAgent({ request }) {
  console.log("[agent-yuxiaohe] createAgent", {
    hasAppKey: !!process.env.ADP_APP_KEY,
    envId: process.env.TCB_ENV || process.env.ENV_ID || "",
    method: request?.method,
    path: request?.path,
  });

  const agent = new MyAgent({
    adpConfig: {
      appKey: process.env.ADP_APP_KEY || "",
      credential: {
        secretId: process.env.TENCENTCLOUD_SECRETID || "",
        secretKey: process.env.TENCENTCLOUD_SECRETKEY || "",
        token: process.env.TENCENTCLOUD_SESSIONTOKEN || "",
      },
      enableUpload: false,
    },
  });

  agent.use(new DetectCloudbaseUserMiddleware(request));
  return agent;
}

function createBaseAgent() {
  return new MyAgent({
    adpConfig: {
```

```
      appKey: process.env.ADP_APP_KEY || "",
      credential: {
        secretId: process.env.TENCENTCLOUD_SECRETID || "",
        secretKey: process.env.TENCENTCLOUD_SECRETKEY || "",
        token: process.env.TENCENTCLOUD_SESSIONTOKEN || "",
      },
      enableUpload: false,
    },
  });
}

function extractCloudbaseUserId(request) {
  const authorization =
    request?.headers?.authorization ||
    request?.headers?.Authorization ||
    request?.headers?.get?.("authorization") ||
    request?.headers?.get?.("Authorization") ||
    "";

  const token = authorization.startsWith("Bearer ")
    ? authorization.slice(7)
    : "";

  if (!token) {
    return "";
  }

  try {
    const payload = jwtDecode(token);
    return payload?.uid || payload?.sub || payload?.openid || payload?.user_id || "";
  } catch (error) {
    console.warn(
      "[agent-yuxiaohe] jwt decode failed",
      error?.message || error
    );
    return "";
  }
}

function createWxAgent({ request, options }) {
  console.log("[agent-yuxiaohe] createWxAgent", {
    agentId: options?.agentId,
    method: request?.method,
    path: request?.path,
  });

  const baseAgent = createAgent({ request });
  const envId = process.env.TCB_ENV || process.env.ENV_ID || "";
  const authorization =
    request.headers.authorization ||
    request.headers.Authorization ||
    request.headers.get?.("authorization") ||
    request.headers.get?.("Authorization") ||
    "";
  const accessToken = authorization.startsWith("Bearer ")
    ? authorization.slice(7)
    : "";

  return {
    agent: new PatchedWeChatAgent({
      agentId: options?.agentId || process.env.AGENT_ID || "agent-yuxiaohe",
      agent: baseAgent,
```

```
185       wechatConfig: {
186         sendMode: "aitools",
187         context: {
188           extendedContext: {
189             envId,
190             accessToken,
191           },
192         },
193       },
194       historyManager: new WeChatHistoryManager({
195         envId,
196       }),
197     }),
198   };
199 }
200
201 function createRunId() {
202   try {
203     return randomUUID();
204   } catch (error) {
205     return `run-${Date.now()}-${Math.floor(Math.random() * 100000)}`;
206   }
207 }
208
209 function normalizeBotHistory(history = []) {
210   if (!Array.isArray(history)) {
211     return [];
212   }
213
214   return history
215     .map((item, index) => {
216       const role = item?.role === "assistant" ? "assistant" : "user";
217       const content = String(item?.content || "").trim();
218
219       if (!content) {
220         return null;
221       }
222
223       return {
224         id: `history-${index}-${Date.now()}`,
225         role,
226         content,
227       };
228     })
229     .filter(Boolean);
230 }
231
232 async function buildAgentInputFromBotBody(body = {}, cloudbaseUserId = "") {
233   const historyMessages = normalizeBotHistory(body?.history);
234   const msg = String(body?.msg || "").trim();
235
236   if (!msg) {
237     throw new Error("msg is required");
238   }
239
240   const workflowContext = await buildConversationWorkflow({
241     question: msg,
242     contextPayload: body?.contextPayload || {},
243     cloudbaseUserId,
244   });
245   const unifiedTaskState = await buildUnifiedTaskState({
246     question: msg,
```

```
    contextPayload: body?.contextPayload || {},
    cloudbaseUserId,
  });
  const guideFields =
    workflowContext?.mainline === "guide_customization" &&
    (workflowContext?.fields || unifiedTaskState?.collected)
      ? workflowContext?.fields || unifiedTaskState?.collected || {}
      : null;
  const groundingPayload = guideFields
    ? buildGuideGroundingPayload({
        question: msg,
        contextPayload: body?.contextPayload || {},
        guideFields,
      })
    : body?.contextPayload || {};
  const groundingContext = await buildPlatformGroundingContext(groundingPayload);
  const groundedMessageContent = buildAgentUserPrompt({
    question: msg,
    groundingContext,
    workflowContext,
    contextPayload: body?.contextPayload || {},
  });

  return {
    messages: historyMessages.concat({
      id: `user-${Date.now()}`,
      role: "user",
      content: groundedMessageContent,
    }),
    threadId:
      String(body?.conversationId || "").trim() || `conversation-${Date.now()}`,
    runId: createRunId(),
    tools: [],
    context: [],
    state: {},
    forwardedProps: {
      groundingRegion: groundingContext?.regionLabel || "",
      groundingCandidateCount: groundingContext?.candidates?.length || 0,
      mainline: workflowContext?.mainline || "",
      unifiedTaskState,
      unifiedTaskStateText: JSON.stringify(unifiedTaskState),
    },
  };
}

function writeSseChunk(res, data) {
  res.write(`data: ${JSON.stringify(data)}\n\n`);
}

function writeDirectTextResponse(res, text) {
  const content = String(text || "").trim();
  if (content) {
    writeSseChunk(res, { content });
  }
  res.write("data: [DONE]\n\n");
  res.end();
}

async function handleBotSendMessage(req, res) {
  console.log("[agent-yuxiaohe] bot send-message request", {
    method: req.method,
    path: req.path,
```

```
    bodyKeys: req.body ? Object.keys(req.body) : [],
    hasMsg: !!req.body?.msg,
    historyLength: Array.isArray(req.body?.history) ? req.body.history.length : 0,
  });

  let agent;
  let textBuffer = "";
  let chunkCount = 0;

  try {
    const cloudbaseUserId = extractCloudbaseUserId(req);
    const directAnswer = await buildDirectAnswer({
      question: req.body?.msg || "",
      contextPayload: req.body?.contextPayload || {},
      cloudbaseUserId,
    });

    if (directAnswer) {
      res.setHeader("Content-Type", "text/event-stream");
      res.setHeader("Cache-Control", "no-cache, no-transform");
      res.setHeader("Connection", "keep-alive");
      res.flushHeaders?.();
      writeDirectTextResponse(res, directAnswer);
      return;
    }

    const input = await buildAgentInputFromBotBody(req.body || {}, cloudbaseUserId);
    input.forwardedProps = {
      ...(input.forwardedProps || {}),
      cloudbaseUserId,
    };
    agent = createBaseAgent();

    console.log("[agent-yuxiaohe] bot send-message createAgent", {
      threadId: input.threadId,
      runId: input.runId,
      messageCount: input.messages.length,
      hasCloudbaseUserId: !!cloudbaseUserId,
      groundingRegion: input.forwardedProps?.groundingRegion || "",
      groundingCandidateCount: input.forwardedProps?.groundingCandidateCount || 0,
      unifiedTaskState: input.forwardedProps?.unifiedTaskState || {},
    });

    res.setHeader("Content-Type", "text/event-stream");
    res.setHeader("Cache-Control", "no-cache, no-transform");
    res.setHeader("Connection", "keep-alive");
    res.flushHeaders?.();

    const events = agui.sendMessageAGUI.handler(input, agent);

    for await (const event of events) {
      console.log("[agent-yuxiaohe] bot send-message event", {
        type: event?.type,
        hasDelta: !!event?.delta,
        hasContent: !!event?.content,
      });

      if (event?.type === "TEXT_MESSAGE_CONTENT") {
        const content = event.delta || event.content || "";
        if (content) {
          textBuffer += content;
          chunkCount += 1;
```

```
371           console.log("[agent-yuxiaohe] bot send-message chunk", {
372             chunkCount,
373             contentLength: content.length,
374             totalLength: textBuffer.length,
375           });
376           writeSseChunk(res, { content });
377         }
378         continue;
379       }
380
381       if (event?.type === "TEXT_MESSAGE_END") {
382         break;
383       }
384     }
385
386     if (!chunkCount && textBuffer) {
387       writeSseChunk(res, { content: textBuffer });
388     }
389
390     console.log("[agent-yuxiaohe] bot send-message finish", {
391       chunkCount,
392       totalLength: textBuffer.length,
393     });
394     res.write("data: [DONE]\n\n");
395     res.end();
396   } catch (error) {
397     console.error("[agent-yuxiaohe] bot send-message setup failed", error);
398     if (!res.headersSent) {
399       res.status(400).json({
400         error: "invalid send-message request",
401         message: error?.message || String(error),
402       });
403     } else if (!res.writableEnded) {
404       res.end();
405     }
406   }
407 }
408
409 async function handleBuddyMatch(req, res) {
410   try {
411     const body = req.body || {};
412     const departure = String(body.departure || "").trim();
413     const destination = String(body.destination || "").trim();
414     const time = String(body.time || "").trim();
415     const companionPreference = String(body.companionPreference || "").trim();
416     const cloudbaseUserId =
417       String(body.cloudbaseUserId || "").trim() || extractCloudbaseUserId(req);
418
419     const result = await buildBuddyMatchResult({
420       departure,
421       destination,
422       time,
423       companionPreference,
424       cloudbaseUserId,
425       contextPayload: body.contextPayload || {},
426     });
427
428     res.json(result);
429   } catch (error) {
430     console.error("[agent-yuxiaohe] buddy-match failed", error);
431     res.status(500).json({
432       success: false,
```

```
      message: "          ",
      error: error?.message || String(error),
    });
  }
}

async function handleGuideCustomization(req, res) {
  try {
    const body = req.body || {};
    const result = await buildGuideCustomizationResult({
      time: String(body.time || "").trim(),
      peopleCount: String(body.peopleCount || "").trim(),
      relationship: String(body.relationship || "").trim(),
      budget: String(body.budget || "").trim(),
      destination: String(body.destination || "").trim(),
      contextPayload: body.contextPayload || {},
    });

    res.json(result);
  } catch (error) {
    console.error("[agent-yuxiaohe] guide-customization failed", error);
    res.status(500).json({
      success: false,
      message: "           ",
      error: error?.message || String(error),
    });
  }
}

async function handleConversationState(req, res) {
  try {
    const body = req.body || {};
    const cloudbaseUserId =
      String(body.cloudbaseUserId || "").trim() || extractCloudbaseUserId(req);
    const result = await buildConversationStateResult({
      question: String(body.question || "").trim(),
      contextPayload: body.contextPayload || {},
      cloudbaseUserId,
    });

    res.json(result);
  } catch (error) {
    console.error("[agent-yuxiaohe] conversation-state failed", error);
    res.status(500).json({
      success: false,
      message: "     €           ",
      error: error?.message || String(error),
    });
  }
}

async function handleWeatherQuery(req, res) {
  try {
    const body = req.body || {};
    const result = await buildWeatherQueryResult({
      question: String(body.question || "").trim(),
      contextPayload: body.contextPayload || {},
    });

    res.json(result);
  } catch (error) {
    console.error("[agent-yuxiaohe] weather-query failed", error);
```

```
    res.status(500).json({
      success: false,
      subType: "weather",
      answer: "                              ",
      error: error?.message || String(error),
    });
  }
}

async function handleLocationQuery(req, res) {
  try {
    const body = req.body || {};
    const result = await buildLocationQueryResult({
      contextPayload: body.contextPayload || {},
    });

    res.json(result);
  } catch (error) {
    console.error("[agent-yuxiaohe] location-query failed", error);
    res.status(500).json({
      success: false,
      subType: "location",
      answer: "                                 ",
      error: error?.message || String(error),
    });
  }
}

const app = express();

app.use((req, res, next) => {
  console.log("[agent-yuxiaohe] incoming", {
    method: req.method,
    path: req.path,
  });
  next();
});

app.post("/send-message", express.json(), handleBotSendMessage);

app.post("/api/buddy-match", express.json(), handleBuddyMatch);

app.post("/api/guide-customization", express.json(), handleGuideCustomization);

app.post("/api/conversation-state", express.json(), handleConversationState);

app.post("/api/weather-query", express.json(), handleWeatherQuery);

app.post("/api/location-query", express.json(), handleLocationQuery);

app.post(
  "/v1/aibot/bots/:agentId/send-message",
  express.json(),
  (req, res, next) => {
    console.log("[agent-yuxiaohe] /v1/aibot/bots/:agentId/send-message hit", {
      method: req.method,
      path: req.path,
      agentId: req.params?.agentId,
    });
    next();
  },
  handleBotSendMessage
```

```
557 );
558
559 createExpressRoutes({
560   createAgent,
561   express: app,
562 });
563
564 app.post(
565   "/wx-send-message",
566   express.json(),
567   (req, res, next) => {
568     console.log("[agent-yuxiaohe] /wx-send-message hit", {
569       hasBody: !!req.body,
570     });
571     next();
572   },
573   createWxMessageHandler(createWxAgent)
574 );
575
576 app.post(
577   "/v1/aibot/bots/:agentId/wx-send-message",
578   express.json(),
579   (req, res, next) => {
580     console.log("[agent-yuxiaohe] /v1/aibot/bots/:agentId/wx-send-message hit", {
581       agentId: req.params?.agentId,
582       hasBody: !!req.body,
583     });
584     next();
585   },
586   createWxMessageHandler(createWxAgent)
587 );
588
589 app.listen(PORT, () => {
590   console.log("[agent-yuxiaohe] boot", {
591     port: PORT,
592     hasAppKey: !!process.env.ADP_APP_KEY,
593     envId: process.env.TCB_ENV || process.env.ENV_ID || "",
594   });
595 });
```

Path: cloudfunctions/agent-yuxiaohe-1grmumm967563411/src/direct-answer.js

---

```
   1 import https from "https";
   2 import {
   3   MAINLINE,
   4   buildConversationWorkflow,
   5   detectMainline,
   6   normalizeText,
   7 } from "./utils.js";
   8
   9 const WEATHER_KEYWORDS = [
  10   "  ",
  11   "  ",
  12   "  ",
  13   "  ",
  14   "  ",
  15   "  ",
  16   "   ",
  17   "   ",
  18   "   ",
  19   "  ",
  20   "    ",
  21 ];
  22
  23 const LOCATION_KEYWORDS = [
  24   "   ",
  25   "     ",
  26   "    ",
  27   "  ",
  28   "    ",
  29 ];
  30
  31 function requestJson(url) {
  32   return new Promise((resolve, reject) => {
  33     https
  34       .get(url, (res) => {
  35         let raw = "";
  36         res.on("data", (chunk) => {
  37           raw += chunk;
  38         });
  39         res.on("end", () => {
  40           try {
  41             resolve(JSON.parse(raw));
  42           } catch (error) {
  43             reject(error);
  44           }
  45         });
  46       })
  47       .on("error", reject);
  48   });
  49 }
  50
  51 function formatNumber(value) {
  52   const number = Number(value);
  53   return Number.isFinite(number) ? String(Math.round(number)) : "";
  54 }
  55
  56 function routeIntent(question) {
  57   const text = normalizeText(question);
  58   if (!text) return { intent: "empty" };
  59   if (WEATHER_KEYWORDS.some((keyword) => text.includes(keyword))) return { intent: "weat
     her" };
```

```
  if (LOCATION_KEYWORDS.some((keyword) => text.includes(keyword))) return { intent: "where_am_i" };
  return { intent: "other" };
}

function stripWeatherTerms(question = "") {
  return normalizeText(question)
    .replace(/(       |     |    |    )/g, "")
    .replace(/(  |  |   |   |  |  |  |   |  |   |   |   |  |  |     )/g, "")
    .replace(/^(  |    |    |    |    )/g, "")
    .trim();
}

function detectExplicitPlace(question = "") {
  const cleaned = stripWeatherTerms(question);
  if (!cleaned) return "";

  const exactAdminMatch = cleaned.match(/([\u4e00-\u9fa5]{2,12}(?: | | | | ))/u);
  if (exactAdminMatch?.[1]) return exactAdminMatch[1];

  return cleaned.split(/[,\uFF0C\u3002\uFF1F?!()\s]/u).find(Boolean) || "";
}

async function placeSuggestion(keyword, region = "") {
  const apiKey = process.env.TENCENT_MAP_KEY || process.env.TENCENT_MAP_API_KEY || "";
  if (!apiKey || !keyword) return [];

  const params = new URLSearchParams({
    key: apiKey,
    keyword,
    region,
    region_fix: "0",
    page_size: "10",
  });

  const url = `https://apis.map.qq.com/ws/place/v1/suggestion?${params.toString()}`;
  const result = await requestJson(url);
  if (result?.status !== 0 || !Array.isArray(result?.data)) return [];

  return result.data.map((item) => ({
    title: item.title || "",
    adcode: item.adcode || "",
    province: item.province || "",
    city: item.city || "",
    district: item.district || "",
    latitude: item.location?.lat || "",
    longitude: item.location?.lng || "",
  }));
}

async function reverseGeocoder({ latitude, longitude }) {
  const apiKey = process.env.TENCENT_MAP_KEY || process.env.TENCENT_MAP_API_KEY || "";
  if (!apiKey || !latitude || !longitude) return null;

  const params = new URLSearchParams({
    key: apiKey,
    location: `${latitude},${longitude}`,
    get_poi: "0",
  });

  const url = `https://apis.map.qq.com/ws/geocoder/v1/?${params.toString()}`;
  const result = await requestJson(url);
```

```
  if (result?.status !== 0 || !result?.result) return null;

  const component = result.result.address_component || {};
  const location = result.result.location || {};
  return {
    province: component.province || "",
    city: component.city || "",
    district: component.district || "",
    locationText: result.result.address || "",
    adcode: component.adcode || "",
    location: {
      lat: location.lat || "",
      lng: location.lng || "",
    },
  };
}

async function ipLocation() {
  const apiKey = process.env.TENCENT_MAP_KEY || process.env.TENCENT_MAP_API_KEY || "";
  if (!apiKey) return null;

  const url = `https://apis.map.qq.com/ws/location/v1/ip?key=${encodeURIComponent(apiKey)}`;
  const result = await requestJson(url);
  if (result?.status !== 0 || !result?.result) return null;

  const adInfo = result.result.ad_info || {};
  const location = result.result.location || {};
  return {
    province: adInfo.province || "",
    city: adInfo.city || "",
    district: adInfo.district || "",
    locationText: [adInfo.province, adInfo.city, adInfo.district].filter(Boolean).join(" "),
    adcode: adInfo.adcode || "",
    location: {
      lat: location.lat || "",
      lng: location.lng || "",
    },
  };
}

async function resolveLocation(contextPayload = {}, question = "") {
  const location = contextPayload.location || {};
  const explicitPlace = detectExplicitPlace(question);
  const region = [location.province, location.city, location.district].filter(Boolean).join("");

  if (explicitPlace) {
    const suggestion = await placeSuggestion(explicitPlace, region || explicitPlace);
    if (suggestion.length) {
      const picked = suggestion[0];
      return {
        label: [picked.province, picked.city, picked.district].filter(Boolean).join("") || picked.title,
        province: picked.province,
        city: picked.city,
        district: picked.district,
        adcode: picked.adcode,
        latitude: picked.latitude,
        longitude: picked.longitude,
      };
```

```
    }
  }

  if (location.latitude && location.longitude) {
    const reversed = await reverseGeocoder({
      latitude: location.latitude,
      longitude: location.longitude,
    });
    if (reversed) {
      return {
        label: [reversed.province, reversed.city, reversed.district].filter(Boolean).joi
n("") || reversed.locationText,
        province: reversed.province,
        city: reversed.city,
        district: reversed.district,
        adcode: reversed.adcode,
        latitude: reversed.location?.lat,
        longitude: reversed.location?.lng,
      };
    }
  }

  if (location.city || location.district) {
    const fallbackKeyword = normalizeText(location.city || location.district);
    const suggestion = await placeSuggestion(fallbackKeyword, region);
    if (suggestion.length) {
      const picked = suggestion[0];
      return {
        label: [picked.province, picked.city, picked.district].filter(Boolean).join("")
|| picked.title,
        province: picked.province,
        city: picked.city,
        district: picked.district,
        adcode: picked.adcode,
        latitude: picked.latitude,
        longitude: picked.longitude,
      };
    }
  }

  const ipResolved = await ipLocation();
  if (ipResolved) {
    return {
      label: [ipResolved.province, ipResolved.city, ipResolved.district].filter(Boolean)
.join("") || ipResolved.locationText,
      province: ipResolved.province,
      city: ipResolved.city,
      district: ipResolved.district,
      adcode: ipResolved.adcode,
      latitude: ipResolved.location?.lat,
      longitude: ipResolved.location?.lng,
    };
  }

  return null;
}

async function fetchOpenMeteoWeather(latitude, longitude) {
  const url =
    "https://api.open-meteo.com/v1/forecast?forecast_days=1&timezone=Asia%2FShanghai" +
    `&latitude=${encodeURIComponent(latitude)}` +
    `&longitude=${encodeURIComponent(longitude)}` +
```

```
    "&current=temperature_2m,apparent_temperature,weather_code,wind_speed_10m,precipitation" +
    "&daily=temperature_2m_max,temperature_2m_min,precipitation_probability_max";

  return requestJson(url);
}

function mapWeatherCode(code) {
  const mapping = {
    0: " ",
    1: "    ",
    2: "  ",
    3: " ",
    45: "  ",
    48: "  ",
    51: "    ",
    53: "   ",
    55: "     ",
    61: "  ",
    63: "  ",
    65: "  ",
    71: "  ",
    73: "  ",
    75: "  ",
    80: "   ",
    81: "  ",
    82: "   ",
    95: "   ",
  };
  return mapping[Number(code)] || "       ";
}

function buildWeatherAnswer(locationLabel, weatherData) {
  const current = weatherData?.current || {};
  const daily = weatherData?.daily || {};
  const weatherText = mapWeatherCode(current.weather_code);
  const temp = formatNumber(current.temperature_2m);
  const feelTemp = formatNumber(current.apparent_temperature);
  const minTemp = Array.isArray(daily.temperature_2m_min) ? formatNumber(daily.temperature_2m_min[0]) : "";
  const maxTemp = Array.isArray(daily.temperature_2m_max) ? formatNumber(daily.temperature_2m_max[0]) : "";
  const rainProb = Array.isArray(daily.precipitation_probability_max)
    ? formatNumber(daily.precipitation_probability_max[0])
    : "";
  const wind = formatNumber(current.wind_speed_10m);

  const metrics = [];
  if (temp) metrics.push(`    ${temp}°C`);
  if (feelTemp) metrics.push(`    ${feelTemp}°C`);
  if (minTemp && maxTemp) metrics.push(`   ${minTemp}°C - ${maxTemp}°C`);
  if (wind) metrics.push(`    ${wind} km/h`);

  const parts = [];
  parts.push(`${locationLabel}  ${weatherText ? `   ${weatherText}` : "     "}${metrics.length ? ` ${metrics.join(" ")}` : ""} `);

  if (rainProb) {
    const rainNumber = Number(rainProb);
    if (Number.isFinite(rainNumber) && rainNumber >= 60) {
      parts.push(`            ${rainProb}%       `);
    } else if (Number.isFinite(rainNumber) && rainNumber >= 30) {
```

```
      parts.push(`             ${rainProb}%              `);
    } else {
      parts.push(`            ${rainProb}% `);
    }
  }

  if (temp) {
    const tempNumber = Number(temp);
    if (tempNumber <= 8) {
      parts.push("                   ");
    } else if (tempNumber >= 28) {
      parts.push("               ");
    } else {
      parts.push("                     ");
    }
  }

  return parts.join("");
}

async function buildWeatherDirectResponse(question, contextPayload = {}) {
  const result = await buildWeatherQueryResult({
    question,
    contextPayload,
  });
  return result.answer;
}

async function buildWhereAmIDirectResponse(contextPayload = {}) {
  const result = await buildLocationQueryResult({
    contextPayload,
  });
  return result.answer;
}

export async function buildWeatherQueryResult({
  question = "",
  contextPayload = {},
} = {}) {
  const resolved = await resolveLocation(contextPayload, question);
  if (!resolved) {
    return {
      success: false,
      subType: "weather",
      resolvedLocation: null,
      weather: null,
      answer: "                                        “         ” ",
    };
  }

  if (!resolved.latitude || !resolved.longitude) {
    return {
      success: false,
      subType: "weather",
      resolvedLocation: {
        label: normalizeText(resolved.label),
        province: normalizeText(resolved.province),
        city: normalizeText(resolved.city),
        district: normalizeText(resolved.district),
        adcode: normalizeText(resolved.adcode),
        latitude: normalizeText(resolved.latitude),
        longitude: normalizeText(resolved.longitude),
```

```
      },
      weather: null,
      answer: `                ${resolved.label || "    "}
        `,
    };
  }

  try {
    const weatherData = await fetchOpenMeteoWeather(resolved.latitude, resolved.longitude);
    const locationLabel = normalizeText(resolved.label) || "       ";
    const current = weatherData?.current || {};
    const daily = weatherData?.daily || {};

    return {
      success: true,
      subType: "weather",
      resolvedLocation: {
        label: normalizeText(resolved.label),
        province: normalizeText(resolved.province),
        city: normalizeText(resolved.city),
        district: normalizeText(resolved.district),
        adcode: normalizeText(resolved.adcode),
        latitude: normalizeText(resolved.latitude),
        longitude: normalizeText(resolved.longitude),
      },
      weather: {
        weatherText: mapWeatherCode(current.weather_code),
        temperature: formatNumber(current.temperature_2m),
        apparentTemperature: formatNumber(current.apparent_temperature),
        minTemperature: Array.isArray(daily.temperature_2m_min) ? formatNumber(daily.temperature_2m_min[0]) : "",
        maxTemperature: Array.isArray(daily.temperature_2m_max) ? formatNumber(daily.temperature_2m_max[0]) : "",
        precipitationProbability: Array.isArray(daily.precipitation_probability_max)
          ? formatNumber(daily.precipitation_probability_max[0])
          : "",
        windSpeed: formatNumber(current.wind_speed_10m),
      },
      answer: buildWeatherAnswer(locationLabel, weatherData),
    };
  } catch (error) {
    return {
      success: false,
      subType: "weather",
      resolvedLocation: {
        label: normalizeText(resolved.label),
        province: normalizeText(resolved.province),
        city: normalizeText(resolved.city),
        district: normalizeText(resolved.district),
        adcode: normalizeText(resolved.adcode),
        latitude: normalizeText(resolved.latitude),
        longitude: normalizeText(resolved.longitude),
      },
      weather: null,
      answer: "                              ",
    };
  }
}

export async function buildLocationQueryResult({
  contextPayload = {},
```

```
416 } = {}) {
417   const resolved = await resolveLocation(contextPayload, "");
418   if (!resolved) {
419     return {
420       success: false,
421       subType: "location",
422       resolvedLocation: null,
423       answer: "                                          ",
424     };
425   }
426
427   const label =
428     normalizeText(resolved.label) ||
429     [resolved.province, resolved.city, resolved.district].filter(Boolean).join("") ||
430     "       ";
431
432   return {
433     success: true,
434     subType: "location",
435     resolvedLocation: {
436       label: normalizeText(label),
437       province: normalizeText(resolved.province),
438       city: normalizeText(resolved.city),
439       district: normalizeText(resolved.district),
440       adcode: normalizeText(resolved.adcode),
441       latitude: normalizeText(resolved.latitude),
442       longitude: normalizeText(resolved.longitude),
443     },
444     answer: `         ${label}                        `,
445   };
446 }
447
448 function buildBuddyMissingFieldResponse(workflow = {}) {
449   return workflow?.missingField?.ask || "                            ";
450 }
451
452 function buildBuddyNoCandidateResponse() {
453   return "                                       ";
454 }
455
456 function buildGuideMissingFieldResponse(workflow = {}) {
457   return workflow?.missingField?.ask || "                        ";
458 }
459
460 function buildFeedbackDirectResponse(workflow = {}) {
461   const feedbackType = workflow?.feedbackType || "    ";
462   if (feedbackType === "    ") {
463     return "                                          ";
464   }
465
466   if (feedbackType === "    ") {
467     return "                                          ";
468   }
469
470   return "                                         ";
471 }
472
473 export async function buildDirectAnswer({
474   question = "",
475   contextPayload = {},
476   cloudbaseUserId = "",
477 } = {}) {
```

```
  const route = routeIntent(question);
  if (route.intent === "weather") {
    return buildWeatherDirectResponse(question, contextPayload);
  }
  if (route.intent === "where_am_i") {
    return buildWhereAmIDirectResponse(contextPayload);
  }

  const mainline = detectMainline(question, contextPayload);
  const workflow = await buildConversationWorkflow({
    question,
    contextPayload,
    cloudbaseUserId,
  });

  console.log("[agent-yuxiaohe] buildDirectAnswer", {
    question: normalizeText(question),
    mainline,
    isReady: workflow?.isReady,
    missingField: workflow?.missingField?.key || workflow?.missingField?.label || "",
    collected: workflow?.fields || {},
    feedbackType: workflow?.feedbackType || "",
  });

  if (mainline === MAINLINE.BUDDY) {
    if (!workflow.isReady) {
      return buildBuddyMissingFieldResponse(workflow);
    }

    if (!workflow.buddyCandidates?.length) {
      return buildBuddyNoCandidateResponse();
    }

    return "";
  }

  if (mainline === MAINLINE.GUIDE && !workflow.isReady) {
    return buildGuideMissingFieldResponse(workflow);
  }

  if (mainline === MAINLINE.FEEDBACK && normalizeText(question).length <= 18) {
    return buildFeedbackDirectResponse(workflow);
  }

  return "";
}
```

Path: cloudfunctions/agent-yuxiaohe-1grmumm967563411/src/utils.js

---

```
import { jwtDecode } from "jwt-decode";
import tcb from "@cloudbase/node-sdk";

const LOCATION_WORD_REGEXP = /(  |  |  |  |  |  |    |  )/u;
const DEFAULT_REGION_LABEL = "     ";
const MAX_DATASET_FETCH = 60;
const MAX_CANDIDATE_PER_TYPE = 3;
const BUDDY_MAX_CANDIDATES = 3;
const GUIDE_MAX_CANDIDATES = 6;

export const MAINLINE = {
  BUDDY: "buddy_matching",
  GUIDE: "guide_customization",
  WEATHER: "weather_location",
  FEEDBACK: "xiaohe_feedback",
  GENERIC: "generic_guard",
};

const MAINLINE_SKILL_MODE_MAP = {
  buddy_matching: MAINLINE.BUDDY,
  guide_customization: MAINLINE.GUIDE,
  xiaohe_feedback: MAINLINE.FEEDBACK,
};

const BUDDY_KEYWORDS = ["  ", "  ", "   ", "    ", "  ", "  ", "  ", "   ", "    ", "
    "];
const GUIDE_KEYWORDS = ["  ", "  ", "  ", "   ", "  ", "    ", "  ", "  "];
const WEATHER_KEYWORDS = [
  "  ",
  "  ",
  "  ",
  "  ",
  "  ",
  "   ",
  "  ",
  "  ",
  "  ",
  "   ",
  "    ",
];
const FEEDBACK_KEYWORDS = [
  "  ",
  "  ",
  "  ",
  "  ",
  "  ",
  "  ",
  "  ",
  "bug",
  "  ",
  " ",
  "  ",
  "  ",
  "emo",
];

let cloudbaseApp = null;

export function normalizeText(value) {
  return String(value || "").trim();
```

```
}

function normalizeTextLower(value) {
  return normalizeText(value).toLowerCase();
}

function normalizeArray(value) {
  return Array.isArray(value)
    ? value.map((item) => normalizeText(item)).filter(Boolean)
    : [];
}

function uniqueList(list = []) {
  return Array.from(new Set((list || []).filter(Boolean)));
}

function maskIdentifier(value = "") {
  const normalized = normalizeText(value);
  if (!normalized) return "";
  if (normalized.length <= 8) return normalized;
  return `${normalized.slice(0, 4)}***${normalized.slice(-4)}`;
}

function includesAny(text = "", keywords = []) {
  return keywords.some((keyword) => text.includes(keyword));
}

function isBuddyIntentText(text = "") {
  const normalized = normalizeText(text);
  if (!normalized) return false;
  if (includesAny(normalized, BUDDY_KEYWORDS)) return true;

  return (
    /( |  |   ).*(  |  |  )/u.test(normalized) ||
    /(  ).*( |  |   )/u.test(normalized) ||
    /( |  |   ).*(  |  |   |   )/u.test(normalized) ||
    /(  |  |   |   ).*(  |  |  )/u.test(normalized)
  );
}

function removeRegionSuffix(value) {
  return normalizeText(value)
    .replace(/(     |   |   |  )$/u, "")
    .replace(/( | | | | )$/u, "");
}

function buildRegionTokens(location = {}) {
  return uniqueList(
    [
      location.province,
      location.city,
      location.district,
      location.displayName,
      location.locationText,
    ]
      .map(removeRegionSuffix)
      .filter(Boolean)
  );
}

function buildExplicitRegionTokens(region = "") {
  const normalizedRegion = normalizeText(region);
```

```
  if (!normalizedRegion) return [];

  return uniqueList(
    [normalizedRegion, removeRegionSuffix(normalizedRegion)].filter(Boolean)
  );
}

function buildRegionLabel(location = {}) {
  const displayName = normalizeText(location.displayName);
  if (displayName) {
    return displayName;
  }

  const region = [location.province, location.city, location.district]
    .map(removeRegionSuffix)
    .filter(Boolean)
    .join(" / ");

  return region || normalizeText(location.locationText) || DEFAULT_REGION_LABEL;
}

function getConversationHistory(contextPayload = {}) {
  if (!Array.isArray(contextPayload?.history)) {
    return [];
  }

  return contextPayload.history
    .map((item) => ({
      role: item?.role === "assistant" || item?.role === "ai" ? "assistant" : "user",
      text: normalizeText(item?.text || item?.content || ""),
    }))
    .filter((item) => item.text)
    .slice(-20);
}

function inferFieldFromAssistant(mainline = "", assistantText = "") {
  const text = normalizeText(assistantText);
  if (!text) return "";

  if (mainline === MAINLINE.GUIDE) {
    if (text.includes("     ")) return "destination";
    if (text.includes("     ")) return "time";
    if (text.includes("     ")) return "peopleCount";
    if (text.includes("    ")) return "relationship";
    if (text.includes("  ")) return "budget";
  }

  if (mainline === MAINLINE.BUDDY) {
    if (text.includes("     ")) return "departure";
    if (text.includes("     ")) return "destination";
    if (text.includes("      ")) return "time";
    if (text.includes("    ")) return "companionPreference";
  }

  return "";
}

function inferMainlineFromHistory(contextPayload = {}) {
  const history = getConversationHistory(contextPayload);
  if (!history.length) return "";

  const lastAssistantText =
```

```
    [...history].reverse().find((item) => item.role === "assistant")?.text || "";
  const combinedText = history.map((item) => item.text).join("\n");

  if (
    lastAssistantText.includes("     ") ||
    lastAssistantText.includes("     ") ||
    lastAssistantText.includes("     ") ||
    lastAssistantText.includes("    ") ||
    lastAssistantText.includes("  ")
  ) {
    return MAINLINE.GUIDE;
  }

  if (
    lastAssistantText.includes("     ") ||
    lastAssistantText.includes("      ") ||
    lastAssistantText.includes("    ")
  ) {
    return MAINLINE.BUDDY;
  }

  if (
    lastAssistantText.includes("     ") ||
    lastAssistantText.includes("      ") ||
    lastAssistantText.includes("      ") ||
    lastAssistantText.includes("       ")
  ) {
    return MAINLINE.FEEDBACK;
  }

  if (includesAny(combinedText, GUIDE_KEYWORDS)) return MAINLINE.GUIDE;
  if (isBuddyIntentText(combinedText)) return MAINLINE.BUDDY;
  if (includesAny(combinedText, FEEDBACK_KEYWORDS)) return MAINLINE.FEEDBACK;
  if (includesAny(combinedText, WEATHER_KEYWORDS)) return MAINLINE.WEATHER;

  return "";
}

function inferTaskStateFromHistory(contextPayload = {}) {
  const explicitState = contextPayload?.currentTaskState || {};
  const mainline =
    normalizeText(explicitState?.mainline) || inferMainlineFromHistory(contextPayload);

  if (!mainline) {
    return explicitState || {};
  }

  const history = getConversationHistory(contextPayload);
  const inferredState = {
    ...explicitState,
    mainline,
    collected: {
      ...(explicitState?.collected || {}),
    },
    lastAskedField: normalizeText(explicitState?.lastAskedField),
    feedbackType: normalizeText(explicitState?.feedbackType),
  };

  const userTexts = history
    .filter((item) => item.role === "user")
    .map((item) => item.text);

```

```
  if (mainline === MAINLINE.GUIDE) {
    const collected = { ...(inferredState.collected || {}) };
    const historyInferenceContext = {
      ...(contextPayload || {}),
      disableHistoryInference: true,
      currentTaskState: {
        ...(explicitState || {}),
        mainline,
        collected,
      },
      skillContext: {
        ...(contextPayload?.skillContext || {}),
        collected: {
          ...(contextPayload?.skillContext?.collected || {}),
          ...collected,
        },
      },
    };

    userTexts.forEach((text) => {
      historyInferenceContext.currentTaskState.collected = collected;
      historyInferenceContext.skillContext.collected = {
        ...(contextPayload?.skillContext?.collected || {}),
        ...collected,
      };

      if (!collected.time) collected.time = extractTime(text, historyInferenceContext);
      if (!collected.peopleCount) {
        collected.peopleCount = extractCount(
          text,
          historyInferenceContext,
          ["peopleCount"],
          /([           \d]+(?: | ))/u
        );
      }
      if (!collected.relationship) {
        collected.relationship = extractRelationship(text, historyInferenceContext);
      }
      if (!collected.peopleCount) {
        collected.peopleCount = normalizePeopleCountFromRelationship(
          collected.relationship,
          collected.peopleCount
        );
      }
      if (!collected.budget) collected.budget = extractGuideBudgetValue(text, historyInferenceContext);
      if (!collected.destination) collected.destination = extractDestination(text, historyInferenceContext);
    });
    inferredState.collected = collected;
  }

  if (mainline === MAINLINE.BUDDY) {
    const collected = { ...(inferredState.collected || {}) };
    const historyInferenceContext = {
      ...(contextPayload || {}),
      disableHistoryInference: true,
      currentTaskState: {
        ...(explicitState || {}),
        mainline,
        collected,
      },
```

```
      skillContext: {
        ...(contextPayload?.skillContext || {}),
        collected: {
          ...(contextPayload?.skillContext?.collected || {}),
          ...collected,
        },
      },
    };

    userTexts.forEach((text) => {
      historyInferenceContext.currentTaskState.collected = collected;
      historyInferenceContext.skillContext.collected = {
        ...(contextPayload?.skillContext?.collected || {}),
        ...collected,
      };

      if (!collected.departure) {
        const location = contextPayload?.location || {};
        collected.departure = normalizeText(location.city || location.displayName);
      }
      if (!collected.destination) collected.destination = extractDestination(text, historyInferenceContext);
      if (!collected.time) collected.time = extractTime(text, historyInferenceContext);
      if (!collected.companionPreference) {
        collected.companionPreference = extractCompanionPreference(text, historyInferenceContext);
      }
    });
    inferredState.collected = collected;
  }

  if (mainline === MAINLINE.FEEDBACK && !inferredState.feedbackType) {
    const latestUserText = [...userTexts].reverse().find(Boolean) || "";
    inferredState.feedbackType = classifyFeedbackType(latestUserText);
  }

  if (!inferredState.lastAskedField) {
    const lastAssistantText =
      [...history].reverse().find((item) => item.role === "assistant")?.text || "";
    inferredState.lastAskedField = inferFieldFromAssistant(mainline, lastAssistantText);
  }

  return inferredState;
}

function getCurrentTaskState(contextPayload = {}) {
  if (contextPayload?.disableHistoryInference) {
    return contextPayload?.currentTaskState || {};
  }

  return inferTaskStateFromHistory(contextPayload);
}

function getMergedCollected(contextPayload = {}) {
  const skillCollected = contextPayload?.skillContext?.collected || {};
  const taskCollected = getCurrentTaskState(contextPayload)?.collected || {};

  return {
    ...skillCollected,
    ...taskCollected,
  };
}
```

```

function readStateValue(contextPayload = {}, keys = []) {
  const collected = getMergedCollected(contextPayload);
  for (const key of keys) {
    const value = normalizeText(collected?.[key]);
    if (value) return value;
  }
  return "";
}

function mergeSkillLocation(location = {}, contextPayload = {}) {
  const skillMode = normalizeText(contextPayload?.skillContext?.mode);
  const collected = getMergedCollected(contextPayload);
  const explicitRegion =
    skillMode === MAINLINE.GUIDE
      ? normalizeText(collected.destination || collected.region)
      : normalizeText(collected.region);

  if (!explicitRegion) {
    return location || {};
  }

  return {
    ...(location || {}),
    displayName: explicitRegion,
    locationText: explicitRegion,
  };
}

function safeNumber(value) {
  const number = Number(value);
  return Number.isFinite(number) ? number : 0;
}

function getPriceText(item = {}, type = "activity") {
  const price = safeNumber(item.priceFrom || item.price || 0);
  if (!price) {
    return type === "hotel" ? "    " : "   ";
  }
  return type === "hotel" ? `¥${price}/ ` : `¥${price} `;
}

function getItemRegionText(item = {}) {
  return [item.province, item.city, item.district]
    .map(removeRegionSuffix)
    .filter(Boolean)
    .join(" / ");
}

function joinSearchText(item = {}) {
  return normalizeTextLower(
    [
      item.title,
      item.name,
      item.summary,
      item.content,
      item.detail,
      item.desc,
      item.description,
      item.locationName,
      item.address,
      item.province,
```

```
      item.city,
      item.district,
      item.transport,
      item.stay,
      ...normalizeArray(item.tags),
      ...normalizeArray(item.travelModeTags),
      ...normalizeArray(item.playTags),
      ...normalizeArray(item.suitableGroups),
      ...normalizeArray(item.highlights),
      ...normalizeArray(item.itinerary),
      ...normalizeArray(item.categoryTags),
      ...normalizeArray(item.dnaTags),
    ].join(" ")
  );
}

function buildIntentTokens(question = "", userProfile = {}) {
  const source = normalizeText(question);
  const tokens = [];
  const keywordMap = [
    "  ",
    "  ",
    "  ",
    "  ",
    "  ",
    "  ",
    "  ",
    "  ",
    "  ",
    "  ",
    "  ",
    "  ",
    "  ",
    "  ",
    "  ",
    "  ",
    "  ",
    "  ",
    "  ",
    "  ",
    "  ",
  ];

  keywordMap.forEach((keyword) => {
    if (source.includes(keyword)) {
      tokens.push(keyword);
    }
  });

  normalizeArray(userProfile.dnaTags).forEach((tag) => tokens.push(tag));
  return uniqueList(tokens);
}

function detectRequestedTypes(question = "") {
  const text = normalizeText(question);
  const requested = [];

  if (/  |  |  |  |  |  |  | /u.test(text)) {
    requested.push("activity");
  }
  if (/  |  |  |  |  |  /u.test(text)) {
    requested.push("scenic");
  }
```

```
  if (/  |  |  | /u.test(text)) {
    requested.push("hotel");
  }
  if (/  |  |  |   /u.test(text)) {
    requested.push("product");
  }

  return requested.length ? requested : ["activity", "scenic", "hotel", "product"];
}

function scoreLocation(item = {}, regionTokens = []) {
  if (!regionTokens.length) return 0;

  const text = joinSearchText(item);
  let score = 0;
  regionTokens.forEach((token, index) => {
    if (token && text.includes(normalizeTextLower(token))) {
      score += 15 - Math.min(index * 2, 8);
    }
  });
  return score;
}

function scoreIntent(item = {}, intentTokens = []) {
  if (!intentTokens.length) return 0;

  const text = joinSearchText(item);
  return intentTokens.reduce((score, token) => {
    if (!token) return score;
    return score + (text.includes(normalizeTextLower(token)) ? 8 : 0);
  }, 0);
}

function scoreNearbyIntent(question = "", item = {}, location = {}) {
  if (!LOCATION_WORD_REGEXP.test(question)) return 0;

  const regionText = buildRegionLabel(location);
  const itemRegion =
    getItemRegionText(item) || normalizeText(item.locationName) || normalizeText(item.address);

  return itemRegion && regionText !== DEFAULT_REGION_LABEL && itemRegion.includes(removeRegionSuffix(regionText))
    ? 10
    : 0;
}

function scoreSourceType(type, requestedTypes = []) {
  if (!requestedTypes.length) return 0;
  if (requestedTypes[0] === type) return 20;
  return requestedTypes.includes(type) ? 10 : -10;
}

function sortByScore(list = []) {
  return [...list].sort((a, b) => {
    if (b.score !== a.score) {
      return b.score - a.score;
    }

    return safeNumber(b.raw.updatedAt || b.raw.createTime || 0) - safeNumber(a.raw.updatedAt || a.raw.createTime || 0);
  });
```

```
549 }
550
551 function summarizeItem(type, item = {}) {
552   const title = normalizeText(
553     item.title ||
554       item.name ||
555       item.locationName ||
556       {
557         activity: "    ",
558         scenic: "    ",
559         hotel: "    ",
560         product: "    ",
561       }[type]
562   );
563
564   return {
565     id: normalizeText(item._id || item.id),
566     type,
567     title,
568     region: getItemRegionText(item) || normalizeText(item.locationName) || normalizeText
    (item.address) || "     ",
569     priceText: getPriceText(item, type),
570     summary: normalizeText(item.summary || item.content || item.desc || item.description
     || item.detail),
571     tags: uniqueList([
572       ...normalizeArray(item.tags),
573       ...normalizeArray(item.travelModeTags),
574       ...normalizeArray(item.playTags),
575       ...normalizeArray(item.suitableGroups),
576       ...normalizeArray(item.categoryTags),
577     ]).slice(0, 4),
578   };
579 }
580
581 function formatCandidateLine(candidate, index) {
582   const summary = candidate.summary ? `    ${candidate.summary}` : "";
583   const tags = candidate.tags.length ? `    ${candidate.tags.join(" ")}` : "";
584   return `${index + 1}. [${candidate.type}] ${candidate.title} ID: ${candidate.id || " "
    }    ${candidate.region}    ${candidate.priceText}${tags}${summary} `;
585 }
586
587 function itemMatchesExplicitRegion(item = {}, regionTokens = []) {
588   if (!regionTokens.length) return true;
589
590   const text = joinSearchText(item);
591   return regionTokens.some((token) => text.includes(normalizeTextLower(token)));
592 }
593
594 function getCloudbaseApp() {
595   if (cloudbaseApp) return cloudbaseApp;
596
597   const env = process.env.TCB_ENV || process.env.ENV_ID || "";
598   cloudbaseApp = tcb.init({
599     env,
600     secretId: process.env.TENCENTCLOUD_SECRETID || "",
601     secretKey: process.env.TENCENTCLOUD_SECRETKEY || "",
602     sessionToken: process.env.TENCENTCLOUD_SESSIONTOKEN || "",
603   });
604
605   return cloudbaseApp;
606 }
607
```

```
608 async function fetchCollection(collectionName, query = null) {
609   const app = getCloudbaseApp();
610   let request = app.database().collection(collectionName);
611   if (query) {
612     request = request.where(query);
613   }
614
615   const result = await request.limit(MAX_DATASET_FETCH).get();
616   return Array.isArray(result?.data) ? result.data : [];
617 }
618
619 async function fetchUserByOpenid(openid = "") {
620   const normalizedOpenid = normalizeText(openid);
621   if (!normalizedOpenid) return null;
622
623   const result = await fetchCollection("users", { openid: normalizedOpenid });
624   console.log("[agent-yuxiaohe] fetchUserByOpenid", {
625     requestedOpenid: maskIdentifier(normalizedOpenid),
626     matchedCount: result.length,
627     matchedByOpenid: result.some(
628       (item) => normalizeText(item?.openid) === normalizedOpenid
629     ),
630     matchedBy_Openid: result.some(
631       (item) => normalizeText(item?._openid) === normalizedOpenid
632     ),
633   });
634   return result[0] || null;
635 }
636
637 async function loadPlatformDataset() {
638   const [activities, hotels, scenics, products] = await Promise.all([
639     fetchCollection("activities", { status: "published" }),
640     fetchCollection("hotels", { status: true }),
641     fetchCollection("scenics"),
642     fetchCollection("products"),
643   ]);
644
645   return { activities, hotels, scenics, products };
646 }
647
648 function buildRankedCandidates({
649   question = "",
650   location = {},
651   userProfile = {},
652   datasets = {},
653   explicitRegion = "",
654 }) {
655   const regionTokens = buildRegionTokens(location);
656   const explicitRegionTokens = buildExplicitRegionTokens(explicitRegion);
657   const requestedTypes = detectRequestedTypes(question);
658   const intentTokens = buildIntentTokens(question, userProfile);
659
660   const candidates = []
661     .concat((datasets.activities || []).map((item) => ({ type: "activity", raw: item }))
    )
662     .concat((datasets.scenics || []).map((item) => ({ type: "scenic", raw: item })))
663     .concat((datasets.hotels || []).map((item) => ({ type: "hotel", raw: item })))
664     .concat((datasets.products || []).map((item) => ({ type: "product", raw: item })))
665     .map((entry) => {
666       const summary = summarizeItem(entry.type, entry.raw);
667       const score =
668         scoreSourceType(entry.type, requestedTypes) +
```

```
        scoreLocation(entry.raw, regionTokens) +
        scoreIntent(entry.raw, intentTokens) +
        scoreNearbyIntent(question, entry.raw, location);

      return {
        ...summary,
        raw: entry.raw,
        score,
      };
    })
    .filter((item) => item.score > 0);

  const regionConstrainedCandidates = explicitRegionTokens.length
    ? candidates.filter((item) => itemMatchesExplicitRegion(item.raw, explicitRegionTokens))
    : candidates;

  const sorted = sortByScore(regionConstrainedCandidates);
  const grouped = { activity: [], scenic: [], hotel: [], product: [] };

  sorted.forEach((candidate) => {
    if (grouped[candidate.type].length >= MAX_CANDIDATE_PER_TYPE) return;
    grouped[candidate.type].push(candidate);
  });

  const selected = requestedTypes
    .flatMap((type) => grouped[type] || [])
    .concat(
      Object.keys(grouped)
        .filter((type) => !requestedTypes.includes(type))
        .flatMap((type) => grouped[type] || [])
    )
    .slice(0, 6);

  return {
    requestedTypes,
    intentTokens,
    regionLabel: explicitRegion || buildRegionLabel(location),
    selected,
  };
}

export async function buildPlatformGroundingContext(payload = {}) {
  const question = normalizeText(payload.question);
  if (!question) {
    return {
      question: "",
      regionLabel: DEFAULT_REGION_LABEL,
      candidates: [],
      prompt: "",
    };
  }

  const location = mergeSkillLocation(payload.location || {}, payload);
  const userProfile = payload.userProfile || {};
  const preferences = payload.preferences || {};
  const datasets = payload.platformDataset || (await loadPlatformDataset());
  const explicitRegion = readStateValue(payload, ["destination", "region", "place"]);
  const ranked = buildRankedCandidates({
    question,
    location,
    userProfile,
```

```
    datasets,
    explicitRegion,
  });

  const preferenceText = [
    preferences.distance ? `     ${preferences.distance}` : "",
    preferences.budget ? `     ${preferences.budget}` : "",
    preferences.detailLevel ? `      ${preferences.detailLevel}` : "",
  ]
    .filter(Boolean)
    .join(" ");

  const profileText = normalizeArray(userProfile.dnaTags).join(" ");
  const lines = ranked.selected.map((item, index) => formatCandidateLine(item, index));
  const dataBlock = lines.length ? lines.join("\n") : "             ";

  const prompt = [
    "                           ",
    `       ${question}`,
    `     ${ranked.regionLabel}`,
    profileText ? `       ${profileText}` : "",
    preferenceText ? `     ${preferenceText}` : "",
    ranked.intentTokens.length ? `        ${ranked.intentTokens.join(" ")}` : "",
    "                         ",
    dataBlock,
  ]
    .filter(Boolean)
    .join("\n");

  return {
    question,
    regionLabel: ranked.regionLabel,
    candidates: ranked.selected,
    prompt,
  };
}

export function detectMainline(question = "", contextPayload = {}) {
  const currentTaskMainline = normalizeText(getCurrentTaskState(contextPayload)?.mainline);
  if (currentTaskMainline) {
    return currentTaskMainline;
  }

  const inferredMainline = inferMainlineFromHistory(contextPayload);
  if (inferredMainline) {
    return inferredMainline;
  }

  const skillMode = normalizeText(contextPayload?.skillContext?.mode);
  if (skillMode && MAINLINE_SKILL_MODE_MAP[skillMode]) {
    return MAINLINE_SKILL_MODE_MAP[skillMode];
  }

  const text = normalizeText(question);
  if (!text) return MAINLINE.GENERIC;
  if (includesAny(text, WEATHER_KEYWORDS)) return MAINLINE.WEATHER;
  if (isBuddyIntentText(text)) return MAINLINE.BUDDY;
  if (includesAny(text, GUIDE_KEYWORDS)) return MAINLINE.GUIDE;
  if (includesAny(text, FEEDBACK_KEYWORDS)) return MAINLINE.FEEDBACK;
  return MAINLINE.GENERIC;
}
```

```

function extractDestination(question = "", contextPayload = {}) {
  const fromCollected = readStateValue(contextPayload, ["destination", "region", "place"
]);
  if (fromCollected) return fromCollected;

  const normalizedQuestion = normalizeText(question);
  const patterns = [
    / ([\u4e00-\u9fa5A-Za-z0-9]{2,12}?)(?: | |  |  | |  | | | |\s|$)/u,
    / ([\u4e00-\u9fa5A-Za-z0-9]{2,12}?)(?: | |  |  | |  | | | |\s|$)/u,
    / ([\u4e00-\u9fa5A-Za-z0-9]{2,12}?)(?: | |  |  )/u,
  ];

  for (const pattern of patterns) {
    const matched = normalizedQuestion.match(pattern);
    if (matched?.[1]) return matched[1];
  }

  if (
    /^[\u4e00-\u9fa5A-Za-z0-9]{2,12}$/u.test(normalizedQuestion) &&
    !extractTime(normalizedQuestion, contextPayload) &&
    !extractGuideBudgetValue(normalizedQuestion, contextPayload) &&
    !extractCount(normalizedQuestion, contextPayload, ["peopleCount"], /([           \d]
+(?: | ))/u)
  ) {
    return normalizedQuestion;
  }

  return "";
}

function extractGuideBudgetValue(question = "", contextPayload = {}) {
  const fromCollected = readStateValue(contextPayload, ["budget"]);
  if (fromCollected) return fromCollected;

  const normalizedQuestion = normalizeText(question);
  const amountWithUnit = normalizedQuestion.match(/(\d{2,5}\s*(?: | |w| ))/u);
  if (amountWithUnit?.[1]) return amountWithUnit[1];

  const embeddedBudget = normalizedQuestion.match(
    /(?:  |  |  |  |  )(?:  |  |   |   )?\s*(\d{2,5})(?:\s*(?: | ))?(?:  |  |  |  |    )
?/u
  );
  if (embeddedBudget?.[1]) {
    return `${embeddedBudget[1]} `;
  }

  const standaloneBudget = normalizedQuestion.match(
    /^(?:  )?\s*(\d{2,5})(?:\s*(?: | ))?(?:  |  |  |  |  |  )?$/u
  );
  if (standaloneBudget?.[1]) {
    return `${standaloneBudget[1]} `;
  }

  const currentTaskState = getCurrentTaskState(contextPayload);
  if (normalizeText(currentTaskState?.lastAskedField) === "budget") {
    const plainNumber = normalizedQuestion.match(/^(\d{2,5})(?:  |  |  |  )?$/u);
    if (plainNumber?.[1]) {
      return `${plainNumber[1]} `;
    }
  }

```

```
  return "";
}

function extractCount(question = "", contextPayload = {}, keys = [], pattern) {
  const fromCollected = readStateValue(contextPayload, keys);
  if (fromCollected) return fromCollected;
  const normalizedQuestion = normalizeText(question);
  const matched = normalizedQuestion.match(pattern);
  if (matched?.[1]) return matched[1];

  const extendedMatched = normalizedQuestion.match(/([           \d]+)\s* ? /u);
  if (extendedMatched?.[1]) {
    return `${extendedMatched[1]} `;
  }

  if (/^(  |  )$/u.test(normalizedQuestion)) return "2 ";
  if (/^(   |  |2  |2 |     |    )$/u.test(normalizedQuestion)) return "2 ";
  if (/^  (?:   |   |  |  |  |  |  |  |  |  )/u.test(normalizedQuestion)) {
    return "2 ";
  }

  return "";
}

function extractTime(question = "", contextPayload = {}) {
  const fromCollected = readStateValue(contextPayload, ["time", "travelTime", "date", "days"]);
  if (fromCollected) return fromCollected;

  const matched = normalizeText(question).match(
    /(  |  |  |  |  |  |  |  |  |  |  |  |  | [        ]|\d{1,2} \d{1,2} |\d{1,2} )/u
  );
  return matched?.[1] || "";
}

function extractCompanionPreference(question = "", contextPayload = {}) {
  const fromCollected = readStateValue(contextPayload, ["companionPreference", "groupPreference", "buddyPreference"]);
  if (fromCollected) return fromCollected;

  const normalizedQuestion = normalizeText(question);
  const directPatterns = [
    /(    |    |    |    |    |    |     )/u,
    /(   |   |   |   |   |    |    |   |   )/u,
  ];

  for (const pattern of directPatterns) {
    const matched = normalizedQuestion.match(pattern);
    if (matched?.[1]) return matched[1];
  }

  const normalizedGender =
    /(  |   )/u.test(normalizedQuestion) ? "  " : /(  |   )/u.test(normalizedQuestion) ? "  " : "";
  const normalizedTrait =
    /(   |   |  )/u.test(normalizedQuestion)
      ? "   "
      : /(   |   )/u.test(normalizedQuestion)
        ? "   "
        : /   /u.test(normalizedQuestion)
          ? "   "
          : /   /u.test(normalizedQuestion)
```

```
            ? "   "
            : /   /u.test(normalizedQuestion)
              ? "   "
              : /   /u.test(normalizedQuestion)
                ? "   "
                : /  /u.test(normalizedQuestion)
                  ? "    "
                  : /  /u.test(normalizedQuestion)
                    ? "    "
                    : "";

  if (normalizedTrait && normalizedGender) {
    return `${normalizedTrait} ${normalizedGender}`;
  }

  if (normalizedTrait) return normalizedTrait;
  if (normalizedGender) return normalizedGender;
  return "";
}

function extractRelationship(question = "", contextPayload = {}) {
  const fromCollected = readStateValue(contextPayload, ["relationship", "groupType"]);
  if (fromCollected) return fromCollected;

  const normalizedQuestion = normalizeText(question);
  const matched = normalizedQuestion.match(
    /(  |  |  |  |  |  |  |  |   |  |   |   |  |  |  )/u
  );
  if (!matched?.[1]) return "";

  if (/(   |   |  |  |  )/u.test(matched[1])) {
    return "  ";
  }

  return matched[1];
}

function classifyFeedbackType(question = "") {
  const text = normalizeText(question);
  if (!text) return "    ";
  if (/(  |  |   |    |  )/u.test(text)) return "    ";
  if (/(  | |bug|  |   |   |  )/u.test(text)) return "    ";
  if (/( |  |  |  |emo|  | )/u.test(text)) return "    ";
  return "    ";
}

function normalizePeopleCountFromRelationship(relationship = "", currentPeopleCount = ""
) {
  if (normalizeText(currentPeopleCount)) {
    return normalizeText(currentPeopleCount);
  }

  const normalizedRelationship = normalizeText(relationship);
  if (normalizedRelationship === "  " || normalizedRelationship === "  ") {
    return "2 ";
  }

  return "";
}

function buildBuddyWorkflow(question = "", contextPayload = {}) {
  const location = contextPayload?.location || {};
```

```
  const fields = {
    departure:
      readStateValue(contextPayload, ["departure", "origin"]) ||
      normalizeText(location.city || location.displayName),
    destination: extractDestination(question, contextPayload),
    time: extractTime(question, contextPayload),
    companionPreference: extractCompanionPreference(question, contextPayload),
  };

  const fieldDefs = [
    { key: "departure", label: "   ", ask: "           " },
    { key: "destination", label: "   ", ask: "         " },
    { key: "time", label: "  ", ask: "          " },
    { key: "companionPreference", label: "    ", ask: "             " },
  ];

  return {
    mainline: MAINLINE.BUDDY,
    fields,
    missingField: fieldDefs.find((item) => !fields[item.key]) || null,
  };
}

function buildGuideWorkflow(question = "", contextPayload = {}) {
  const relationship = extractRelationship(question, contextPayload);
  const fields = {
    time: extractTime(question, contextPayload),
    peopleCount: extractCount(question, contextPayload, ["peopleCount"], /([           \d]+(?: | ))/u),
    relationship,
    budget: extractGuideBudgetValue(question, contextPayload),
    destination: extractDestination(question, contextPayload),
  };
  fields.peopleCount = normalizePeopleCountFromRelationship(relationship, fields.peopleCount);

  const fieldDefs = [
    { key: "destination", label: "   ", ask: "         " },
    { key: "time", label: "  ", ask: "         " },
    { key: "peopleCount", label: "  ", ask: "          " },
    { key: "relationship", label: "  ", ask: "              " },
    { key: "budget", label: "  ", ask: "               " },
  ];

  return {
    mainline: MAINLINE.GUIDE,
    fields,
    missingField: fieldDefs.find((item) => !fields[item.key]) || null,
  };
}

function buildBuddyCandidateCard(currentUser = {}, candidate = {}, fields = {}) {
  const currentTags = new Set(normalizeArray(currentUser.dnaTags));
  const candidateTags = normalizeArray(candidate.dnaTags);
  const sharedTags = candidateTags.filter((tag) => currentTags.has(tag));
  const currentCity = removeRegionSuffix(currentUser.city || currentUser.province);
  const candidateCity = removeRegionSuffix(candidate.city || candidate.province);
  const destination = normalizeText(fields.destination);
  const companionPreference = normalizeText(fields.companionPreference);
  const candidateText = joinSearchText(candidate);

  let score = 40;
```

```
  if (currentCity && candidateCity && currentCity === candidateCity) score += 18;
  score += Math.min(sharedTags.length * 10, 30);
  if (companionPreference && candidateText.includes(normalizeTextLower(companionPreference))) score += 15;
  if (destination && candidateText.includes(normalizeTextLower(destination))) score += 10;
  if (candidate.profileCompleted) score += 5;
  if (candidate.dnaCompleted) score += 5;

  const reasons = [];
  if (currentCity && candidateCity && currentCity === candidateCity) reasons.push("    ");
  if (sharedTags.length) reasons.push(`DNA      ${Math.min(sharedTags.length, 3)}  `);
  if (companionPreference && candidateText.includes(normalizeTextLower(companionPreference))) {
    reasons.push(`    “${companionPreference}”`);
  }
  if (!reasons.length) reasons.push("           ");

  return {
    id: normalizeText(candidate._id || candidate.id),
    type: "buddy",
    title: normalizeText(candidate.nickName) || "    ",
    region:
      [candidate.province, candidate.city, candidate.district]
        .map(removeRegionSuffix)
        .filter(Boolean)
        .join(" / ") || "     ",
    priceText: "    ",
    summary: normalizeText(candidate.signature || candidate.bio || "") || "    ",
    tags: uniqueList(sharedTags.concat(candidateTags)).slice(0, 4),
    matchReason: reasons.join(" "),
    score,
  };
}

async function buildBuddyCandidates(contextPayload = {}, cloudbaseUserId = "", workflow = {}) {
  const currentUser = await fetchUserByOpenid(cloudbaseUserId);
  if (!currentUser) {
    const users = await fetchCollection("users");
    console.warn("[agent-yuxiaohe] buildBuddyCandidates current user missing", {
      cloudbaseUserId: maskIdentifier(cloudbaseUserId),
      totalUsers: users.length,
      usersWithOpenid: users.filter((item) => normalizeText(item?.openid)).length,
      usersWith_Openid: users.filter((item) => normalizeText(item?._openid)).length,
      usersWithMatchingOpenid: users.filter(
        (item) => normalizeText(item?.openid) === normalizeText(cloudbaseUserId)
      ).length,
      usersWithMatching_Openid: users.filter(
        (item) => normalizeText(item?._openid) === normalizeText(cloudbaseUserId)
      ).length,
    });
    return [];
  }

  const users = await fetchCollection("users");
  const normalizedCloudbaseUserId = normalizeText(cloudbaseUserId);
  const withOpenid = users.filter((item) => normalizeText(item?.openid));
  const excludingSelf = withOpenid.filter(
    (item) => normalizeText(item?.openid) !== normalizedCloudbaseUserId
```

```
  );
  const profileCompletedUsers = excludingSelf.filter((item) => item?.profileCompleted);
  const dnaCompletedUsers = profileCompletedUsers.filter((item) => item?.dnaCompleted);

  console.log("[agent-yuxiaohe] buildBuddyCandidates stats", {
    cloudbaseUserId: maskIdentifier(normalizedCloudbaseUserId),
    currentUserFound: true,
    currentUserOpenid: maskIdentifier(currentUser?.openid || currentUser?._openid || "")
,
    currentUserProfileCompleted: Boolean(currentUser?.profileCompleted),
    currentUserDnaCompleted: Boolean(currentUser?.dnaCompleted),
    workflowFields: {
      departure: normalizeText(workflow?.fields?.departure),
      destination: normalizeText(workflow?.fields?.destination),
      time: normalizeText(workflow?.fields?.time),
      companionPreference: normalizeText(workflow?.fields?.companionPreference),
    },
    totalUsers: users.length,
    usersWithOpenid: withOpenid.length,
    usersWith_Openid: users.filter((item) => normalizeText(item?._openid)).length,
    excludingSelf: excludingSelf.length,
    profileCompleted: profileCompletedUsers.length,
    dnaCompleted: dnaCompletedUsers.length,
    finalCandidatePool: dnaCompletedUsers.length,
    sampleCandidateOpenids: dnaCompletedUsers
      .slice(0, 5)
      .map((item) => maskIdentifier(item?.openid || item?._openid || "")),
  });

  return dnaCompletedUsers
    .map((item) => buildBuddyCandidateCard(currentUser, item, workflow.fields || {}))
    .sort((left, right) => right.score - left.score)
    .slice(0, BUDDY_MAX_CANDIDATES);
}

export async function buildConversationWorkflow({
  question = "",
  contextPayload = {},
  cloudbaseUserId = "",
} = {}) {
  const mainline = detectMainline(question, contextPayload);
  const currentTaskState = getCurrentTaskState(contextPayload);

  if (mainline === MAINLINE.BUDDY) {
    const workflow = buildBuddyWorkflow(question, contextPayload);
    const buddyCandidates = workflow.missingField
      ? []
      : await buildBuddyCandidates(contextPayload, cloudbaseUserId, workflow);

    return {
      ...workflow,
      isReady: !workflow.missingField,
      buddyCandidates,
      lastAskedField: normalizeText(currentTaskState.lastAskedField),
    };
  }

  if (mainline === MAINLINE.GUIDE) {
    const workflow = buildGuideWorkflow(question, contextPayload);
    return {
      ...workflow,
      isReady: !workflow.missingField,
```

```
      lastAskedField: normalizeText(currentTaskState.lastAskedField),
    };
  }

  if (mainline === MAINLINE.FEEDBACK) {
    return {
      mainline,
      feedbackType:
        normalizeText(currentTaskState.feedbackType) ||
        classifyFeedbackType(question),
    };
  }

  return {
    mainline,
    lastAskedField: normalizeText(currentTaskState.lastAskedField),
  };
}

export async function buildUnifiedTaskState({
  question = "",
  contextPayload = {},
  cloudbaseUserId = "",
} = {}) {
  const workflow = await buildConversationWorkflow({
    question,
    contextPayload,
    cloudbaseUserId,
  });
  const currentTaskState = getCurrentTaskState(contextPayload);

  return {
    mainline: normalizeText(workflow?.mainline || currentTaskState?.mainline),
    collected: workflow?.fields || currentTaskState?.collected || {},
    missingField: workflow?.missingField
      ? {
          key: normalizeText(workflow.missingField.key),
          label: normalizeText(workflow.missingField.label),
          ask: normalizeText(workflow.missingField.ask),
        }
      : null,
    lastAskedField: normalizeText(
      workflow?.missingField?.key || workflow?.lastAskedField || currentTaskState?.lastAskedField
    ),
    feedbackType: normalizeText(workflow?.feedbackType || currentTaskState?.feedbackType),
    isReady: Boolean(workflow?.isReady),
  };
}

export async function buildBuddyMatchResult({
  departure = "",
  destination = "",
  time = "",
  companionPreference = "",
  cloudbaseUserId = "",
  contextPayload = {},
} = {}) {
  const mergedContextPayload = {
    ...(contextPayload || {}),
    currentTaskState: {
```

```
      ...(contextPayload?.currentTaskState || {}),
      mainline: MAINLINE.BUDDY,
      collected: {
        ...(contextPayload?.currentTaskState?.collected || {}),
        departure: normalizeText(departure),
        destination: normalizeText(destination),
        time: normalizeText(time),
        companionPreference: normalizeText(companionPreference),
      },
    },
  };

  const workflow = await buildConversationWorkflow({
    question: "",
    contextPayload: mergedContextPayload,
    cloudbaseUserId,
  });

  return {
    success: true,
    mainline: workflow?.mainline || MAINLINE.BUDDY,
    isReady: !!workflow?.isReady,
    missingField: workflow?.missingField
      ? {
          key: normalizeText(workflow.missingField.key),
          label: normalizeText(workflow.missingField.label),
          ask: normalizeText(workflow.missingField.ask),
        }
      : null,
    collected: workflow?.fields || {},
    candidates: normalizeBuddyCandidates(workflow?.buddyCandidates),
  };
}

export function buildGuideRecallQuestion(fields = {}) {
  const segments = [
    normalizeText(fields.destination),
    normalizeText(fields.time),
    normalizeText(fields.peopleCount),
    normalizeText(fields.relationship),
    normalizeText(fields.budget),
    "  ",
    "  ",
    "  ",
    "  ",
  ].filter(Boolean);

  return segments.join(" ");
}

export function buildGuideGroundingPayload({
  question = "",
  contextPayload = {},
  guideFields = {},
} = {}) {
  const normalizedFields =
    guideFields && typeof guideFields === "object" ? guideFields : {};

  return {
    ...(contextPayload || {}),
    question: buildGuideRecallQuestion(normalizedFields) || normalizeText(question),
    location: contextPayload?.location || {},
```

```
    userProfile: contextPayload?.userProfile || {},
    preferences: contextPayload?.preferences || {},
    skillContext: {
      ...(contextPayload?.skillContext || {}),
      mode: MAINLINE.GUIDE,
      title: normalizeText(contextPayload?.skillContext?.title) || "    ",
      collected: {
        ...(contextPayload?.skillContext?.collected || {}),
        ...normalizedFields,
      },
    },
    currentTaskState: {
      ...(contextPayload?.currentTaskState || {}),
      mainline: MAINLINE.GUIDE,
      collected: {
        ...(contextPayload?.currentTaskState?.collected || {}),
        ...normalizedFields,
      },
    },
  };
}

function normalizeGuideCandidates(candidates = []) {
  return (Array.isArray(candidates) ? candidates : [])
    .filter((item) => normalizeText(item.type) !== "product")
    .slice(0, GUIDE_MAX_CANDIDATES)
    .map((item) => ({
      id: normalizeText(item.id),
      type: normalizeText(item.type),
      title: normalizeText(item.title),
      region: normalizeText(item.region),
      priceText: normalizeText(item.priceText),
      summary: normalizeText(item.summary),
      tags: normalizeArray(item.tags),
    }));
}

function normalizeBuddyCandidates(candidates = []) {
  return (Array.isArray(candidates) ? candidates : []).map((item) => ({
    id: normalizeText(item.id),
    nickname: normalizeText(item.title),
    region: normalizeText(item.region),
    summary: normalizeText(item.summary),
    tags: normalizeArray(item.tags),
    matchReason: normalizeText(item.matchReason),
  }));
}

export async function buildGuideCustomizationResult({
  time = "",
  peopleCount = "",
  relationship = "",
  budget = "",
  destination = "",
  contextPayload = {},
} = {}) {
  const mergedContextPayload = {
    ...(contextPayload || {}),
    currentTaskState: {
      ...(contextPayload?.currentTaskState || {}),
      mainline: MAINLINE.GUIDE,
      collected: {
```

```
        ...(contextPayload?.currentTaskState?.collected || {}),
        time: normalizeText(time),
        peopleCount: normalizeText(peopleCount),
        relationship: normalizeText(relationship),
        budget: normalizeText(budget),
        destination: normalizeText(destination),
      },
    },
  };

  const workflow = await buildConversationWorkflow({
    question: "",
    contextPayload: mergedContextPayload,
    cloudbaseUserId: "",
  });

  if (!workflow?.isReady) {
    return {
      success: true,
      mainline: workflow?.mainline || MAINLINE.GUIDE,
      isReady: false,
      missingField: workflow?.missingField
        ? {
            key: normalizeText(workflow.missingField.key),
            label: normalizeText(workflow.missingField.label),
            ask: normalizeText(workflow.missingField.ask),
          }
        : null,
      collected: workflow?.fields || {},
      candidates: [],
    };
  }

  const groundingContext = await buildPlatformGroundingContext(
    buildGuideGroundingPayload({
      question: buildGuideRecallQuestion(workflow.fields || {}),
      contextPayload: mergedContextPayload,
      guideFields: workflow.fields || {},
    })
  );

  return {
    success: true,
    mainline: workflow?.mainline || MAINLINE.GUIDE,
    isReady: true,
    missingField: null,
    collected: workflow?.fields || {},
    candidates: normalizeGuideCandidates(groundingContext?.candidates || []),
  };
}

export async function buildConversationStateResult({
  question = "",
  contextPayload = {},
  cloudbaseUserId = "",
} = {}) {
  const workflow = await buildConversationWorkflow({
    question,
    contextPayload,
    cloudbaseUserId,
  });
  const unifiedTaskState = await buildUnifiedTaskState({
```

```
    question,
    contextPayload,
    cloudbaseUserId,
  });

  let candidates = [];

  if (
    normalizeText(unifiedTaskState?.mainline) === MAINLINE.GUIDE &&
    Boolean(unifiedTaskState?.isReady)
  ) {
    const guideFields = workflow?.fields || unifiedTaskState?.collected || {};
    const groundingContext = await buildPlatformGroundingContext(
      buildGuideGroundingPayload({
        question,
        contextPayload,
        guideFields,
      })
    );

    candidates = normalizeGuideCandidates(groundingContext?.candidates || []);
  }

  if (
    normalizeText(unifiedTaskState?.mainline) === MAINLINE.BUDDY &&
    Boolean(unifiedTaskState?.isReady)
  ) {
    candidates = normalizeBuddyCandidates(workflow?.buddyCandidates);
  }

  return {
    success: true,
    mainline: normalizeText(unifiedTaskState?.mainline),
    collected: unifiedTaskState?.collected || {},
    missingField: unifiedTaskState?.missingField || null,
    lastAskedField: normalizeText(unifiedTaskState?.lastAskedField),
    feedbackType: normalizeText(unifiedTaskState?.feedbackType),
    isReady: Boolean(unifiedTaskState?.isReady),
    candidates,
  };
}

export function buildAgentUserPrompt({
  question = "",
  groundingContext = {},
  workflowContext = {},
  contextPayload = {},
} = {}) {
  const normalizedQuestion = normalizeText(question);
  const regionLabel = normalizeText(groundingContext?.regionLabel) || DEFAULT_REGION_LABEL;
  const profileTags = normalizeArray(contextPayload?.userProfile?.dnaTags).join(" ");
  const preferencePairs = Object.entries(contextPayload?.preferences || {})
    .map(([key, value]) => [normalizeText(key), normalizeText(value)])
    .filter(([, value]) => value)
    .map(([key, value]) => `${key}: ${value}`);
  const skillMode = normalizeText(contextPayload?.skillContext?.mode);
  const skillTitle = normalizeText(contextPayload?.skillContext?.title);
  const skillPairs = Object.entries(getMergedCollected(contextPayload))
    .map(([key, value]) => [normalizeText(key), normalizeText(value)])
    .filter(([, value]) => value)
    .map(([key, value]) => `${key}: ${value}`);
```

```
  const mainline = workflowContext?.mainline || detectMainline(question, contextPayload);
  const shouldAttachPlatformGrounding =
    mainline === MAINLINE.GUIDE || mainline === MAINLINE.GENERIC;

  const buddyFields = workflowContext?.fields || {};
  const guideFields = workflowContext?.fields || {};
  const buddyCandidates = Array.isArray(workflowContext?.buddyCandidates)
    ? workflowContext.buddyCandidates
    : [];

  const mainlineInstruction =
    mainline === MAINLINE.BUDDY
      ? [
          "         ",
          "      1         ",
          "                      ",
          `        =${buddyFields.departure || "   "}    =${buddyFields.destination || "   "}   =${buddyFields.time || "   "}     =${buddyFields.companionPreference || "   "} `,
          workflowContext?.missingField
            ? `                ${workflowContext.missingField.ask}`
            : buddyCandidates.length
              ? `                           \n${buddyCandidates.map((item, index) => `${index + 1}. ${item.title} ${item.region} ${item.matchReason}`).join("\n")}`
              : "                    “        ”                  ",
          "               ",
          "    “  ”            ",
        ].join("\n")
      : mainline === MAINLINE.GUIDE
        ? [
            "          ",
            "                    ",
            `       =${guideFields.time || "   "}   =${guideFields.peopleCount || "   "}   =${guideFields.relationship || "   "}   =${guideFields.budget || "   "}    =${guideFields.destination || "   "} `,
            workflowContext?.missingField
              ? `                 ${workflowContext.missingField.ask}`
              : "                    “       ” ",
            "              ",
          ].join("\n")
        : mainline === MAINLINE.FEEDBACK
          ? [
              "          ",
              `       ${workflowContext?.feedbackType || "    "} `,
              "             ",
              "                          ",
            ].join("\n")
          : mainline === MAINLINE.WEATHER
            ? "       /                        "
            : [
                "             ",
                "            ",
                "            ",
                "             ",
                "                ",
              ].join("\n");

  const sections = [
    "     “   ”      “  ” ",
    "“   ”                    ",
    "                            ",
```

```
    "               ",
    skillMode ? `           ${skillTitle || skillMode}` : "",
    skillPairs.length ? `        ${skillPairs.join(" ")}` : "",
    `       ${mainline}`,
    mainlineInstruction,
    `       ${normalizedQuestion}`,
    `       ${regionLabel}`,
    profileTags ? `       ${profileTags}` : "",
    preferencePairs.length ? `     ${preferencePairs.join(" ")}` : "",
    shouldAttachPlatformGrounding ? groundingContext?.prompt || "" : "",
    mainline === MAINLINE.GUIDE && workflowContext?.isReady
      ? "                                     1. 2. 3.
            "
      : "",
    "           ",
  ];

  return sections.filter(Boolean).join("\n\n");
}

export class DetectCloudbaseUserMiddleware {
  constructor(request) {
    this.request = request;
  }

  async beforeAgent({ input }) {
    const authorization =
      this.request?.headers?.authorization ||
      this.request?.headers?.Authorization ||
      this.request?.headers?.get?.("authorization") ||
      this.request?.headers?.get?.("Authorization") ||
      "";

    const token = authorization.startsWith("Bearer ") ? authorization.slice(7) : "";
    if (!token) return;

    try {
      const payload = jwtDecode(token);
      input.forwardedProps = {
        ...(input.forwardedProps || {}),
        cloudbaseUserId: payload?.uid || payload?.sub || payload?.openid || payload?.user_id || "",
      };
    } catch (error) {
      console.warn("[DetectCloudbaseUserMiddleware] jwt decode failed", error?.message || error);
    }
  }
}
```

Path: cloudfunctions/agent-yuxiaohe-1grmumm967563411/package.json

---

```
1 {
2   "name": "agent-yuxiaohe",
3   "version": "1.0.0",
4   "description": "CloudBase Agent service for    ",
5   "main": "src/index.js",
6   "type": "module",
7   "scripts": {
8     "start": "node src/index.js"
9   },
10  "dependencies": {
11    "@cloudbase/agent-adapter-adp": "^1.0.1-alpha.30",
12    "@cloudbase/agent-adapter-wx": "^1.0.1-alpha.30",
13    "@cloudbase/agent-server": "^0.0.20",
14    "@cloudbase/node-sdk": "^3.18.1",
15    "@dotenvx/dotenvx": "^1.51.1",
16    "express": "^5.2.1",
17    "jwt-decode": "^4.0.0",
18    "rxjs": "7.8.1"
19  },
20  "overrides": {
21    "rxjs": "7.8.1"
22  }
23 }
```